

RESTEasy

Version 8.0.0.Alpha1, 2026-11-09

Table of Contents

1. Overview . . . 1
2. License . . . 2
3. Installation/Configuration . . . 3
   3.1. RESTEasy modules in WildFly . . . 3
   3.2. Deploying a RESTEasy application to WildFly . . . 5
   3.3. Deploying to other servlet containers . . . 6
   3.4. Configuration . . . 9
   3.5. Configuration switches . . . 15
   3.6. `jakarta.ws.rs.core.Application` . . . 23
   3.7. Client side . . . 23
   3.8. Configuring Providers . . . 23
4. Using `@Path` and `@GET`, `@POST`, etc. . . . 25
   4.1. `@Path` and regular expression mappings . . . 26
5. @PathParam . . . 27
   5.1. Advanced @PathParam and Regular Expressions . . . 27
   5.2. @PathParam and PathSegment . . . 28
6. @QueryParam . . . 29
7. @HeaderParam . . . 30
   7.1. Header Delegates . . . 30
8. Linking resources . . . 32
   8.1. Link Headers . . . 32
   8.2. Atom links in the resource representations . . . 32
9. @MatrixParam . . . 43
10. @CookieParam . . . 44
11. @FormParam . . . 45
12. @Form . . . 46
13. Improved `@···Param` annotations . . . 49
14. Optional parameter types . . . 50
15. @DefaultValue . . . 52
16. @Encoded and encoding . . . 53
17. @Context . . . 54
18. Jakarta RESTful Web Services Resource Locators and Sub Resources . . . 55
19. Resources metadata configuration . . . 58
20. Jakarta RESTful Web Services Content Negotiation . . . 60
   20.1. URL-based negotiation . . . 61
   20.2. Query String Parameter-based negotiation . . . 62
21. Content Marshalling/Providers . . . 63
   21.1. Default Providers and default Jakarta RESTful Web Services Content Marshalling . . . 63

21.2. Content Marshalling with @Provider classes . . . 63
21.3. Providers Utility Class . . . 65
21.4. Configuring Document Marshalling . . . 66
21.5. Text media types and character sets . . . 67
22. Jakarta XML Binding providers . . . 69
22.1. Jakarta XML Binding Decorators . . . 70
22.2. Pluggable JAXBContext's with ContextResolvers . . . 71
22.3. Jakarta XML Binding + XML provider . . . 72
22.4. Jakarta XML Binding + JSON provider . . . 73
22.5. Jakarta XML Binding + FastinfoSet provider . . . 75
22.6. Arrays and Collections of Jakarta XML Binding Objects . . . 75
22.7. Maps of XML Objects . . . 78
22.8. Interfaces, Abstract Classes, and Jakarta XML Binding . . . 81
22.9. Configuring Jakarta XML Binding Marshalling . . . 82
23. RESTEasy Atom Support . . . 85
23.1. RESTEasy Atom API and Provider . . . 85
23.2. Using Jakarta XML Binding with the Atom Provider . . . 86
24. JSON Support via Jackson . . . 88
24.1. Using Jackson 2 Outside of WildFly . . . 88
24.2. Jakarta XML Binding Annotations . . . 88
24.3. Additional RESTEasy Specifics . . . 89
24.4. JSONP Support . . . 89
24.5. Jackson JSON Decorator . . . 90
24.6. JSON Filter Support . . . 91
25. Polymorphic Typing deserialization . . . 93
26. JSON Support via Jakarta EE JSON-P API . . . 94
27. Multipart Providers . . . 95
27.1. Multipart/mixed . . . 95
27.2. Multipart/related . . . 103
27.3. Multipart/form-data . . . 107
27.4. Note about multipart parsing and working with other frameworks . . . 117
27.5. Overwriting the default fallback content type for multipart messages . . . 118
27.6. Overwriting the content type for multipart messages . . . 118
27.7. Overwriting the default fallback charset for multipart messages . . . 119
28. Jakarta RESTful Web Services 2.1 Additions . . . 120
28.1. `CompletionStage` support . . . 120
29. Reactive Clients API . . . 121
29.1. Server-Sent Events (SSE) . . . 121
29.2. Java API for JSON Binding . . . 123
29.3. JSON Patch and JSON Merge Patch . . . 123
30. String marshalling for String based @*Param . . . 127

30.1. Simple conversion . . . 127
30.2. ParamConverter . . . 127
30.3. StringParameterUnmarshaller . . . 129
30.4. Collections . . . 130
30.5. Extension to `ParamConverter` semantics . . . 134
30.6. Default multiple valued `ParamConverter` . . . 137
31. Responses using jakarta.ws.rs.core.Response . . . 141
32. Exception Handling . . . 142
32.1. Exception Mappers . . . 142
32.2. RESTEasy Built-in Internally-Thrown Exceptions . . . 142
32.3. Resteasy WebApplicationExceptions . . . 144
32.4. Overriding RESTEasy Builtin Exceptions . . . 146
33. Content encoding . . . 147
33.1. GZIP Compression/Decompression . . . 147
33.2. General content encoding . . . 149
34. CORS . . . 151
34.1. Credentials . . . 151
35. Content-Range Support . . . 152
36. RESTEasy Caching Features . . . 153
36.1. @Cache and @NoCache Annotations . . . 153
36.2. Client "Browser" Cache . . . 153
36.3. Local Server-Side Response Cache . . . 155
36.4. HTTP preconditions . . . 156
37. Filters and Interceptors . . . 157
37.1. Server Side Filters . . . 157
37.2. Client Side Filters . . . 159
37.3. Reader and Writer Interceptors . . . 159
37.4. Per Resource Method Filters and Interceptors . . . 160
37.5. Ordering . . . 160
38. Asynchronous HTTP Request Processing . . . 161
38.1. Using the `@Suspended` annotation . . . 161
38.2. Using Reactive return types . . . 162
38.3. Asynchronous filters . . . 163
38.4. Asynchronous IO . . . 163
39. Asynchronous Job Service . . . 165
39.1. Using Async Jobs . . . 165
39.2. Oneway: Fire and Forget . . . 166
39.3. Setup and Configuration . . . 166
40. Asynchronous Injection . . . 168
40.1. org.jboss.resteasy.spi.ContextInjector Interface . . . 168
40.2. Single<Foo> Example . . . 168

40.3. Async Injector With Annotations Example . . . 169
41. Reactive programming support . . . 171
41.1. CompletionStage . . . 171
41.2. CompletionStage in Jakarta RESTful Web Services . . . 173
41.3. Beyond CompletionStage . . . 178
41.4. Pluggable reactive types: RxJava 2 in RESTEasy . . . 179
41.5. Proxies . . . 189
41.6. Adding extensions . . . 191
42. Jakarta RESTful Web Services SeBootstrap . . . 194
42.1. Overview . . . 194
42.2. Usage . . . 195
42.3. Configuration Options . . . 195
42.4. Custom EmbeddedServer Implementations . . . 196
43. Embedded Containers . . . 198
43.1. Undertow . . . 198
43.2. Oracle JDK HTTP Server . . . 200
43.3. Netty . . . 200
43.4. Reactor-Netty . . . 201
43.5. Jetty . . . 202
43.6. Vert.x . . . 202
43.7. EmbeddedJaxrsServer . . . 202
44. Server-side Mock Framework . . . 203
45. Securing Jakarta RESTful Web Services and RESTEasy . . . 204
46. JSON Web Signature and Encryption (JOSE-JWT) . . . 206
47. JSON Web Signature (JWS) . . . 207
48. JSON Web Encryption (JWE) . . . 208
49. Doseta Digital Signature Framework . . . 210
49.1. Maven settings . . . 211
49.2. Signing API . . . 212
49.3. Signature Verification API . . . 213
49.4. Managing Keys via a KeyRepository . . . 215
50. Body Encryption and Signing via SMIME . . . 219
50.1. Maven settings . . . 219
50.2. Message Body Encryption . . . 219
50.3. Message Body Signing . . . 221
50.4. application/pkcs7-signature . . . 223
51. Jakarta Enterprise Beans Integration . . . 224
51.1. Automatic Discovery via CDI (Recommended) . . . 224
51.2. Manual JNDI Registration (Legacy) . . . 225
52. Spring Integration . . . 226
52.1. Overview . . . 226

52.2. Basic Integration . . . 227
52.3. Customized Configuration . . . 229
52.4. Spring MVC Integration . . . 230
52.5. Undertow Embedded Spring Container . . . 233
52.6. Processing Spring Web REST annotations in RESTEasy . . . 235
52.7. Spring Boot starter . . . 236
52.8. Upgrading in Wildfly . . . 237
53. CDI Integration . . . 238
53.1. Using CDI beans as Jakarta RESTful Web Services components . . . 238
53.2. Default scopes . . . 238
53.3. Constructor Injection . . . 239
53.4. Parameter Injection with CDI Qualifiers . . . 240
53.5. Configuration within WildFly . . . 243
53.6. Configuration with different distributions . . . 243
54. RESTEasy Client API . . . 244
54.1. Jakarta RESTful Web Services Client API . . . 244
54.2. RESTEasy Proxy Framework . . . 245
54.3. Apache HTTP Client 4.x and other backends . . . 251
54.4. Client Utilities . . . 260
55. MicroProfile Rest Client . . . 261
55.1. Client proxies . . . 261
55.2. Concepts imported from Jakarta RESTful Web Services . . . 264
55.3. Beyond Jakarta RESTful Web Services and RESTEasy . . . 264
56. AJAX Client . . . 275
56.1. Generated JavaScript API . . . 275
56.2. Using the JavaScript API to build AJAX queries . . . 282
56.3. Caching Features . . . 283
57. RESTEasy WADL Support . . . 285
57.1. RESTEasy WADL Support for Servlet Container **(Deprecated)** . . . 285
57.2. RESTEasy WADL Support for Servlet Container(Updated) . . . 285
57.3. RESTEasy WADL support for Sun JDK HTTP Server . . . 286
57.4. RESTEasy WADL support for Netty Container . . . 287
57.5. RESTEasy WADL Support for Undertow Container . . . 288
58. RESTEasy Tracing Feature . . . 290
58.1. Overview . . . 290
58.2. Tracing Info Mode . . . 290
58.3. Tracing Info Level . . . 290
58.4. Basic Usages . . . 290
58.5. Client Side Tracing Info . . . 293
58.6. Json Formatted Response . . . 293
58.7. ON_DEMAND Mode Usage . . . 295

58.8. List Of Tracing Events . . . 300
58.9. Tracing Example . . . 301
59. Validation . . . 303
59.1. Violation reporting . . . 304
Validation Service Providers . . . 307
59.2. Validation Implementations . . . 310
60. Internationalization and Localization . . . 312
60.1. Internationalization . . . 312
61. Localization . . . 314
62. Maven and RESTEasy . . . 315
63. Migrating to RESTEasy 8 . . . 317
63.1. Changes that fail the deployment . . . 317
63.2. Default scopes and warnings . . . 318
63.3. Behavior changes . . . 319
63.4. Configuration . . . 321
63.5. Requirements . . . 321
63.6. Changes for integrations . . . 321
64. Migration from older versions . . . 323
64.1. Migration to RESTEasy 3.0 series . . . 323
64.2. Migration to RESTEasy 3.1 series . . . 323
64.3. Migration to RESTEasy 3.5+ series . . . 325
64.4. Migration to RESTEasy 4 series . . . 325
65. Books You Can Read . . . 329

Chapter 1. Overview



This version, 8.0.0.Alpha1, of RESTEasy supports Jakarta RESTful Web Services 5.0.

Originally released in 2009, RESTEasy has tracked and implemented a series of official specifications that provide a Java API for RESTful Web Services over the HTTP protocol:

RESTEasy version	Specification	JavaDoc/Changes
7.0+	Jakarta RESTful Web Services 4.0	• JavaDoc • Changes
6.1+	Jakarta RESTful Web Services 3.1	• JavaDoc • Changes
6.0	Jakarta RESTful Web Services 3.0	• JavaDoc • Changes
3.5+	Jakarta RESTful Web Services 2.1	• JavaDoc
3+	JAX-RS 2.0	
2+	JAX-RS 1.0	

JAX-RS 1.0 (JSR-311), JAX-RS 2.0 (JSR-339), and JAX-RS 2.1 (JSR-370) are Java Community Process (JCP) specifications. In 2017, the Java Enterprise Edition (Java EE) specifications, including JAX-RS, were transferred to the Eclipse Foundation (Java EE Moves to the Eclipse Foundation), Java EE became Jakarta Enterprise Edition, and new governing committees under the umbrella of the Eclipse Jakarta EE Platform were constituted. For JAX-RS, the specifications are provided by the Jakarta RESTful Web Services committee. The first specification released under the authority of Jakarta RESTful Web Services was Jakarta RESTful Web Services 2.1, that being essentially identical to JAX-RS 2.1.

RESTEasy is a portable implementation of these specifications which can run in any Servlet container. Tighter integration with the WildFly application server is also available to make the user experience nicer in that environment. RESTEasy also comes with additional features on top of the plain old specifications.



References in this document to JAX-RS refer to the Jakarta RESTful Web Services unless otherwise noted.

Chapter 2. License

RESTEasy is distributed under the Apache License 2.0. Some dependencies are covered by other open source licenses.

Chapter 3. Installation/Configuration

RESTEasy is installed and configured in different ways depending on which environment you are running in. If you are running in WildFly, RESTEasy is already bundled and integrated completely so there is very little you have to do. If you are running in a different environment, there is some manual installation and configuration you will have to do.

3.1. RESTEasy modules in WildFly

In WildFly, RESTEasy and the Jakarta RESTful Web Services API are automatically loaded into your deployment's classpath if and only if you are deploying a Jakarta RESTful Web Services application (as determined by the presence of Jakarta RESTful Web Services annotations). However, only some RESTEasy features are automatically loaded. See the modules table. If you need any of those libraries which are not loaded automatically, you'll have to bring them in with a `jboss-deployment-structure.xml` file in the WEB-INF directory of your WAR file. Here's an example:

```
<jboss-deployment-structure>
    <deployment>
        <dependencies>
            <module name="org.jboss.resteasy.resteasy-jackson2-provider" services=
"import"/>
        </dependencies>
    </deployment>
</jboss-deployment-structure>
```

The 'services` attribute must be set to "import" for modules that have default providers in a `META-INF/services/jakarta.ws.rs.ext.Providers` file.



Using the `META-INF/services/jakarta.ws.rs.ext.Providers` is RESTEasy specific and not considered portable.

To get an idea of which RESTEasy modules are loaded by default when services are deployed, please see the table below, which refers to a recent WildFly ditribution patched with the current RESTEasy distribution. Clearly, future and unpatched WildFly distributions might differ a bit in terms of modules enabled by default, as the container actually controls this too.

Table 1. Modules

Module Name	Loaded by Default	Description
org.jboss.resteasy.resteasy-atom-provider	yes	RESTEasy's atom library
org.jboss.resteasy.resteasy-cdi	yes	RESTEasy CDI integration
org.jboss.resteasy.resteasy-crypto	yes	S/MIME, DKIM, and support for other security formats.

Module Name	Loaded by Default	Description
org.jboss.resteasy.resteasy-jackson2-provider	yes	Integration with the JSON parser and object mapper Jackson 2
org.jboss.resteasy.resteasy-jaxb-provider	yes	Jakarta XML Binding integration.
org.jboss.resteasy.resteasy-core	yes	Core RESTEasy libraries for server.
org.jboss.resteasy.resteasy-client	yes	Core RESTEasy libraries for client.
org.jboss.resteasy.jose-jwt	no	JSON Web Token support.
org.jboss.resteasy.resteasy-jsapi	yes	RESTEasy's Javascript API
org.jboss.resteasy.resteasy-json-p-provider	yes	JSON parsing API
org.jboss.resteasy.resteasy-json-binding-provider	yes	JSON binding API
jakarta.json.bind-api	yes	JSON binding API
org.eclipse.yasson	yes	RI implementation of JSON binding API
org.jboss.resteasy.resteasy-multipart-provider	yes	Support for multipart formats
org.jboss.resteasy.resteasy-spring	yes (if Spring is present in the deployment)	Spring provider
org.jboss.resteasy.resteasy-validator-provider	yes	RESTEasy's interface to Hibernate Bean Validation

3.1.1. Other RESTEasy modules

Not all RESTEasy modules are bundled with WildFly. For example, `resteasy-fastinfoset-provider` and `resteasy-wadl` are not included among the modules listed in RESTEasy modules in WildFly. If you want to use them in your application, you can include them in your WAR as you would if you were deploying outside of WildFly. See Deploying to other servlet containers for more information.

3.1.2. Upgrading RESTEasy within WildFly

RESTEasy is bundled with WildFly. However, you may wish to upgrade to the latest version. With Galleon this makes upgrading RESTEasy in WildFly quite easy.

Using Maven to provision WildFly you can simply use the RESTEasy Channel Manifest.

```
<plugin>
    <groupId>org.wildfly.plugins</groupId>
    <artifactId>wildfly-maven-plugin</artifactId>
```

```
    <executions>
        <execution>
            <id>server-provisioning</id>
            <phase>generate-test-resources</phase>
            <goals>
                <goal>provision</goal>
            </goals>
            <configuration>
                <feature-packs>
                    <feature-pack>
                        <groupId>org.wildfly</groupId>
                        <artifactId>wildfly-ee-galleon-pack</artifactId>
                    </feature-pack>
                </feature-packs>
                <channels>
                    <channel>
                        <manifest>
                            <groupId>org.wildfly.channels</groupId>
                            <artifactId>wildfly-ee</artifactId>
                        </manifest>
                    </channel>
                    <channel>
                        <manifest>
                            <groupId>org.jboss.resteasy.channels</groupId>
                            <artifactId>resteasy-8.0</artifactId>
                        </manifest>
                    </channel>
                </channels>
            </configuration>
        </execution>
    </executions>
</plugin>
```



Please note for versions 6.2.15.Final and 7.0.1.Final the `groupId` for the channel has changed from `dev.resteasy.channels` to `org.jboss.resteasy.channels`.

3.2. Deploying a RESTEasy application to WildFly

RESTEasy is bundled with WildFly and completely integrated as per the requirements of Jakarta EE. A Jakarta RESTful Web Services application can contain Jakarta Enterprise Beans and CDI. WildFly scans the WAR file for the Jakarta RESTful Web Services services and provider classes packaged in the WAR either as POJOs, CDI beans, or Jakarta Enterprise Beans.

The web.xml can supply to RESTEasy init-params and context-params (see Configuration switches) if you want to tweak or turn on/off any specific RESTEasy feature.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
```

```
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
   version="5.0">
</web-app>
```

When a servlet-mapping element is not declared in the web.xml, then a class must be provided that implements `jakarta.ws.rs.core.Application` class (see `jakarta.ws.rs.core.Application`). This class must be annotated with the `jakarta.ws.rs.ApplicationPath` annotation. If this implementation class returns an empty set for classes and singletons, the WAR will be scanned for resource and provider classes as indicated by the presence of Jakarta RESTful Web Services annotations.

```
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@ApplicationPath("/root-path")
public class MyApplication extends Application {
}
```



If the application WAR contains an `Application` class (or a subclass thereof) which is annotated with an `ApplicationPath` annotation, a `web.xml` file is not required. If the application WAR contains an `Application` class but the class doesn't have a declared `@ApplicationPath` annotation, then the web.xml must at least declare a servlet-mapping element.



As mentioned in Other RESTEasy modules, not all RESTEasy modules are bundled with WildFly. For example, resteasy-fastinfoset-provider and resteasy-wadl are not included among the modules listed in RESTEasy modules in WildFly. If they are required by the application, they can be included in the WAR as is done if you were deploying outside of WildFly. See Deploying to other servlet containers for more information.

3.3. Deploying to other servlet containers

If you are using RESTEasy outside of WildFly, in a standalone servlet container like Tomcat or Jetty, for example, you will need to include the appropriate RESTEasy jars in your WAR file. You will need the core classes in the resteasy-core and resteasy-client modules, and you may need additional facilities like the resteasy-jaxb-provider module. We strongly suggest that you use Maven to build your WAR files as RESTEasy is split into a bunch of different modules:

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-core</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
<dependency>
```

```
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-client</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-jaxb-provider</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

You can see sample Maven projects in https://github.com/resteasy/resteasy-examples.

If not using Maven, include the necessary jars by hand. If downloading RESTEasy (from https://resteasy.dev/downloads.html, for example) you will get a file, resteasy-8.0.0.Alpha1-all.zip. Unzip the file. The resulting directory will contain a lib/ directory that contains the libraries needed by RESTEasy. Copy these, as needed, into your /WEB-INF/lib directory. Place your Jakarta RESTful Web Services annotated class resources and providers within one or more jars within /WEB-INF/lib or your raw class files within /WEB-INF/classes.

3.3.1. Servlet Containers

RESTEasy provides an implementation of the Servlet `ServletContainerInitializer` integration interface for containers to use in initializing an application. The container calls this interface during the application's startup phase. The RESTEasy implementation performs automatic scanning for resources and providers, and programmatic registration of a servlet. RESTEasy's implementation is provided in maven artifact, `resteasy-servlet-initializer`. Add this artifact dependency to your project's pom.xml file so the JAR file will be included in your WAR file.

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-servlet-initializer</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

3.3.2. Defining the Servlet in a web.xml

You can manually declare the RESTEasy servlet in the `WEB-INF/web.xml` file of your WAR project, and provide an `Application` class (see `jakarta.ws.rs.core.Application`). For example:

```
<web-app>
    <display-name>Archetype Created Web Application</display-name>

    <servlet>
        <servlet-name>Resteasy</servlet-name>
        <servlet-class>
            org.jboss.resteasy.plugins.server.servlet.HttpServletDispatcher
        </servlet-class>
        <init-param>
```

```
            <param-name>jakarta.ws.rs.Application</param-name>
            <param-value>com.restfully.shop.services.ShoppingApplication</param-value>
        </init-param>
    </servlet>

    <servlet-mapping>
        <servlet-name>Resteasy</servlet-name>
        <url-pattern>/*</url-pattern>
    </servlet-mapping>

</web-app>
```

The RESTEasy servlet is responsible for initializing some basic components of RESTEasy.

3.3.3. RESTEasy as a ServletContextListener

Initialization of RESTEasy can be performed within a 'servletContextListener` instead of within the Servlet. You may need this if you are writing custom Listeners that need to interact with RESTEasy at boot time. An example of this is the RESTEasy Spring integration that requires a Spring ServletContextListener. The `org.jboss.resteasy.plugins.server.servlet.ResteasyBootstrap` class is a `ServletContextListener` that configures an instance of an `ResteasyProviderFactory` and Registry. You can obtain instances of a `ResteasyProviderFactory` and Registry from the 'servletContext` attributes `org.jboss.resteasy.spi.ResteasyProviderFactory` and `org.jboss.resteasy.spi.Registry`. From these instances you can programmatically interact with RESTEasy registration interfaces.

```
<web-app>
   <listener>
      <listener-class>
         org.jboss.resteasy.plugins.server.servlet.ResteasyBootstrap
      </listener-class>
   </listener>

  <!-- ** INSERT YOUR LISTENERS HERE!!!! -->

   <servlet>
      <servlet-name>Resteasy</servlet-name>
      <servlet-class>
         org.jboss.resteasy.plugins.server.servlet.HttpServletDispatcher
      </servlet-class>
   </servlet>

   <servlet-mapping>
      <servlet-name>Resteasy</servlet-name>
      <url-pattern>/Resteasy/*</url-pattern>
   </servlet-mapping>

</web-app>
```

3.3.4. RESTEasy as a Servlet Filter

A downside of running RESTEasy as a Servlet is that you cannot have static resources like .html and .jpeg files in the same path as your Jakarta RESTful Web Services services. RESTEasy allows you to run as a `Filter` instead. If a Jakarta RESTful Web Services resource is not found under the URL requested, RESTEasy will delegate back to the base servlet container to resolve URLs.

```
<web-app>
    <filter>
        <filter-name>Resteasy</filter-name>
        <filter-class>
            org.jboss.resteasy.plugins.server.servlet.FilterDispatcher
        </filter-class>
        <init-param>
            <param-name>jakarta.ws.rs.Application</param-name>
            <param-value>com.restfully.shop.services.ShoppingApplication</param-value>
        </init-param>
    </filter>

    <filter-mapping>
        <filter-name>Resteasy</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>

</web-app>
```

3.4. Configuration

RESTEasy has two mutually exclusive mechanisms for retrieving configuration parameters (see Configuration switches). The classic mechanism depends on context-params and init-params in a web.xml file. Alternatively, the Eclipse MicroProfile Config project (https://github.com/eclipse/microprofile-config) provides a flexible parameter retrieval mechanism that RESTEasy will use if the necessary dependencies are available. See Configuring MicroProfile Config for more about that. If they are not available, it will fall back to an extended form of the classic mechanism.

3.4.1. RESTEasy with MicroProfile Config

In the presence of the Eclipse MicroProfile Config API jar and an implementation of the API (see Configuring MicroProfile Config), RESTEasy will use the facilities of MicroProfile Config for accessing configuration properties (see Configuration switches). MicroProfile Config offers to both RESTEasy users and RESTEasy developers a great deal of flexibility in controlling runtime configuration.

In MicroProfile Config, a `ConfigSource` represents a `Map<String, String>` of property names to values, and a `Config` represents a sequence of ConfigSource's, ordered by priority. The priority of a `ConfigSource` is given by an ordinal (represented by an `int`), with a higher value indicating a higher priority. For a given property name, the ConfigSource's are searched in order until a value is found.

MicroProfile Config mandates the presence of the following `ConfigSource's:

1. a `ConfigSource` based on 'system.getProperties()` (ordinal = 400)
2. a `ConfigSource` based on 'system.getenv()` (ordinal = 300)
3. a `ConfigSource` for each `META-INF/microprofile-config.properties` file on the class path, separately configurable via a config_ordinal property inside each file (default ordinal = 100)

Note that a property which is found among the System properties and which is also in the System environment will be assigned the System property value because of the relative priorities of the ConfigSource's.

The set of config sources is extensible. For example, smallrye-config (https://github.com/smallrye/smallrye-config), the implementation of the MicroProfile Config specification currently used by RESTEasy, adds the following kinds of ConfigSource's:

1. `PropertiesConfigSource` creates a `ConfigSource` from a Java `Properties` object or a `Map<String, String>` object or a properties file (referenced by its URL) (default ordinal = 100).
2. `DirConfigSource` creates a `ConfigSource` that will look into a directory where each file corresponds to a property (the file name is the property key and its textual content is the property value). This `ConfigSource` can be used to read configuration from Kubernetes ConfigMap (default ordinal = 100).
3. `ZkMicroProfileConfig` creates a `ConfigSource` that is backed by Apache Zookeeper (ordinal = 150).

These can be registered programmatically by using an instance of `ConfigProviderResolver`:

```
Config config = new PropertiesConfigSource("file:/// ...");
ConfigProviderResolver.instance().registerConfig(config, getClass().getClassLoader());
```

where `ConfigProviderResolver` is part of the Eclipse API.

If the application is running in Wildfly, then Wildfly provides another set of ConfigSource's, as described in the "MicroProfile Config Subsystem Configuration" section of the WildFly Admin guide (https://docs.wildfly.org/32/Admin_Guide.html#MicroProfile_Config_SmallRye).

Finally, RESTEasy MicroProfile automatically provides three more ConfigSource's:

- `org.jboss.resteasy.microprofile.config.ServletConfigSource` represents a servlet's init-params from web.xml (ordinal = 60).
- `org.jboss.resteasy.microprofile.config.FilterConfigSource` represents a filter's `<init-param>` from `web.xml` (ordinal = 50). (See RESTEasy as a Servlet Filter for more information.)
- `org.jboss.resteasy.microprofile.config.ServletContextConfigSource` represents context-params from web.xml (ordinal = 40).



As stated by the MicroProfile Config specification, a special property `config_ordinal` can be set within any RESTEasy built-in ConfigSource's. The default implementation of `getOrdinal()` will attempt to read this value. If found and a

> valid integer, the value will be used. Otherwise, the respective default value will be used.

3.4.2. Using pure MicroProfile Config

The MicroProfile Config API is very simple. A `Config` may be obtained either programmatically:

```
Config config = ConfigProvider.getConfig();
```

or, in the presence of CDI, by way of injection:

```
@Inject
Config config;
```

Once a `Config` has been obtained, a property can be queried. For example,

```
String s = config.getValue("prop_name", String.class);
```

or

```
String s = config.getOptionalValue("prop_name", String.class).orElse("d'oh");
```

Now, consider a situation in which "prop_name" has been set by 'System.setProperty("prop_name", "system")` and also in the application's `web.xml` in element `context-param`.

```
<context-param>
  <param-name>prop_name</param-name>
  <param-value>context</param-value>
</context-param>
```

Since the system parameter `ConfigSource` (ordinal = 400) has precedence over `servletContextConfigSource` (ordinal = 40), `config.getValue("prop_name", String.class)` will return "system" rather than "context".

3.4.3. Using RESTEasy's extension of MicroProfile Config

RESTEasy offers a general purpose parameter retrieval mechanism which incorporates MicroProfile Config if the necessary dependencies are available, and which falls back to an extended version of the classic RESTEasy mechanism (see RESTEasy's classic configuration mechanism) otherwise.

Calling:

```
final var config = ConfigurationFactory.getInstance().getConfiguration();
```

will return an instance of `org.jboss.resteasy.spi.config.Configuration`:

```
public interface Configuration {

    /**
     * Returns the resolved value for the specified type of the named property.
     *
     * @param name the name of the parameter
     * @param type the type to convert the value to
     * @param T  the property type
     *
     * @return the resolved optional value
     *
     * @throws IllegalArgumentException if the type is not supported
     */
    <T> Optional<T> getOptionalValue(String name, ClassT type);

    /**
     * Returns the resolved value for the specified type of the named property.
     *
     * @param name the name of the parameter
     * @param type the type to convert the value to
     * @param T  the property type
     *
     * @return the resolved value
     *
     * @throws IllegalArgumentException         if the type is not supported
     * @throws java.util.NoSuchElementException if there is no property associated
with the name
     */
    <T> T getValue(String name, ClassT type);
}
```

For example,

```
String value = ConfigurationFactory.getInstance().getConfiguration().getOptionalValue
("prop_name", String.class).orElse("d'oh");
```

If MicroProfile Config is available, that would be equivalent to

```
String value = ConfigProvider.getConfig().getOptionalValue("prop_name", String.class)
.orElse("d'oh");
```

If MicroProfile Config is not available, then an attempt is made to retrieve the parameter from the

following sources in this order:

1. system variables, followed by
2. environment variables, followed by
3. web.xml parameters, as described in RESTEasy's classic configuration mechanism

3.4.4. Configuring MicroProfile Config

If an application is running inside Wildfly, then all of the dependencies are automatically available. Outside of Wildfly, an application will need the Eclipse MicroProfile API at compile time.

As of RESTEasy 5.0 you will first need to add the RESTEasy MicroProfile Config dependency to the project.

```
<dependency>
    <groupId>org.jboss.resteasy.microprofile</groupId>
    <artifactId>microprofile-config</artifactId>
</dependency>
```

You will also need the MicroProfile Config API and an implementation, in our case SmallRye.

```
<dependency>
    <groupId>org.eclipse.microprofile.config</groupId>
    <artifactId>microprofile-config-api</artifactId>
</dependency>
<dependency>
    <groupId>io.smallrye</groupId>
    <artifactId>smallrye-config</artifactId>
</dependency>
```

3.4.5. RESTEasy's classic configuration mechanism

Prior to the incorporation of MicroProfile Config, nearly all of RESTEasy's parameters were retrieved from servlet `init-params` and `context-params`. Which ones are available depends on how a web application invokes RESTEasy.

If RESTEasy is invoked as a servlet, as in

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
   version="5.0">
   <context-param>
      <param-name>system</param-name>
      <param-value>system-context</param-value>
```

```
   </context-param>

   <servlet>
      <servlet-name>Resteasy</servlet-name>
      <servlet-class>
org.jboss.resteasy.plugins.server.servlet.HttpServlet30Dispatcher</servlet-class>

      <init-param>
         <param-name>system</param-name>
         <param-value>system-init</param-value>
      </init-param>

   </servlet>

   <servlet-mapping>
      <servlet-name>Resteasy</servlet-name>
      <url-pattern>/*</url-pattern>
   </servlet-mapping>
</web-app>
```

then the servlet specific init-params and the general context-params are available, with the former taking precedence over the latter. For example, the property "system" would have the value "system-init".

If RESTEasy is invoked by way of a filter (see RESTEasy as a Servlet Filter), as in

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
   version="5.0">

   <context-param>
      <param-name>system</param-name>
      <param-value>system-context</param-value>
   </context-param>

   <filter>
      <filter-name>Resteasy</filter-name>
      <filter-class>
org.jboss.resteasy.plugins.server.servlet.FilterDispatcher</filter-class>

      <init-param>
         <param-name>system</param-name>
         <param-value>system-filter</param-value>
      </init-param>

    </filter>
```

```
    <filter-mapping>
        <filter-name>Resteasy</filter-name>
        <url-pattern>/*</url-pattern>
    </filter-mapping>

</web-app>
```

then the filter specific init-params and the general context-params are available, with the former taking precedence over the latter. For example, the property "system" would have the value "system-filter".

Finally, if RESTEasy is invoked by way of a `ServletContextListener` (see RESTEasy as a ServletContextListener), as in

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
   version="5.0">

   <listener>
      <listener-class>
         org.jboss.resteasy.plugins.server.servlet.ResteasyBootstrap
      </listener-class>
   </listener>

   <context-param>
      <param-name>system</param-name>
      <param-value>system-context</param-value>
   </context-param>
</web-app>
```

where `ResteasyBootstrap` is a `servletContextListener`, then the context-params are available.

3.4.6. Overriding RESTEasy's configuration mechanism

Before adopting the default behavior, with or without MicroProfile Config, as described in previous sections, RESTEasy will use service loading to look for one or more implementations of the interface `org.jboss.resteasy.spi.config.ConfigurationFactory`, selecting one with the highest priority as determined by the value returned by `ConfigurationFactory.priority()`. Smaller numbers indicate higher priority. The default `ConfigurationFactory` is `org.jboss.resteasy.core.config.DefaultConfigurationFactory` with a priority of 500.

3.5. Configuration switches

RESTEasy can receive the following configuration options from any ConfigSource's that are available at runtime:

Option Name	Default Value	Description
resteasy.servlet.mapping.prefix	no default	If the url-pattern for the RESTEasy servlet-mapping is not /*
resteasy.providers	no default	A comma delimited list of fully qualified @Provider class names to be register
resteasy.disable.providers	no default	A comma delimited list of fully qualified Jakarta RESTful Web Services `@Provider` class names that will be disabled.
resteasy.use.builtin.providers	true	Whether or not to register default, built-in @Provider classes
resteasy.resources	no default	A comma delimited list of fully qualified Jakarta RESTful Web Services resource class names to be register
resteasy.jndi.resources	no default	A comma delimited list of JNDI names which reference objects to be registered as Jakarta RESTful Web Services resources
jakarta.ws.rs.Application	no default	Fully qualified name of Application class to bootstrap in a spec portable way
resteasy.media.type.mappings	no default	Replaces the need for an Accept header by mapping file name extensions (like .xml or .txt) to a media type. Used when the client is unable to use an Accept header to choose a representation (i.e. a browser). See Jakarta RESTful Web Services Content Negotiation for more details.

Option Name	Default Value	Description
resteasy.language.mappings	no default	Replaces the need for an Accept-Language header by mapping file name extensions (like .en or .fr) to a language. Used when the client is unable to use an Accept-Language header to choose a language (i.e. a browser). See Jakarta RESTful Web Services Content Negotiation for more details.
resteasy.media.type.param.mapping	no default	Names a query parameter that can be set to an acceptable media type, enabling content negotiation without an Accept header. See Jakarta RESTful Web Services Content Negotiation for more details.
resteasy.role.based.security	false	Enables role based security. See Securing Jakarta RESTful Web Services and RESTEasy for more details.
resteasy.document.expand.entity.references	false	Expand external entities in org.w3c.dom.Document documents and Jakarta XML Binding object representations
resteasy.document.secure.processing.feature	true	Impose security constraints in processing org.w3c.dom.Document documents and Jakarta XML Binding object representations. When enabled, sets `XMLConstants.FEATURE_SECURE_PROCESSING` which applies JVM limits including `jdk.xml.entityExpansionLimit` (2,500 in Java 24+, 64,000 in earlier versions), `jdk.xml.totalEntitySizeLimit` (50,000,000), and other XML processing restrictions. See https://docs.oracle.com/en/java/javase/25/security/java-api-xml-processing-jaxp-security-guide.html

Option Name	Default Value	Description
resteasy.document.secure.disableDTDs	true	Prohibit DTDs in org.w3c.dom.Document documents and Jakarta XML Binding object representations
resteasy.wider.request.matching	false	Turns off the Jakarta RESTful Web Services spec defined class-level expression filtering and instead tries to match every method's full path.
resteasy.proxy.implement.all.interfaces	false	When RESTEasy injects a context type which is an interface, it injects a proxy which delegates to the context object of the current request. By default the proxy only implements the context type. When enabled, the proxy also implements all interfaces of the context object, so it can be cast to them. This does not apply to context types injected by CDI, which only implement the context type. Deprecated: this switch will be removed along with RESTEasy's legacy context injection.
resteasy.use.container.form.params	false	Obtain form parameters by using `HttpServletRequest.getParameterMap()`. Use this switch if you are calling this method within a servlet filter or consuming the input stream within the filter.
resteasy.rfc7232preconditions	false	Enables RFC7232 compliant HTTP preconditions handling.
resteasy.gzip.max.input	10000000	Imposes maximum size on decompressed gzipped .
resteasy.secure.random.max.use	100	The number of times a SecureRandom can be used before reseeding.

Option Name	Default Value	Description
resteasy.buffer.exception.entity	true	Upon receiving an exception, the client side buffers any response entity before closing the connection.
resteasy.add.charset	true	If a resource method returns a text/* or application/xml* media type without an explicit charset, RESTEasy adds "charset=UTF-8" to the returned Content-Type header. To disable this behavior set this switch to false.
resteasy.disable.html.sanitizer	false	Normally, a response with media type "text/html" and a status of 400 will be processed so that the characters "/", "/", "<", ">", "&", """ (double quote), and "'" (single quote) are escaped to prevent an XSS attack. Setting this parameter to "true", escaping will not occur.
resteasy.patchfilter.disabled	false	RESTEasy provides class PatchMethodFilter to handle JSON patch and JSON Merge Patch requests. It is active by default. This filter can be disabled by setting this switch to "true" and a customized patch method filter can be provided to serve the JSON patch and JSON merge patch request instead.
resteasy.patchfilter.legacy	true	**Deprecated**: Use `resteasy.preferJacksonOverJsonB` instead. When set to `false`, the Jakarta JSON Processing provider is used for PATCH filter processing. When set to `true`, the Jackson provider is used. This parameter will be removed in a future release.

Option Name	Default Value	Description
resteasy.original.webapplicationexception.behavior	false	Set to "true", this parameter will restore the original behavior in which a Client running in a resource method will throw a Jakarta RESTful Web Services WebApplicationException instead of a Resteasy version with a sanitized `Response`. For more information, see section Resteasy WebApplicationExceptions
dev.resteasy.throw.options.exception	false	Setting this value to true will throw a `org.jboss.resteasy.spi.DefaultOptionsMethodException` if the HTTP method "OPTIONS" is sent and the matching method is not annotated with `@OPTIONS`. This is the original behavior of RESTEasy. However, this has been changed to return the response so that its processed with an `ExceptionMapper`.
dev.resteasy.provider.jackson.disable.default.object.mapper	false	Setting this value to true will disable RESTEasy creating a `com.fasterxml.jackson.databind.ObjectMapper` if the Jackson Provider is being used and there is no `jakarta.ws.rs.ext.ContextResolver` for an `ObjectMapper`.
`dev.resteasy.entity.memory.threshold`	5MB	The threshold to use for the amount of data to store in memory for entities.
`dev.resteasy.entity.file.buffer.size`	8192	The size of the buffer used when writing the entity to a file. This can be disabled by setting a value of less than 1. If disabled, ensure a larger value is set in `-XX:MaxDirectMemorySize` JVM argument for large entities.

Option Name	Default Value	Description
`dev.resteasy.entity.file.threshold`	50MB	The threshold to use for the amount of data that can be stored in a file for entities. If the threshold is reached an `IllegalStateException` will be thrown. A value of -1 means no limit.
`dev.resteasy.entity.tmpdir`	The value of the `java.io.tmpdir` system property.	The temporary directory to use when the `dev.resteasy.entity.memory.threshold` was reached and the entity must be written to a file. Note that the directory must already exist.
`dev.resteasy.client.ssl.context.algorithm`	`TLS`	When a `ClientBuilder.keyStore()` or `ClientBuilder.trustStore()` is set and no `SSLContext` is set on a client builder this value is used to set the `SSLContext` algorithm to use when invoking `SSLContext.getInstance()` for client connections.

Option Name	Default Value	Description
`dev.resteasy.image.threshold`	200MB	The maximum estimated size of a decoded image, including its thumbnails, which the `org.jboss.resteasy.plugins.providers.IIOImageProvider` will read. Images whose estimated size exceeds this value are rejected with a 400 before being decoded. Please note this is the estimated size of the decoded image in memory, not the size of the request body, and it is a per-image estimate rather than a limit on the memory used across concurrent requests. A value of `-1` disables the validation. The value is a number followed by an optional unit of `B`, `KB`, `MB`, `GB`, `TB`, `PB`, `EB` or `ZB`. The unit is case-insensitive and may be separated from the number by a space, so `200MB`, `200mb` and `200 MB` are equivalent. The units are binary, making `200MB` 209715200 bytes. A value with no unit, or with a unit other than those listed, is read as a number of bytes; `200` and `200MiB` both mean 200 bytes rather than 200 megabytes.



The resteasy.servlet.mapping.prefix context param variable must be set if the servlet-mapping for the RESTEasy servlet has a url-pattern other than `/*`. For example, if the url-pattern is

```
<servlet-mapping>
    <servlet-name>Resteasy</servlet-name>
    <url-pattern>/restful-services/*</url-pattern>
</servlet-mapping>
```

Then the value of resteasy.servlet.mapping.prefix must be:

```
<context-param>
```

```
    <param-name>resteasy.servlet.mapping.prefix</param-name>
    <param-value>/restful-services</param-value>
</context-param>
```

Resteasy internally uses a cache to find the resource invoker for the request url. The cache size and enablement can be controlled with these system properties.

System Property Name	Default Value	Description
resteasy.match.cache.enabled	true	If the match cache is enabled or not
resteasy.match.cache.size	2048	The size of this match cache

3.6. `jakarta.ws.rs.core.Application`

The `jakarta.ws.rs.core.Application` class is a standard Jakarta RESTful Web Services class that may be implemented to provide information about your deployment. It is simply a class the lists all Jakarta RESTful Web Services root resources and providers.



If the application's web.xml file does not have a servlet-mapping element, you must provide an `Application` class annotated with `@ApplicationPath`.

3.7. Client side

Jakarta RESTful Web Services conforming implementations, such as RESTEasy, support a client side framework which simplifies communicating with restful applications. In RESTEasy, the minimal set of modules needed for the client framework consists of resteasy-core and resteasy-client. You can access them by way of maven:

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-client</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

Other modules, such as resteasy-jaxb-provider, may be brought in as needed.

3.8. Configuring Providers

There are a number of ways in which Providers can be supplied to RESTEasy.

- The Jakarta RESTful Web Services specification mandates a number of built-in providers.
- `Application.getClasses()` may supply provider classes.
- The configuration parameter "resteasy.providers" may supply a comma delimited list of fully qualified provider class names.

- If an `Application` returns empty sets from `getClasses()` and `getSingletons()`, classes annotated with `@Provider` are discovered automatically.

RESTEasy also implements the configuration parameter "resteasy.disable.providers", which can be set to a comma delimited list of fully qualified class names of providers that **are not** meant to be made available. That list may include any providers supplied by any of the means listed above, and it will override them.

Chapter 4. Using @Path and @GET, @POST, etc.

```
@Path("/library")
public class Library {

   @GET
   @Path("/books")
   public String getBooks() {}

   @GET
   @Path("/book/{isbn}")
   public String getBook(@PathParam("isbn") String id) {
      // search my database and get a string representation and return it
   }

   @PUT
   @Path("/book/{isbn}")
   public void addBook(@PathParam("isbn") String id, @QueryParam("name") String name)
{}

   @DELETE
   @Path("/book/{id}")
   public void removeBook(@PathParam("id") String id) {}

}
```

In the class above, the RESTEasy servlet is configured and reachable at a root path of http://localhost/services. The requests handled by class, Library, are:

- GET http://localhost/services/library/books
- GET http://localhost/services/library/book/333
- PUT http://localhost/services/library/book/333
- DELETE http://localhost/services/library/book/333

The `@jakarta.ws.rs.Path` annotation must exist on either the class and/or a resource method. If it exists on both the class and method, the relative path to the resource method is a concatenation of the class and method.

In the `jakarta.ws.rs` package there are annotations for each HTTP method. `@GET`, `@POST`, `@PUT`, `@DELETE`, and `@HEAD`. Place these on public methods that you want to map to that certain kind of HTTP method. As long as there is a @Path annotation on the class, a @Path annotation is not required on the method you are mapping. There can be more than one HTTP method as long as they can be distinguished from other methods.

When a `@Path` annotation is on a method without an HTTP method, these are called `JAXRSResourceLocators`.

4.1. @Path and regular expression mappings

The @Path annotation is not limited to simple path expressions. Regular expressions can be inserted into a `@Path` value attribute. For example:

```
@Path("/resources)
public class MyResource {

   @GET
   @Path("{var:.*}/stuff")
   public String get() {}
}
```

The following GETs will route to the `getResource()` method:

```
GET /resources/stuff
GET /resources/foo/stuff
GET /resources/on/and/on/stuff
```

The format of the expression is:

```
"{" variable-name [ ":" regular-expression ] "}"
```

The regular-expression part is optional. When the expression is not provided, it defaults to a wildcard matching of one particular segment. In regular-expression terms, the expression defaults to

```
"([]*)"
```

For example:

```
@Path("/resources/{var}/stuff")
```

will match these:

```
GET /resources/foo/stuff
GET /resources/bar/stuff
```

but will not match:

```
GET /resources/a/bunch/of/stuff
```

Chapter 5. @PathParam



RESTEasy supports `@PathParam` annotations with no parameter name..

`@PathParam` is a parameter annotation which allows you to map variable URI path fragments into your method call.

```
@Path("/library")
public class Library {

   @GET
   @Path("/book/{isbn}")
   public String getBook(@PathParam("isbn") String id) {
      // search my database and get a string representation and return it
   }
}
```

What this allows you to do is embed variable identification within the URIs of your resources. In the above example, an isbn URI parameter is used to pass information about the book we want to access. The parameter type you inject into can be any primitive type, a String, or any Java object that has a constructor that takes a String parameter, ozr a static valueOf method that takes a String asa parameter. For example, lets say we wanted isbn to be a real object. We could do:

```
@GET
@Path("/book/{isbn}")
public String getBook(@PathParam("isbn") ISBN id) {}

public class ISBN {
   public ISBN(String str) {}
}
```

Or instead of a public String constructors, have a valueOf method:

```
public class ISBN {
    public static ISBN valueOf(String isbn) {}
}
```

5.1. Advanced @PathParam and Regular Expressions

There are a few more complicated uses of @PathParams not discussed in the previous section.

It is allowed to specify one or more path params embedded in one URI segment. Here are some examples:

1. `@Path("/aaa{param}bbb")`

2. `@Path("/{name}-{zip}")`
3. `@Path("/foo{name}-{zip}bar")`

So, a URI of "/aaa111bbb" would match #1. "/bill-02115" would match #2. "foobill-02115bar" would match #3.

It was discussed before how to use regular expression patterns within @Path values.

```
@GET
@Path("/aaa{param:b+}/{many:.*}/stuff")
public String getIt(@PathParam("param") String bs, @PathParam("many") String many) {}
```

For the following requests, lets see what the values of the "param" and "many" `@PathParams` would be:

Request	Param	Many
GET /aaabb/some/stuff	bb	some
GET /aaab/a/lot/of/stuff	b	a/lot/of

5.2. @PathParam and PathSegment

The specification has a very simple abstraction for examining a fragment of the URI path being invoked on jakarta.ws.rs.core.PathSegment.

RESTEasy can inject a `PathSegment` instead of a value with the `@PathParam`.

```
@GET
@Path("/book/{id}")
public String getBook(@PathParam("id") PathSegment id) {}
```

This is very useful if you have a bunch of `@PathParams` that use matrix parameters. The idea of matrix parameters is that they are an arbitrary set of name-value pairs embedded in a uri path segment. The PathSegment object gives you access to these parameters. See also `MatrixParam`.

A matrix parameter example is:

```
GET http://localhost/library/book;name=EJB 3.0;author=Bill Burke
```

The idea of matrix parameters is that it represents resources that are addressable by their attributes as well as their raw id.

Chapter 6. @QueryParam



RESTEasy supports `@QueryParam` annotations with no parameter name.

The `@QueryParam` annotation allows mapping a URI query string parameter or url form encoded parameter to a method invocation.

```
GET /books?num=5
```

```
@GET
public String getBooks(@QueryParam("num") int num) {
}
```

Since RESTEasy is built on top of a Servlet, it does not distinguish between URI query strings or url form encoded parameters. Like `PathParam`, the parameter type can be a String, primitive, or class that has a String constructor or static valueOf() method.

Chapter 7. @HeaderParam



RESTEasy supports `@HeaderParam` annotations with no parameter name..

The `@HeaderParam` annotation allows you to map a request HTTP header to a method invocation.

```
GET /books?num=5
```

```
@GET
public String getBooks(@HeaderParam("From") String from) {
}
```

Like `PathParam`, a parameter type can be a String, primitive, or class that has a String constructor or static valueOf() method. For example, MediaType has a valueOf() method and you could do:

```
@PUT
public void put(@HeaderParam("Content-Type") MediaType contentType);
```

7.1. Header Delegates

In addition to the usual methods for translating parameters to and from strings, parameters annotated with `@HeaderParam` have another option: implementations of `RuntimeDelegate$HeaderDelegate`.

`HeaderDelegate` is similar to `ParamConverter`, but it is not very convenient to register a `HeaderDelegate` since, unlike, for example, `ParamConverterProvider`, it is not treated by the specification as a provider. The class `jakarta.ws.rs.core.Configurable`, which is subclassed by, for example, `org.jboss.resteasy.spi.ResteasyProviderFactory` has methods like

```
    /**
     * Register a class of a custom  component (such as an extension provider or
     * a {@link jakarta.ws.rs.core.Feature feature} meta-provider) to be instantiated
     * and used in the scope of this configurable context.
     *
     * ...
     *
     * @param componentClass  component class to be configured in the scope of this
     *                        configurable context.
     * @return the updated configurable context.
     */
    public C register(Class<?> componentClass);
```

but it is not clear that they are applicable to `HeaderDelegate`s.

RESTEasy approaches this problem by allowing `HeaderDelegate` to be annotated with `@Provider`. Not only will `ResteasyProviderFactory.register()` process a `HeaderDelegate`, but another useful consequence is that a `HeaderDelegate` can be discovered automatically at runtime.

Chapter 8. Linking resources

There are two mechanisms available in RESTEasy to link a resource to another, and to link resources to operations: the Link HTTP header, and Atom links inside the resource representations.

8.1. Link Headers

RESTEasy has both client and server side support for the Link header specification. See the javadocs for `org.jboss.resteasy.spi.LinkHeader`, `org.jboss.resteasy.specimpl.LinkImpl`, and `org.jboss.resteasy.client.ClientResponse`.

The main advantage of Link headers over Atom links in the resource is that those links are available without parsing the entity body.

8.2. Atom links in the resource representations

RESTEasy allows the injection of Atom links directly inside the entity objects that are sending to the client, via auto-discovery.



This is only available when using the Jackson2 or Jakarta XML Binding providers (for JSON and XML).

The main advantage over Link headers is that there can be any number of Atom links directly over the concerned resources, for any number of resources in the response. For example, you can have Atom links for the root response entity, and also for each of its children entities.

8.2.1. Configuration

There is no configuration required to be able to inject Atom links into a resource representation, you just have to have this maven artifact in the application's path:

Group	Artifact	Version
org.jboss.resteasy	resteasy-links	8.0.0.Alpha1

8.2.2. Your first links injected

Three things are needed in order to tell RESTEasy to inject Atom links into an entity:

- Annotate the Jakarta RESTful Web Services method with `@AddLinks` to indicate that Atom links are to be injected into the response entity.
- Add `RESTServiceDiscovery` fields to the resource classes where Atom links are to be injected.
- Annotate the Jakarta RESTful Web Services methods you want Atom links for with `@LinkResource`, so that RESTEasy knows which links to create for which resources.

The following example illustrates how to declare everything in order to get the Atom links injected in the book store example:

```
@Path("/")
@Consumes({"application/xml", "application/json"})
@Produces({"application/xml", "application/json"})
public interface BookStore {

    @AddLinks
    @LinkResource(value = Book.class)
    @GET
    @Path("books")
    Collection<Book> getBooks();

    @LinkResource
    @POST
    @Path("books")
    void addBook(Book book);

    @AddLinks
    @LinkResource
    @GET
    @Path("book/{id}")
    Book getBook(@PathParam("id") String id);

    @LinkResource
    @PUT
    @Path("book/{id}")
    void updateBook(@PathParam("id") String id, Book book);

    @LinkResource(value = Book.class)
    @DELETE
    @Path("book/{id}")
    void deleteBook(@PathParam("id") String id);
}
```

And this is the definition of the Book resource:

```
@Mapped(namespaceMap = @XmlNsMap(jsonName = "atom", namespace =
"http://www.w3.org/2005/Atom"))
@XmlRootElement
@XmlAccessorType(XmlAccessType.NONE)
public class Book {
    @XmlAttribute
    private String author;

    @XmlID
    @XmlAttribute
    private String title;

    @XmlElementRef
    private RESTServiceDiscovery rest;
```

```
}
```

If you do a GET /order/foo, this XML representation will be returned:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<book xmlns:atom="http://www.w3.org/2005/Atom" title="foo" author="bar">
 <atom:link href="http://localhost:8081/books" rel="list"/>
 <atom:link href="http://localhost:8081/books" rel="add"/>
 <atom:link href="http://localhost:8081/book/foo" rel="self"/>
 <atom:link href="http://localhost:8081/book/foo" rel="update"/>
 <atom:link href="http://localhost:8081/book/foo" rel="remove"/>
</book>
```

And in JSON format:

```
{
 "book":
 {
  "@title":"foo",
  "@author":"bar",
  "atom.link":
   [
    {"@href":"http://localhost:8081/books","@rel":"list"},
    {"@href":"http://localhost:8081/books","@rel":"add"},
    {"@href":"http://localhost:8081/book/foo","@rel":"self"},
    {"@href":"http://localhost:8081/book/foo","@rel":"update"},
    {"@href":"http://localhost:8081/book/foo","@rel":"remove"}
   ]
 }
}
```

8.2.3. Customising Atom link serialization

The `RESTServiceDiscovery` is a Jakarta XML Binding type which inherits from `List`. You are free to annotate it in order to customise the Jakarta XML Binding serialisation, or just rely on the default with `@XmlElementRef`.

8.2.4. Specifying which Jakarta RESTful Web Services methods are tied to which resources

This is done by annotating the methods with the `@LinkResource` annotation. It supports the following optional parameters:

Parameter	Type	Function	Default
value	`Class`	Declares an Atom link for the given type of resources.	Defaults to the entity body type (non-annotated parameter), or the method's return type. This default does not work with `Response` or `Collection` types, they need to be explicitly specified.
rel	`String`	The Atom link relation	• `list`: For `GET` methods returning a `Collection` • `self`: For `GET` methods returning a non-`Collection` • `remove`: For `DELETE` methods • `update`: For `PUT` methods • `add`: For `POST` methods

Several `@LinkResource` annotations can be added on a single method by enclosing them in a `@LinkResources` annotation. This allows the adding of links to the same method on several resource types. For example the `/order/foo/comments` operation can belong on the `Order` resource with the `comments` relation, and on t he `Comment` resource with the `list` relation.

8.2.5. Specifying path parameter values for URI templates

When RESTEasy adds links to your resources it needs to insert the right values in the URI template. This is done either automatically by guessing the list of values from the entity, or by specifying the values in the `@LinkResource(pathParameters)` parameter.

Loading URI template values from the entity

URI template values are extracted from the entity from fields or Java Bean properties annotated with `@ResourceID`, Jakarta XML Binding's `@XmlID` or Jakarta Persistence's `@Id`. If there is more than one URI template value to find in a given entity, the entity can be annotated with `@ResourceIDs` to list the names of fields or properties that make up this entity's Id. If there are other URI template values required from a parent entity, RESTEasy tries to find the parent on a field or Java Bean property annotated with `@ParentResource`. The list of URI template values extracted up every `@ParentResource` is then reversed and used as the list of values for the URI template.

For example, consider the previous Book example, and a list of comments:

```
@XmlRootElement
@XmlAccessorType(XmlAccessType.NONE)
public class Comment {
    @ParentResource
    private Book book;

    @XmlElement
    private String author;

    @XmlID
    @XmlAttribute
    private String id;

    @XmlElementRef
    private RESTServiceDiscovery rest;
}
```

Given the previous book store service augmented with comments:

```
@Path("/")
@Consumes({"application/xml", "application/json"})
@Produces({"application/xml", "application/json"})
public interface BookStore {

    @AddLinks
    @LinkResources({
        @LinkResource(value = Book.class, rel = "comments"),
        @LinkResource(value = Comment.class)
    })
    @GET
    @Path("book/{id}/comments")
    Collection<Comment> getComments(@PathParam("id") String bookId);

    @AddLinks
    @LinkResource
    @GET
    @Path("book/{id}/comment/{cid}")
    Comment getComment(@PathParam("id") String bookId, @PathParam("cid") String
commentId);

    @LinkResource
    @POST
    @Path("book/{id}/comments")
    void addComment(@PathParam("id") String bookId, Comment comment);

    @LinkResource
    @PUT
    @Path("book/{id}/comment/{cid}")
    void updateComment(@PathParam("id") String bookId, @PathParam("cid") String
```

```
commentId, Comment comment);

    @LinkResource(Comment.class)
    @DELETE
    @Path("book/{id}/comment/{cid}")
    void deleteComment(@PathParam("id") String bookId, @PathParam("cid") String
commentId);

}
```

Whenever we need to make links for a `Book` entity, we look up the ID in the `Book` `@XmlID` property. Whenever we make links for `Comment` entities, we have a list of values taken from the `Comment` `@XmlID` and its `@ParentResource`: the `Book` and its `@XmlID`.

For a `Comment` with `id "1"` on a `Book` with `title "foo"` we will therefore get a list of URI template values of `{"foo", "1"}`, to be replaced in the URI template, thus obtaining either `"/book/foo/comments"` or `"/book/foo/comment/1"`.

Specifying path parameters manually

An alternative to annotating entities with resource ID annotations (`@ResourceID`, `@ResourceIDs`, `@XmlID` or `@Id`) and `@ParentResource`, you can specify the URI template values inside the `@LinkResource` annotation, using Unified Expression Language expressions:

Parameter	Type	Function	Default
pathParameters	`String[]`	Declares a list of UEL expressions to obtain the URI template values.	Defaults to using `@ResourceID`, `@ResourceIDs`,`@XmlID` or `@Id` and `@ParentResource` annotations to extract the values from the model.

The UEL expressions are evaluated in the context of the entity, which means that any unqualified variable will be taken as a property for the entity itself, with the special variable `this` bound to the entity for which links are being generated.

The previous example of `Comment` service could be declared as such:

```
@Path("/")
@Consumes({"application/xml", "application/json"})
@Produces({"application/xml", "application/json"})
public interface BookStore {

    @AddLinks
    @LinkResources({
        @LinkResource(value = Book.class, rel = "comments", pathParameters = "
${title}"),
```

```
        @LinkResource(value = Comment.class, pathParameters = {"${book.title}", "
${id}"})
    })
    @GET
    @Path("book/{id}/comments")
    Collection<Comment> getComments(@PathParam("id") String bookId);

    @AddLinks
    @LinkResource(pathParameters = {"${book.title}", "${id}"})
    @GET
    @Path("book/{id}/comment/{cid}")
    Comment getComment(@PathParam("id") String bookId, @PathParam("cid") String
commentId);

    @LinkResource(pathParameters = {"${book.title}", "${id}"})
    @POST
    @Path("book/{id}/comments")
    void addComment(@PathParam("id") String bookId, Comment comment);

    @LinkResource(pathParameters = {"${book.title}", "${id}"})
    @PUT
    @Path("book/{id}/comment/{cid}")
    void updateComment(@PathParam("id") String bookId, @PathParam("cid") String
commentId, Comment comment);

    @LinkResource(value = Comment.class, pathParameters = {"${book.title}", "${id}"})
    @DELETE
    @Path("book/{id}/comment/{cid}")
    void deleteComment(@PathParam("id") String bookId, @PathParam("cid") String
commentId);

}
```

8.2.6. Securing entities

The user can restrict which links are injected in the resource based on security restrictions for the client, so that if the current client doesn't have permission to delete a resource he will not be presented with the `"delete"` link relation.

Security restrictions can either be specified on the `@LinkResource` annotation, or using RESTEasy and Jakarta Enterprise Beans security annotation `@RolesAllowed` on the Jakarta RESTful Web Services method.

Parameter	Type	Function	Default
constraint	`String`	A UEL expression which must evaluate to true to inject this method's link in the response entity.	Defaults to using `@RolesAllowed` from the Jakarta RESTful Web Services method.

8.2.7. Extending the UEL context

It has been shown that both the URI template values and the security constraints of `@LinkResource` use UEL to evaluate expressions. RESTEasy provides a basic UEL context with access only to the entity we are injecting links in, and nothing more.

More variables or functions can be added in this context, by adding a `@LinkELProvider` annotation on the Jakarta RESTful Web Services method, its class, or its package. This annotation's value should point to a class that implements the `ELProvider` interface, which wraps the default `ELContext` in order to add any missing functions.

For example, to support the Seam annotation `s:hasPermission(target, permission)` in your security constraints, add a `package-info.java` file like this:

```
@LinkELProvider(SeamELProvider.class)
package org.jboss.resteasy.links.test;

import org.jboss.resteasy.links.*;
```

With the following provider implementation:

```
package org.jboss.resteasy.links.test;

import jakarta.el.ELContext;
import jakarta.el.ELResolver;
import jakarta.el.FunctionMapper;
import jakarta.el.VariableMapper;

import org.jboss.seam.el.SeamFunctionMapper;

import org.jboss.resteasy.links.ELProvider;

public class SeamELProvider implements ELProvider {

    public ELContext getContext(final ELContext ctx) {
        return new ELContext() {

            private SeamFunctionMapper functionMapper;

            @Override
            public ELResolver getELResolver() {
                return ctx.getELResolver();
            }

            @Override
            public FunctionMapper getFunctionMapper() {
                if (functionMapper == null)
                    functionMapper = new SeamFunctionMapper(ctx
                            .getFunctionMapper());
```

```
                return functionMapper;
            }

            @Override
            public VariableMapper getVariableMapper() {
                return ctx.getVariableMapper();
            }
        };
    }

}
```

And then use it as such:

```
@Path("/")
@Consumes({"application/xml", "application/json"})
@Produces({"application/xml", "application/json"})
public interface BookStore {

    @AddLinks
    @LinkResources({
        @LinkResource(value = Book.class, rel = "comments", constraint =
"${s:hasPermission(this, 'add-comment')}"),
        @LinkResource(value = Comment.class, constraint = "${s:hasPermission(this,
'insert')}")
    })
    @GET
    @Path("book/{id}/comments")
    Collection<Comment> getComments(@PathParam("id") String bookId);

    @AddLinks
    @LinkResource(constraint = "${s:hasPermission(this, 'read')}")
    @GET
    @Path("book/{id}/comment/{cid}")
    Comment getComment(@PathParam("id") String bookId, @PathParam("cid") String
commentId);

    @LinkResource(constraint = "${s:hasPermission(this, 'insert')}")
    @POST
    @Path("book/{id}/comments")
    void addComment(@PathParam("id") String bookId, Comment comment);

    @LinkResource(constraint = "${s:hasPermission(this, 'update')}")
    @PUT
    @Path("book/{id}/comment/{cid}")
    void updateComment(@PathParam("id") String bookId, @PathParam("cid") String
commentId, Comment comment);

    @LinkResource(value = Comment.class, constraint = "${s:hasPermission(this,
'delete')}")
```

```
    @DELETE
    @Path("book/{id}/comment/{cid}")
    void deleteComment(@PathParam("id") String bookId, @PathParam("cid") String
commentId);

}
```

8.2.8. Resource facades

Sometimes it is useful to add resources which are just containers or layers on other resources, for example to represent a collection of `Comment` with a start index and a certain number of entries, in order to implement paging. Such a collection is not really an entity in the model, but it should obtain the `"add"` and `"list"` link relations for the `Comment` entity.

This is possibly using resource facades. A resource facade is a resource which implements the `ResourceFacade<T>` interface for the type `T`, and as such, should receive all links for that type.

Since in most cases the instance of the `T` type is not directly available in the resource facade, another way is needed to extract its URI template values. This is done by calling the resource facade's `pathParameters()` method to obtain a map of URI template values by name. This map will be used to fill in the URI template values for any link generated for `T`, if there are enough values in the map.

Here is an example of such a resource facade for a collection of `Comment`s:

```
@XmlRootElement
@XmlAccessorType(XmlAccessType.NONE)
public class ScrollableCollection implements ResourceFacade<Comment> {

    private String bookId;
    @XmlAttribute
    private int start;
    @XmlAttribute
    private int totalRecords;
    @XmlElement
    private List<Comment> comments = new ArrayList<Comment>();
    @XmlElementRef
    private RESTServiceDiscovery rest;

    public Class<Comment> facadeFor() {
        return Comment.class;
    }

    public Map<String, Object> pathParameters() {
        HashMap<String, String> map = new HashMap<String, String>();
        map.put("id", bookId);
        return map;
    }
}
```

This will produce such an XML collection:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<collection xmlns:atom="http://www.w3.org/2005/Atom" totalRecords="2" start="0">
 <atom.link href="http://localhost:8081/book/foo/comments" rel="add"/>
 <atom.link href="http://localhost:8081/book/foo/comments" rel="list"/>
 <comment xmlid="0">
  <text>great book</text>
  <atom.link href="http://localhost:8081/book/foo/comment/0" rel="self"/>
  <atom.link href="http://localhost:8081/book/foo/comment/0" rel="update"/>
  <atom.link href="http://localhost:8081/book/foo/comment/0" rel="remove"/>
  <atom.link href="http://localhost:8081/book/foo/comments" rel="add"/>
  <atom.link href="http://localhost:8081/book/foo/comments" rel="list"/>
 </comment>
 <comment xmlid="1">
  <text>terrible book</text>
  <atom.link href="http://localhost:8081/book/foo/comment/1" rel="self"/>
  <atom.link href="http://localhost:8081/book/foo/comment/1" rel="update"/>
  <atom.link href="http://localhost:8081/book/foo/comment/1" rel="remove"/>
  <atom.link href="http://localhost:8081/book/foo/comments" rel="add"/>
  <atom.link href="http://localhost:8081/book/foo/comments" rel="list"/>
 </comment>
</collection>
```

Chapter 9. @MatrixParam



RESTEasy supports `@MatrixParam` annotations with no parameter name..

The idea of matrix parameters is that they are an arbitrary set of name-value pairs embedded in a uri path segment. A matrix parameter example is:

```
GET http://host.com/library/book;name=EJB 3.0;author=Bill Burke
```

The idea of matrix parameters is that it represents resources that are addressable by their attributes as well as their raw id. The `@MatrixParam` annotation allows URI matrix parameters to be injected into a method invocation.

```
@GET
public String getBook(@MatrixParam("name") String name, @MatrixParam("author") String
author) {}
```

There is one big problem with `@MatrixParam` that the current version of the specification does not resolve. What if the same `MatrixParam` exists twice in different path segments? In this case, it is better to use `PathParam` combined with `PathSegment`.

Chapter 10. @CookieParam



RESTEasy supports `@CookieParam` annotations with no parameter name..

The `@CookieParam` annotation allows the injection of the value of a cookie or an object representation of an HTTP request cookie into a method invocation.

```
GET /books?num=5
```

```
@GET
public String getBooks(@CookieParam("sessionid") int id) {
}

@GET
public String getBooks(@CookieParam("sessionid") jakarta.ws.rs.core.Cookie id) {
}
```

Like `PathParam`, the parameter type can be a String, primitive, or class that has a String constructor or static valueOf() method. It can also get an object representation of the cookie via the `jakarta.ws.rs.core.Cookie` class.

Chapter 11. @FormParam



RESTEasy supports `@FormParam` annotations with no parameter name..

When the input request body is of the type "application/x-www-form-urlencoded", a.k.a. an HTML Form, individual form parameters can be injected from the request body into method parameter values.

```
<form method="POST" action="/resources/service">
First name:
<input type="text" name="firstname">
<br>
Last name:
<input type="text" name="lastname">
</form>
```

If posted through that form, this is what the service might look like:

```
@Path("/")
public class NameRegistry {

   @Path("/resources/service")
   @POST
   public void addName(@FormParam("firstname") String first, @FormParam("lastname")
String last) {}
}
```

A `@FormParam` cannot be combined with the default "application/x-www-form-urlencoded" that unmarshalls to a `MultivaluedMap<String, String>`. i.e. This is illegal:

```
@Path("/")
public class NameRegistry {

   @Path("/resources/service")
   @POST
   @Consumes("application/x-www-form-urlencoded")
   public void addName(@FormParam("firstname") String first, MultivaluedMap<String,
String> form) {}
}
```

Chapter 12. @Form

`@Form` is a RESTEasy specific annotation that allows the re-use of any `@*Param` annotation within an injected class. RESTEasy will instantiate the class and inject values into any annotated `@*Param` or `@Context` property. This is useful when there are a lot of parameters on a method and it is wanted to condense them into a value object.

```
public class MyForm {

    @FormParam("stuff")
    private int stuff;

    @HeaderParam("myHeader")
    private String header;

    @PathParam("foo")
    public void setFoo(String foo) {}
}


@POST
@Path("/myservice")
public void post(@Form MyForm form) {}
```

When somebody posts to /myservice, RESTEasy will instantiate an instance of MyForm and inject the form parameter "stuff" into the "stuff" field, the header "myheader" into the header field, and call the setFoo method with the path param variable of "foo".

`@Form` has some expanded `@FormParam` features. If a prefix is specified within the Form param, this will prepend a prefix to any form parameter lookup. For example, say you have one Address class, but want to reference invoice and shipping addresses from the same set of form parameters:

```
public static class Person {
    @FormParam("name")
    private String name;

    @Form(prefix = "invoice")
    private Address invoice;

    @Form(prefix = "shipping")
    private Address shipping;
}

public static class Address {
    @FormParam("street")
    private String street;
}
```

```
@Path("person")
public static class MyResource {
    @POST
    @Produces(MediaType.TEXT_PLAIN)
    @Consumes(MediaType.APPLICATION_FORM_URLENCODED)
    public String post(@Form Person p) {
        return p.toString();
    }
}
```

In this example, the client could send the following form parameters:

```
name=bill
invoice.street=xxx
shipping.street=yyy
```

The `Person.invoice` and `Person.shipping` fields would be populated appropriately. Note, prefix mappings also support lists and maps:

```
public static class Person {
    @Form(prefix="telephoneNumbers") List<TelephoneNumber> telephoneNumbers;
    @Form(prefix="address") Map<String, Address> addresses;
}

public static class TelephoneNumber {
    @FormParam("countryCode") private String countryCode;
    @FormParam("number") private String number;
}

public static class Address {
    @FormParam("street") private String street;
    @FormParam("houseNumber") private String houseNumber;
}

@Path("person")
public static class MyResource {

    @POST
    @Consumes(MediaType.APPLICATION_FORM_URLENCODED)
    public void post (@Form Person p) {}
}
```

The following form params could be submitted and the `Person.telephoneNumbers` and `Person.addresses` fields would be populated appropriately

```
request.addFormHeader("telephoneNumbers[0].countryCode", "31");
request.addFormHeader("telephoneNumbers[0].number", "0612345678");
```

```
request.addFormHeader("telephoneNumbers[1].countryCode", "91");
request.addFormHeader("telephoneNumbers[1].number", "9717738723");
request.addFormHeader("address[INVOICE].street", "Main Street");
request.addFormHeader("address[INVOICE].houseNumber", "2");
request.addFormHeader("address[SHIPPING].street", "Square One");
request.addFormHeader("address[SHIPPING].houseNumber", "13");
```

Chapter 13. Improved `@···Param` annotations

The Jakarta RESTful Web Services specification defines annotations `@PathParam`, `@QueryParam`, `@FormParam`, `@CookieParam`, `@HeaderParam` and `@MatrixParam`. Each annotation requires a parameter name. RESTEasy provides a parallel set of annotations, `@PathParam`, `@QueryParam`, `@FormParam`, `@CookieParam`, `@HeaderParam` and `@MatrixParam` which do not require a parameter name. To use this RESTEasy feature, replace the annotation's package name, `jakarta.ws.rs` with, `org.jboss.resteasy.annotations.jaxrs`.

Note that you can omit the annotation name for annotated method parameters as well as annotated fields or JavaBean properties.

Usage:

```
import org.jboss.resteasy.annotations.jaxrs.*;

@Path("/library")
public class Library {

   @GET
   @Path("/book/{isbn}")
   public String getBook(@PathParam String isbn) {
      // search my database and get a string representation and return it
   }
}
```

If an annotated variable does not have the same name as the path parameter, the name can still be specified:

```
import org.jboss.resteasy.annotations.jaxrs.*;

@Path("/library")
public class Library {

   @GET
   @Path("/book/{isbn}")
   public String getBook(@PathParam("isbn") String id) {
      // search my database and get a string representation and return it
   }
}
```

Chapter 14. Optional parameter types

RESTEasy offers a mechanism to support a series of `java.util.Optional` types as a wrapper object types. This will give users the ability to use optional typed parameters, and eliminate all null checks by using methods like `Optional.orElse()`.

Here is the sample:

```
@Path("/double")
@GET
public String optDouble(@QueryParam("value") OptionalDouble value) {
    return Double.toString(value.orElse(4242.0));
}
```

From the above sample code we can see that the `OptionalDouble` can be used as parameter type, and when users don't provide a value in `@QueryParam`, then the default value will be returned.

Here is the list of supported optional parameter types:

- `@QueryParam`
- `@FormParam`
- `@MatrixParam`
- `@HeaderParam`
- `@CookieParam`

As the list shown above, those parameter types support the Java-provided `Optional` types. Please note that the `@PathParam` is an exception for which `Optional` is not available. The reason is that `Optional` for the `@PathParam` use case would just be a NO-OP, since an element of the path cannot be omitted.

The `Optional` types can also be used as type of the fields of a `@BeanParam's class.

Here is an example of endpoint with a `@BeanParam`:

```
@Path("/double")
@GET
public String optDouble(@BeanParam Bean bean) {
    return Double.toString(bean.value.orElse(4242.0));
}
```

The corresponding class `Bean`:

```
public class Bean {
    @QueryParam("value")
    OptionalDouble value;
```

```
}
```

Finally, the `Optional` types can be used directly as type of the fields of a Jakarta RESTful Web Services resource class.

Here is an example of a Jakarta RESTful Web Services resource class with an `Optional` type:

```
@RequestScoped
public class OptionalResource {
    @QueryParam("value")
    Optional<String> value;
}
```

Chapter 15. @DefaultValue

`@DefaultValue` is a parameter annotation that can be combined with any of the other @*Param annotations to define a default value when the HTTP request item does not exist.

```
@GET
public String getBooks(@QueryParam("num") @DefaultValue("10") int num) {}
```

Chapter 16. @Encoded and encoding

Jakarta RESTful Web Services allows encoded or decoded `@*Params`, the specification of path definitions and parameter names using encoded or decoded strings.

The `@jakarta.ws.rs.Encoded` annotation can be used on a class, method, or param. By default, inject `@PathParam` and `@QueryParams` are decoded. Additionally adding the `@Encoded` annotation, the value of these params will be provided in encoded form.

```
@Path("/")
public class MyResource {

    @Path("/{param}")
    @GET
    public String get(@PathParam("param") @Encoded String param) {}
}
```

In the above example, the value of the `@PathParam` injected into the param of the `get()` method will be URL encoded. Adding the `@Encoded` annotation as a parameter annotation triggers this affect.

The `@Encoded` annotation may also be used on the entire method and any combination of `@QueryParam` or `@PathParam's values will be encoded.

```
@Path("/")
public class MyResource {

    @Path("/{param}")
    @GET
    @Encoded
    public String get(@QueryParam("foo") String foo, @PathParam("param") String param)
{}
}
```

In the above example, the values of the foo query param and param path param will be injected as encoded values.

The default can also be encoded for the entire class.

```
@Path("/")
@Encoded
public class ClassEncoded {

    @GET
    public String get(@QueryParam("foo") String foo) {}
}
```

Chapter 17. @Context



Support for `@Context` inject has been deprecated. RESTEasy supports using the `@Inject` annotation on instance variables. Support for method and constructor injection will be available in a future release.

The `@Context` annotation allows the injection of instances of the following types:

- `jakarta.ws.rs.core.HttpHeaders`
- `jakarta.ws.rs.core.UriInfo`
- `jakarta.ws.rs.core.Request`
- `jakarta.servlet.http.HttpServletRequest`
- `jakarta.servlet.http.HttpServletResponse`
- `jakarta.servlet.ServletConfig`
- `jakarta.servlet.ServletContext`
- `jakarta.ws.rs.core.SecurityContext`

Chapter 18. Jakarta RESTful Web Services Resource Locators and Sub Resources

Resource classes are able to partially process a request and provide another sub resource object that can process the remainder of the request. For example:

```
@Path("/")
public class ShoppingStore {

    @Inject
    CustomerRegistry customerRegistry;

    @Path("/customers/{id}")
    public Customer getCustomer(@PathParam("id") int id) {
        return customerRegistry.find(id);
    }
}

public class Customer {

    @GET
    public String get() {}

    @Path("/address")
    public String getAddress() {}
}
```

Resource methods that have a `@Path` annotation, but no HTTP method are considered sub-resource locators. Their job is to provide an object that can process the request. In the above example `ShoppingStore` is a root resource because its class is annotated with `@Path`. The `getCustomer()` method is a sub-resource locator method.

If the client invoked:

```
GET /customer/123
```

The `ShoppingStore.getCustomer()` method would be invoked first. This method provides a Customer object that can service the request. The http request will be dispatched to the `Customer.get()` method. Another example is:

```
GET /customer/123/address
```

In this request, again, first the `ShoppingStore.getCustomer()` method is invoked. A customer object is returned, and the rest of the request is dispatched to the `Customer.getAddress()` method.

Another interesting feature of Sub-resource locators is that the locator method result is dynamically processed at runtime to figure out how to dispatch the request. So, the `ShoppingStore.getCustomer()` method does not have to declare any specific type.

```
@Path("/")
public class ShoppingStore {

    @Inject
    CustomerRegistry customerRegistry;

    @Path("/customers/{id}")
    public java.lang.Object getCustomer(@PathParam("id") int id) {
        return customerRegistry.find(id);
    }
}

public class Customer {

    @GET
    public String get() {}

    @Path("/address")
    public String getAddress() {}
}
```

In the above example, `getCustomer()` returns a `java.lang.Object`. Per request, at runtime, the Jakarta RESTful Web Services server will determine how to dispatch the request based on the object returned by `getCustomer()`. Possible uses of this are:

- There maybe a class hierarchy for your customers. Customer is the abstract base class, `CorporateCustomer` and `IndividualCustomer` are subclasses.
- The `getCustomer()` method might be doing a Hibernate polymorphic query and doesn't know, or care, what concrete class is it querying for, or what it returns.

```
@Path("/")
public class ShoppingStore {

   @Path("/customers/{id}")
   public java.lang.Object getCustomer(@PathParam("id") int id) {
      return entityManager.find(Customer.class, id);
   }
}

public class Customer {

    @GET
    public String get() {}
```

```
    @Path("/address")
    public String getAddress() {}
}

public class CorporateCustomer extends Customer {

    @Path("/businessAddress")
    public String getAddress() {}
}
```

Chapter 19. Resources metadata configuration

When processing Jakarta RESTful Web Services deployments, RESTEasy relies on `ResourceBuilder` to create metadata for each resource. Such metadata is defined using the metadata SPI in package `org.jboss.resteasy.spi.metadata`, in particular the `ResourceClass` interface:

```
package org.jboss.resteasy.spi.metadata;

public interface ResourceClass {
  String getPath();

  Class<?> getClazz();

  ResourceConstructor getConstructor();

  FieldParameter[] getFields();

  SetterParameter[] getSetters();

  ResourceMethod[] getResourceMethods();

  ResourceLocator[] getResourceLocators();
}
```

Among the other classes and interfaces defining metadata SPI, the following interfaces are worth a mention here:

```
public interface ResourceConstructor {
  ResourceClass getResourceClass();

  Constructor getConstructor();

  ConstructorParameter[] getParams();
}

public interface ResourceMethod extends ResourceLocator {
  SetString getHttpMethods();

  MediaType[] getProduces();

  MediaType[] getConsumes();

  boolean isAsynchronous();

  void markAsynchronous();
}
```

```
public interface ResourceLocator {
  ResourceClass getResourceClass();

  Class<?> getReturnType();

  Type getGenericReturnType();

  Method getMethod();

  Method getAnnotatedMethod();

  MethodParameter[] getParams();

  String getFullpath();

  String getPath();

}
```

The interesting point is that RESTEasy allows tuning the metadata generation by providing implementations of the `ResourceClassProcessor` interface:

```
package org.jboss.resteasy.spi.metadata;

public interface ResourceClassProcessor {

  /**
   * Allows the implementation of this method to modify the resource metadata
represented by
   * the supplied {@link ResourceClass} instance. Implementation will typically create
   * wrappers which modify only certain aspects of the metadata.
   *
   * @param clazz The original metadata
   * @return the (potentially modified) metadata (never null)
   */
  ResourceClass process(ResourceClass clazz);

}
```

The processors are meant to be, and are resolved as, regular Jakarta RESTful Web Services annotated providers. They allow for wrapping resource metadata classes with custom versions that can be used for various advanced scenarios like

- adding additional resource method/locators to the resource
- altering the http methods
- altering the @Produces / @Consumes media types

Chapter 20. Jakarta RESTful Web Services Content Negotiation

The HTTP protocol has built in content negotiation headers that allow the client and server to specify what content they are transferring and what content they would prefer to get. The server declares content preferences via the `@Produces` and `@Consumes` headers.

`@Consumes` is an array of media types that a particular resource or resource method consumes. For example:

```
@Consumes("text/*")
@Path("/library")
public class Library {

    @POST
    public String stringBook(String book) {}

    @Consumes("text/xml")
    @POST
    public String jaxbBook(Book book) {}
}
```

When a client makes a request, Jakarta RESTful Web Services first finds all methods that match the path, then, it those matches based on the content-type header sent by the client. So, if a client sent:

```
POST /library
Content-Type: text/plain

This is a nice book
```

The `stringBook()` method would be invoked because it matches to the default `text/*` media type. Now, if the client instead sends XML:

```
POST /library
Content-Type: text/xml

book name="EJB 3.0" author="Bill Burke"/
```

The `jaxbBook()` method would be invoked.

The `@Produces` is used to map a client request and match it up to the client's Accept header. The Accept HTTP header is sent by the client and defines the media types the client prefers to receive from the server.

```
@Produces("text/*")
```

```
@Path("/library")
public class Library {

@GET
@Produces("application/json")
public String getJSON() {}

@GET
public String get() {}
```

So, if the client sends:

```
GET /library
Accept: application/json
```

The `getJSON()` method would be invoked.

`@Consumes` and `@Produces` can list multiple media types that they support. The client's Accept header can also send multiple types it might like to receive. More specific media types are chosen first. The client Accept header or @Produces @Consumes can also specify weighted preferences that are used to match requests with resource methods. This is best explained by RFC 2616 section 14.1. RESTEasy supports this complex way of doing content negotiation.

A variant in Jakarta RESTful Web Services is a combination of media type, content-language, and content encoding as well as etags, last modified headers, and other preconditions. This is a more complex form of content negotiation that is done programmatically by the application developer using the `jakarta.ws.rs.core.Variant`, `VariantListBuilder`, and `Request` objects. Request is injected via `@Context`. Read the javadoc for more info on these.

20.1. URL-based negotiation

Some clients, like browsers, cannot use the Accept and Accept-Language headers to negotiation the representation's media type or language. RESTEasy allows the mapping of file name suffixes like (.xml, .txt, .en, .fr) to media types and languages. These file name suffixes take the place and override any Accept header sent by the client. They are configured using the `resteasy.media.type.mappings` and `resteasy.language.mappings` parameters. If configured within the application's web.xml, it would look like:

```
<web-app>
    <display-name>Archetype Created Web Application</display-name>
    <context-param>
        <param-name>resteasy.media.type.mappings</param-name>
        <param-value>html : text/html, json : application/json, xml :
application/xml</param-value>
    </context-param>

   <context-param>
```

```
        <param-name>resteasy.language.mappings</param-name>
        <param-value>en : en-US, es : es, fr : fr</param-value>
    </context-param>
</web-app>
```

See Configuration for more information about application configuration.

Mappings are a comma delimited list of suffix/mediatype or suffix/language mappings. Each mapping is delimited by a ':'. If GET /foo/bar.xml.en is invoked , this would be equivalent to invoking the following request:

```
GET /foo/bar
Accept: application/xml
Accept-Language: en-US
```

The mapped file suffixes are stripped from the target URL path before the request is dispatched to a corresponding Jakarta RESTful Web Services resource.

20.2. Query String Parameter-based negotiation

RESTEasy can do content negotiation based in a parameter in the query string. To enable this, the parameter `resteasy.media.type.param.mapping` should be configured in the web.xml file. It would look like the following:

```
<web-app>
    <display-name>Archetype Created Web Application</display-name>
    <context-param>
        <param-name>resteasy.media.type.param.mapping</param-name>
        <param-value>someName</param-value>
    </context-param>
</web-app>
```

See Configuration for more information about application configuration.

The param-value is the name of the query string parameter that RESTEasy will use in the place of the Accept header.

Invoking http://localhost/resouce?someName=application/xml, will give the application/xml media type the highest priority in the content negotiation.

In cases where the request contains both the parameter and the Accept header, the parameter will be more relevant.

It is possible to leave the param-value empty, that will cause the processor to look for a parameter named 'accept'.

Chapter 21. Content Marshalling/Providers

21.1. Default Providers and default Jakarta RESTful Web Services Content Marshalling

RESTEasy can automatically marshal and unmarshal a few different message bodies.

Media Types	Java Type
application/*+xml, text/*\+xml, application/*\+json, application/*\+fastinfoset, application/atom\+*	Jakarta XML Binding annotated classes
application/*+xml, text/*+xml	org.w3c.dom.Document
/	java.lang.String
/	java.io.InputStream
text/plain	primitives, java.lang.String, or any type that has a String constructor, or static valueOf(String) method for input, toString() for output
/	jakarta.activation.DataSource
/	java.io.File
/	byte[]
application/x-www-form-urlencoded	jakarta.ws.rs.core.MultivaluedMap



When a java.io.File is created, as in

```
@Path("/test")
public class TempFileDeletionResource {
   @POST
   @Path("post")
   public Response post(File file)  {
      return Response.ok(file.getPath()).build();
   }
}
```

a temporary file is created in the file system. On the server side, that temporary file will be deleted at the end of the invocation. On the client side, however, it is the responsibility of the user to delete the temporary file.

21.2. Content Marshalling with @Provider classes

The Jakarta RESTful Web Services specification allows the user to plug in their own request/response body reader and writers. To do this, the user annotates a class with @Provider and

specify the `@Produces` types for a writer and `@Consumes` types for a reader. The user must also implement a MessageBodyReader/Writer interface respectively. Here is an example:

```
@Provider
@Produces("text/plain")
@Consumes("text/plain")
public class DefaultTextPlain implements MessageBodyReader, MessageBodyWriter {

    @Override
    public boolean isReadable(Class<?> type, Type genericType, Annotation[]
annotations, MediaType mediaType) {
       // StringTextStar should pick up strings
       return !String.class.equals(type)  TypeConverter.isConvertable(type);
    }

    @Override
    public Object readFrom(Class<?> type, Type genericType, Annotation[] annotations,
MediaType mediaType, MultivaluedMap httpHeaders, InputStream entityStream) throws
IOException, WebApplicationException {
       InputStream delegate = NoContent.noContentCheck(httpHeaders, entityStream);
       String value = ProviderHelper.readString(delegate, mediaType);
       return TypeConverter.getType(type, value);
    }

    @Override
    public boolean isWriteable(Class<?> type, Type genericType, Annotation[]
annotations, MediaType mediaType) {
       // StringTextStar should pick up strings
       return !String.class.equals(type) && !type.isArray();
    }

    @Override
    public long getSize(Object o, Class<?> type, Type genericType, Annotation[]
annotations, MediaType mediaType) {
       String charset = mediaType.getParameters().get("charset");
       if (charset != null)
          try {
             return o.toString().getBytes(charset).length;
          } catch (UnsupportedEncodingException e) {
             // Use default encoding.
          }
       return o.toString().getBytes(StandardCharsets.UTF_8).length;
    }

    @Override
    public void writeTo(Object o, Clas<?> type, Type genericType, Annotation[]
annotations, MediaType mediaType, MultivaluedMap httpHeaders, OutputStream
entityStream) throws IOException, WebApplicationException {
       String charset = mediaType.getParameters().get("charset");
       if (charset == null) entityStream.write(o.toString().getBytes(StandardCharsets
```

```
.UTF_8));
       else entityStream.write(o.toString().getBytes(charset));
    }
}
```

Note that in order to support Async IO, the user must implement the `AsyncMessageBodyWriter` interface, which requires the implementation of this extra method:

```
 @Provider
 @Produces("text/plain")
 @Consumes("text/plain")
 public class DefaultTextPlain implements MessageBodyReader, AsyncMessageBodyWriter {
    // ...
    public CompletionStage<Void> asyncWriteTo(Object o, Class<?> type, Type
genericType, Annotation[] annotations, MediaType mediaType, MultivaluedMap
httpHeaders, AsyncOutputStream entityStream) {
       String charset = mediaType.getParameters().get("charset");
       if (charset == null)
          return entityStream.asyncWrite(o.toString().getBytes(StandardCharsets.UTF_8
));
       else
          return entityStream.asyncWrite(o.toString().getBytes(charset));
    }
 }
```

The RESTEasy ServletContextLoader will automatically scan the application's `WEB-INF/lib` and classes directories for classes annotated with `@Provider` or the user can manually configure them in web.xml. See Installation/Configuration.

21.3. Providers Utility Class

`jakarta.ws.rs.ext.Providers` is a simple injectable interface that allows you to look up `MessageBodyReaders`, Writers, ContextResolvers, and ExceptionMappers. It is very useful, for instance, for implementing multipart providers. Content types that embed other random content types.

A Providers instance is injectable into MessageBodyReader or Writers:

```
@Provider
@Consumes("multipart/fixed")
public class MultipartProvider implements MessageBodyReader {

    @Context
    private Providers providers;

}
```

21.4. Configuring Document Marshalling

XML document parsers are subject to a form of attack known as the XXE (Xml eXternal Entity) Attack (https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing), in which expanding an external entity causes an unsafe file to be loaded. For example, the document

```
<?xml version="1.0"?>
<!DOCTYPE foo
[<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<search>
    <user>bill</user>
    <file>&xxe;<file>
</search>
```

could cause the passwd file to be loaded.

Similarly, RESTEasy's built-in provider for `javax.xml.transform.Source` (used when resource methods accept or return `Source` objects) applies XXE protections when writing `StreamSource` instances. The `StreamSource` is wrapped with a securely configured `XMLReader` that disables external entity resolution by default. The `TransformerFactory` used for serialization is also configured with secure processing enabled.

These protections are controlled by the same configuration parameters described below (`resteasy.document.expand.entity.references`, `resteasy.document.secure.processing.feature`, and `resteasy.document.secure.disableDTDs`). Setting these parameters to allow entity expansion or disable secure processing will weaken the XXE protections for `Source` types as well.



If a custom `MessageBodyReader` produces a non-`StreamSource` type (e.g. `SAXSource`), the application is responsible for ensuring the source is securely configured. RESTEasy only applies XXE hardening to `StreamSource` instances produced by its built-in reader.

By default, RESTEasy's built-in unmarshaller for `org.w3c.dom.Document` documents will not expand external entities, replacing them by the empty string instead. It can be configured to replace external entities by values defined in the DTD by setting the parameter `resteasy.document.expand.entity.references` to "true". If configured in the `web.xml` file, it would be:

```
<context-param>
    <param-name>resteasy.document.expand.entity.references</param-name>
    <param-value>true</param-value>
</context-param>
```

See Configuration for more information about application configuration.

Another way of dealing with the problem is by prohibiting DTDs, which RESTEasy does by default. This behavior can be changed by setting the parameter `resteasy.document.secure.disableDTDs` to

"false".

Documents are also subject to Denial of Service (DoS) Attacks when buffers are overrun by large entities or too many attributes. This is known as the "Billion Laughs" or "XML Bomb" attack. For example, if a DTD defined the following entities:

```
<!ENTITY foo "foo">
<!ENTITY foo1 "&foo;&foo;&foo;&foo;&foo;&foo;&foo;&foo;&foo;&foo;">
<!ENTITY foo2 "&foo1;&foo1;&foo1;&foo1;&foo1;&foo1;&foo1;&foo1;&foo1;&foo1;">
<!ENTITY foo3 "&foo2;&foo2;&foo2;&foo2;&foo2;&foo2;&foo2;&foo2;&foo2;&foo2;">
<!ENTITY foo4 "&foo3;&foo3;&foo3;&foo3;&foo3;&foo3;&foo3;&foo3;&foo3;&foo3;">
<!ENTITY foo5 "&foo4;&foo4;&foo4;&foo4;&foo4;&foo4;&foo4;&foo4;&foo4;&foo4;">
<!ENTITY foo6 "&foo5;&foo5;&foo5;&foo5;&foo5;&foo5;&foo5;&foo5;&foo5;&foo5;">
```

then the expansion of <code>&foo6;</code> would result in 1,000,000 "foo" strings, consuming significant memory from a small XML input.

By default, RESTEasy enables `XMLConstants.FEATURE_SECURE_PROCESSING` which limits the number of entity expansions, total entity size, element depth, and number of attributes per entity. The parameter `resteasy.document.secure.processing.feature` controls this behavior (default is `true`).



On Java 24+, the default entity expansion limit was reduced from 64,000 to 2,500 for improved security. If you encounter `SAXParseException` errors related to entity expansion limits, see the JAXB providers section for details on how to handle this change.

21.5. Text media types and character sets

The Jakarta RESTful Web Services specification says

> When writing responses, implementations SHOULD respect application-supplied character set metadata and SHOULD use UTF-8 if a character set is not specified by the application or if the application specifies a character set that is unsupported.

On the other hand, the HTTP specification says

> When no explicit charset parameter is provided by the sender, media subtypes of the "text" type are defined to have a default charset value of "ISO-8859-1" when received via HTTP. Data in character sets other than "ISO-8859-1" or its subsets MUST be labeled with an appropriate charset value.

It follows that, in the absence of a character set specified by a resource or resource method, RESTEasy SHOULD use UTF-8 as the character set for text media types, and, if it does, it MUST add an explicit charset parameter to the Content-Type response header. RESTEasy started adding the

explicit charset parameter in releases 3.1.2.Final and 3.0.22.Final, and that new behavior could cause some compatibility problems. To specify the previous behavior, in which UTF-8 was used for text media types, but the explicit charset was not appended, the parameter "resteasy.add.charset" may be set to "false". It defaults to "true".



By "text" media types, we mean

- a media type with type "text" and any subtype;
- a media type with type ""application" and subtype beginning with "xml".

The latter set includes "application/xml-external-parsed-entity" and "application/xml-dtd".

Chapter 22. Jakarta XML Binding providers



In Jakarta RESTful Web Services 4.0 the Jakarta XML Binding support was removed from specification. RESTEasy still plans on supporting Jakarta XML Binding for the time being. This may not be supported in a future release. Please plan on migrating to JSON or some other supported data type.

As required by the specification, RESTEasy includes support for (un)marshalling Jakarta XML Binding annotated classes. RESTEasy provides multiple providers to address some subtle differences between classes generated by XJC and classes which are simply annotated with @XmlRootElement, or working with JAXBElement classes directly.

For the most part, developers using the Jakarta RESTful Web Services API, the selection of which provider is invoked will be completely transparent. For developers wishing to access the providers directly (which most folks won't need to do), this document describes which provider is best suited for different configurations.

A Jakarta XML Binding Provider is selected by RESTEasy when a parameter or return type is an object that is annotated with annotations Jakarta XML Binding (such as `@XmlRootEntity` or `@XmlType`) or if the type is a JAXBElement. Additionally, the resource class or resource method will be annotated with either a `@Consumes` or `@Produces` annotation and contain one or more of the following values:

- text/*+xml
- application/*+xml
- application/*+fastinfoset
- application/*+json

RESTEasy will select a different provider based on the return type or parameter type used in the resource. This section describes how the selection process works.

`@XmlRootEntity` When a class is annotated with a `@XmlRootElement` annotation, RESTEasy will select the `JAXBXmlRootElementProvider`. This provider handles basic marshaling and unmarshalling of custom Jakarta XML Binding entities.

`@XmlType` Classes which have been generated by XJC will most likely not contain an `@XmlRootEntity` annotation. In order for these classes to marshalled, they must be wrapped within a `JAXBElement` instance. This is typically accomplished by invoking a method on the class which serves as the `XmlRegistry` and is named `ObjectFactory`.

The `JAXBXmlTypeProvider` provider is selected when the class is annotated with an `XmlType` annotation and not an `XmlRootElement` annotation.

This provider simplifies this task by attempting to locate the `XmlRegistry` for the target class. By default, a Jakarta XML Binding implementation will create a class called `ObjectFactory` and is located in the same package as the target class. When this class is located, it will contain a create method that takes the object instance as a parameter. For example, if the target type is called

`Contact`, then the `ObjectFactory` class will have a method:

```
public JAXBElement<?> createContact(Contact value);
```

`JAXBElement<?>` If your resource works with the JAXBElement class directly, the RESTEasy runtime will select the `JAXBElementProvider`. This provider examines the `ParameterizedType` value of the `JAXBElement` in order to select the appropriate `JAXBContext`.

22.1. Jakarta XML Binding Decorators

Resteasy's Jakarta XML Binding providers have a pluggable way to decorate Marshaller and Unmarshaller instances. The way it works is that you can write an annotation that can trigger the decoration of a Marshaller or Unmarshaller. Your decorators can do things like set Marshaller or Unmarshaller properties, set up validation, and the like. Here's an example. Say we want to have an annotation that will trigger pretty-printing, nice formatting, of an XML document. If we were using raw Jakarta XML Binding, we would set a property on the Marshaller of Marshaller.JAXB_FORMATTED_OUTPUT. Let's write a Marshaller decorator.

First define an annotation:

```
import org.jboss.resteasy.annotations.Decorator;

@Target({ElementType.TYPE, ElementType.METHOD, ElementType.PARAMETER, ElementType
.FIELD})
@Retention(RetentionPolicy.RUNTIME)
@Decorator(processor = PrettyProcessor.class, target = Marshaller.class)
public @interface Pretty {}
```

For this to work, we must annotate the `@Pretty` annotation with a meta-annotation called @Decorator. The `target()` attribute must be the Jakarta XML Binding Marshaller class. The `processor()` attribute is a class we will write next.

```
import org.jboss.resteasy.core.interception.DecoratorProcessor;
import org.jboss.resteasy.annotations.DecorateTypes;

import jakarta.xml.bind.Marshaller;
import jakarta.xml.bind.PropertyException;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.Produces;
import java.lang.annotation.Annotation;

/**
 * @author <a href="mailto:bill@burkecentral.com">Bill Burke</a>
 * @version $Revision: 1 $
 */
@DecorateTypes({"text/*+xml", "application/*+xml"})
public class PrettyProcessor implements DecoratorProcessor<Marshaller, Pretty> {
```

```
    public Marshaller decorate(Marshaller target, Pretty annotation,
                  Class<?> type, Annotation[] annotations, MediaType mediaType) {
       target.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, Boolean.TRUE);
    }
}
```

The processor implementation must implement the `DecoratorProcessor` interface and should also be annotated with `@DecorateTypes`. This annotation specifies what media types the processor can be used with. Now that we've defined our annotation and our Processor, we can use it on our Jakarta RESTful Web Services resource methods or Jakarta XML Binding types as follows:

```
@GET
@Pretty
@Produces("application/xml")
public SomeJAXBObject get() {}
```

Check the RESTEasy source code for the implementation of @XmlHeader for more detailed information

22.2. Pluggable JAXBContext's with ContextResolvers

Do not use this feature unless you are knowledgeable about using it.

Based on the class being marshalling/unmarshalling, RESTEasy will, by default create and cache JAXBContext instances per class type. If you do not want RESTEasy to create JAXBContexts, plug in your own by implementing an instance of `jakarta.ws.rs.ext.ContextResolver`

```
public interface ContextResolver<T> {
    T getContext(Class<?> type);
}

@Provider
@Produces("application/xml")
public class MyJAXBContextResolver implements ContextResolver<JAXBContext> {
    JAXBContext getContext(Class<?> type) {
        if (type.equals(WhateverClassIsOverridedFor.class)) return JAXBContext
.newInstance();
        return null;
    }
}
```

A `@Produces` annotation must be provided to specify the media type the context is meant for. An implementation of `ContextResolver<JAXBContext>` must be provided and that class must have the `@Provider` annotation. This helps the runtime match to the correct context resolver.

There are multiple ways to make this ContextResolver available.

1. Return it as a class or instance from a jakarta.ws.rs.core.Application implementation
2. List it as a provider with resteasy.providers
3. Let RESTEasy automatically scan for it within the WAR file. See Configuration Guide
4. Manually add it via `ResteasyProviderFactory.getInstance().registerProvider(Class)` or `registerProviderInstance(Object)`

22.3. Jakarta XML Binding + XML provider

RESTEasy is required to provide Jakarta XML Binding provider support for XML. It has a few extra annotations that can help code the application.

22.3.1. @XmlHeader and @Stylesheet

Sometimes when outputting XML documents you may want to set an XML header. RESTEasy provides the `@org.jboss.resteasy.annotations.providers.jaxb.XmlHeader` annotation for this. For example:

```
@XmlRootElement
public static class Thing {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

@Path("/test")
public static class TestService {
    @GET
    @Path("/header")
    @Produces("application/xml")
    @XmlHeader("<?xml-stylesheet type='text/xsl' href='${baseuri}foo.xsl' ?>")
    public Thing get() {
        Thing thing = new Thing();
        thing.setName("bill");
        return thing;
    }
}
```

The `@XmlHeader` forces the XML output to have a xml-stylesheet header. This header could also have been put on the Thing class to get the same result. See the javadocs for more details on how to use substitution values provided by resteasy.

RESTEasy also has a convenience annotation for stylesheet headers. For example:

```
@XmlRootElement
public static class Thing {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

@Path("/test")
public static class TestService {
    @GET
    @Path("/stylesheet")
    @Produces("application/xml")
    @Stylesheet(type="text/css", href="${basepath}foo.xsl")
    @Junk
    public Thing getStyle() {
        Thing thing = new Thing();
        thing.setName("bill");
        return thing;
    }
}
```

22.4. Jakarta XML Binding + JSON provider

RESTEasy supports the marshalling of Jakarta XML Binding annotated POJOs to and from JSON. This provider wraps the Jackson2 library to accomplish this.

To use this integration with Jackson import the resteasy-jackson2-provider Maven module.

For example, consider this Jakarta XML Binding class:

```
@XmlRootElement(name = "book")
public class Book {
    private String author;
    private String ISBN;
    private String title;

    public Book() {
    }

    public Book(String author, String ISBN, String title) {
        this.author = author;
```

```
        this.ISBN = ISBN;
        this.title = title;
    }

    @XmlElement
    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
    this.author = author;
    }

    @XmlElement
    public String getISBN() {
        return ISBN;
    }

    public void setISBN(String ISBN) {
        this.ISBN = ISBN;
    }

    @XmlAttribute
    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }
}
```

And we can write a method to use the above entity:

```
@Path("/test_json")
@GET
@Produces(MediaType.APPLICATION_JSON)
public Book test_json() {
    Book book = new Book();
    book.setTitle("EJB 3.0");
    book.setAuthor("Bill Burke");
    book.setISBN("596529260");
    return book;
}
```

When making a requesting of the above method, the default Jackson2 marshaller would return JSON output that looked like this:

```
$ http localhost:8080/dummy/test_json
```

```
HTTP/1.1 200
...
Content-Type: application/json

{
"ISBN": "596529260",
"author": "Bill Burke",
"title": "EJB 3.0"
}
```

22.5. Jakarta XML Binding + FastinfoSet provider

RESTEasy supports the FastinfoSet mime type with Jakarta XML Binding annotated classes. Fast infoset documents are faster to serialize and parse, and smaller in size, than logically equivalent XML documents. Thus, fast infoset documents may be used whenever the size and processing time of XML documents is an issue. It is configured the same way the provider is.

To use this integration with Fastinfoset import the resteasy-fastinfoset-provider Maven module.

22.6. Arrays and Collections of Jakarta XML Binding Objects

RESTEasy will automatically marshal arrays, java.util.Set's, and java.util.List's of Jakarta XML Binding objects to and from XML, JSON, Fastinfoset.

```
@XmlRootElement(name = "customer")
@XmlAccessorType(XmlAccessType.FIELD)
public class Customer {
    @XmlElement
    private String name;

    public Customer() {
    }

    public Customer(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

@Path("/")
public class MyResource {
    @PUT
    @Path("array")
    @Consumes("application/xml")
```

```
    public void putCustomers(Customer[] customers) {
        Assertions.assertEquals("bill", customers[0].getName());
        Assertions.assertEquals("monica", customers[1].getName());
    }

    @GET
    @Path("set")
    @Produces("application/xml")
    public Set<Customer> getCustomerSet() {
        return Set.of(new Customer("bill"), new Customer("monica"));
    }

    @PUT
    @Path("list")
    @Consumes("application/xml")
    public void putCustomers(List<Customer> customers) {
        Assertions.assertEquals("bill", customers.get(0).getName());
        Assertions.assertEquals("monica", customers.get(1).getName());
    }
}
```

The above resource can publish and receive Jakarta XML Binding objects. It is assumed they are wrapped in a collection element

```
<collection>
    <customer><name>bill</name></customer>
    <customer><name>monica</name></customer>
<collection>
```

The namespace URI, namespace tag, and collection element name can be changed by using the `@org.jboss.resteasy.annotations.providers.jaxb.Wrapped` annotation on a parameter or method

```
@Target({ElementType.PARAMETER, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface Wrapped {
    String element() default "collection";

    String namespace() default "http://jboss.org/resteasy";

    String prefix() default "resteasy";
}
```

To output this XML

```
<foo:list xmlns:foo="https://jboss.org">
    <customer><name>bill</name></customer>
    <customer><name>monica</name></customer>
```

```
</foo:list>
```

The `@Wrapped` annotation would be used as follows:

```
@GET
@Path("list")
@Produces("application/xml")
@Wrapped(element="list", namespace="https://jboss.org", prefix="foo")
public List<Customer> getCustomerSet() {
    return List.of(new Customer("bill"), new Customer("monica"));
}
```

22.6.1. Retrieving Collections on the client side

To retrieve a `List` or `Set` of Jakarta XML Binding objects on the client side, the element type returned in the List or Set must be identified. Below the call to `readEntity()` will fail because the class type, Customer has not been properly identified:

```
Response response = request.get();
List<Customer> list = response.readEntity(List.class);
```

Use `jakarta.ws.rs.core.GenericType` to declare the data type,`Customer`, returned within the List.

```
Response response = request.get();
GenericType<List<Customer>> genericType = new GenericType<>() {};
List<Customer> list = response.readEntity(genericType);
```

For more information about `GenericType`, please see its javadoc.

The same strategy applies to retrieving a `Set`:

```
Response response = request.get();
GenericType<Set<Customer>> genericType = new GenericType<>() {};
Set<Customer> set = response.readEntity(genericType);
```

`GenericType` is not necessary to retrieve an array of Jakarta XML Binding objects:

```
Response response = request.get();
Customer[] array = response.readEntity(Customer[].class);
```

22.6.2. JSON and Jakarta XML Binding Collections/arrays

RESTEasy supports using collections with JSON. It encloses List, Set, or arrays of returned XML objects within a simple JSON array. For example:

```
@XmlRootElement
@XmlAccessorType(XmlAccessType.FIELD)
public static class Foo {
    @XmlAttribute
    private String test;

    public Foo() {
    }

    public Foo(String test) {
        this.test = test;
    }

    public String getTest() {
        return test;
    }

    public void setTest(String test) {
        this.test = test;
    }
}
```

A List or array of Foo class would be represented in JSON like this:

```
[{"foo":{"@test":"bill"}},{"foo":{"@test":"monica}"}}]
```

It also expects this format for input

22.7. Maps of XML Objects

RESTEasy automatically marshals maps of Jakarta XML Binding objects to and from XML, JSON, Fastinfoset (or any other new Jakarta XML Binding mapper). The parameter or method return type must be a generic with a String as the key and the Jakarta XML Binding object's type.

```
@XmlRootElement(namespace = "http://foo.com")
public static class Foo {
    @XmlAttribute
    private String name;

    public Foo() {
    }

    public Foo(String name) {
        this.name = name;
    }

    public String getName() {
```

```
        return name;
    }
}

@Path("/map")
public static class MyResource {
    @POST
    @Produces("application/xml")
    @Consumes("application/xml")
    public Map<String, Foo> post(Map<String, Foo> map) {
        Assertions.assertEquals(2, map.size());
        Assertions.assertNotNull(map.get("bill"));
        Assertions.assertNotNull(map.get("monica"));
        Assertions.assertEquals(map.get("bill").getName(), "bill");
        Assertions.assertEquals(map.get("monica").getName(), "monica");
        return map;
    }
}
```

The above resource can publish and receive XML objects within a map. By default, they are wrapped in a "map" element in the default namespace. Also, each "map" element has zero or more "entry" elements with a "key" attribute.

```
<map>
    <entry key="bill" xmlns="http://foo.com">
        <foo name="bill"/>
    </entry>
    <entry key="monica" xmlns="http://foo.com">
        <foo name="monica"/>
    </entry>
</map>
```

The namespace URI, namespace prefix and map, entry, and key element and attribute names can be changed by using the `@org.jboss.resteasy.annotations.providers.jaxb.WrappedMap` annotation on a parameter or method

```
@Target({ElementType.PARAMETER, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface WrappedMap {
    /**
     * map element name
     */
    String map() default "map";

    /**
     * entry element name *
     */
    String entry() default "entry";
```

```
    /**
     * entry's key attribute name
     */
    String key() default "key";

    String namespace() default "";

    String prefix() default "";
}
```

To output this XML

```
<hashmap>
    <hashentry hashkey="bill" xmlns:foo="http://foo.com">
        <foo:foo name="bill"/>
    </hashentry>
</map>
```

Use the `@WrappedMap` annotation as follows:

```
@Path("/map")
public static class MyResource {
    @GET
    @Produces("application/xml")
    @WrappedMap(map="hashmap", entry="hashentry", key="hashkey")
    public Map<String, Foo> get() {
        return Map.of("bill", new Foo("bill"));
    }
}
```

22.7.1. Retrieving Maps on the client side

To retrieve a `Map` of XML objects on the client side, the element types returned in the Map must be identified. Below the call to `readEntity()` will fail because the class types, String and Customer have not been properly identified:

```
Response response = request.get();
Map<String, Customer> map = response.readEntity(Map.class);
```

Use `jakarta.ws.rs.core.GenericType` to declare the data types, `String` and `Customer`, returned within the Map.

```
Response response = request.get();
GenericType<Map<String, Customer> genericType = new GenericType<>() {};
```

```
Map<String, Customer> map = response.readEntity(genericType);
```

For more information about `GenericType`, please see its javadoc.

22.7.2. JSON and XML maps

RESTEasy supports using maps with JSON. It encloses maps returned XML objects within a simple JSON map. For example:

```
@XmlRootElement
@XmlAccessorType(XmlAccessType.FIELD)
public static class Foo {
    @XmlAttribute
    private String test;

    public Foo() {
    }

    public Foo(String test) {
        this.test = test;
    }

    public String getTest() {
        return test;
    }

    public void setTest(String test) {
        this.test = test;
    }
}
```

A List or array of this Foo class would be represented in JSON like this:

```
{ "entry1" : {"foo":{"@test":"bill"}}, "entry2" : {"foo":{"@test":"monica}"}}}
```

It also expects this format for input

22.8. Interfaces, Abstract Classes, and Jakarta XML Binding

Some objects models use abstract classes and interfaces heavily. Unfortunately, Jakarta XML Binding doesn't work with interfaces that are root elements and RESTEasy can't unmarshal parameters that are interfaces or raw abstract classes because it doesn't have enough information to create a JAXBContext. For example:

```
public interface IFoo {}
```

```
@XmlRootElement
public class RealFoo implements IFoo {}

@Path("/xml")
public class MyResource {

    @PUT
    @Consumes("application/xml")
    public void put(IFoo foo) {}
}
```

In this example, RESTEasy will report error, "Cannot find a MessageBodyReader for...". This is because RESTEasy does not know that implementations of IFoo are Jakarta XML Binding classes and doesn't know how to create a JAXBContext for it. As a workaround, RESTEasy allows the use of Jakarta XML Binding annotation @XmlSeeAlso on the interface to correct the problem. (NOTE, this will not work with manual, hand-coded).

```
@XmlSeeAlso(RealFoo.class)
public interface IFoo {}
```

The extra @XmlSeeAlso on IFoo allows RESTEasy to create a JAXBContext that knows how to unmarshal RealFoo instances.

22.9. Configuring Jakarta XML Binding Marshalling

As a consumer of XML datasets, Jakarta XML Binding is subject to a form of attack known as the XXE (Xml eXternal Entity) Attack (https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing), in which expanding an external entity causes an unsafe file to be loaded. Preventing the expansion of external entities is discussed in Configuring Document Marshalling. The same parameter, `resteasy.document.expand.entity.references`

applies to Jakarta XML Binding unmarshallers.

Configuring Document Marshalling also discusses the prohibition of DTDs and the imposition of limits on entity expansion and the number of attributes per element. The parameters `resteasy.document.secure.disableDTDs` and `resteasy.document.secure.processing.feature` discussed there, and their default values, also apply to the representation of Jakarta XML Binding objects.

22.9.1. XML Entity Expansion Limits

When `resteasy.document.secure.processing.feature=true` (the default), `XMLConstants.FEATURE_SECURE_PROCESSING` is enabled, which enforces JVM-level XML processing limits.

What FEATURE_SECURE_PROCESSING Protects Against

`FEATURE_SECURE_PROCESSING` protects against XML-based Denial of Service (DoS) attacks, particularly:

Billion Laughs Attack (XML Bomb): Exponential entity expansion where a small XML document (< 1 KB) can expand to gigabytes in memory, causing OutOfMemoryError and server crashes. For example:

```
<!DOCTYPE root [
  <!ENTITY lol "lol">
  <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
]>
<root>&lol3;</root>  <!-- Expands to 1,000 "lol" strings -->
```

Without limits, this pattern can continue (lol4, lol5, etc.) creating billions of expansions from a tiny input document.

Quadratic Blowup Attack: Using large entity definitions referenced many times, causing excessive memory consumption even without exponential expansion.

Deep Nesting Attacks: Extremely deep element nesting that can cause stack overflow errors.

Attribute Overflow: Elements with hundreds or thousands of attributes that slow parsing.



The `resteasy.document.expand.entity.references` setting controls **external** entity expansion (XXE protection - reading files or making network requests), while `resteasy.document.secure.processing.feature` controls **internal** entity expansion limits (DoS protection). Both work together for comprehensive XML security.



On Java 24+, the `jdk.xml.entityExpansionLimit` may be enforced as a JVM-level security policy even when `resteasy.document.secure.processing.feature=false`. The exact behavior depends on the JVM implementation and security configuration. Setting the JVM property is the most reliable way to adjust this limit if needed.

If you encounter `SAXParseException` with "entity expansion limit" on Java 24+, you can:

1. **Recommended**: Reduce entity usage in your XML documents to stay under 2,500 expansions
2. **Alternative**: Increase the JVM limit (security tradeoff):

   ```
   -Djdk.xml.entityExpansionLimit=10000
   ```

   This property should be set as a JVM argument when starting your application server.

3. **Not recommended**: Disable secure processing (may not bypass JVM-level limits on Java 24+):

   ```
   <context-param>
       <param-name>resteasy.document.secure.processing.feature</param-name>
       <param-value>false</param-value>
   ```

```
</context-param>
```



Disabling this feature exposes your application to XML entity expansion attacks and may not bypass the `jdk.xml.entityExpansionLimit` on Java 24+. Use only for testing or when you have other security controls in place.

For more information: https://docs.oracle.com/en/java/javase/25/security/java-api-xml-processing-jaxp-security-guide.html

Chapter 23. RESTEasy Atom Support

From W3.org (http://tools.ietf.org/html/rfc4287):

"Atom is an XML-based document format that describes lists of related information known as "feeds". Feeds are composed of a number of items, known as "entries", each with an extensible set of attached metadata. For example, each entry has a title. The primary use case that Atom addresses is the syndication of Web content such as weblogs and news headlines to Websites as well as directly to user agents."

23.1. RESTEasy Atom API and Provider

RESTEasy has defined a simple object model in Java to represent Atom and uses Jakarta XML Binding to marshal and unmarshal it. The main classes are in the `org.jboss.resteasy.plugins.providers.atom` package and are `Feed`, `Entry`, `Content`, and `Link`. If you look at the source, you will see that these are annotated with Jakarta XML Binding annotations. The distribution contains the javadocs for this project and are a must to learn the model. Here is a simple example of sending an atom feed using the RESTEasy API.

```
import org.jboss.resteasy.plugins.providers.atom.Content;
import org.jboss.resteasy.plugins.providers.atom.Entry;
import org.jboss.resteasy.plugins.providers.atom.Feed;
import org.jboss.resteasy.plugins.providers.atom.Link;
import org.jboss.resteasy.plugins.providers.atom.Person;

@Path("atom")
public class MyAtomService {
    @GET
    @Path("feed")
    @Produces("application/atom+xml")
    public Feed getFeed() {
        Feed feed = new Feed();
        feed.setId(new URI("http://example.com/42"));
        feed.setTitle("My Feed");
        feed.setUpdated(new Date());
        Link link = new Link();
        link.setHref(new URI("http://localhost"));
        link.setRel("edit");
        feed.getLinks().add(link);
        feed.getAuthors().add(new Person("Bill Burke"));
        Entry entry = new Entry();
        entry.setTitle("Hello World");
        Content content = new Content();
        content.setType(MediaType.TEXT_HTML_TYPE);
        content.setText("Nothing much");
        entry.setContent(content);
        feed.getEntries().add(entry);
        return feed;
```

```
    }
}
```

RESTEasy's atom provider is Jakarta XML Binding based, there are no limits to sending atom objects using XML. All the other Jakarta XML Binding providers that RESTEasy has like JSON and fastinfoset can automatically be re-use. Just add "atom+" in front of the main subtype. i.e. s

```
@Produces("application/atom+json") or @Consumes("application/atom+fastinfoset")
```

23.2. Using Jakarta XML Binding with the Atom Provider

The `org.jboss.resteasy.plugins.providers.atom.Content` class is used to unmarshal and marshal Jakarta XML Binding annotated objects that are the body of the content. Here's an example of sending an `Entry` with a Customer object attached as the body of the entry's content.

```
@XmlRootElement(namespace = "https://jboss.org/Customer")
@XmlAccessorType(XmlAccessType.FIELD)
public class Customer {
    @XmlElement
    private String name;

    public Customer() {
    }

    public Customer(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

@Path("atom")
public static class AtomServer {
    @GET
    @Path("entry")
    @Produces("application/atom+xml")
    public Entry getEntry() {
        Entry entry = new Entry();
        entry.setTitle("Hello World");
        Content content = new Content();
        content.setJAXBObject(new Customer("bill"));
        entry.setContent(content);
        return entry;
    }
```

```
}
```

The `Content.setJAXBObject()` method is used to tell the content object, an object is being sent back and it is to be marshalled appropriately. If using a different base format than XML, i.e. "application/atom+json", this attached object will be marshalled into that same format.

If the input is an atom document, Jakarta XML Binding objects can be extracted from Content using the `Content.getJAXBObject(Class<?> clazz)` method. Here is an example of an input atom document and extracting a Customer object from the content.

```
@Path("atom")
public static class AtomServer {
    @PUT
    @Path("entry")
    @Produces("application/atom+xml")
    public void putCustomer(Entry entry) {
        Content content = entry.getContent();
        Customer cust = content.getJAXBObject(Customer.class);
    }
}
```

Chapter 24. JSON Support via Jackson



This chapter documents the `resteasy-jackson2-provider` for Jackson 2. For Jackson 3 support, see the standalone RESTEasy Jackson Provider project (`dev.resteasy.providers:resteasy-jackson-provider`).

RESTEasy supports integration with the Jackson project. For more on Jackson 2, see https://github.com/FasterXML/jackson-databind/wiki. Besides having Jakarta XML Binding like APIs, it has a JavaBean based model, described at https://github.com/FasterXML/jackson-databind/wiki/Databind-annotations, which allows the marshalling of Java objects to and from JSON. RESTEasy integrates with the JavaBean model. While Jackson does come with its own Jakarta RESTful Web Services integration, RESTEasy expanded it a little, as described below.



As of RESTEasy 6.2.11.Final a default `com.fasterxml.jackson.databind.ObjectMapper` is created if there is **not** a `jakarta.ws.rs.ext.ContextResolver` for the `ObjectMapper`. This can be disabled with the configuration parameter `dev.resteasy.provider.jackson.disable.default.object.mapper` set to `true`.

24.1. Using Jackson 2 Outside of WildFly

If deploying RESTEasy outside WildFly, add the RESTEasy Jackson provider to the pom.xml:

```
<dependency>
   <groupId>org.jboss.resteasy</groupId>
   <artifactId>resteasy-jackson2-provider</artifactId>
   <version>8.0.0.Alpha1</version>
</dependency>
```

24.2. Jakarta XML Binding Annotations

In Jackson 2.13+ `com.fasterxml.jackson.module.jakarta.xmlbind.JakartaXmlBindAnnotationModule` must be registered in order to use XML binding annotations for Jackson to marshall/unmarshall them in JSON. To do this, `com.fasterxml.jackson.databind.ObjectMapper` must be provided via an implementation of `jakarta.ws.rs.ext.ContextResolver`.

An example `jakarta.ws.rs.ext.ContextResolver`:

```
@Provider
public class ObjectMapperProvider implements ContextResolver<ObjectMapper> {
    static final JsonMapper MAPPER = JsonMapper.builder()
            .addModule(new JakartaXmlBindAnnotationModule())
            .build();

    @Override
    public ObjectMapper getContext(final Class<?> type) {
```

```
        return MAPPER;
    }
}
```

24.3. Additional RESTEasy Specifics

RESTEasy added support for "application/*+json". Jackson supports "application/json" and "text/json" as valid media types. This allows the creation of json-based media types and still lets Jackson perform the marshalling. For example:

```
@Path("/customers")
public class MyService {

    @GET
    @Produces("application/vnd.customer+json")
    public Customer[] getCustomers() {}
}
```

24.4. JSONP Support

RESTEasy supports JSONP. It can be enabled by adding provider `org.jboss.resteasy.plugins.providers.jackson.Jackson2JsonpInterceptor` to the deployment. Jackson and JSONP can be used together. If the media type of the response is json and a callback query parameter is given, the response will be a JavaScript snippet with a method call of the method defined by the callback parameter. For example:

```
GET /resources/stuff?callback=processStuffResponse
```

will produce this response:

```
processStuffResponse(nomal JSON body)
```

This supports the default behavior of jQuery. To enable Jackson2JsonpInterceptor in WildFly, annotations must be imported from the `org.jboss.resteasy.resteasy-jackson2-provider` module. To do that a `jboss-deployment-structure.xml` file must be added to the application's WAR:

```
<jboss-deployment-structure>
  <deployment>
    <dependencies>
      <module name="org.jboss.resteasy.resteasy-jackson2-provider" annotations="true
"/>
    </dependencies>
  </deployment>
```

```
</jboss-deployment-structure>
```

The name of the callback parameter can be changed by setting the `callbackQueryParameter` property.

`Jackson2JsonpInterceptor` can wrap the response into a try-catch block:

```
try{processStuffResponse(normal JSON body)}catch(e){}
```

This feature can be enabled by setting the resteasy.jsonp.silent property to true



Because JSONP can be used in **Cross Site Scripting Inclusion (XSSI) attacks**, `Jackson2JsonpInterceptor` is disabled by default. Two steps are necessary to enable it:

1. As noted above, `Jackson2JsonpInterceptor` must be included in the deployment. For example, a service file `META-INF/services/jakarta.ws.rs.ext.Providers` with the line

   ```
   org.jboss.resteasy.plugins.providers.jackson.Jackson2JsonpInterceptor
   ```

   may be included on the classpath

2. Parameter "resteasy.jsonp.enable" must be set to "true". [See Configuration for more information about application configuration.]

24.5. Jackson JSON Decorator

If using the Jackson 2 provider, RESTEasy has provided a pretty-printing annotation similar with the one in Jakarta XML Binding provider: `org.jboss.resteasy.annotations.providers.jackson.Formatted`

Here is an example:

```
@GET
@Produces("application/json")
@Path("/formatted/{id}")
@Formatted
public Product getFormattedProduct() {
    return new Product(333, "robot");
}
```

As the example shown above, the @Formatted annotation will enable the underlying Jackson option `SerializationFeature.INDENT_OUTPUT`.

24.6. JSON Filter Support

Jackson2 provides annotation, JsonFilter. `@JsonFilter` is used to apply filtering during serialization/de-serialization for example which properties are to be used or not. Here is an example which defines mapping from "nameFilter" to filter instances and filter bean properties when serialized to json format:

```
@JsonFilter(value="nameFilter")
public class Jackson2Product {
    protected String name;
    protected int id;
    public Jackson2Product() {
    }
    public Jackson2Product(final int id, final String name) {
        this.id = id;
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
}
```

`@JsonFilter` annotates a resource class to filter out some property not to be serialized in the json response. To map the filter id and instance we need to create another Jackson class to add the id and filter instance map:

```
public class ObjectFilterModifier extends ObjectWriterModifier {
    public ObjectFilterModifier() {
    }
    @Override
    public ObjectWriter modify(EndpointConfigBase<?> endpoint,
            MultivaluedMap<String, Object> httpHeaders, Object valueToWrite,
            ObjectWriter w, JsonGenerator jg) throws IOException {

        FilterProvider filterProvider = new SimpleFilterProvider().addFilter(
                "nameFilter",
                SimpleBeanPropertyFilter.filterOutAllExcept("name"));
        return w.with(filterProvider);
```

```
    }
}
```

Here the method `modify()` takes care of filtering all properties except "name" property before a write. To make this work, the mapping information must be accessible to RESTEasy. This can be made available through a WriterInterceptor that uses Jackson's `ObjectWriterInjector`:

```
@Provider
public class JsonFilterWriteInterceptor implements WriterInterceptor{

    private ObjectFilterModifier modifier = new ObjectFilterModifier();
    @Override
    public void aroundWriteTo(WriterInterceptorContext context)
            throws IOException, WebApplicationException {
        //set a threadlocal modifier
        ObjectWriterInjector.set(modifier);
        context.proceed();
    }

}
```

Alternatively, Jackson's documentation suggest doing the same in a servlet filter; that however potentially leads to issues in RESTEasy. The ObjectFilterModifier is stored using a ThreadLocal object and there's no guarantee the same thread serving the servlet filter will be running the resource endpoint execution. For the servlet filter scenario, RESTEasy offers its own injector that relies on the current thread context classloader for carrying the specified modifier:

```
public class ObjectWriterModifierFilter implements Filter {
    private static ObjectFilterModifier modifier = new ObjectFilterModifier();

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
            FilterChain chain) throws IOException, ServletException {
        ResteasyObjectWriterInjector.set(Thread.currentThread().getContextClassLoader
(), modifier);
        chain.doFilter(request, response);
    }

    @Override
    public void destroy() {
    }

}
```

Chapter 25. Polymorphic Typing deserialization

Due to numerous CVEs for a specific kind of Polymorphic Deserialization (see details in FasterXML Jackson documentation), starting from Jackson 2.10 users have a means to allow only specified classes to be deserialized. RESTEasy enables this feature by default. It allows controlling the content of the whitelist of allowed classes/packages.

Property	**Description**	**resteasy.jackson.deserialization.whitelist.allowIfBaseType.prefix**
Method for appending a matcher that will allow all subtypes in cases where nominal base type's class name starts with specific prefix. "*" can be used for allowing any class.	resteasy.jackson.deserialization.whitelist.allowIfSubType.prefix	Method for appending a matcher that will allow specific subtype (regardless of declared base type) in cases where subclass name starts with specified prefix. "*" can be used for allowing any class.

Chapter 26. JSON Support via Jakarta EE JSON-P API

JSON-P is a Jakarta EE parsing API. RESTEasy provides an archive for it. It has support for `JsonObject`, `JsonArray`, and `JsonStructure` as request or response entities. It should not conflict with Jackson if it is on the classpath too. RESTEasy's provider is provided by default in WildFly. If using another server, use this maven dependency.

```
<dependency>
   <groupId>org.jboss.resteasy</groupId>
   <artifactId>resteasy-json-p-provider</artifactId>
   <version>8.0.0.Alpha1</version>
</dependency>
```

Chapter 27. Multipart Providers

RESTEasy has rich support for the "multipart/*" and "multipart/form-data" mime types. The multipart mime format is used to pass lists of content bodies. Multiple content bodies are embedded in one message. "multipart/form-data" is often found in web application HTML Form documents and is generally used to upload files. The form-data format is the same as other multipart formats, except that each inlined piece of content has a name associated with it.

RESTEasy provides a custom API for reading and writing multipart types as well as marshalling arbitrary List (for any multipart type) and Map (multipart/form-data only) objects

Classes `MultipartInput` and `MultipartOutput` provide read and write support for mime type "multipart/mixed" messages respectively. They provide for multiple part messages, in which one or more different sets of data are combined in a single body.

`MultipartRelatedInput` and `MultipartRelatedOutput` classes provide read and write support for mime type "multipart/related" messages. These are messages that contain multiple body parts that are inter-related.

`MultipartFormDataInput` and `MultipartFormDataOutput` classes provide read and write support for mine type "multipart/form-data". This type is used when returning a set of values as the the result of a user filling out a form or for uploading files.

27.1. Multipart/mixed

27.1.1. Writing multipart/mixed messages

`MultipartOutput` provides a set of addPart methods for registering message content and specifying special marshalling requirements. In all cases the addPart methods require an input parameter, Object and a MediaType that declares the mime type of the object. Sometimes there may be an object in which marshalling is sensitive to generic type metadata. In such cases, use an addPart method in which you declare the GenericType of the entity Object. Perhaps a file will be passed as content and it will require UTF-8 encoding. Setting input parameter, utf8Encode to `true` will indicate to RESTEasy to process the filename according to the character set and language encoding rules of `rfc5987`. This flag is only processed when mime type "multipart/form-data" is specified.

`MultipartOutput` automatically generates a unique message boundary identifier when it is created. Method setBoundary is provided to declare a different identifier.

```
public class MultipartOutput {
   public OutputPart addPart(Object entity, MediaType mediaType);
   public OutputPart addPart(Object entity, MediaType mediaType,
        String filename);
   public OutputPart addPart(Object entity, MediaType mediaType,
        String filename, boolean utf8Encode);
   public OutputPart addPart(Object entity, GenericType<?> type,
        MediaType mediaType);
   public OutputPart addPart(Object entity, GenericType<?> type,
```

```
        MediaType mediaType, String filename);
    public OutputPart addPart(Object entity, GenericType<?> type,
        MediaType mediaType, String filename, boolean utf8Encode);
    public OutputPart addPart(Object entity, Class<?> type, Type genericType,
        MediaType mediaType);
    public OutputPart addPart(Object entity, Class<?> type, Type genericType,
        MediaType mediaType, String filename);
    public OutputPart addPart(Object entity, Class<?> type, Type genericType,
        MediaType mediaType, String filename, boolean utf8Encode);
    public List<OutputPart> getParts();
    public String getBoundary();
    public void setBoundary(String boundary);
}
```

Each message part registered with `MultipartOutput` is represented by an `OutputPart` object. Class `MultipartOutput` generates an `OutputPart` object for each addPart method call.

```
public class OutputPart {
    public OutputPart(final Object entity, final Class<?> type,
        final Type genericType, final MediaType mediaType);
    public OutputPart(final Object entity, final Class<?> type,
        final Type genericType, final MediaType mediaType,
        final String filename);
    public OutputPart(final Object entity, final Class<?> type,
        final Type genericType, final MediaType mediaType,
        final String filename, final boolean utf8Encode);
    public MultivaluedMap<String, Object> getHeaders();
    public Object getEntity();
    public Class<?> getType();
    public Type getGenericType();
    public MediaType getMediaType();
    public String getFilename();
    public boolean isUtf8Encode();
}
```

27.1.2. Reading multipart/mixed messages

`MultipartInput` and `InputPart` are interface classes that provide access to multipart/mixed message data. RESTEasy provides an implementation of these classes. They perform the work to retrieve message data.

```
package org.jboss.resteasy.plugins.providers.multipart;

import java.util.List;

public interface MultipartInput {
    List<InputPart> getParts();
    String getPreamble();
```

```
    /**
     * Call this method to delete any temporary files created from unmarshalling
     * this multipart message
     * Otherwise they will be deleted on Garbage Collection or JVM exit.
     */
    void close();
}
```

```
package org.jboss.resteasy.plugins.providers.multipart;

import jakarta.ws.rs.core.GenericType;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.MultivaluedMap;
import java.io.IOException;
import java.lang.reflect.Type;

/**
 * Represents one part of a multipart message.
 */
public interface InputPart {
    /**
     * If no content-type header is sent in a multipart message part
     * "text/plain; charset=ISO-8859-1" is assumed.
     *
     * This can be overwritten by setting a different String value in
     * {@link org.jboss.resteasy.spi.HttpRequest#setAttribute(String, Object)}
     * with this ("resteasy.provider.multipart.inputpart.defaultContentType")
     * String as key. It should be done in a
     * {@link jakarta.ws.rs.container.ContainerRequestFilter}.
     */
    String DEFAULT_CONTENT_TYPE_PROPERTY =
     "resteasy.provider.multipart.inputpart.defaultContentType";

    /**
     * If there is a content-type header without a charset parameter,
     * charset=US-ASCII is assumed.
     *
     * This can be overwritten by setting a different String value in
     * {@link org.jboss.resteasy.spi.HttpRequest#setAttribute(String, Object)}
     * with this ("resteasy.provider.multipart.inputpart.defaultCharset")
     * String as key. It should be done in a
     * {@link jakarta.ws.rs.container.ContainerRequestFilter}.
     */
    String DEFAULT_CHARSET_PROPERTY =
     "resteasy.provider.multipart.inputpart.defaultCharset";

    /**
     * @return headers of this part
     */
    MultivaluedMap<String, String> getHeaders();
```

```
    String getBodyAsString() throws IOException;
    <T> T getBody(Class<T> type, Type genericType) throws IOException;
    <T> T getBody(GenericType<T> type) throws IOException;

    /**
     * @return "Content-Type" of this part
     */
    MediaType getMediaType();

    /**
     * @return true if the Content-Type was resolved from the message, false if
     *         it was resolved from the server default
     */
    boolean isContentTypeFromMessage();

    /**
     * Change the media type of the body part before you extract it.
     * Useful for specifying a charset.
     * @param mediaType media type
     */
    void setMediaType(MediaType mediaType);
}
```

27.1.3. Simple multipart/mixed message example

The following example shows how to read and write a simple multipart/mixed message.

The data to be transferred is a very simple class, Soup.

```
package org.jboss.resteasy.test.providers.multipart.resource;

import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlRootElement;
import jakarta.xml.bind.annotation.XmlElement;

@XmlRootElement(name = "soup")
@XmlAccessorType(XmlAccessType.FIELD)
public class Soup {
    @XmlElement
    private String id;

    public Soup(){}
    public Soup(final String id){this.id = id;}
    public String getId(){return id;}
}
```

This code fragment creates a multipart/mixed message passing Soup information using class, `MultipartOutput`.

```
MultipartOutput multipartOutput = new MultipartOutput();
multipartOutput.addPart(new Soup("Chicken Noodle"),
    MediaType.APPLICATION_XML_TYPE);
multipartOutput.addPart(new Soup("Vegetable"),
    MediaType.APPLICATION_XML_TYPE);
multipartOutput.addPart("Granny's Soups", MediaType.TEXT_PLAIN_TYPE);
```

This code fragment uses class `MultipartInput` to extract the Soup information provided by multipartOutput above.

```
public void process() {
    // MultipartInput, the entity returned in the client in a
    // Response object or the input value of an endpoint method parameter.
    for (InputPart inputPart : multipartInput.getParts()) {
        if (MediaType.APPLICATION_XML_TYPE.equals(inputPart.getMediaType())) {
          Soup c = inputPart.getBody(Soup.class, null);
          String name = c.getId();
        } else {
          String s = inputPart.getBody(String.class, null);
        }
    }
}
```

Returning a multipart/mixed message from an endpoint can be done in two ways. `MultipartOutput` can be returned as the method's return object or as an entity in a `Response` object.

```
@GET
@Path("soups/obj")
@Produces("multipart/mixed")
public MultipartOutput soupsObj() {
    return multipartOutput;
}

@GET
@Path("soups/resp")
@Produces("multipart/mixed")
public Response soupsResp() {
  return Response.ok(multipartOutput, MediaType.valueOf("multipart/mixed"))
                 .build();
}
```

There is no difference in the way a client retrieves the message from the endpoint. It is done as follows.

```
public void process() {
    try (Client client = ClientBuilder.newClient()) {
```

```
        WebTarget target = client.target(THE_URL);
        try (Response response = target.request().get()) {
            MultipartInput multipartInput = response.readEntity(MultipartInput.class);
            for (InputPart inputPart : multipartInput.getParts()) {
                if (MediaType.APPLICATION_XML_TYPE.equals(inputPart.getMediaType())) {
                  Soup c = inputPart.getBody(Soup.class, null);
                  String name = c.getId();
                } else {
                  String s = inputPart.getBody(String.class, null);
                }
            }
        }
    }
}
```

A client sends the message, multipartOutput, to an endpoint as an entity object in an HTTP method call in this code fragment.

```
Client client = ClientBuilder.newClient();
WebTarget target = client.target(SOME_URL + "/register/soups");
Entity<MultipartOutput> entity = Entity.entity(multipartOutput,
        new MediaType("multipart", "mixed"));
Response response = target.request().post(entity);
```

Here is the endpoint receiving the message and extracting the contents.

```
 @POST
 @Consumes("multipart/mixed")
 @Path("register/soups")
  public void registerSoups(MultipartInput multipartInput) throws IOException {

      for (InputPart inputPart : multipartInput.getParts()) {
         if (MediaType.APPLICATION_XML_TYPE.equals(inputPart.getMediaType())) {
                Soup c = inputPart.getBody(Soup.class, null);
                String name = c.getId();
         } else {
                String s = inputPart.getBody(String.class, null);
         }
      }
  }
```

27.1.4. Multipart/mixed message with GenericType example

This example shows how to read and write a multipart/mixed message whose content consists of a generic type, in this case a List<Soup>. The `MultipartOutput` and `MultipartIntput` methods that use `GenericType` parameters are used.

The multipart/mixed message is created using `MultipartOutput` as follows.

```
public void process() {
    MultipartOutput multipartOutput = new MultipartOutput();
    List<Soup> soupList = new ArrayList<Soup>();
    soupList.add(new Soup("Chicken Noodle"));
    soupList.add(new Soup("Vegetable"));
    multipartOutput.addPart(soupList, new GenericType<List<Soup>>(){},
           MediaType.APPLICATION_XML_TYPE );
    multipartOutput.addPart("Granny's Soups", MediaType.TEXT_PLAIN_TYPE);
}
```

The message data is extracted with `MultipartInput`. Note there are two `MultipartInput` getBody methods that can be used to retrieve data specifying `GenericType`. This code fragment uses the second one but shows the first one in comments.

```
<T> T getBody(Class<T> type, Type genericType) throws IOException;
<T> T getBody(GenericType<T> type) throws IOException;
```

```
public void process() {

   // MultipartInput multipartInput, the entity returned in the client in a
   // Response object or the input value of an endpoint method parameter.
   GenericType<List<Soup>> gType = new GenericType<List<Soup>>(){};

   for (InputPart inputPart : multipartInput.getParts()) {
      if (MediaType.APPLICATION_XML_TYPE.equals(inputPart.getMediaType())) {
         List<Soup> c = inputPart.getBody(gType);
      } else {
         String s = inputPart.getBody(String.class, null);;
      }
   }
}
```

27.1.5. java.util.List with multipart/mixed data example

When a set of message parts are uniform they do not need to be written using `MultipartOutput` or read with `MultipartInput`. They can be sent and received as a `List`. RESTEasy performs the necessary work to read and write the message data.

For this example the data to be transmitted is class, `ContextProvidersCustomer`

```
package org.jboss.resteasy.test.providers.multipart.resource;

import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlElement;
import jakarta.xml.bind.annotation.XmlRootElement;
```

```
@XmlRootElement(name = "customer")
@XmlAccessorType(XmlAccessType.FIELD)
public class ContextProvidersCustomer {
   @XmlElement
   private String name;

   public ContextProvidersCustomer() { }
   public ContextProvidersCustomer(final String name) {
      this.name = name;
  }
   public String getName() { return name;}
}
```

In this code fragment the client creates and sends a list of `ContextProvidersCustomers`.

```
List<ContextProvidersCustomer> customers =
    new ArrayList<ContextProvidersCustomer>();
customers.add(new ContextProvidersCustomer("Bill"));
customers.add(new ContextProvidersCustomer("Bob"));

Entity<ContextProvidersCustomer> entity = Entity.entity(customers,
new MediaType("multipart", "mixed"));

Client client = ClientBuilder.newClient();
WebTarget target = client.target(SOME_URL);
Response response = target.request().post(entity);
```

The endpoint receives the list, alters the contents and returns a new list.

```
@POST
@Consumes("multipart/mixed")
@Produces(MediaType.APPLICATION_XML)
@Path("post/list")
public List<ContextProvidersName> postList(
     List<ContextProvidersCustomer> customers) throws IOException {

  List<ContextProvidersName> names = new ArrayList<ContextProvidersName>();

  for (ContextProvidersCustomer customer : customers) {
     names.add(new ContextProvidersName("Hello " + customer.getName()));
  }
  return names;
}
```

The client receives the altered message data and processes it.

```
Response response = target.request().post(entity);
List<ContextProvidersCustomer> rtnList =
  response.readEntity(new GenericType<List<ContextProvidersCustomer>>(){});
```

27.2. Multipart/related

The Multipart/Related mime type is intended for compound objects consisting of several inter-related body parts, (RFC2387). There is a root or start part. All other parts are referenced from the root part. Each part has a unique id. The type and the id of the start part is presented in parameters in the message content-type header.

27.2.1. Writing multipart/related messages

RESTEasy provides class `MultipartRelatedOutput` to assist the user in specifying the required information and generating a properly formatted message. `MultipartRelatedOutput` is a subclass of `MultipartOutput`.

```
package org.jboss.resteasy.plugins.providers.multipart;

import jakarta.ws.rs.core.MediaType;

public class MultipartRelatedOutput extends MultipartOutput {
   private String startInfo;

   /**
    * The part used as the root.
    */
   public OutputPart getRootPart();

   /**
    * entity  object representing the part's body
    * mediaType Content-Type of the part
    * contentId  Content-ID to be used as identification for the current
    *            part, optional, if null one will be generated
    * contentTransferEncoding
    *            value used for the Content-Transfer-Encoding header
    *            field of the part. It's optional, if you don't want to set
    *            this pass null. Example values are: "7bit",
    *            "quoted-printable", "base64", "8bit", "binary"
    */
   public OutputPart addPart(Object entity, MediaType mediaType,
         String contentId, String contentTransferEncoding);

   /**
    * start-info parameter of the Content-Type.  An optional parameter.
    * As described in RFC2387, section 3.3. The Start-Info Parameter
    */
   public String getStartInfo();
```

```
}
```

27.2.2. Reading multipart/related messages

`MultipartRelatedInput` is an interface class that provides access to multipart/related message data. It is a subclass of `MultipartInput`. RESTEasy provides an implementation of this class. It performs the work to retrieve message data.

```
package org.jboss.resteasy.plugins.providers.multipart;

import jakarta.ws.rs.core.MediaType;

public class MultipartRelatedOutput extends MultipartOutput {
   private String startInfo;

   /**
    * The part used as the root.
    */
   public OutputPart getRootPart();

   /**
    * entity  object representing the part's body
    * mediaType Content-Type of the part
    * contentId  Content-ID to be used as identification for the current
    *            part, optional, if null one will be generated
    * contentTransferEncoding
    *            value used for the Content-Transfer-Encoding header
    *            field of the part. It's optional, if you don't want to set
    *            this pass null. Example values are: "7bit",
    *            "quoted-printable", "base64", "8bit", "binary"
    */
   public OutputPart addPart(Object entity, MediaType mediaType,
         String contentId, String contentTransferEncoding);

   /**
    * start-info parameter of the Content-Type.  An optional parameter.
    * As described in RFC2387, section 3.3. The Start-Info Parameter
    */
   public String getStartInfo();
}
```

27.2.3. Multipart/related message example

The client in this example creates a multipart/related message, POSTs it to the endpoint and processes the multipart/related message returned by the endpoint.

```
public void process() {
    MultipartRelatedOutput mRelatedOutput = new MultipartRelatedOutput();
```

```
    mRelatedOutput.setStartInfo("text/html");
    mRelatedOutput.addPart("Bill", new MediaType("image", "png"), "bill", "binary");
    mRelatedOutput.addPart("Bob", new MediaType("image", "png"), "bob", "binary");

    Entity<MultipartRelatedOutput> entity = Entity.entity(mRelatedOutput,
        new MediaType("multipart", "related"));

    try (Client client = ClientBuilder.newClient()) {
        WebTarget target = client.target(SOME_URL);
        try (Response response = target.request().post(entity)) {
            MultipartRelatedInput result = response.readEntity(
                  MultipartRelatedInput.class);
            Map<String, InputPart> map = result.getRelatedMap();
            Set<String> keys = map.keySet();
            boolean a = keys.contains("Bill");
            boolean b = keys.contains("Bob");
            for (InputPart inputPart : map.values()) {
                String alterName = inputPart.getBody(String.class, null);
            }
        }
    }
}
```

Here is the endpoint the client above is calling.

```
@POST
@Consumes("multipart/related")
@Produces("multipart/related")
@Path("post/related")
public MultipartRelatedOutput postRelated(MultipartRelatedInput input)
        throws IOException {

  MultipartRelatedOutput rtnMRelatedOutput = new MultipartRelatedOutput();
        rtnMRelatedOutput.setStartInfo("text/html");

  for (Iterator<InputPart> it = input.getParts().iterator(); it.hasNext(); ) {
      InputPart part = it.next();
      String name = part.getBody(String.class, null);
      rtnMRelatedOutput.addPart("Hello " + name,
                    new MediaType("image", "png"), name, null);
  }
  return rtnMRelatedOutput;
}
```

27.2.4. XML-binary Optimized Packaging (XOP)

RESTEasy supports XOP messages packaged as multipart/related messages (http://www.w3.org/TR/xop10/). A Jakarta XML Binding annotated POJO that also holds binary content can be transmitted using XOP. XOP allows the binary data to skip going through the XML serializer because binary data

can be serialized differently from text and this can result in faster transport time.

RESTEasy requires annotation `@XopWithMultipartRelated` to be placed on any endpoint method that returns an object that is to be processed with XOP and on any endpoint input parameter that is to be processed by XOP.

RESTEasy highly recommends, if you know the exact mime type of the POJO's binary data, tag the field with annotation @XmlMimeType. This annotation tells Jakarta XML Binding the mime type of the binary content, however this is not required in order to do XOP packaging.

27.2.5. @XopWithMultipartRelated return object example

The data to be transmitted is class, `ContextProvidersXop`. Note that field, `bytes`, is identified as an application/octet-stream mime type using annotation @XmlMimeType

```
package org.jboss.resteasy.test.providers.multipart.resource;

import jakarta.ws.rs.core.MediaType;
import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlMimeType;
import jakarta.xml.bind.annotation.XmlRootElement;

@XmlRootElement
@XmlAccessorType(XmlAccessType.FIELD)
public class ContextProvidersXop {

   @XmlMimeType(MediaType.APPLICATION_OCTET_STREAM)
   private byte[] bytes;

   public ContextProvidersXop(final byte[] bytes) {
      this.bytes = bytes;
   }

   public ContextProvidersXop() {}
   public byte[] getBytes() {return bytes;}
   public void setBytes(byte[] bytes) {this.bytes = bytes;}
}
```

The endpoint returns an instance of `ContextProvidersXop`. Note annotation `@XopWithMultipartRelated` is declared on the method because we want the return object to use XOP packaging.

```
@GET
@Path("get/xop")
@Produces("multipart/related")
@XopWithMultipartRelated
public ContextProvidersXop getXop() {
    return new ContextProvidersXop("goodbye world".getBytes());
```

```
}
```

The client retrieves the data as follows

```
public void process() {
    try (Client client = ClientBuilder.newClient()) {
        WebTarget target = client.target(SOME_URL);
        try (Response response = target.request().get()) {
            ContextProvidersXo entity = response.readEntity(ContextProvidersXop.class
);
        }
    }
}
```

27.2.6. @XopWithMultipartRelated input parameter example

Here is an endpoint that has an input parameter that is transmitted as an XOP package. Note the @XopWithMultipartRelated annotation on input parameter xop.

```
@POST
@Path("post/xop")
@Consumes("multipart/related")
public String postXop(@XopWithMultipartRelated ContextProvidersXop xop) {
    return new String(xop.getBytes());
}
```

This client is sending the data to the endpoint above.

```
ContextProvidersXop xop = new ContextProvidersXop("hello world".getBytes());
Entity<ContextProvidersXop> entity = Entity.entity(xop,
    new MediaType("multipart", "related"));

Client client = ClientBuilder.newClient();
WebTarget target = client.target(SOME_URL);
Response response = target.request().post(entity);
```

27.3. Multipart/form-data

The MultiPart/Form-Data mime type is used in sending form data (rfc2388). It can include data generated by user input, information that is typed, or included from files that the user has selected. "multipart/form-data" is often found in web application HTML Form documents and is generally used to upload files. The form-data format is the same as other multi-part formats, except that each inlined piece of content has a name associated with it.

27.3.1. Multipart/form-data with EntityParts

In Jakarta RESTful Web Services 3.1 the `jakarta.ws.rs.core.EntityPart` API was introduced. This can be used to read and write `multipart/form-data` with a standardized API.

The content of the `EntityPart` has two definable size limits. It also allows you to override the temporary directory where files will be written to. These can be set as system properties, servlet context properties or a MicroProfile Config compatible value.

Property Name	Description	Default Value	Example
`dev.resteasy.entity.memory.threshold`	The threshold to use for the amount of data to store in memory for entities.	5MB	512MB is equivalent to 512 Megabytes
`dev.resteasy.entity.file.buffer.size`	The size of the buffer used when writing the entity to a file. This can be disabled by setting a value of less than 1. If disabled, ensure a larger value is set in `-XX:MaxDirectMemorySize` JVM argument for large entities.	8192	-1 would disable the buffer, a value of 16384 processes the entities and writes them to the file 16384 bytes at a time.
`dev.resteasy.entity.file.threshold`	The threshold to use for the amount of data that can be stored in a file for entities. If the threshold is reached an `IllegalStateException` will be thrown. A value of -1 means no limit.	50MB	1GB is equivalent to 1 Gigabyte
`dev.resteasy.entity.tmpdir`	The temporary directory to use when the `dev.resteasy.entity.memory.threshold` was reached and the entity must be written to a file. Note that the directory must already exist. `IllegalStateException` will be thrown. A value of -1 means no limit.	The value of the `java.io.tmpdir` system property.	`/tmp/entity`

You can read, write and inject (with `@FormParam`) an `EntityPart` in the following forms;

- `EntityPart`
- `List<EntityPart>`
- `InputStream`

Example reading data:

```
public void process() {
    try (Client client = ClientBuilder.newClient()) {
        final ListEntityPart multipart = new ArrayList();
        multipart.add(
                EntityPart.withName("content")
                        .content("Example Content")
                        .mediaType(MediaType.TEXT_PLAIN_TYPE)
                        .build()
        );
        try (
                Response response = client.target(INSTANCE.configuration()
.baseUriBuilder().path("test/injected"))
                        .request(MediaType.MULTIPART_FORM_DATA_TYPE)
                        .post(Entity.entity(new GenericEntity(multipart) {
                        }, MediaType.MULTIPART_FORM_DATA))
        ) {
            Assert.assertEquals(Response.Status.OK, response.getStatusInfo());
            final ListEntityPart entityParts = response.readEntity(new GenericType() {
            });
        }
    }
}
```

Example receiving and writing data.

```
@POST
@Consumes(MediaType.MULTIPART_FORM_DATA)
@Produces(MediaType.MULTIPART_FORM_DATA)
@Path("/injected")
public ListEntityPart injected(@FormParam("content") final String string,
                        @FormParam("content") final EntityPart entityPart,
                        @FormParam("content") final InputStream in) throws
IOException {
    final ListEntityPart multipart = new ArrayList();
    multipart.add(
            EntityPart.withName("received-entity-part")
                    .content(entityPart.getContent(String.class))
                    .mediaType(entityPart.getMediaType())
                    .fileName(entityPart.getFileName().orElse(null))
                    .build()
    );
    multipart.add(
```

```
            EntityPart.withName("received-input-stream")
                    .content(MultipartEntityPartProviderTest.toString(in).getBytes
(StandardCharsets.UTF_8))
                    .mediaType(MediaType.APPLICATION_OCTET_STREAM_TYPE)
                    .build()
    );
    multipart.add(
            EntityPart.withName("received-string")
                    .content(string)
                    .mediaType(MediaType.TEXT_PLAIN_TYPE)
                    .build()
    );
    return multipart;
}
```

27.3.2. Writing multipart/form-data messages

Form data consists of key/value pairs. RESTEasy provides class `MultipartFormDataOutput` to assist the user in specifying the required information and generating a properly formatted message. It is a subclass of `MultipartOutput`. And as with multipart/mixed data sometimes there may be marshalling which is sensitive to generic type metadata, in those cases use the methods containing input parameter GenericType.

```
package org.jboss.resteasy.plugins.providers.multipart;

public class MultipartFormDataOutput extends MultipartOutput {
    public OutputPart addFormData(String key, Object entity,
            MediaType mediaType)
    public OutputPart addFormData(String key, Object entity, GenericType type,
            MediaType mediaType)
    public OutputPart addFormData(String key, Object entity, Class type,
            Type genericType, MediaType mediaType)
    public Map<String, OutputPart> getFormData()
    public Map<String, List<OutputPart>> getFormDataMap()
}
```

27.3.3. Reading multipart/form-data messages

`MultipartFormDataInput` is an interface class that provides access to multipart/form-data message data. It is a subclass of `MultipartInput`. RESTEasy provides an implementation of this class. It performs the work to retrieve message data.

```
package org.jboss.resteasy.plugins.providers.multipart;

import java.io.IOException;
import java.lang.reflect.Type;
import java.util.List;
import java.util.Map;
```

```
import jakarta.ws.rs.core.GenericType;

public interface MultipartFormDataInput extends MultipartInput {
   /**
    * @return A parameter map containing a list of values per name.
    */
   Map<String, List<InputPart>> getFormDataMap();
   <T> T getFormDataPart(String key, Class<T> rawType, Type genericType)
         throws IOException;
   <T> T getFormDataPart(String key, GenericType<T> type) throws IOException;
}
```

27.3.4. Simple multipart/form-data message example

The following example show how to read and write a simple multipart/form-data message.

The multipart/mixed message is created on the clientside using the `MultipartFormDataOutput` object. One piece of form data to be transfered is a very simple class, `ContextProvidersName`.

```
package org.jboss.resteasy.test.providers.multipart.resource;

import jakarta.xml.bind.annotation.XmlAccessType;
import jakarta.xml.bind.annotation.XmlAccessorType;
import jakarta.xml.bind.annotation.XmlElement;
import jakarta.xml.bind.annotation.XmlRootElement;

@XmlRootElement(name = "name")
@XmlAccessorType(XmlAccessType.FIELD)
public class ContextProvidersName {
   @XmlElement
   private String name;

   public ContextProvidersName() {}
   public ContextProvidersName(final String name) {this.name = name;}
   public String getName() {return name;}
}
```

The client creates and sends the message as follows:

```
public void process() {
    MultipartFormDataOutput output = new MultipartFormDataOutput();
    output.addFormData("bill", new ContextProvidersCustomer("Bill"),
     MediaType.APPLICATION_XML_TYPE);
    output.addFormData("bob", "Bob", MediaType.TEXT_PLAIN_TYPE);

    Entity<MultipartFormDataOutput> entity = Entity.entity(output,
      new MediaType("multipart", "related"));
```

```
    try (Client client = ClientBuilder.newClient()) {
        WebTarget target = client.target(SOME_URL);
        try (Response response = target.request().post(entity)) {
        }
    }
}
```

The endpoint receives the message and processes it.

```
@POST
@Consumes("multipart/form-data")
@Produces(MediaType.APPLICATION_XML)
@Path("post/form")
public Response postForm(MultipartFormDataInput input)
     throws IOException {

  Map<String, List<InputPart>> map = input.getFormDataMap();
  List<ContextProvidersName> names = new ArrayList<ContextProvidersName>();

  for (Iterator<String> it = map.keySet().iterator(); it.hasNext(); ) {
     String key = it.next();
     InputPart inputPart = map.get(key).iterator().next();
     if (MediaType.APPLICATION_XML_TYPE.equals(inputPart.getMediaType())) {
        names.add(new ContextProvidersName(inputPart.getBody(
              ContextProvidersCustomer.class, null).getName()));
     } else {
        names.add(new ContextProvidersName(inputPart.getBody(
              String.class, null)));
     }
  }
  return Response.ok().build();
}
```

27.3.5. java.util.Map with multipart/form-data

When the data of a multipart/form-data message is uniform it does not need to be written in a `MultipartFormDataOutput` object. It can be sent and received as a `java.util.Map` object. RESTEasy performs the necessary work to read and write the message data, however the Map object must declare the type it is unmarshalling via the generic parameters in the Map type declaration.

Here is an example of a client creating and sending a multipart/form-data message.

```
public void process() {
    Map<String, ContextProvidersCustomer> customers =
          new HashMap<String, ContextProvidersCustomer>();
    customers.put("bill", new ContextProvidersCustomer("Bill"));
    customers.put("bob", new ContextProvidersCustomer("Bob"));
```

```
    Entity<Map<String, ContextProvidersCustomer>> entity =
    Entity.entity(customers, new MediaType("multipart", "form-data"));

    try (Client client = ClientBuilder.newClient()) {
        WebTarget target = client.target(SOME_URL);
        try (Response response = target.request().post(entity)) {
        }
    }
}
```

This is the endpoint the client above is calling. It receives the message and processes it.

```
@POST
@Consumes("multipart/form-data")
@Produces(MediaType.APPLICATION_XML)
@Path("post/map")
public Response postMap(Map<String, ContextProvidersCustomer> customers)
        throws IOException {

  List<ContextProvidersName> names = new ArrayList<ContextProvidersName>();
  for (Iterator<String> it = customers.keySet().iterator(); it.hasNext(); ) {
      String key = it.next();
      ContextProvidersCustomer customer = customers.get(key);
      names.add(new ContextProvidersName(key + ":" + customer.getName()));
  }
  return Response.ok().build();
}
```

27.3.6. Multipart/form-data java.util.Map as method return type

A `java.util.Map` object representing a multipart/form-data message can be returned from an endpoint as long as the message data is uniform, however the endpoint method MUST be annotated with @PartType which declares the media type of the Map entries and the Map object must declare the type it is unmarshalling via the generic parameters in the Map type declaration. RESTEasy requires this information so it can generate the message properly.

Here is an example of an endpoint returning a Map of `ContextProvidersCustomer` to the client.

```
@GET
@Produces("multipart/form-data")
@PartType("application/xml")
@Path("get/map")
public Map<String, ContextProvidersCustomer> getMap() {

  Map<String, ContextProvidersCustomer> map =
      new HashMap<String, ContextProvidersCustomer>();
  map.put("bill", new ContextProvidersCustomer("Bill"));
```

```
  map.put("bob", new ContextProvidersCustomer("Bob"));
  return map;
}
```

The client would retrieve the data as follows.

```
public void process() {
    try (Client client = ClientBuilder.newClient()) {
        WebTarget target = client.target(SOME_URL);
        try (Response response = target.request().get()) {
            MultipartFormDataInput entity = response.readEntity(
MultipartFormDataInput.class);
        }
    }

    ContextProvidersCustomer bill = entity.getFormDataPart("bill",
        ContextProvidersCustomer.class, null);
    ContextProvidersCustomer bob = entity.getFormDataPart("bob",
        ContextProvidersCustomer.class, null);
}
```

27.3.7. @MultipartForm and POJOs

If you have exact knowledge of the multipart/form-data packets, they can be mapped to and from a POJO class using the annotation `@org.jboss.resteasy.annotations.providers.multipart.MultipartForm` and the Jakarta RESTful Web Services `@FormParam` annotation. Simply define a POJO with at least a default constructor and annotate its fields and/or properties with `@FormParams`. These `@FormParams` must also be annotated with `@org.jboss.resteasy.annotations.providers.multipart.PartType` if doing output. For example:

```
public class CustomerProblemForm {
    @FormParam("customer")
    @PartType("application/xml")
    private Customer customer;

    @FormParam("problem")
    @PartType("text/plain")
    private String problem;

    public Customer getCustomer() { return customer; }
    public void setCustomer(Customer cust) { this.customer = cust; }
    public String getProblem() { return problem; }
    public void setProblem(String problem) { this.problem = problem; }
}
```

After defining the POJO class it can be used to represent multipart/form-data. Here's an example of sending a `CustomerProblemForm` using the RESTEasy client framework:

```
@Path("portal")
public class CustomerPortal {

   @Path("issues/{id}")
   @Consumes("multipart/form-data")
   @PUT
   public void putProblem(@MultipartForm CustomerProblemForm form,
                          @PathParam("id") int id) {
      CustomerPortal portal = ProxyFactory.create(
            CustomerPortal.class, "http://example.com");
      portal.putProblem(form, 333);
   }
}
```

Note that the `@MultipartForm` annotation was used to tell RESTEasy that the object has a `@FormParam` and that it should be marshalled from that. The same object can be used to receive multipart data. Here is an example of the server side counterpart of our customer portal.

```
@Path("portal")
public class CustomerPortalServer {

    @Path("issues/{id}")
    @Consumes("multipart/form-data")
    @PUT
    public void putIssue(@MultipartForm CustoemrProblemForm form,
                         @PathParam("id") int id) {
    }
}
```

In addition to the XML data format, JSON formatted data can be used to represent POJO classes. To achieve this, plug in a JSON provider into your project. For example, add the RESTEasy Jackson2 Provider into the project's dependency scope:

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-jackson2-provider</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

Now an ordinary POJO class can be written, which Jackson2 will automatically serialize/deserialize into JSON format:

```
public class JsonUser {
   private String name;

   public JsonUser() {}
```

```
    public JsonUser(final String name) { this.name = name; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

The resource class can be written like this:

```
import org.jboss.resteasy.annotations.providers.multipart.MultipartForm;
import org.jboss.resteasy.annotations.providers.multipart.PartType;

import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.FormParam;
import jakarta.ws.rs.PUT;
import jakarta.ws.rs.Path;

@Path("/")
public class JsonFormResource {

    public JsonFormResource() {
    }

    public static class Form {

    @FormParam("user")
    @PartType("application/json")
    private JsonUser user;

    public Form() {
    }

    public Form(final JsonUser user) {
    this.user = user;
    }

    public JsonUser getUser() {
      return user;
    }
}

    @PUT
    @Path("form/class")
    @Consumes("multipart/form-data")
    public String putMultipartForm(@MultipartForm Form form) {
         return form.getUser().getName();
    }
}
```

In the code shown above, it can be seen the PartType of JsonUser is marked as "application/json", and it's included in the "@MultipartForm Form" class instance.

To send the request to the resource method, send the JSON formatted data that corresponds with the JsonUser class. The easiest way to do this is to use a proxy class that has the same definition of the resource class. Here is the sample code of the proxy class that is corresponding with the JsonFormResource class:

```
import org.jboss.resteasy.annotations.providers.multipart.MultipartForm;

import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.PUT;
import jakarta.ws.rs.Path;

@Path("/")
public interface JsonForm {

    @PUT
    @Path("form/class")
    @Consumes("multipart/form-data")
    String putMultipartForm(@MultipartForm JsonFormResource.Form form);
}
```

And then use the proxy class above to send the request to the resource method correctly. Here is the sample code:

```
ResteasyClient client = (ResteasyClient)ClientBuilder.newClient();
JsonForm proxy = client.target("your_request_url_address")
                       .proxy(JsonForm.class);
String name = proxy.putMultipartForm(new JsonFormResource
                   .Form(new JsonUser("bill")));
```

If the client side has the Jackson2 provider included, the request will be marshaled correctly. The JsonUser data will be converted into JSON format and sent to the server side. Hand-crafted JSON data can be use in the request and sent to the server side. It must be made sure the request data is in the correct form.

27.4. Note about multipart parsing and working with other frameworks

There are many frameworks doing multipart parsing automatically with the help of filters and interceptors, like `org.jboss.seam.web.MultipartFilter` in Seam and `org.springframework.web.multipart.MultipartResolver` in Spring, however these incoming multipart request stream can be parsed only once. RESTEasy users working with multipart should make sure that nothing parses the stream before RESTEasy gets it.

27.5. Overwriting the default fallback content type for multipart messages

By default if no Content-Type header is present in a part `"text/plain; charset=us-ascii"` is used as the fallback. This is the value defined by the MIME RFC, however some web clients, and web browsers do not send Content-Type headers for all fields in a multipart/form-data request. They send them only for the file parts. This can cause character encoding and unmarshalling errors on the server side. To correct this there is an option to define another, non-rfc compliant fallback value. This can be done dynamically per request with the filter facility of Jakarta RESTful Web Services 3.0. In the following example we will set `"*/*; charset=UTF-8"` as the new default fallback:

```
import org.jboss.resteasy.plugins.providers.multipart.InputPart;

@Provider
public class InputPartDefaultCharsetOverwriteContentTypeCharsetUTF8
   implements ContainerRequestFilter {

   @Override
   public void filter(ContainerRequestContext requestContext) throws IOException {
      requestContext.setProperty(InputPart.DEFAULT_CONTENT_TYPE_PROPERTY, "*/*;
charset=UTF-8");
   }
}
```

27.6. Overwriting the content type for multipart messages

Using attribute, InputPart.DEFAULT_CONTENT_TYPE_PROPERTY and a filter enables the setting of a default Content-Type, It is also possible to override the Content-Type by setting a different media type with method `InputPart.setMediaType()`. Here is an example:

```
@POST
@Path("query")
@Consumes(MediaType.MULTIPART_FORM_DATA)
@Produces(MediaType.TEXT_PLAIN)
public Response setMediaType(MultipartInput input) throws IOException {
    List<InputPart> parts = input.getParts();
    InputPart part = parts.get(0);
    part.setMediaType(MediaType.valueOf("application/foo+xml"));
    String s = part.getBody(String.class, null);
}
```

27.7. Overwriting the default fallback charset for multipart messages

Sometimes, a part may have a Content-Type header with no charset parameter. If the `InputPart.DEFAULT_CONTENT_TYPE_PROPERTY` property is set and the value has a charset parameter, that value will be appended to an existing Content-Type header that has no charset parameter. It is also possible to specify a default charset using the constant `InputPart.DEFAULT_CHARSET_PROPERTY` (actual value "resteasy.provider.multipart.inputpart.defaultCharset"):

```
import org.jboss.resteasy.plugins.providers.multipart.InputPart;

@Provider
public class InputPartDefaultCharsetOverwriteContentTypeCharsetUTF8
   implements ContainerRequestFilter {

   @Override
   public void filter(ContainerRequestContext requestContext) throws IOException {
      requestContext.setProperty(InputPart.DEFAULT_CHARSET_PROPERTY, "UTF-8");
   }
}
```

If both `InputPart.DEFAULT_CONTENT_TYPE_PROPERTY` and `InputPart.DEFAULT_CHARSET_PROPERTY` are set, then the value of `InputPart.DEFAULT_CHARSET_PROPERTY` will override any charset in the value of `InputPart.DEFAULT_CONTENT_TYPE_PROPERTY`.

Chapter 28. Jakarta RESTful Web Services 2.1 Additions

Jakarta RESTful Web Services 2.1 adds more asynchronous processing support in both the Client and the Server API. The specification adds a Reactive programming style to the Client side and Server-Sent Events (SSE) protocol support to both client and server.

28.1. `CompletionStage` support

The specification adds support for declaring asynchronous resource methods by returning a `CompletionStage` instead of using the `@Suspended` annotation.



RESTEasy supports more reactive types than the specification.

Chapter 29. Reactive Clients API

The specification defines a new type of invoker named `RxInvoker`, and a default implementation of this type named `CompletionStageRxInvoker`. `CompletionStageRxInvoker` implements Java 8's interface `CompletionStage`. This interface declares a large number of methods dedicated to managing asynchronous computations.

There is also a new rx method which is used in a similar manner to async.

29.1. Server-Sent Events (SSE)

SSE is part of HTML standard, currently supported by many browsers. It is a server push technology, which provides a way to establish a one-way channel to continuously send data to clients. SSE events are pushed to the client via a long-running HTTP connection. In case of lost connection, clients can retrieve missed events by setting a "Last-Event-ID" HTTP header in a new request.

SSE stream has text/event-stream media type and contains multiple SSE events. SSE event is a data structure encoded with UTF-8 and contains fields and comment. The field can be event, data, id, retry and other kinds of field will be ignored.

From Jakarta RESTful Web Services 2.1, Server-sent Events APIs are introduced to support sending, receiving and broadcasting SSE events.

29.1.1. SSE Server

As shown in the following example, a SSE resource method has the text/event-stream produce media type and an injected context parameter SseEventSink. The injected SseEventSink is the connected SSE stream where events can be sent. Another injected context Sse is an entry point for creating and broadcasting SSE events. Here is an example to demonstrate how to send SSE events every 200ms and close the stream after a "done" event.

```
@Resource
private ManagedExecutorService executor;

@GET
@Path("domains/{id}")
@Produces(MediaType.SERVER_SENT_EVENTS)
public void startDomain(@PathParam("id") final String id, @Context SseEventSink sink,
@Context Sse sse) {
  executor.execute(() -> {
        try {
           sink.send(sse.newEventBuilder().name("domain-progress")
                 .data(String.class, "starting domain " + id + " ...").build());
           Thread.sleep(200);
           sink.send(sse.newEvent("domain-progress", "50%"));
           Thread.sleep(200);
           sink.send(sse.newEvent("domain-progress", "60%"));
```

```
            Thread.sleep(200);
            sink.send(sse.newEvent("domain-progress", "70%"));
            Thread.sleep(200);
            sink.send(sse.newEvent("domain-progress", "99%"));
            Thread.sleep(200);
            sink.send(sse.newEvent("domain-progress", "Done.")).thenAccept((Object obj)
-> {
               sink.close();
            });
         } catch (Exception e) {
            logger.error(e.getMessage(), e);
         }
   });
}
```



RESTEasy supports sending SSE events via reactive types.

29.1.2. SSE Broadcasting

With SseBroadcaster, SSE events can broadcast to multiple clients simultaneously. It will iterate over all registered SseEventSinks and send events to all requested SSE Stream. An application can create a SseBroadcaster from an injected context Sse. The broadcast method on a SseBroadcaster is used to send SSE events to all registered clients. The following code snippet is an example on how to create SseBroadcaster, subscribe and broadcast events to all subscribed consumers.

```
@GET
@Path("/subscribe")
@Produces(MediaType.SERVER_SENT_EVENTS)
public void subscribe(@Context SseEventSink sink) throws IOException {
   if (sink == null) {
      throw new IllegalStateException("No client connected.");
   }
   if (sseBroadcaster == null) {
      sseBroadcaster = sse.newBroadcaster();
   }
   sseBroadcaster.register(sink);
}

@POST
@Path("/broadcast")
public void broadcast(String message) throws IOException {
   if (sseBroadcaster == null) {
      sseBroadcaster = sse.newBroadcaster();
   }
   sseBroadcaster.broadcast(sse.newEvent(message));

}
```

29.1.3. SSE Client

SseEventSource is the entry point to read and process incoming SSE events. A SseEventSource instance can be initialized with a WebTarget. Once SseEventSource is created and connected to a server, registered event consumer will be invoked when an inbound event arrives. In case of errors, an exception will be passed to a registered consumer so that it can be processed. SseEventSource can automatically reconnect the server and continuously receive pushed events after the connection has been lost. SseEventSource can send lastEventId to the server by default when it is reconnected, and server may use this id to replay all missed events. But reply event is really upon on SSE resource method implementation. If the server responds HTTP 503 with a RETRY_AFTER header, SseEventSource will automatically schedule a reconnect task with this RETRY_AFTER value. The following code snippet is to create a SseEventSource and print the inbound event data value and error if it happens.

```
public void printEvent() throws Exception {
  WebTarget target = client.target("http://localhost:8080/service/server-sent-events"
);
  try (SseEventSource eventSource = SseEventSource.target(target).build()) {
     eventSource.register(event -> System.out.println(event.readData(String.class)),
            ex -> ex.printStackTrace());
     eventSource.open();
  }
}
```

29.2. Java API for JSON Binding

RESTEasy supports both JSON-B and JSON-P. In accordance with the specification, entity providers for JSON-B take precedence over those for JSON-P for all types except JsonValue and its sub-types.

The support for JSON-B is provided by the `JsonBindingProvider` from `resteasy-json-binding-provider` module. To satisfy Jakarta RESTful Web Services" 2.1 requirements, JsonBindingProvider takes precedence over the other providers for dealing with JSON payloads, in particular the Jackson one. The JSON outputs (for the same input) from Jackson and JSON-B reference implementation can be slightly different. As a consequence, in order to allow retaining backward compatibility, RESTEasy offers a `resteasy.preferJacksonOverJsonB` context property that can be set to `true` to disable JsonBindingProvider for the current deployment.

WildFly 14 supports specifying the default value for the `resteasy.preferJacksonOverJsonB` context property by setting a system property with the same name. Moreover, if no value is set for the context and system properties, it scans Jakarta RESTful Web Services deployments for Jackson annotations and sets the property to `true` if any of those annotations is found.

29.3. JSON Patch and JSON Merge Patch

RESTEasy supports applying partial modifications to target resources with JSON Patch/JSON Merge Patch. Instead of sending a json request which represents the whole modified resource with an HTTP PUT method, the json request only contains the modified part with an HTTP PATCH method to

do the same job.

JSON Patch request has an array of json objects and each JSON object gives the operation to execute against the target resource. Here is an example to modify the target Student resource which has these fields and values: `\{"firstName":"Alice","id":1,"school":"MiddleWood School"}`:

```
PATCH /StudentPatchTest/students/1 HTTP/1.1
Content-Type: application/json-patch+json
Content-Length: 184
Host: localhost:8090
Connection: Keep-Alive

[{"op":"copy","from":"/firstName","path":"/lastName"},
 {"op":"replace","path":"/firstName","value":"John"},
 {"op":"remove","path":"/school"},
 {"op":"add","path":"/gender","value":"male"}]
```

This JSON Patch request will copy the firstName to lastName field , then change the firstName value to "John". The next operation will remove the school value and add male gender to this "id=1" student resource. After this JSON Path is applied. The target resource will be modified to: `\{"firstName":"John","gender":"male","id":1,"lastName":"Taylor"}`. The operation keyword here can be "add", "remove", "replace", "move", "copy", or "test". The "path" value must be a JSON Pointer value that points to the part to apply this JSON Patch.

Unlike using the operation keyword to patch the target resource, JSON Merge Patch request directly sends the expected json change. RestEasy merges this change into the target resource which is identified by the request URI. Like the JSON Merge Patch request below, it removes the "school" value and changes the "firstName" to "Green".

```
PATCH /StudentPatchTest/students/1 HTTP/1.1
Content-Type: application/merge-patch+json
Content-Length: 34
Host: localhost:8090
Connection: Keep-Alive
{"firstName":"Green","school":null}
```

Enabling JSON Patch or JSON Merge Patch only requires correctly annotating the resource method with these mediaTypes. `@Consumes(MediaType.APPLICATION_JSON_PATCH_JSON)` enables JSON Patch:

```
@GET
@Path("/{id}")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Student getStudent(@PathParam("id") long id) {
    Student student = studentsMap.get(id);
    if (student == null) {
        throw new NotFoundException();
```

```
    }
    return student;
}
@PATCH
@Path("/{id}")
@Consumes(MediaType.APPLICATION_JSON_PATCH_JSON)
@Produces(MediaType.APPLICATION_JSON)
    public Student patchStudent(@PathParam("id") long id, Student student) {
    if (studentsMap.get(id) == null) {
        throw new NotFoundException();
    }
    studentsMap.put(id, student);
    return student;
}
@PATCH
@Path("/{id}")
@Consumes("application/merge-patch+json")
@Produces(MediaType.APPLICATION_JSON)
public Student mergePatchStudent(@PathParam("id") long id, Student student) {
    if (studentsMap.get(id) == null) {
        throw new NotFoundException();
    }
    studentsMap.put(id, student);
    return student;
}
```



Before creating a JSON Patch or JSON Merge Patch resource method, there must be a GET method to get the target resource. In the above code example, the first resource method is responsible for getting the target resource to apply the patch.

It requires the patch filter to enable JSON Patch or JSON Merge Patch. The RESTEasy `PatchMethodFilter` is enabled by default. This filter can be disabled by setting "resteasy.patchfilter.disabled" to true as described in Configuration switches.

The client side needs to create these json objects and send them with a http PATCH method.

```
//send JSON Patch request
WebTarget patchTarget = client.target(
"http://localhost:8090/StudentPatchTest/students/1"));
JsonArray patchRequest = Json.createArrayBuilder()
.add(Json.createObjectBuilder().add("op", "copy").add("from", "/firstName").add("path
", "/lastName").build())
.build();
patchTarget.request().build(HttpMethod.PATCH, Entity.entity(patchRequest, MediaType
.APPLICATION_JSON_PATCH_JSON)).invoke();
//send JSON Merge Patch request
WebTarget patchTarget = client.target(
"http://localhost:8090/StudentPatchTest/students/1");
```

```
JsonObject object = Json.createObjectBuilder().add("lastName", "Green").addNull(
"school").build();
Response result = patchTarget.request().build(HttpMethod.PATCH, Entity.entity(object,
"application/merge-patch+json")).invoke();
```

Chapter 30. String marshalling for String based @*Param

30.1. Simple conversion

Parameters and properties annotated with `@CookieParam`, `@HeaderParam`, `@MatrixParam`, `@PathParam`, or `@QueryParam` are represented as strings in a raw HTTP request. The specification says that any of these injected parameters can be converted to an object if the object's class has a `valueOf(String)` static method or a constructor that takes one `String` parameter. In the following, for example,

```
public static class Customer {
   private String name;

   public Customer(String name) {
      this.name = name;
   }

   public String getName() {
      return name;
   }
}

@Path("test")
public static class TestResource {

   @GET
   @Path("")
   public Response test(@QueryParam("cust") Customer cust) {
      return Response.ok(cust.getName()).build();
   }
}

@Test
public void testQuery() throws Exception {
   Invocation.Builder request = ClientBuilder.newClient().target(
"http://localhost:8081/test?cust=Bill").request();
   Response response = request.get();
}
```

the query "?cust=Bill" will be transformed automatically to an instance of `Customer` with name == "Bill".

30.2. ParamConverter

What if you have a class where `valueOf()` or this string constructor don't exist or is inappropriate for an HTTP request? Jakarta RESTful Web Services has the

`jakarta.ws.rs.ext.ParamConverterProvider` to help in this situation.

A `ParamConverterProvider` is a provider defined as follows:

```
public interface ParamConverterProvider {

   <T> ParamConverter<T> getConverter(Class<T> rawType, Type genericType, Annotation[]
annotations);
}
```

where a `ParamConverter` is defined:

```
public interface ParamConverter<T> {
   T fromString(String value);
   String toString(T value);
}
```

For example, consider `DateParamConverterProvider` and `DateParamConverter`:

```
@Provider
public class DateParamConverterProvider implements ParamConverterProvider {

   @SuppressWarnings("unchecked")
   @Override
   public T ParamConverterT getConverter(ClassT rawType, Type genericType,
Annotation[] annotations) {
      if (rawType.isAssignableFrom(Date.class)) {
         return (ParamConverterT) new DateParamConverter();
      }
      return null;
   }
}

public class DateParamConverter implements ParamConverterDate {

   public static final String DATE_PATTERN = "yyyyMMdd";

   @Override
   public Date fromString(String param) {
      try {
         return new SimpleDateFormat(DATE_PATTERN).parse(param.trim());
      } catch (ParseException e) {
         throw new BadRequestException(e);
      }
   }

   @Override
   public String toString(Date date) {
```

```
      return new SimpleDateFormat(DATE_PATTERN).format(date);
   }
}
```

Sending a `Date` in the form of a query, e.g., "?date=20161217" will cause the string "20161217" to be converted to a `Date` on the server.

30.3. StringParameterUnmarshaller

In addition to the Jakarta RESTful Web Services `jakarta.ws.rs.ext.ParamConverterProvider`, RESTEasy also has its own `org.jboss.resteasy.StringParameterUnmarshaller`, defined

```
public interface StringParameterUnmarshaller<T> {
   void setAnnotations(Annotation[] annotations);

   T fromString(String str);
}
```

It is similar to `jakarta.ws.rs.ext.ParamConverter` except that

- it converts only from String's;
- it is configured with the annotations on the injected parameter, which allows for fine-grained control over the injection; and
- it is bound to a given parameter by an annotation that is annotated with the meta-annotation `org.jboss.resteasy.annotations.StringParameterUnmarshallerBinder:`

```
@Target({ElementType.ANNOTATION_TYPE})
@Retention(RetentionPolicy.RUNTIME)
public @interface StringParameterUnmarshallerBinder {
   Class<? extends StringParameterUnmarshaller> value();
}
```

For example,

```
@Retention(RetentionPolicy.RUNTIME)
@StringParameterUnmarshallerBinder(TestDateFormatter.class)
public @interface TestDateFormat {
    String value();
}

public static class TestDateFormatter implements StringParameterUnmarshallerDate {
  private SimpleDateFormat formatter;

  public void setAnnotations(Annotation[] annotations) {
     TestDateFormat format = FindAnnotation.findAnnotation(annotations,
TestDateFormat.class);
```

```
      formatter = new SimpleDateFormat(format.value());
   }

   public Date fromString(String str) {
      try {
         return formatter.parse(str);
      } catch (ParseException e) {
         throw new RuntimeException(e);
      }
   }
}

@Path("/")
public static class TestResource {

   @GET
   @Produces("text/plain")
   @Path("/datetest/{date}")
   public String get(@PathParam("date") @TestDateFormat("MM-dd-yyyy") Date date) {
      Calendar c = Calendar.getInstance();
      c.setTime(date);
      return date.toString();
   }
}
```

Note that the annotation `@StringParameterUnmarshallerBinder` on the annotation `@TestDateFormat` binds the formatter `TestDateFormatter` to a parameter annotated with `@TestDateFormat`. In this example, `TestDateFormatter` is used to format the `Date` parameter. Note also that the parameter "MM-dd-yyyy" to `@TestDateFormat` is accessible from `TestDateFormatter.setAnnotations()`.

30.4. Collections

For parameters and properties annotated with `@CookieParam`, `@HeaderParam`, `@MatrixParam`, `@PathParam`, or `@QueryParam`, the Jakarta RESTful Web Services specification [https://jakarta.ee/specifications/restful-ws/5.0/jakarta-restful-ws-spec-5.0] allows conversion as defined in the Javadoc of the corresponding annotation. In general, the following types are supported:

1. Types for which a `ParamConverter` is available via a registered `ParamConverterProvider`. See Javadoc for these classes for more information.
2. Primitive types.
3. Types that have a constructor that accepts a single `String` argument.
4. Types that have a static method named `valueOf` or `fromString` with a single `String` argument that return an instance of the type. If both methods are present then `valueOf` MUST be used unless the type is an enum in which case `fromString` MUST be used.
5. `List<T>`, `Set<T>`, or `SortedSet<T>`, where `T` satisfies 3 or 4 above.

Items 1, 3, and 4 have been discussed above, and item 2 is obvious. Note that item 5 allows for

collections of parameters. How these collections are expressed in HTTP messages depends, by default, on the particular kind of parameter. In most cases, the notation for collections is based on convention rather than a specification.

30.4.1. @QueryParam

For example, a multivalued query parameter is conventionally expressed like this:

```
http://bluemonkeydiamond.com?q=1q=2q=3
```

In this case, there is a query with name "q" and value {1, 2, 3}. This notation is further supported in Jakarta RESTful Web Services by the method

```
public MultivaluedMap<String, String> getQueryParameters();
```

in `jakarta.ws.rs.core.UriInfo`.

30.4.2. @MatrixParam

There is no specified syntax for collections derived from matrix parameters, but

1. matrix parameters in a URL segment are conventionally separated by ";", and
2. the method

   ```
   MultivaluedMap<String, String> getMatrixParameters();
   ```

   in `jakarta.ws.rs.core.PathSegment` supports extraction of collections from matrix parameters.

RESTEasy adopts the convention that multiple instances of a matrix parameter with the same name are treated as a collection. For example,

```
http://bluemonkeydiamond.com/sippycup;m=1;m=2;m=3
```

is interpreted as a matrix parameter on path segment "sippycup" with name "m" and value {1, 2, 3}.

30.4.3. @HeaderParam

The HTTP 1.1 specification doesn't exactly specify that multiple components of a header value should be separated by commas, but commas are used in those headers that naturally use lists, e.g. Accept and Allow. Also, note that the method

```
public MultivaluedMap<String, String> getRequestHeaders();
```

in `jakarta.ws.rs.core.HttpHeaders` returns a `MultivaluedMap`. It is natural, then, for RESTEasy to treat

```
x-header: a, b, c
```

as mapping name "x-header" to set {a, b, c}.

30.4.4. @CookieParam

The syntax for cookies is specified, but, unfortunately, it is specified in multiple competing specifications. Typically, multiple name=value cookie pairs are separated by ";". However, unlike the case with query and matrix parameters, there is no specified Jakarta RESTful Web Services method that returns a collection of cookie values. Consequently, if two cookies with the same name are received on the server and directed to a collection typed parameter, RESTEasy will inject only the second one. Note, in fact, that the method

```
public Map<String, Cookie> getCookies();
```

in `jakarta.ws.rs.core.HttpHeaders` returns a `Map` rather than a `MultivaluedMap`.

30.4.5. @PathParam

Deriving a collection from path segments is somewhat less natural than it is for other parameters, but Jakarta RESTful Web Services supports the injection of multiple `jakarta.ws.rs.core.PathSegment`. There are a couple of ways of obtaining multiple `PathSegment`. One is through the use of multiple path variables with the same name. For example, the result of calling `testTwoSegmentsArray()` and `testTwoSegmentsList()` in

```
@Path("")
public static class TestResource {

   @GET
   @Path("{segment}/{other}/{segment}/array")
   public Response getTwoSegmentsArray(@PathParam("segment") PathSegment[] segments) {
      System.out.println("array segments: " + segments.length);
      return Response.ok().build();
   }

   @GET
   @Path("{segment}/{other}/{segment}/list")
   public Response getTwoSegmentsList(@PathParam("segment") ListPathSegment segments)
{
      System.out.println("list segments: " + segments.size());
      return Response.ok().build();
   }
}

   @Test
   public void testTwoSegmentsArray() throws Exception {
      Invocation.Builder request = client.target("http://localhost:8081/a/b/c/array")
```

```
.request();
      Response response = request.get();
      Assert.assertEquals(200, response.getStatus());
      response.close();
   }

   @Test
   public void testTwoSegmentsList() throws Exception {
      Invocation.Builder request = client.target("http://localhost:8081/a/b/c/list")
.request();
      Response response = request.get();
      Assert.assertEquals(200, response.getStatus());
      response.close();
   }
```

is

```
array segments: 2
list segments: 2
```

An alternative is to use a wildcard template parameter. For example, the output of calling `testWildcardArray()` and `testWildcardList()` in

```
@Path("")
public static class TestResource {

   @GET
   @Path("{segments:.*}/array")
   public Response getWildcardArray(@PathParam("segments") PathSegment[] segments) {
      System.out.println("array segments: " + segments.length);
      return Response.ok().build();
   }

   @GET
   @Path("{segments:.*}/list")
   public Response getWildcardList(@PathParam("segments") ListPathSegment segments) {
      System.out.println("list segments: " + segments.size());
      return Response.ok().build();
   }

   @Test
   public void testWildcardArray() throws Exception {
      Invocation.Builder request = client.target("http://localhost:8081/a/b/c/array")
.request();
      Response response = request.get();
      response.close();
   }

   @Test
```

```
   public void testWildcardList() throws Exception {
      Invocation.Builder request = client.target("http://localhost:8081/a/b/c/list")
.request();
      Response response = request.get();
      response.close();
   }
}
```

is

```
array segments: 3
list segments: 3
```

30.5. Extension to `ParamConverter` semantics

In the Jakarta RESTful Web Services semantics, a `ParamConverter` is supposed to convert a single `String` that represents an individual object. RESTEasy extends the semantics to allow a `ParamConverter` to parse the `String` representation of multiple objects and generate a `List<T>`, `Set<T>`, `SortedSet<T>`, array, or, indeed, any multivalued data structure whatever. First, consider the resource

```
@Path("queryParam")
public static class TestResource {

   @GET
   @Path("")
   public Response conversion(@QueryParam("q") List<String> list) {
      return Response.ok(stringify(list)).build();
   }
}

private static <T> String stringify(List<T> list) {
   StringBuilder sb = new StringBuilder();
   for (T s : list) {
      sb.append(s).append(',');
   }
   return sb.toString();
}
```

Calling `TestResource` as follows, using the standard notation,

```
@Test
public void testQueryParamStandard() throws Exception {
   Client client = ClientBuilder.newClient();
   Invocation.Builder request = client.target(
"http://localhost:8081/queryParam?q=20161217q=20161218q=20161219").request();
```

```
   Response response = request.get();
   System.out.println("response: " + response.readEntity(String.class));
}
```

results in

```
response: 20161217,20161218,20161219,
```

Suppose, instead, that we want to use a comma separated notation. We can add

```
public static class MultiValuedParamConverterProvider implements
ParamConverterProvider {

   @SuppressWarnings("unchecked")
   @Override
   public <T> ParamConverter<T> getConverter(Class<T> rawType, Type genericType,
Annotation[] annotations) {
      if (List.class.isAssignableFrom(rawType)) {
         return (ParamConverterT) new MultiValuedParamConverter();
      }
      return null;
   }
}

public static class MultiValuedParamConverter implements ParamConverter<List<?>> {

   @Override
   public List<?> fromString(String param) {
      if (param == null || param.trim().isEmpty()) {
         return null;
      }
      return parse(param.split(","));
   }

   @Override
   public String toString(List<?> list) {
      if (list == null || list.isEmpty()) {
         return null;
      }
      return stringify(list);
   }

   private static List<String> parse(String[] params) {
      List<String> list = new ArrayList<String>();
      for (String param : params) {
         list.add(param);
      }
      return list;
   }
```

```
}
```

Now we can call

```
@Test
public void testQueryParamCustom() throws Exception {
   Client client = ClientBuilder.newClient();
   Invocation.Builder request = client.target(
"http://localhost:8081/queryParam?q=20161217,20161218,20161219").request();
   Response response = request.get();
   System.out.println("response: " + response.readEntity(String.class));
}
```

and get

```
response: 20161217,20161218,20161219,
```

Note that in this case, `MultiValuedParamConverter.fromString()` creates and returns an `ArrayList`, so `TestResource.conversion()` could be rewritten

```
@Path("queryParam")
public static class TestResource {

   @GET
   @Path("")
   public Response conversion(@QueryParam("q") ArrayList<String> list) {
      return Response.ok(stringify(list)).build();
   }
}
```

On the other hand, `MultiValuedParamConverter` could be rewritten to return a `LinkList` and the parameter list in `TestResource.conversion()` could be either a `List` or a `LinkedList`.

Finally, note that this extension works for arrays as well. For example,

```
  public static class Foo {
      private String foo;
      public Foo(String foo) {this.foo = foo;}
      public String getFoo() {return foo;}
   }

   public static class FooArrayParamConverter implements ParamConverter<Foo[]> {

      @Override
      public Foo[] fromString(String value) {
         String[] ss = value.split(",");
```

```
         Foo[] fs = new Foo[ss.length];
         int i = 0;
         for (String s : ss) {
            fs[i++] = new Foo(s);
         }
         return fs;
      }

      @Override
      public String toString(Foo[] values) {
         StringBuffer sb = new StringBuffer();
          for (Foo value : values) {
              sb.append(value.getFoo()).append(",");
          }
         if (sb.length() < 0) {
            sb.deleteCharAt(sb.length() - 1);
         }
         return sb.toString();
      }
   }

   @Provider
   public static class FooArrayParamConverterProvider implements
ParamConverterProvider {

      @SuppressWarnings("unchecked")
      @Override
      public <T> ParamConverter<T> getConverter(Class<T> rawType, Type genericType,
Annotation[] annotations) {
         if (rawType.equals(Foo[].class))
            return (ParamConverter<T>) new FooArrayParamConverter();
         return null;
      }
   }

   @Path("")
   public static class ParamConverterResource {

      @GET
      @Path("test")
      public Response test(@QueryParam("foos") Foo[] foos) {
         return Response.ok(new FooArrayParamConverter().toString(foos)).build();
      }
   }
```

30.6. Default multiple valued `ParamConverter`

RESTEasy includes two built-in ParamConverter's in the resteasy-core module, one for collections:

```
org.jboss.resteasy.plugins.providers.MultiValuedCollectionParamConverter,
```

and one for arrays:

```
org.jboss.resteasy.plugins.providers.MultiValuedArrayParamConverter,
```

which implement the concepts in the previous section.

In particular, `MultiValued*ParamConverter.fromString()` can transform a string representation coming over the network into a `Collection` or array, and `MultiValued*ParamConverter.toString()` can be used by a client side proxy to transform `Collection`s or arrays into a string representation.

String representations are determined by `org.jboss.resteasy.annotations.Separator`, a parameter annotation in the resteasy-core module:

```
@Target({ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
public @interface Separator {
   String value() default "";
}
```

The value of `Separator.value()` is used to separate individual elements of a `Collection` or array. For example, a proxy implementing

```
@Path("path/separator/multi/{p}")
@GET
public String pathMultiSeparator(@PathParam("p") @Separator("-") List<String> ss);
```

will turn

```
List<String> list = new ArrayList<String>();
list.add("abc");
list.add("xyz");
proxy.pathMultiSeparator(list);
```

and "path/separator/multi/{p}" into ".../path/separator/multi/abc-xyz". On the server side, the RESTEasy runtime will turn "abc-xyz" back into a list consisting of elements "abc" and "xyz" for

```
@Path("path/separator/multi/{p}")
@GET
public String pathMultiSeparator(@PathParam("p") @Separator("-") List<String> ss) {
    return String.join("|", ss);
}
```

which will return "abc|xyz|".

In fact, the value of the `Separator` annotations may be a more general regular expression, which is passed to `String.split()`. For example, "[-,;]" tells the server side to break up a string using either "-", ",", or ";". On the client side, a string will be created using the first element, "-" in this case.

If a parameter is annotated with `@Separator` with no value, then the default value is

- "," for a `@HeaderParam`, `@MatrixParam`, `@PathParam`, or `@QueryParam`, and
- "-" for a `@CookieParam`.

The MultiValued*ParamConverter's depend on existing facilities for handling the individual elements. On the server side, once it has parsed the incoming string into substrings, `MultiValued*ParamConverter` turns each substring into an Java object according to Section 3.2 "Fields and Bean Properties" of the Jakarta RESTful Web Services specification. On the client side, `MultiValued*ParamConverter` turns a Java object into a string as follows:

1. look for a `ParamConverter`;
2. if there is no suitable `ParamConverter` and the parameter is labeled `@HeaderParam`, look for a `HeaderDelegate`; or
3. call `toString()`.

These `ParamConverter`s are meant to be fairly general, but there are a number of restrictions:

1. They don't handle nested `Collections` or arrays. That is, `List<String>` and `String[]` are OK, but `List<List<String>>` and `String[][]` are not.
2. The regular expression used in Separator must match the regular expression

   ```
   "\\p{Punct}|\\[\\p{Punct}+\\]"
   ```

   That is, it must be either a single instance of a punctuation symbol, i.e., a symbol in the set

   ```
   !"#$%'()*+,-./:;=?@[\]^_`{|}~
   ```

   or a class of punctuation symbols like "[-,;]".
3. For either of these ParamConverter's to be available for use with a given parameter, that parameter must be annotated with `@Separator`.

There are also some logical restrictions:

1. Cookie syntax, as specified in https://tools.ietf.org/html/rfc6265#section-4.1.1, assigns a meaning to ";", so it cannot be used as a separator.
2. If a separator character appears in the content of an element, then there will be problems. For example, if "," is used as a separator, then, if a proxy sends the array `["a","b,c","d"]`, it will turn into the string "a,b,c,d" on the wire and be reconstituted on the server as four elements.

These built-in `ParamConverter`s have the lowest priority, so any user supplied ParamConverter's will be tried first.

Chapter 31. Responses using jakarta.ws.rs.core.Response

Custom responses can be built using the `jakarta.ws.rs.core.Response` and `ResponseBuilder` classes. To do your own streaming, your entity response must be an implementation of `jakarta.ws.rs.core.StreamingOutput`.

Chapter 32. Exception Handling

32.1. Exception Mappers

ExceptionMappers are custom, application provided, components that can catch thrown application exceptions and write specific HTTP responses. They are classes annotated with `@Provider` and implement the `ExceptionMapper` interface.

When an application exception is thrown it will be caught by the Jakarta RESTful Web Services runtime. Jakarta RESTful Web Services will then scan registered ExceptionMappers to see which one support marshalling the exception type thrown. Here is an example of ExceptionMapper

```
@Provider
public class EJBExceptionMapper implements ExceptionMapper<EJBException> {

    @Override
    public Response toResponse(EJBException exception) {
       return Response.status(500).build();
    }
}
```

ExceptionMappers are registered the same way as MessageBodyReader/Writers. By scanning for `@Provider` annotated classes, or programmatically through the `ResteasyProviderFactory` class.

As of RESTEasy 6.1 if a default `ExceptionMapper` is registered. It handles all uncaught exceptions and returns a response with a status of 500 and no entity body, so that internal exception details are not disclosed to clients. If the exception is a `WebApplicationException` the response from the exception is returned. This can be turned off by setting the `dev.resteasy.exception.mapper` to `false`.

The default `ExceptionMapper` will also log the exception at a error level. The logger name is `org.jboss.resteasy.core.providerfactory.DefaultExceptionMapper` which can be used to disable these log messages.

32.2. RESTEasy Built-in Internally-Thrown Exceptions

RESTEasy has a set of built-in exceptions that are thrown when it encounters errors during dispatching or marshalling. They all revolve around specific HTTP error codes. They can be found in RESTEasy's javadoc under the package org.jboss.resteasy.spi. Here's a list of them:

Exception	HTTP Code	Description
`ReaderException`	400	All exceptions thrown from MessageBodyReaders are wrapped within this exception. If there is no `ExceptionMapper` for the wrapped exception or if the exception isn't a `WebApplicationException`, then resteasy will return a 400 code by default.
`WriterException`	500	All exceptions thrown from MessageBodyWriters are wrapped within this exception. If there is no `ExceptionMapper` for the wrapped exception or if the exception isn't a `WebApplicationExceptio`n, then resteasy will return a 400 code by default.
`org.jboss.resteasy.plugins.providers.jaxb.`JAXBUnmarshalException	400	The Jakarta RESTful Web Services providers throw this exception on reads. They may be wrapping `JAXBExceptions`. This class extends `ReaderException`.
`org.jboss.resteasy.plugins.providers.jaxb.JAXBMarshalException`	500	The Jakarta RESTful Web Services providers throw this exception on writes. They may be wrapping `JAXBExceptions`. This class extends `WriterException`.
`ApplicationException`	N/A	This exception wraps all exceptions thrown from application code. It functions much in the same way as `InvocationTargetException`. If there is an `ExceptionMapper` for wrapped exception, then that is used to handle the request.
`Failure`	N/A	Internal RESTEasy. Not logged
`LoggableFailure`	N/A	Internal RESTEasy error. Logged

Exception	HTTP Code	Description
`DefaultOptionsMethodException`	N/A	If the user invokes HTTP OPTIONS and no Jakarta RESTful Web Services method for it, RESTEasy provides a default behavior by throwing this exception. This is only done if the property `dev.resteasy.throw.options.exception` is set to true.
`JsonProcessingException`	400	A Jackson provider throws this exception when JSON data is determined to be invalid.

32.3. Resteasy WebApplicationExceptions

Suppose a client at local.com calls the following resource method:

```
@GET
@Path("remote")
public String remote() {
    Client client = ClientBuilder.newClient();
    return client.target("http://localhost/exception").request().get(String.class);
}
```

If the call to http://localhost returns a status code 3xx, 4xx, or 5xx, then the `Client` is obliged by the Jakarta RESTful Web Services specification to throw a `WebApplicationException`. Moreover, if the `WebApplicationException` contains a `Response`, which it normally would in RESTEasy, the server runtime is obliged by the Jakarta RESTful Web Services specification to return that `Response`. As a result, information from the server at third.party.com, e.g., headers and body, will get sent back to local.com. The problem is that that information could be, at best, meaningless to the client and, at worst, a security breach.

RESTEasy has a solution that works around the problem and still conforms to the Jakarta RESTful Web Services specification. In particular, for each `WebApplicationException` it defines a new subclass:

```
WebApplicationException
+-ResteasyWebApplicationException
+-ClientErrorException
| +-ResteasyClientErrorException
| +-BadRequestException
| | +-ResteasyBadRequestException
| +-ForbiddenException
| | +-ResteasyForbiddenException
| +-NotAcceptableException
```

```
| | +-ResteasyNotAcceptableException
| +-NotAllowedException
| | +-ResteasyNotAllowedException
| +-NotAuthorizedException
| | +-ResteasyNotAuthorizedException
| +-NotFoundException
| | +-ResteasyNotFoundException
| +-NotSupportedException
| | +-ResteasyNotSupportedException
+-RedirectionException
| +-ResteasyRedirectionException
+-ServerErrorException
| +-ResteasyServerErrorException
| +-InternalServerErrorException
| | +-ResteasyInternalServerErrorException
| +-ServiceUnavailableException
| | +-ResteasyServiceUnavailableException
```

The new exceptions play the same role as the original ones, but RESTEasy treats them slightly differently. When a `Client` detects that it is running in the context of a resource method, it will throw one of the new `Exception`. However, instead of storing the original `Response`, it stores a "sanitized" version of the `Response`, in which only the status and the Allow and Content-Type headers are preserved. The original `WebApplicationException`, and therefore the original `Response`, can be accessed in one of two ways:

```
// Create a NotAcceptableException.
NotAcceptableException nae = new NotAcceptableException(Response.status(406).entity(
"ooops").build());

// Wrap the NotAcceptableException in a ResteasyNotAcceptableException.
ResteasyNotAcceptableException rnae = (ResteasyNotAcceptableException)
WebApplicationExceptionWrapper.wrap(nae);

// Extract the original NotAcceptableException using instance method.
NotAcceptableException nae2 = rnae.unwrap();
Assertions.assertEquals(nae, nae2);

// Extract the original NotAcceptableException using class method.
NotAcceptableException nae3 = (NotAcceptableException) WebApplicationExceptionWrapper
.unwrap(nae); // second way
Assertions.assertEquals(nae, nae3);
```

Note that this change is intended to introduce a safe default behavior in the case that the `Exception` generated by the remote call is allowed to make its way up to the server runtime. It is considered a good practice, though, to catch the `Exception` and treat it in some appropriate manner:

```
@GET
@Path("remote/{i}")
```

```
public String remote(@PathParam("i") String i) throws Exception {
  Client client = ClientBuilder.newClient();
  try {
     return client.target("http://localhost/exception/" + i).request().get(String
.class);
  } catch (WebApplicationException wae) {
      LOGGER.errorf(wae, "Failed to execute %s", i);
  }
}
```



While RESTEasy will default to the new, safer behavior, the original behavior can be restored by setting the configuration parameter `resteasy.original.webapplicationexception.behavior` to "true".

32.4. Overriding RESTEasy Builtin Exceptions

RESTEasy built-in exceptions can be overridden by writing an `ExceptionMapper` for the exception. You can write an `ExceptionMapper` for any thrown exception including `WebApplicationException`.

Chapter 33. Content encoding

33.1. GZIP Compression/Decompression

RESTEasy supports (though not by default - see below) GZIP decompression. If properly configured, the client framework or a Jakarta RESTful Web Services service, upon receiving a message body with a Content-Encoding of "gzip", will automatically decompress it. The client framework can (though not by default - see below) automatically set the Accept-Encoding header to be "gzip, deflate", so you do not have to set this header yourself.

RESTEasy also supports (though not by default - see below) automatic compression. If the client framework is sending a request or the server is sending a response with the Content-Encoding header set to "gzip", RESTEasy will (if properly configured) do the compression. So that you do not have to set the Content-Encoding header directly, you can use the @org.jboss.resteasy.annotation.GZIP annotation.

```
@Path("/")
public interface MyProxy {

   @Consumes("application/xml")
   @PUT
   void put(@GZIP Order order);
}
```

In the above example, we tag the outgoing message body, order, to be gzip compressed. The same annotation can be used to tag server responses

```
@Path("/")
public class MyService {

   @GET
   @Produces("application/xml")
   @GZIP
   public String getData() {}
}
```

33.1.1. Configuring GZIP compression / decompression



Decompression carries a risk of attack from a bad actor that can package an entity that will expand greatly. Consequently, RESTEasy disables GZIP compression / decompression by default.

There are three interceptors that are relevant to GZIP compression / decompression:

1. `org.jboss.resteasy.plugins.interceptors.GZIPDecodingInterceptor`: If the Content-Encoding header is present and has the value "gzip", `GZIPDecodingInterceptor` will install an `InputStream`

that decompresses the message body.

2. `org.jboss.resteasy.plugins.interceptors.GZIPEncodingInterceptor`: If the Content-Encoding header is present and has the value "gzip", `GZIPEncodingInterceptor` will install an `OutputStream` that compresses the message body.
3. `org.jboss.resteasy.plugins.interceptors.AcceptEncodingGZIPFilter`: If the Accept-Encoding header does not exist, `AcceptEncodingGZIPFilter` will add Accept-Encoding with the value "gzip, deflate". If the Accept-Encoding header exists but does not contain "gzip", `AcceptEncodingGZIPFilter` will append ", gzip". Note that enabling GZIP compression / decompression does not depend on the presence of this interceptor.

If GZIP decompression is enabled, an upper limit is imposed on the number of bytes `GZIPDecodingInterceptor` will extract from a compressed message body. The default limit is 10,000,000, but a different value can be configured. See below.

Server side configuration

The interceptors may be enabled by including their classnames in a META-INF/services/jakarta.ws.rs.ext.Providers file on the classpath. The upper limit on deflated files may be configured by setting the parameter "resteasy.gzip.max.input". [See Configuration for more information about application configuration.] If the limit is exceeded on the server side, `GZIPDecodingInterceptor` will return a `Response` with status 413 ("Request Entity Too Large") and a message specifying the upper limit.



As of release 3.1.0.Final, the GZIP interceptors have moved from package `org.jboss.resteasy.plugins.interceptors.encoding` to `org.jboss.resteasy.plugins.interceptors` and they should be named accordingly in `jakarta.ws.rs.ext.Providers`.

Client side configuration

The interceptors may be enabled by registering them with a `Client` or `WebTarget`. For example,

```
Client client = ClientBuilder.newBuilder() // Activate gzip compression on client:
                    .register(AcceptEncodingGZIPFilter.class)
                    .register(GZIPDecodingInterceptor.class)
                    .register(GZIPEncodingInterceptor.class)
                    .build();
```

The upper limit on deflated files may be configured by creating an instance of `GZIPDecodingInterceptor` with a specific value:

```
Client client = ClientBuilder.newBuilder() // Activate gzip compression on client:
                    .register(AcceptEncodingGZIPFilter.class)
                    .register(new GZIPDecodingInterceptor(256))
                    .register(GZIPEncodingInterceptor.class)
                    .build();
```

If the limit is exceeded on the client side, `GZIPDecodingInterceptor` will throw a `ProcessingException` with a message specifying the upper limit.

33.2. General content encoding

The designation of a compressible entity by the use of the `@GZIP` annotation is a built-in, specific instance of a more general facility supported by RESTEasy. There are three components to this facility.

1. The annotation `org.jboss.resteasy.annotations.ContentEncoding` is a "meta-annotation" used on other annotations to indicate that they represent a Content-Encoding. For example, @GZIP is defined

   ```
   @Target({ElementType.TYPE, ElementType.METHOD, ElementType.PARAMETER})
   @Retention(RetentionPolicy.RUNTIME)
   @ContentEncoding("gzip")
   public @interface GZIP {
   }
   ```

   The value of @ContentEncoding indicates the represented Content-Encoding. For `@GZIP` it is "gzip".

2. `ClientContentEncodingAnnotationFeature` and `ServerContentEncodingAnnotationFeature`, two `DynamicFeatures` in package `org.jboss.resteasy.plugins.interceptors`, examine resource methods for annotations decorated with `@ContentEncoding`.

3. For each value found in a `@ContentEncoding` decorated annotation on a resource method, an instance of `ClientContentEncodingAnnotationFilter` or `ServerContentEncodingAnnotationFilter`, `jakarta.ws.rs.ext.WriterInterceptors` in package `org.jboss.resteasy.plugins.interceptors`, is registered. They are responsible for adding an appropriate Content-Encoding header. For example, `ClientContentEncodingAnnotationFilter` is defined.

   ```
   @ConstrainedTo(RuntimeType.CLIENT)
   @Priority(Priorities.HEADER_DECORATOR)
   public class ClientContentEncodingAnnotationFilter implements WriterInterceptor {
      protected String encoding;

      public ClientContentEncodingAnnotationFilter(String encoding) {
         this.encoding = encoding;
      }

      @Override
      public void aroundWriteTo(WriterInterceptorContext context) {
         context.getHeaders().putSingle(HttpHeaders.CONTENT_ENCODING, encoding);
         context.proceed();
      }
   }
   ```

The annotation `@GZIP` is built into RESTEasy, but `ClientContentEncodingAnnotationFeature` and `ServerContentEncodingAnnotationFeature` will also recognize application defined annotations. For example,

```
@Target({ElementType.TYPE, ElementType.METHOD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@ContentEncoding("compress")
public @interface Compress {
}

@Path("")
public static class TestResource {

  @GET
  @Path("a")
  @Compress
  public String a() {
     return "a";
  }
}
```

If `TestResource.a()` is invoked as follows

```
@Test
public void testCompress() throws Exception {
  try (Client client = ClientBuilder.newClient()) {
      Invocation.Builder request = client.target("http://localhost:8081/a").request();
      request.acceptEncoding("gzip,compress");
      Response response = request.get();
      System.out.println("content-encoding: "+ response.getHeaderString("Content-
Encoding"));
  }
}
```

the output will be

```
content-encoding: compress
```

Chapter 34. CORS

RESTEasy has a `ContainerRequestFilter` that can be used to handle CORS preflight and actual requests. `org.jboss.resteasy.plugins.interceptors.CorsFilter`. You must allocate this and register it as a singleton provider from your Application class. See the javadoc or its various settings.

```
CorsFilter filter = new CorsFilter();
filter.getAllowedOrigins().add("http://localhost");
```

34.1. Credentials

Credentials are disabled by default. `Access-Control-Allow-Credentials` is only sent if `CorsFilter.setAllowCredentials(true)` is called.

```
CorsFilter filter = new CorsFilter();
filter.getAllowedOrigins().add("http://localhost");
filter.setAllowCredentials(true);
```



In previous releases credentials were enabled by default. A deployment which relies on `Access-Control-Allow-Credentials` being sent must now opt in explicitly.



Credentials cannot be combined with a wildcard origin. If `"*"` is added to the allowed origins, `Access-Control-Allow-Origin: *` is returned and `Access-Control-Allow-Credentials` is never sent, even if credentials are enabled. A warning is logged the first time this combination is used. Configure an explicit list of allowed origins if credentials are required.

Chapter 35. Content-Range Support

RESTEasy supports `Range` requests for `java.io.File` response entities.

```
@Path("/")
public class Resource {
  @GET
  @Path("file")
  @Produces("text/plain")
  public File getFile(){
     return file;
  }
}
```

```
Response response = client.target(generateURL("/file")).request()
      .header("Range", "1-4").get();
Assertions.assertEquals(response.getStatus(), 206);
Assertions.assertEquals(4, response.getLength());
System.out.println("Content-Range: " + response.getHeaderString("Content-Range"));
```

Chapter 36. RESTEasy Caching Features

RESTEasy provides numerous annotations and facilities to support HTTP caching semantics. Annotations to make setting Cache-Control headers easier and both server-side and client-side in-memory caches are available.

36.1. @Cache and @NoCache Annotations

RESTEasy provides an extension to Jakarta RESTful Web Services that allows you to automatically set Cache-Control headers on a successful GET request. It can only be used on @GET annotated methods. A successful `@GET` request is any request that returns 200 OK response.

```
package org.jboss.resteasy.annotations.cache;

public @interface Cache {
   int maxAge() default -1;
   int sMaxAge() default -1;
   boolean noStore() default false;
   boolean noTransform() default false;
   boolean mustRevalidate() default false;
   boolean proxyRevalidate() default false;
   boolean isPrivate() default false;
}

public @interface NoCache {
   String[] fields() default {};
}
```

While `@Cache` builds a complex Cache-Control header, @NoCache is a simplified notation to indicate no caching is wanted; i.e. Cache-Control: nocache.

These annotations can be put on the resource class or interface and specifies a default cache value for each @GET resource method, or they can be put individually on each @GET resource method.

36.2. Client "Browser" Cache

RESTEasy has the ability to set up a client-side, browser-like, cache. It can be used with the Client Proxy Framework, or with ordinary requests. This cache looks for Cache-Control headers sent back with a server response. If the Cache-Control headers specify that the client is allowed to cache the response, Resteasy caches it within local memory. The cache obeys max-age requirements and will also automatically do HTTP 1.1 cache revalidation if either or both the Last-Modified and/or ETag headers are sent back with the original response. See the HTTP 1.1 specification for details on how Cache-Control or cache revalidation works.

It is very simple to enable caching. Here's an example of using the client cache with the Client Proxy Framework

```
@Path("/orders")
public interface OrderServiceClient {

   @Path("{id}")
   @GET
   @Produces("application/xml")
   Order getOrder(@PathParam("id") String id);
}
```

To create a proxy for this interface and enable caching for that proxy requires only a few simple steps in which the `BrowserCacheFeature` is registered:

```
ResteasyWebTarget target = (ResteasyWebTarget) ClientBuilder.newClient().target(
"http://localhost:8081");
BrowserCacheFeature cacheFeature = new BrowserCacheFeature();
OrderServiceClient orderService = target.register(cacheFeature).proxy
(OrderServiceClient.class);
```

`BrowserCacheFeature` will create a Resteasy `LightweightBrowserCache` by default. It is also possible to configure the cache, or install a completely different cache implementation:

```
ResteasyWebTarget target = (ResteasyWebTarget) ClientBuilder.newClient().target(
"http://localhost:8081");
LightweightBrowserCache cache = new LightweightBrowserCache();
cache.setMaxBytes(20L);
BrowserCacheFeature cacheFeature = new BrowserCacheFeature();
cacheFeature.setCache(cache);
OrderServiceClient orderService = target.register(cacheFeature).proxy
(OrderServiceClient.class);
```

If using the standard Jakarta RESTful Web Services client framework to make invocations rather than the proxy framework, it is just as easy:

```
ResteasyWebTarget target = (ResteasyWebTarget) ClientBuilder.newClient().target(
"http://localhost:8081/orders/{id}");
BrowserCacheFeature cacheFeature = new BrowserCacheFeature();
target.register(cacheFeature);
String rtn = target.resolveTemplate("id", "1").request().get(String.class);
```

The LightweightBrowserCache, by default, has a maximum 2 megabytes of caching space. This can be changed programmatically by calling its `setMaxBytes()` method. If the cache gets full, the cache completely wipes itself of all cached data. This may seem a bit draconian, but the cache was written to avoid unnecessary synchronizations in a concurrent environment where the cache is shared between multiple threads. If a more complex caching solution is desired or a third party cache is to be plugged in please contact our resteasy-developers list and discuss it with the community.

36.3. Local Server-Side Response Cache

RESTEasy has a server-side cache that can sit in front of Jakarta RESTful Web Services services. It automatically caches marshalled responses from HTTP GET Jakarta RESTful Web Services invocations if, and only if the Jakarta RESTful Web Services resource method sets a Cache-Control header. When a GET comes in, the RESTEasy Server Cache checks to see if the URI is stored in the cache. If found, it returns the already marshalled response without invoking the Jakarta RESTful Web Services method. Each cache entry has a max age to whatever is specified in the Cache-Control header of the initial request. The cache also will automatically generate an ETag using an MD5 hash on the response body. This allows the client to do HTTP 1.1 cache revalidation with the IF-NONE-MATCH header. The cache is also smart enough to perform revalidation if there is no initial cache hit, but the Jakarta RESTful Web Services method still returns a body that has the same ETag.

The cache is also automatically invalidated for a particular URI that has PUT, POST, or DELETE invoked on it. A reference to the cache can be obtained by injecting an `org.jboss.resteasy.plugins.cache.ServerCache` via the `@Context` annotation

```
    @Context
    ServerCache cache;

    @GET
    public String get(@Context ServerCache cache) {}
```

To set up the server-side cache an instance of `org.jboss.resteasy.plugins.cache.server.ServerCacheFeature` must be registered via the Application's `getSingletons()` or `getClasses()` methods. The underlying cache is Infinispan. By default, RESTEasy will create an Infinispan cache for you. Alternatively, you can create and pass in an instance of your cache to the `ServerCacheFeature` constructor. Infinispan can also be configured by specifying various parameters. If using Maven, add RESTEasy's cache-core artifact to the project:

```
<dependency>
   <groupId>org.jboss.resteasy.cache</groupId>
   <artifactId>cache-core</artifactId>
   <version>${version.org.jboss.resteasy.cache}</version>
</dependency>
```

Next set up the Infinispan configuration in the application's `web.xml`, it would look like

```
<web-app>
    <context-param>
        <param-name>server.request.cache.infinispan.config.file</param-name>
        <param-value>infinispan.xml</param-value>
    </context-param>

    <context-param>
        <param-name>server.request.cache.infinispan.cache.name</param-name>
        <param-value>MyCache</param-value>
```

```
    </context-param>

</web-app>
```

`server.request.cache.infinispan.config.file` can either be a classpath or a file path. `server.request.cache.infinispan.cache.name` is the name of the cache to reference that is declared in the config file.

See Configuration for more information about application configuration.

36.4. HTTP preconditions

Jakarta RESTful Web Services provides an API for evaluating HTTP preconditions based on `"If-Match"`, `"If-None-Match"`, `"If-Modified-Since"` and `"If-Unmodified-Since"` headers.

```
Response.ResponseBuilder rb = request.evaluatePreconditions(lastModified, etag);
```

By default, RESTEasy will return status code 304 (Not modified) or 412 (Precondition failed) if any of conditions fails, however it is not compliant with RFC 7232 which states that headers `"If-Match"`, `"If-None-Match"` MUST have higher precedence. RFC 7232 compatible mode can be enabled by setting the parameter `resteasy.rfc7232preconditions` to `true`. See Configuration for more information about application configuration.

Chapter 37. Filters and Interceptors

Jakarta RESTful Web Services has two different concepts for interceptions: Filters and Interceptors. Filters are mainly used to modify or process incoming and outgoing request headers or response headers. They execute before and after request and response processing.

37.1. Server Side Filters

On the server-side there are two different types of filters.ContainerRequestFilters run before a Jakarta RESTful Web Services resource method is invoked. ContainerResponseFilters run after a Jakarta RESTful Web Services resource method is invoked. As an added caveat, ContainerRequestFilters come in two flavors: pre-match and post-matching. Pre-matching ContainerRequestFilters are designated with the `@PreMatching` annotation and will execute before the Jakarta RESTful Web Services resource method is matched with the incoming HTTP request. Pre-matching filters often are used to modify request attributes to change how it matches to a specific resource method (i.e. strip .xml and add an Accept header). ContainerRequestFilters can abort the request by calling `ContainerRequestContext.abortWith(Response)`. A filter might want to abort if it implements a custom authentication protocol.

After the resource class method is executed, Jakarta RESTful Web Services will run all ContainerResponseFilters. These filters allow the outgoing response to be modified before it is marshalling and sent to the client. Here is some pseudocode to give some understanding of how it works.

```
public void filter() {
    // execute pre match filters
    for (ContainerRequestFilter filter : preMatchFilters) {
        filter.filter(requestContext);
        if (isAborted(requestContext)) {
           sendAbortionToClient(requestContext);
           return;
        }
    }
    // match the HTTP request to a resource class and method
    JaxrsMethod method = matchMethod(requestContext);

    // Execute post match filters
    for (ContainerRequestFilter filter : postMatchFilters) {
       filter.filter(requestContext);
       if (isAborted(requestContext)) {
          sendAbortionToClient(requestContext);
          return;
       }
    }

    // execute resource class method
    method.execute(request);
```

```
    // execute response filters
    for (ContainerResponseFilter filter : responseFilters) {
       filter.filter(requestContext, responseContext);
    }
}
```

37.1.1. Asynchronous filters

It is possible to turn filters into asynchronous filters, if it is needed to suspend execution of the filter until a certain resource has become available. This makes the request asynchronous, but requires no change to the resource method declaration. In particular, synchronous and asynchronous resource methods continue to work as specified, regardless of whether a filter has made the request asynchronous. Similarly, one filter making the request asynchronous requires no change in the declaration of other filters.

In order to make a filter's execution asynchronous, `ContainerRequestContext` must be cast to a `SuspendableContainerRequestContext` (for pre/post request filters), or cast the `ContainerResponseContext` to a `SuspendableContainerResponseContext` (for response filters).

These context objects can make the current filter's execution asynchronous by calling the `suspend()` method. Once asynchronous, the filter chain is suspended, and will only resume after one of the following methods is called on the context object:

- `abortWith(Response)`

  Terminate the filter chain, return the given `Response` to the client (only for `ContainerRequestFilter`).

- `resume()`

  Resume execution of the filter chain by calling the next filter.

- `resume(Throwable)`

  Abort execution of the filter chain by throwing the given exception. This behaves as if the filter were synchronous and threw the given exception.

Async processing can be done inside an `AsyncWriterInterceptor` (if using Async IO), which is the asynchronous-supporting equivalent to `WriterInterceptor`. In this case, there is no need to manually suspend or resume the request.



`SuspendableContainerRequestContext` and `SuspendableContainerResponseContext` are deprecated for removal. Resuming a suspended filter chain may continue execution on a different thread than the one that suspended it. CDI contexts are bound to a specific thread, so CDI may not behave as expected after such a resumption: a `@RequestScoped`, `@SessionScoped`, or `@ConversationScoped` bean may fail to resolve, or resolve to a different instance than expected, regardless of what scope the matched resource itself uses. `@ApplicationScoped` and `@Dependent` beans are unaffected. This is consistent with the Jakarta Concurrency specification's own

> guidance against relying on those same CDI scopes surviving an asynchronous task boundary, and reliably fixing it is not something RESTEasy can do in a CDI-implementation-agnostic way. Asynchronous processing in resource methods via `AsyncResponse`/`CompletionStage` is not deprecated and remains fully supported as a mechanism, but is equally susceptible to this same caveat - code that doesn't reference any CDI bean after dispatching to another thread is unaffected, but code that does is not guaranteed to work.

37.2. Client Side Filters

The client side also has two types of filters: `ClientRequestFilter` and `ClientResponseFilter`. ClientRequestFilters run before an HTTP request is sent over the wire to the server. ClientResponseFilters run after a response is received from the server, but before the response body is unmarshalled. ClientRequestFilters are allowed to abort the execution of the request and provide a canned response without going over the wire to the server. ClientResponseFilters can modify the Response object before it is handed back to application code. Here's pseudocode to illustrate this.

```
public Response filter() {
    // execute request filters
    for (ClientRequestFilter filter : requestFilters) {
       filter.filter(requestContext);
       if (isAborted(requestContext)) {
          return requestContext.getAbortedResponseObject();
       }
    }

    // send request over the wire
    response = sendRequest(request);

    // execute response filters
    for (ClientResponseFilter filter : responseFilters) {
       filter.filter(requestContext, responseContext);
    }
}
```

37.3. Reader and Writer Interceptors

While filters modify request or response headers, interceptors deal with message bodies. Interceptors are executed in the same call stack as their corresponding reader or writer. ReaderInterceptors wrap around the execution of MessageBodyReaders. WriterInterceptors wrap around the execution of MessageBodyWriters. They can be used to implement a specific content-encoding. They can be used to generate digital signatures or to post or pre-process a Java object model before or after it is marshalled.

Note that in order to support Async IO, `AsyncWriterInterceptor` can be implemented, which is a subtype of `WriterInterceptor`.

37.4. Per Resource Method Filters and Interceptors

Sometimes it is desired to have a filter or interceptor only run for a specific resource method. This can be done in two different ways: register an implementation of `DynamicFeature` or use the `@NameBinding` annotation. The DynamicFeature interface is executed at deployment time for each resource method. Use the Configurable interface to register the filters and interceptors wanted for the specific resource method. `@NameBinding` works a lot like CDI interceptors. Annotate a custom annotation with `@NameBinding` and then apply that custom annotation to the filter and resource method. The custom annotation must use `@Retention(RetentionPolicy.RUNTIME)` in order for the attribute to be picked up by the RESTEasy runtime when it is deployed.

```
@NameBinding
@Retention(RetentionPolicy.RUNTIME)
public @interface DoIt {}

@DoIt
public class MyFilter implements ContainerRequestFilter {}

@Path("/root")
public class MyResource {

   @GET
   @DoIt
   public String get() {}
}
```

37.5. Ordering

Ordering is accomplished by using the `@BindingPriority` annotation on the filter or interceptor classes.

Chapter 38. Asynchronous HTTP Request Processing

Asynchronous HTTP Request Processing is a relatively new technique that allows the processing of a single HTTP request using non-blocking I/O and, if desired in separate threads. Some refer to it as COMET capabilities. The primary use case for Asynchronous HTTP is in the case where the client is polling the server for a delayed response. The usual example is an AJAX chat client where you want to push/pull from both the client and the server. These scenarios have the client blocking a long time on the server's socket waiting for a new message. In synchronous HTTP where the server is blocking on incoming and outgoing I/O is that you have a thread consumed per client connection. This eats up memory and valuable thread resources. Not such a big deal in 90% of applications (in fact using asynchronous processing may actually hurt performance in most common scenarios), but when there are a lot of concurrent clients that are blocking like this, there is a lot of wasted resources and the server does not scale that well.

38.1. Using the `@Suspended` annotation

The Jakarta RESTful Web Services specification includes asynchronous HTTP support via two classes. The `@Suspended` annotation, and AsyncResponse interface.

Injecting an AsyncResponse as a parameter to a Jakarta RESTful Web Services methods tells RESTEasy that the HTTP request/response should be detached from the currently executing thread and that the current thread should not try to automatically process the response.

The AsyncResponse is the callback object. The act of calling one of the `resume()` methods will cause a response to be sent back to the client and will also terminate the HTTP request. Here is an example of asynchronous processing:

```
import jakarta.annotation.Resource;
import jakarta.enterprise.concurrent.ManagedExecutorService;
import jakarta.ws.rs.Suspend;
import jakarta.ws.rs.container.AsyncResponse;

@Path("/")
public class SimpleResource {

    @Resource
    private ManagedExecutorService executor;

   @GET
   @Path("basic")
   @Produces("text/plain")
   public void getBasic(@Suspended final AsyncResponse response) {
       executor.execute(() -> {
           try {
               final Resourse builtResponse = Response.ok("basic").type(MediaType
.TEXT_PLAIN).build();
```

```
                response.resume(builtResponse);
            } catch (Exception e) {
                response.resmume(e);
            }
        });
    }
}
```

`AsyncResponse` also has other methods to cancel the execution. See javadoc for more details.



A `CompletionCallback` registered with `AsyncResponse.register()` is invoked once the processing of the request has ended. This includes the case where the client closes the connection before the response is resumed, in which case the callback receives the error. Closed connections are currently only detected when running in a Servlet container.



In RESTEasy version 4.0.0.Final proprietary annotation `org.jboss.resteasy.annotations.Suspend` was removed and replaced by `jakarta.ws.rs.container.Suspended` and class `org.jboss.resteasy.spi.AsynchronousResponse` was removed and replaced by `jakarta.ws.rs.container.AsyncResponse`.



The `@Suspended` does not have a value field, which represented a timeout limit. Instead, `AsyncResponse.setTimeout()` may be called.

38.2. Using Reactive return types

The Jakarta RESTful Web Services 2.1 specification adds support for declaring asynchronous resource methods by returning a `CompletionStage` instead of using the `@Suspended` annotation.

Whenever a resource method returns a `CompletionStage`, it will be subscribed to, the request will be suspended, and only resumed when the `CompletionStage` is resolved either to a value (which is then treated as the return value for the method), or as an error case, in which case the exception will be processed as if it were thrown by the resource method.

Here is an example of asynchronous processing using `CompletionStage`:

```
import jakarta.annotation.Resource;
import jakarta.enterprise.concurrent.ManagedExecutorService;
import jakarta.ws.rs.Suspend;
import jakarta.ws.rs.container.AsyncResponse;

@Path("/")
public class SimpleResource {

    @Resource
    private ManagedExecutorService executor;
```

```
   @GET
   @Path("basic")
   @Produces("text/plain")
   public CompletionStage<Response> getBasic() {
      final CompletableFuture<Response> response = new CompletableFuture<>();
      executor.execute(() -> {
            try{
               final Response jaxrs = Response.ok("basic").type(MediaType.TEXT_PLAIN)
.build();
               response.complete(jaxrs);
            } catch (Exception e) {
               response.completeExceptionally(e);
            }
      });
      return response;
   }
}
```



RESTEasy supports more reactive types for asynchronous programming.

38.3. Asynchronous filters

It is possible to write filters that also make the request asynchronous. Whether filters make the request asynchronous before execution of a method makes absolutely no difference to a method: it does not need to be declared asynchronous in order to function as specified. Synchronous methods and asynchronous methods will work as specified by the spec.

38.4. Asynchronous IO

Some backends support asynchronous IO operations (Servlet, Undertow, Vert.x, Quarkus, Netty), which are exposed using the `AsyncOutputStream` subtype of `OutputStream`. It includes async variants for writing and flushing the stream.

Some backends have what is called an "Event Loop Thread", which is a thread responsible for doing all IO operations. Those backends require the Event Loop Thread to never be blocked, because it does IO for every other thread. Those backends typically require Jakarta RESTful Web Services endpoints to be invoked on worker threads, to make sure they never block the Event Loop Thread.

Sometimes, with Async programming, it is possible for asynchronous Jakarta RESTful Web Services requests to be resumed from the Event Loop Thread. As a result, Jakarta RESTful Web Services will attempt to serialise the response and send it to the client. But Jakarta RESTful Web Services is written using "Blocking IO" mechanics, such as `OutputStream` (used by `MessageBodyWriter` and `WriterInterceptor`), which means that sending the response will block the current thread until the response is received. This would work on a worker thread, but if it happens on the Event Loop Thread it will block it and prevent it from sending the response, resulting in a deadlock.

As a result, we've decided to support and expose Async IO interfaces in the form of

`AsyncOutputStream`, `AsyncMessageBodyWriter` and `AsyncWriterInterceptor`, to allow users to write Async IO applications in RESTEasy.

Most built-in `MessageBodyWriter` and `WriterInterceptor` support Async IO, with the notable exceptions of:

- `HtmlRenderableWriter`, which is tied to servlet APIs
- `ReaderProvider`
- `StreamingOutputProvider`: use `AsyncStreamingOutput` instead

Async IO will be preferred if the following conditions are met:

- The backend supports it
- The writer supports it
- All writer interceptors support it

If those conditions are not met, and you attempt to use Blocking IO on an Event Loop Thread (as determined by the backend), then an exception will be thrown.

Chapter 39. Asynchronous Job Service

The RESTEasy Asynchronous Job Service is an implementation of the Asynchronous Job pattern defined in O'Reilly's "Restful Web Services" book. The idea of it is to bring asynchronicity to a synchronous protocol.

39.1. Using Async Jobs

While HTTP is a synchronous protocol it does have a faint idea of asynchronous invocations. The HTTP 1.1 response code 202, "Accepted" means that the server has received and accepted the response for processing, but the processing has not yet been completed. The RESTEasy Asynchronous Job Service builds around this idea.

```
POST http://localhost/myservice?asynch=true
```

For example, if making the above post with the asynch query parameter set to true, RESTEasy will return a 202, "Accepted" response code and run the invocation in the background. It also sends back a Location header with a URL pointing to where the response of the background method is located.

```
HTTP/1.1 202 Accepted
Location: http://localhost/asynch/jobs/3332334
```

The URI will have the form of:

```
/asynch/jobs/{job-id}?wait={millisconds}|nowait=true
```

You can perform the GET, POST, and DELETE operations on this job URL. GET returns whatever the Jakarta RESTful Web Services resource method invoked returns as a response if the job was completed. If the job has not completed, this GET will return a response code of 202, Accepted. Invoking GET does not remove the job, so it can be called multiple times. When RESTEasy's job queue gets full, it will evict the least recently used job from memory. Manual clean up can be performed by calling DELETE on the URI. POST does a read of the JOB response and will remove the JOB it has completed.

Both GET and POST allow the specification of a maximum wait time in milliseconds, a "wait" query parameter. Here's an example:

```
POST http://localhost/asynch/jobs/122?wait=3000
```

If a "wait" parameter is not specified, the GET or POST will not wait if the job is not complete.

NOTE!! While GET, DELETE, and PUT methods can be invoked asynchronously, this breaks the HTTP 1.1 contract of these methods. While these invocations may not change the state of the resource if invoked more than once, they do change the state of the server as new Job entries with each

invocation. To be a purist, stick with only invoking POST methods asynchronously.

Security NOTE! RESTEasy's role-based security (annotations) does not work with the Asynchronous Job Service. XML declarative security must be used within the web.xml file, because it is impossible to implement role-based security in a portable way.

NOTE. A `java.security.SecureRandom` object is used to generate unique job ids. For security purposes, the `SecureRandom` is periodically reseeded. By default, it is reseeded after 100 uses. This value may be configured with the servlet init parameter "resteasy.secure.random.max.use".

39.2. Oneway: Fire and Forget

RESTEasy also supports the notion of fire and forget. This will also return a 202, Accepted response, but no Job will be created. This is as simple as using the oneway query parameter instead of asynch. For example:

```
POST http://localhost/myservice?oneway=true
```

Security NOTE! RESTEasy role-based security (annotations) does not work with the Asynchronous Job Service. XML declarative security must be use within the web.xml file, because it is impossible to implement role-based security in a portable way.

39.3. Setup and Configuration

The Asynchronous Job Service must be enabled, as it is not turned on by default. If the relevant configuration properties are configured in web.xml, it would look like the following:

```
<web-app>
    <!-- enable the Asynchronous Job Service -->
    <context-param>
        <param-name>resteasy.async.job.service.enabled</param-name>
        <param-value>true</param-value>
    </context-param>

    <!-- The next context parameters are all optional.
         Their default values are shown as example param-values -->

    <!-- How many jobs results can be held in memory at once? -->
    <context-param>
        <param-name>resteasy.async.job.service.max.job.results</param-name>
        <param-value>100</param-value>
    </context-param>

    <!-- Maximum wait time on a job when a client is querying for it -->
    <context-param>
        <param-name>resteasy.async.job.service.max.wait</param-name>
        <param-value>300000</param-value>
    </context-param>
```

```
    <!-- Thread pool size of background threads that run the job -->
    <context-param>
        <param-name>resteasy.async.job.service.thread.pool.size</param-name>
        <param-value>100</param-value>
    </context-param>

    <!-- Set the base path for the Job uris -->
    <context-param>
        <param-name>resteasy.async.job.service.base.path</param-name>
        <param-value>/asynch/jobs</param-value>
    </context-param>

    ...
</web-app>
```

See Configuration for more information about application configuration.

Chapter 40. Asynchronous Injection

Pluggable Asynchronous Injection, also referred to as Asynch Injection, is a feature that allows users to create custom injectable asynchronous types. For example, it is possible to declare an injector for `Single<Foo>` and inject it into an endpoint as a class variable or as a method parameter using `@Context Foo`. The response will be made asynchronous automatically and the resource method will only be invoked once the `Single<Foo>` object is resolved to `Foo`. Resolution is done in a non-blocking manner.

Note. Asynch injection is only attempted at points where asynchronous injection is permitted, such as on resource creation and resource method invocation. It is not enabled at points where the API does not allow for suspending the request, for example on `ResourceContext.getResource(Foo.class)`.

40.1. org.jboss.resteasy.spi.ContextInjector Interface

The `org.jboss.resteasy.spi.ContextInjector` interface must be implemented on any custom async injector object. The implementation class must be annotated with the `@Provider` annotation.

```
/**
 * @param <WrappedType> A class that wraps a data type or data object
 *                            (e.g. Single<Foo>)
 * @param <UnwrappedType> The data type or data object declared in the
 *                              WrappedType (e.g. Foo)
*/
public interface ContextInjector<WrappedType, UnwrappedType> {
    /**
     * This interface allows users to create custom injectable asynchronous types.
     *
     * Asynch injection is only attempted at points where asynchronous injection is
     * permitted, such as on resource creation and resource method invocation. It
     * is not enabled at points where the API does not allow for suspending the
     * request
     *
     * @param rawType
     * @param genericType
     * @param annotations The annotation list is useful to parametrize the injection.
     * @return
     */
     WrappedType resolve(
                Class<? extends WrappedType> rawType,
                Type genericType,
                Annotation[] annotations);
    }
```

40.2. Single<Foo> Example

```
package my.test;
```

```
public class Foo {
   private String value = "PRE-SET-VALUE";

   public void setValue(String s) {
      this.value = s;
   }

   public String getValue() {
      return this.value;
   }
}
```

```
package my.test.asyc.resources;

import io.reactivex.Single;
import jakarta.ws.rs.ext.Provider;
import org.jboss.resteasy.spi.ContextInjector;
import my.test.Foo;

@Provider
public class FooAsychInjectorProvider implements
            ContextInjector<Single<Foo>,Foo> {

   public Single<Foo> resolve(Class<? extends Single<Foo>> rawType,
            Type genericType,
            Annotation[] annotations) {
      Foo value = new Foo();
      return Single.just(value.setValue("made it"));
   }
}
```

40.3. Async Injector With Annotations Example

A convenience interface to provide annotation parameter designators

```
@Retention(RUNTIME)
@Target({ FIELD, METHOD, PARAMETER })
public @interface AsyncInjectionPrimitiveInjectorSpecifier {
   enum Type {
      VALUE, NULL, NO_RESULT
   }

   Type value() default Type.VALUE;
}
```

```
@Provider
public class AsyncInjectionFloatInjector implements
            ContextInjector<CompletionStage<Float>, Float> {

   @Override
   public CompletionStage<Float> resolve(
      Class<? extends CompletionStage<Float>> rawType,
            Type genericType,
            Annotation[] annotations) {
       for (Annotation annotation : annotations) {
           if(annotation.annotationType() == AsyncInjectionPrimitiveInjectorSpecifier
.class) {
             AsyncInjectionPrimitiveInjectorSpecifier.Type value =
               ((AsyncInjectionPrimitiveInjectorSpecifier)annotation).value();
             switch(value) {
               case NO_RESULT:
                  return null;
               case NULL:
                  return CompletableFuture.completedFuture(null);
               case VALUE:
                  return CompletableFuture.completedFuture(4.2f);
            }
            break;
         }
       }
       return CompletableFuture.completedFuture(4.2f);
    }
}
```

Chapter 41. Reactive programming support

With version 2.1, the Jakarta RESTful Web Services specification (https://jcp.org/en/jsr/detail?id=370) takes its first steps into the world of Reactive Programming. There are many discussions of reactive programming on the internet, and a general introduction is beyond the scope of this document, but there are a few things worth discussing. Some primary aspects of reactive programming are the following:

- Reactive programming supports the declarative creation of rich computational structures. The representations of these structures can be passed around as first class objects such as method parameters and return values.
- Reactive programming supports both synchronous and asynchronous computation, but it is particularly helpful in facilitating, at a relatively high level of expression, asynchronous computation. Conceptually, asynchronous computation in reactive program typically involves pushing data from one entity to another, rather than polling for data.

41.1. CompletionStage

In Java 1.8 and Jakarta RESTful Web Services, the support for reactive programming is fairly limited. Java 1.8 introduces the interface `java.util.concurrent.CompletionStage`, and Jakarta RESTful Web Services mandates support for the `jakarta.ws.rs.client.CompletionStageRxInvoker`, which allows a client to obtain a response in the form of a `CompletionStage`.

One implementation of `CompletionStage` is the `java.util.concurrent.CompleteableFuture`. For example:

```
@Test
public void testCompletionStage() throws Exception {
   CompletionStageString stage = getCompletionStage();
   log.info("result: " + stage.toCompletableFuture().get());
}

private CompletionStageString getCompletionStage() {
   CompletableFutureString future = new CompletableFutureString();
   future.complete("foo");
   return future;
}
```

Here, a `CompleteableFuture` is created with the value "foo", and its value is extracted by the method `CompletableFuture.get()`. That's fine, but consider the altered version:

```
@Test
public void testCompletionStageAsync() throws Exception {
   log.info("start");
   CompletionStageString stage = getCompletionStageAsync();
   String result = stage.toCompletableFuture().get();
   log.info("do some work");
```

```
    log.info("result: " + result);
}

private CompletionStageString getCompletionStageAsync() {
    CompletableFutureString future = new CompletableFutureString();
    Executors.newCachedThreadPool().submit(() -> {sleep(2000); future.complete("foo");
});
    return future;
}

private void sleep(long l) {
    try {
        Thread.sleep(l);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

with output something like:

```
3:10:51 PM INFO: start
3:10:53 PM INFO: do some work
3:10:53 PM INFO: result: foo
```

It also works, but it illustrates the fact that `CompletableFuture.get()` is a blocking call. The `CompletionStage` is constructed and returned immediately, but the value isn't returned for two seconds. A version that is more in the spirit of the reactive style is:

```
@Test
public void testCompletionStageAsyncAccept() throws Exception {
    log.info("start");
    CompletionStageString stage = getCompletionStageAsync();
    stage.thenAccept((String s) -> log.info("s: " + s));
    log.info("do some work");
}
```

In this case, the lambda `(String s) → log.info("s: " + s)` is registered with the `CompletionStage` as a "subscriber", and, when the `CompletionStage` eventually has a value, that value is passed to the lambda. Note that the output is something like

```
3:23:05 INFO: start
3:23:05 INFO: do some work
3:23:07 INFO: s: foo
```

Executing a `CompletionStage` asynchronously is so common that there are several supporting convenience methods. For example:

```
@Test
public void testCompletionStageSupplyAsync() throws Exception {
   CompletionStageString stage = getCompletionStageSupplyAsync();
   stage.thenAccept((String s) -> log.info("s: " + s));
}

private CompletionStageString getCompletionStageSupplyAsync() {
   return CompletableFuture.supplyAsync(() -> "foo");
}
```

The static method `ComputableFuture.supplyAsync()` creates a `ComputableFuture`, the value of which is supplied asynchronously by the lambda `() → "foo"`, running, by default, in the default pool of `java.util.concurrent.ForkJoinPool`.

One final example illustrates a more complex computational structure:

```
@Test
public void testCompletionStageComplex() throws Exception {
   ExecutorService executor = Executors.newCachedThreadPool();
   CompletionStageString stage1 = getCompletionStageSupplyAsync1("foo", executor);
   CompletionStageString stage2 = getCompletionStageSupplyAsync1("bar", executor);
   CompletionStageString stage3 = stage1.thenCombineAsync(stage2, (String s, String t)
-> s + t, executor);
   stage3.thenAccept((String s) -> log.info("s: " + s));
}

private CompletionStageString getCompletionStageSupplyAsync1(String s, ExecutorService
executor) {
   return CompletableFuture.supplyAsync(() -> s, executor);
}
```

`stage1` returns "foo", `stage2` returns "bar", and `stage3`, which runs when both `stage1` and `stage2` have completed, returns the concatenation of "foo" and "bar". Note that, in this example, an explict `ExecutorService` is provided for asynchronous processing.

41.2. CompletionStage in Jakarta RESTful Web Services

On the client side, the Jakarta RESTful Web Services specification mandates an implementation of the interface `jakarta.ws.rs.client.CompletionStageRxInvoker`:

```
public interface CompletionStageRxInvoker extends RxInvoker<CompletionStage> {

    @Override
    CompletionStage<Response> get();

    @Override
    <T> CompletionStage<T> get(Class<T> responseType);
```

```
    @Override
    <T> CompletionStage<T> get(GenericTypeT responseType);
}
```

That is, there are invocation methods for the standard HTTP verbs, just as in the standard `jakarta.ws.rs.client.SyncInvoker`. A `CompletionStageRxInvoker` is obtained by calling `rx()` on a `jakarta.ws.rs.client.Invocation.Builder`, which extends `SyncInvoker`. For example,

```
Invocation.Builder builder = client.target(generateURL("/get/string")).request();
CompletionStageRxInvoker invoker = builder.rx(CompletionStageRxInvoker.class);
CompletionStageResponse stage = invoker.get();
Response response = stage.toCompletableFuture().get();
log.infof("result: %s", response.readEntity(String.class));
```

or

```
CompletionStageRxInvoker invoker = client.target(generateURL("/get/string")).request(
).rx(CompletionStageRxInvoker.class);
CompletionStageString stage = invoker.get(String.class);
String s = stage.toCompletableFuture().get();
log.infof("result: %s", s);
```

On the server side, the Jakarta RESTful Web Services specification requires support for resource methods with return type `CompletionStage<T>`. For example,

```
@GET
@Path("get/async")
public CompletionStageString longRunningOpAsync() {
   CompletableFutureString cs = new CompletableFuture();
   executor.submit(
           (Runnable) () -> {
               executeLongRunningOp();
               cs.complete("Hello async world!");
           });
   return cs;
}
```

The way to think about `longRunningOpAsync()` is that it is asynchronously creating and returning a `String`. After `cs.complete()` is called, the server will return the `String` "Hello async world!" to the client.

An important thing to understand is that the decision to produce a result asynchronously on the server and the decision to retrieve the result asynchronously on the client are independent. Suppose that there is also a resource method

```
@GET
@Path("get/sync")
public String longRunningOpSync() {
   return "Hello async world!";
}
```

Then all three of the following invocations are valid:

```
public void testGetStringAsyncAsync() throws Exception {
   CompletionStageRxInvoker invoker = client.target(generateURL("/get/async")).
request().rx();
   CompletionStageString stage = invoker.get(String.class);
   log.infof("s: %s", stage.toCompletableFuture().get());
}
```

```
public void testGetStringSyncAsync() throws Exception {
   Builder request = client.target(generateURL("/get/async")).request();
   String s = request.get(String.class);
   log.infof("s: %s", s);
}
```

and

```
public void testGetStringAsyncSync() throws Exception {
   CompletionStageRxInvoker invoker = client.target(generateURL("/get/sync")).request
().rx();
   CompletionStageString stage = invoker.get(String.class);
   log.infof("s: %s", stage.toCompletableFuture().get());
}
```



`CompletionStage` in Jakarta RESTful Web Services is also discussed in the chapter Asynchronous HTTP Request Processing.



Since running code asynchronously is so common in this context, it is worth pointing out that objects obtained by way of the annotation `@Context` or by way of calling `ResteasyContext.getContextData()` are sensitive to the executing thread. For example, given resource method

```
@Resource
private ManagedExecutorService executor;

@GET
@Path("test")
@Produces("text/plain")
```

```
public CompletionStageString text(@Context HttpRequest request) {
   CompletableFutureString cs = new CompletableFuture();
   executor.submit(
           (Runnable) () -> {
               try {
                   System.out.println("request (async): " + request);
                   cs.complete("hello");
               } catch (Exception e) {
                   e.printStackTrace();
               }
           });
   return cs;
}
```

the output will look something like

```
application (inline):
org.jboss.resteasy.experiment.Test1798CompletionStage$TestApp@23c57474
request (inline):
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@2ce2
3138
application (async): null
org.jboss.resteasy.spi.LoggableFailure: RESTEASY003880: Unable to find
contextual data of type: org.jboss.resteasy.spi.HttpRequest
```

The point is that it is the developer's responsibility to extract information from these context objects in advance. For example:

```
@Resource
private ManagedExecutorService executor;

@Application
private Application application;

@GET
@Path("test")
@Produces("text/plain")
public CompletionStageString text(@Context HttpRequest request) {
   System.out.println("request (inline): " + request);
   CompletableFutureString cs = new CompletableFuture();
   final String httpMethodFinal = request.getHttpMethod();
   final Map<String, Object> mapFinal = application.getProperties();
   executor.submit(
           (Runnable) () -> {
               System.out.println("httpMethod (async): " +
httpMethodFinal);
               System.out.println("map (async): " + mapFinal);
               cs.complete("hello");
           });
```

```
   return cs;
}
```

Alternatively, RESTEasy's support of MicroProfile Context Propagation can be used by using `ThreadContext.contextualRunnable` around a `Runnable`, which will take care of capturing and restoring all registered contexts (the `org.jboss.resteasy.microprofile:microprofile-context-propagation` module will need to be imported):

```
@Resource
private ManagedExecutorService executor;

@GET
@Path("test")
@Produces("text/plain")
public CompletionStageString text(@Context HttpRequest request) {
   System.out.println("request (inline): " + request);
   CompletableFutureString cs = new CompletableFuture();
   ThreadContext threadContext = ThreadContext.builder()
                                                .propagated
(ThreadContext.ALL_REMAINING)
                                                .unchanged()
                                                .cleared()
                                                .build();
   executor.submit(
         threadContext.contextualRunnable((Runnable) () -> {
             try {
                 System.out.println("request (async): " + request);
                 cs.complete("hello");
             } catch (Exception e) {
                 e.printStackTrace();
             }
         }));
   return cs;
}
```

As another alternative the RESTEasy SPI's `ContextualExecutor` can be used if the MicroProfile Context Propagation is not available. This requires a dependency on `org.jboss.resteasy:resteasy-core`.

```
@GET
@Path("test")
@Produces(MediaType.TEXT_PLAIN)
public CompletionStageString text(@Context UriInfo uriInfo) {
  CompletableFutureString cs = new CompletableFuture();
  ExecutorService executor = ContextualExecutors.threadPool();
  executor.submit(() -> {
     try {
```

```
            cs.complete("hello from: " + uriInfo.getAbsolutePath());
        } catch (Exception e) {
            e.printStackTrace();
        }
    });
    return cs;
}
```

41.3. Beyond CompletionStage

The picture becomes more complex and interesting when sequences are added. A `CompletionStage` holds no more than one potential value, but other reactive objects can hold multiple, even unlimited, values. Currently, most Java implementations of reactive programming are based on the project Reactive Streams (http://www.reactive-streams.org/), which defines a set of four interfaces and a specification, in the form of a set of rules, describing how they interact:

```
public interface Publisher<T> {
    void subscribe(Subscriber<? super T> s);
}

public interface Subscriber<T> {
    void onSubscribe(Subscription s);
    void onNext(T t);
    void onError(Throwable t);
    void onComplete();
}

public interface Subscription {
    void request(long n);
    void cancel();
}

public interface Processor<T, R> extends Subscriber<T>, Publisher<R> {
}
```

A `Producer` pushes objects to a `Subscriber`, a `Subscription` mediates the relationship between the two, and a `Processor` which is derived from both, helps to construct pipelines through which objects pass.

One important aspect of the specification is flow control, the ability of a `Suscriber` to control the load it receives from a `Producer` by calling `Suscription.request()`. The general term in this context for flow control is **backpressure**.

There are a number of implementations of Reactive Streams, including

1. RxJava: https://github.com/ReactiveX/RxJava/tree/1.x
2. RxJava 2: https://github.com/ReactiveX/RxJava/tree/2.x

3. RxJava 3: https://github.com/ReactiveX/RxJava
4. Reactor: http://projectreactor.io/
5. Flow : https://community.oracle.com/docs/DOC-1006738/

RESTEasy currently supports RxJava (deprecated), RxJava2 and Reactor.

41.4. Pluggable reactive types: RxJava 2 in RESTEasy

Jakarta RESTful Web Services doesn't currently require support for any Reactive Streams implementations, but it does allow for extensibility to support various reactive libraries. RESTEasy's optional module `resteasy-rxjava2` adds support for RxJava 2.

More in details, `resteasy-rxjava2` contributes support for reactive types `io.reactivex.Single`, `io.reactivex.Flowable`, and `io.reactivex.Observable`. Of these, `Single` is similar to `CompletionStage` in that it holds at most one potential value. `Flowable` implements `io.reactivex.Publisher`, and `Observable` is very similar to `Flowable` except that it doesn't support backpressure. If importing `resteasy-rxjava2`, you can start returning these reactive types from your resource methods on the server side and receiving them on the client side.



When using RESTEasy's modules for RxJava, the reactive contexts are automatically propagated to all supported RxJava types, which means there is no need to worry about `@Context` injection not working within RxJava lambdas, contrary to `CompletionStage` (as previously noted).

41.4.1. Server side

Given the class `Thing`, which can be represented in JSON:

```
public class Thing {

   private String name;

   public Thing() {
   }

   public Thing(String name) {
      this.name = name;
   }
}
```

the method `postThingList()` in the following is a valid resource method:

```
@POST
@Path("post/thing/list")
@Produces(MediaType.APPLICATION_JSON)
@Stream
public Flowable<List<Thing>> postThingList(String s) {
```

```
   return buildFlowableThingList(s, 2, 3);
}

static Flowable<List<Thing>> buildFlowableThingList(String s, int listSize, int
elementSize) {
   return Flowable.create(
      new FlowableOnSubscribeListThing() {

         @Override
         public void subscribe(FlowableEmitter<List<Thing>> emitter) throws Exception
{
            for (int i = 0; i listSize; i++) {
               List<Thing> list = new ArrayList<>();
               for (int j = 0; j < elementSize; j++) {
                  list.add(new Thing(s));
               }
               emitter.onNext(list);
            }
            emitter.onComplete();
         }
      },
      BackpressureStrategy.BUFFER);
}
```

The method `buildFlowableThingList()` deserves some explanation. First,

```
Flowable<List<Thing>> Flowable.create(FlowableOnSubscribe<List<Thing>> source,
BackpressureStrategy mode);
```

creates a `Flowable<List<Thing>>` by describing what should happen when the `Flowable<List<Thing>>` is subscribed to. `FlowableEmitter<List<Thing>>` extends ` io.reactivex.Emitter<List<Thing>>`:

```
/**
 * Base interface for emitting signals in a push-fashion in various generator-like
source
 * operators (create, generate).
 *
 * @param T the value type emitted
 */
public interface Emitter<T> {

    /**
     * Signal a normal value.
     * @param value the value to signal, not null
     */
    void onNext(@NonNull T value);

    /**
     * Signal a Throwable exception.
```

```
     * @param error the Throwable to signal, not null
     */
    void onError(@NonNull Throwable error);

    /**
     * Signal a completion.
     */
    void onComplete();
}
```

and `FlowableOnSubscribe` uses a `FlowableEmitter` to send out values from the `Flowable<List<Thing>>`:

```
/**
 * A functional interface that has a {@code subscribe()} method that receives
 * an instance of a {@link FlowableEmitter} instance that allows pushing
 * events in a backpressure-safe and cancellation-safe manner.
 *
 * @param T the value type pushed
 */
public interface FlowableOnSubscribe<T> {

    /**
     * Called for each Subscriber that subscribes.
     * @param e the safe emitter instance, never null
     * @throws Exception on error
     */
    void subscribe(@NonNull FlowableEmitter<T> e) throws Exception;
}
```

So, what will happen when a subscription to the `Flowable<List<Thing>>` is created is, the `FlowableEmitter.onNext()` will be called, once for each `<List<Thing>>` created, followed by a call to `FlowableEmitter.onComplete()` to indicate that the sequence has ended. Under the covers, RESTEasy subscribes to the `Flowable<List<Thing>>` and handles each element passed in by way of `onNext()`.

41.4.2. Client side

On the client side, Jakarta RESTful Web Services supports extensions for reactive classes by adding the method

```
/**
 * Access a reactive invoker based on a {@link RxInvoker} subclass provider. Note
 * that corresponding {@link RxInvokerProvider} must be registered in the client
runtime.
 *
 * This method is an extension point for Jakarta RESTful Web Services implementations
to support other types
 * representing asynchronous computations.
 *
```

```
     * @param clazz {@link RxInvoker} subclass.
     * @return reactive invoker instance.
     * @throws IllegalStateException when provider for given class is not registered.
     * @see jakarta.ws.rs.client.Client#register(Class)
     * @since 2.1
     */
    public <T extends RxInvoker> T rx(ClassT clazz);
```

to interface `jakarta.ws.rs.client.Invocation.Builder`. Resteasy module `resteasy-rxjava2` adds support for classes:

1. `org.jboss.resteasy.rxjava2.SingleRxInvoker`
2. `org.jboss.resteasy.rxjava2.FlowableRxInvoker`
3. `org.jbosss.resteasy.rxjava2.ObservableRxInvoker`

which allows accessing `Single`s, `Observable`s, and `Flowable`s on the client side.

For example, given the resource method `postThingList()` above, a `Flowable<List<Thing>>` can be retrieved from the server by calling

```
@SuppressWarnings("unchecked")
@Test
public void testPostThingList() throws Exception {
   CountDownLatch latch = new CountdownLatch(1);
   FlowableRxInvoker invoker = client.target(generateURL("/post/thing/list")).request
().rx(FlowableRxInvoker.class);
   Flowable<List<Thing>> flowable = (Flowable<List<Thing>>) invoker.post(Entity.
entity("a", MediaType.TEXT_PLAIN_TYPE), new GenericType<List<Thing>>() {});
   flowable.subscribe(
         (List<?> l) -> thingListList.add(l),
         (Throwable t) -> latch.countDown(),
         () -> latch.countDown());
   latch.await();
   Assert.assertEquals(aThingListList, thingListList);
}
```

where `aThingListList` is

```
[[Thing[a], Thing[a], Thing[a]], [Thing[a], Thing[a], Thing[a]]]
```

Note the call to `Flowable.suscribe()`. On the server side, RESTEasy subscribes to a returning `Flowable` in order to receive its elements and send them over the wire. On the client side, the user subscribes to the `Flowable` in order to receive its elements and do whatever it wants to with them. In this case, three lambdas determine what should happen 1) for each element, 2) if a `Throwable` is thrown, and 3) when the `Flowable` is done passing elements.

41.4.3. Representation on the wire

Neither Reactive Streams nor Jakarta RESTful Web Services have anything to say about representing reactive types on the network. RESTEasy offers a number of representations, each suitable for different circumstances. The wire protocol is determined by 1) the presence or absence of the `@Stream` annotation on the resource method, and 2) the value of the `value` field in the `@Stream` annotation:

```
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface Stream {
   public enum MODE {RAW, GENERAL};
   public String INCLUDE_STREAMING_PARAMETER = "streaming";
   public MODE value() default MODE.GENERAL;
   public boolean includeStreaming() default false;
}
```

Note that `MODE.GENERAL` is the default value, so `@Stream` is equivalent to `@Stream(Stream.MODE.GENERAL)`.

- No `@Stream` annotation on the resource method:

  `java.util.List` entity and send to the client.

- `@Stream(Stream.MODE.GENERAL)`

  This case uses a variant of the SSE format, modified to eliminate some restrictions inherent in SSE. (See the specification at https://html.spec.whatwg.org/multipage/server-sent-events.html for details.) In particular, 1) SSE events are meant to hold text data, represented in character set UTF-8. In the general streaming mode, certain delimiting characters in the data ('\r', '\n', and '\') are escaped so that arbitrary binary data can be transmitted. Also, 2) the SSE specification requires the client to reconnect if it gets disconnected. If the stream is finite, reconnecting will induce a repeat of the stream, so SSE is really meant for unlimited streams. In general streaming mode, the client will close, rather than automatically reconnect, at the end of the stream. It follows that this mode is suitable for finite streams.

  

  <emphasis role="bold">Note. </emphasis> The Content-Type header in general streaming mode is set to

  ```
  applicaton/x-stream-general;"element-type=element-type"
  ```

  where element-type is the media type of the data elements in the stream. The element media type is derived from the @Produces annotation. For example,

  ```
  @GET
  @Path("flowable/thing")
  @Stream
  @Produces("application/json")
  ```

```
public FlowableThing getFlowable() {}
```

induces the media type

```
application/x-stream-general;"element-type=application/json"
```

which describes a stream of JSON elements.

- `@Stream(Stream.MODE.RAW)`

In this case each value is written directly to the wire, without any formatting, as it becomes available. This is most useful for values that can be cut in pieces, such as strings, bytes, buffers, etc., and then re-concatenated on the client side. Note that without delimiters as in general mode, it isn't possible to reconstruct something like `List<List<String>>`



The Content-Type header in raw streaming mode is derived from the `@Produces` annotation. The `@Stream` annotation offers the possibility of an optional `MediaType` parameter called "streaming". The point is to be able to suggest that the stream of data emanating from the server is unbounded, i.e., that the client shouldn't try to read it all as a single byte array, for example. The parameter is set by explicitly setting the `@Stream` parameter `includeStreaming()` to `true`. For example,

```
@GET
@Path("byte/default")
@Produces("application/octet-stream;x=y")
@Stream(Stream.MODE.RAW)
public FlowableByte aByteDefault() {
  return Flowable.fromArray((byte) 0, (byte) 1, (byte) 2);
}
```

induces the `MediaType` "application/octet-stream;x=y", and

```
@GET
@Path("byte/true")
@Produces("application/octet-stream;x=y")
@Stream(value=Stream.MODE.RAW, includeStreaming=true)
public FlowableByte aByteTrue() {
  return Flowable.fromArray((byte) 0, (byte) 1, (byte) 2);
}
```

induces the `MediaType` "application/octet-stream;x=y;streaming=true".

Note that browsers such as Firefox and Chrome seem to be comfortable with reading unlimited streams without any additional hints.

41.4.4. Examples

Example 1.

```
@POST
@Path("post/thing/list")
@Produces(MediaType.APPLICATION_JSON)
@Stream(Stream.MODE.GENERAL)
public Flowable<List<Thing>> postThingList(String s) {
   return buildFlowableThingList(s, 2, 3);
}

@SuppressWarnings("unchecked")
@Test
public void testPostThingList() throws Exception {
   CountDownLatch latch = new CountdownLatch(1);
   FlowableRxInvoker invoker = client.target(generateURL("/post/thing/list")).request
().rx(FlowableRxInvoker.class);
   Flowable<List<Thing>> flowable = (Flowable<List<Thing>>) invoker.post(Entity.
entity("a", MediaType.TEXT_PLAIN_TYPE), new GenericType<List<Thing>>() {});
   flowable.subscribe(
         (List<?> l) -> thingListList.add(l),
         (Throwable t) -> latch.countDown(),
         () -> latch.countDown());
   latch.await();
   Assert.assertEquals(aThingListList, thingListList);
}
```

This is the example given previously, except that the mode in the `@Stream` annotation (which defaults to MODE.GENERAL) is given explicitly. In this scenario, the `Flowable` emits `List<Thing>` elements on the server, they are transmitted over the wire as SSE events:

```
data: [{"name":"a"},{"name":"a"},{"name":"a"}]
data: [{"name":"a"},{"name":"a"},{"name":"a"}]
```

and the `FlowableRxInvoker` reconstitutes a `Flowable` on the client side.

Example 2.

```
@POST
@Path("post/thing/list")
@Produces(MediaType.APPLICATION_JSON)
public Flowable<List<Thing>> postThingList(String s) {
   return buildFlowableThingList(s, 2, 3);
}

@Test
public void testPostThingList() throws Exception {
```

```
   Builder request = client.target(generateURL("/post/thing/list")).request();
   List<List<Thing>> list = request.post(Entity.entity("a", MediaType.TEXT_PLAIN_TYPE
), new GenericTypeList<List<Thing>>() {});
   Assert.assertEquals(aThingListList, list);
}
```

In this scenario, in which the resource method has no `@Stream` annotation, the `Flowable` emits stream elements which are accumulated by the server until the `Flowable` is done, at which point the entire JSON list is transmitted over the wire:

```
[[{"name":"a"},{"name":"a"},{"name":"a"}],[{"name":"a"},{"name":"a"},{"name":"a"}]]
```

and the list is reconstituted on the client side by an ordinary invoker.

Example 3.

```
@GET
@Path("get/bytes")
@Produces(MediaType.APPLICATION_OCTET_STREAM)
@Stream(Stream.MODE.RAW)
public Flowable<byte[]> getBytes() {
   return Flowable.create(
      new FlowableOnSubscribe<byte[]>() {

         @Override
         public void subscribe(FlowableEmitterbyte[] emitter) throws Exception {
            for (int i = 0; i < 3; i++) {
               byte[] b = new byte[10];
               for (int j = 0; j  <10; j++) {
                  b[j] = (byte) (i + j);
               }
               emitter.onNext(b);
            }
            emitter.onComplete();
         }
      },
      BackpressureStrategy.BUFFER);
}

@Test
public void testGetBytes() throws Exception {
   Builder request = client.target(generateURL("/get/bytes")).request();
   InputStream is = request.get(InputStream.class);
   int n = is.read();
   while (n < -1) {
      System.out.print(n);
      n = is.read();
   }
```

```
}
```

Here, the byte arrays are written to the network as they are created by the `Flowable`. On the network, they are concatenated, so the client sees one stream of bytes.

41.4.5. Rx and SSE

Since general streaming mode and SSE share minor variants of the same wire protocol, they are, modulo the SSE restriction to character data, interchangeable. That is, an SSE client can connect to a resource method that returns a `Flowable` or an `Observable`, and a `FlowableRxInvoker`, for example, can connect to an SSE resource method.



SSE requires a `@Produces("text/event-stream")` annotation, so, unlike the cases of raw and general streaming, the element media type cannot be derived from the `@Produces` annotation. To solve this problem, Resteasy introduces the

```
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface SseElementType {
   String value();
}
```

annotation, from which the element media type is derived.

Example 1.

```
@GET
@Path("eventStream/thing")
@Produces("text/event-stream")
@SseElementType("application/json")
public void eventStreamThing(@Context SseEventSink eventSink, @Context Sse sse) {
   new ScheduledThreadPoolExecutor(5).execute(() -> {
      try (SseEventSink sink = eventSink) {
         OutboundSseEvent.Builder  builder = sse.newEventBuilder();
         eventSink.send(builder.data(new Thing("e1")).build());
         eventSink.send(builder.data(new Thing("e2")).build());
         eventSink.send(builder.data(new Thing("e3")).build());
      }
   });
}

@SuppressWarnings("unchecked")
@Test
public void testFlowableToSse() throws Exception {
   CountDownLatch latch = new CountDownLatch(1);
   final AtomicInteger errors = new AtomicInteger(0);
   FlowableRxInvoker invoker = client.target(generateURL("/eventStream/thing"))
.request().rx(FlowableRxInvoker.class);
```

```
    FlowableThing flowable = (FlowableThing) invoker.get(Thing.class);
    flowable.subscribe(
       (Thing t) -> thingList.add(t),
       (Throwable t) -> errors.incrementAndGet(),
       () -> latch.countDown());
    boolean waitResult = latch.await(30, TimeUnit.SECONDS);
    Assert.assertTrue("Waiting for event to be delivered has timed out.", waitResult);
    Assert.assertEquals(0, errors.get());
    Assert.assertEquals(eThingList, thingList);
}
```

Here, a `FlowableRxInvoker` is connecting to an SSE resource method. On the network, the data looks like

```
data: {"name":"e1"}
data: {"name":"e2"}
data: {"name":"e3"}
```

Note that the character data is suitable for an SSE resource method.

Also, note that the `eventStreamThing()` method in this example induces the media type

```
text/event-stream;element-type="application/json"
```

Example 2.

```
@GET
@Path("flowable/thing")
@Produces("text/event-stream")
@SseElementType("application/json")
public Flowable<Thing> flowableSSE() {
   return Flowable.create(
      new FlowableOnSubscribe<Thing>() {

         @Override
         public void subscribe(FlowableEmitter<Thing> emitter) throws Exception {
            emitter.onNext(new Thing("e1"));
            emitter.onNext(new Thing("e2"));
            emitter.onNext(new Thing("e3"));
            emitter.onComplete();
         }
      },
      BackpressureStrategy.BUFFER);
}

@Test
public void testSseToFlowable() throws Exception {
```

```
   final CountDownLatch latch = new CountDownLatch(3);
   final AtomicInteger errors = new AtomicInteger(0);
   WebTarget target = client.target(generateURL("/flowable/thing"));
   SseEventSource msgEventSource = SseEventSource.target(target).build();
   try (SseEventSource eventSource = msgEventSource) {
      eventSource.register(
         event -> {thingList.add(event.readData(Thing.class, MediaType
.APPLICATION_JSON_TYPE)); latch.countDown();},
         ex -> errors.incrementAndGet());
      eventSource.open();

      boolean waitResult = latch.await(30, TimeUnit.SECONDS);
      Assert.assertTrue("Waiting for event to be delivered has timed out.",
waitResult);
      Assert.assertEquals(0, errors.get());
      Assert.assertEquals(eThingList, thingList);
   }
}
```

Here, an SSE client is connecting to a resource method that returns a `Flowable`. Again, the server is sending character data, which is suitable for the SSE client, and the data looks the same on the network.

41.4.6. To stream or not to stream

Whether or not it is appropriate to stream a list of values is a judgment call. Certainly, if the list is unbounded, then it isn't practical, or even possible, perhaps, to collect the entire list and send it at once. In other cases, the decision is less obvious.

Case 1. Suppose that all of the elements are producible quickly. Then the overhead of sending them independently is probably not worth it.

Case 2. Suppose that the list is bounded but the elements will be produced over an extended period of time. Then returning the initial elements when they become available might lead to a better user experience.

Case 3. Suppose that the list is bounded and the elements can be produced in a relatively short span of time but only after some delay. Here is a situation that illustrates the fact that asynchronous reactive processing and streaming over the network are independent concepts. In this case it's worth considering having the resource method return something like `CompletionStage<List<Thing>>` rather than `Flowable<List<Thing>>`. This has the benefit of creating the list asynchronously but, once it is available, sending it to the client in one piece.

41.5. Proxies

Proxies, discussed in RESTEasy Proxy Framework, are a RESTEasy extension that supports a natural programming style in which generic Jakarta RESTful Web Services invoker calls are replaced by application specific interface calls. The proxy framework is extended to include both `CompletionStage` and the RxJava2 types `Single`, `Observable`, and `Flowable`.

Example 1.

```
@Path("")
public interface RxCompletionStageResource {

   @GET
   @Path("get/string")
   @Produces(MediaType.TEXT_PLAIN)
   CompletionStage<String> getString();
}

@Path("")
public class RxCompletionStageResourceImpl {

   @GET
   @Path("get/string")
   @Produces(MediaType.TEXT_PLAIN)
   public CompletionStage<String> getString() { . }
}

public class RxCompletionStageProxyTest {

   private static ResteasyClient client;
   private static RxCompletionStageResource proxy;

   static {
      client = (ResteasyClient)ClientBuilder.newClient();
      proxy = client.target(generateURL("/")).proxy(RxCompletionStageResource.class);
   }

   @Test
   public void testGet() throws Exception {
      CompletionStage<String> completionStage = proxy.getString();
      Assert.assertEquals("x", completionStage.toCompletableFuture().get());
   }
}
```

Example 2.

```
public interface Rx2FlowableResource {

   @GET
   @Path("get/string")
   @Produces(MediaType.TEXT_PLAIN)
   @Stream
   public Flowable<String> getFlowable();
}

@Path("")
```

```
public class Rx2FlowableResourceImpl {

   @GET
   @Path("get/string")
   @Produces(MediaType.TEXT_PLAIN)
   @Stream
   public Flowable<String> getFlowable() {  }
}

public class Rx2FlowableProxyTest {

   private static ResteasyClient client;
   private static Rx2FlowableResource proxy;

   static {
      client = (ResteasyClient)ClientBuilder.newClient();
      proxy = client.target(generateURL("/")).proxy(Rx2FlowableResource.class);
   }

   @Test
   public void testGet() throws Exception {
      Flowable<String> flowable = proxy.getFlowable();
      flowable.subscribe(
         (String o) -> stringList.add(o),
         (Throwable t) -> errors.incrementAndGet(),
         () -> latch.countDown());
      boolean waitResult = latch.await(30, TimeUnit.SECONDS);
      Assert.assertTrue("Waiting for event to be delivered has timed out.",
waitResult);
      Assert.assertEquals(0, errors.get());
      Assert.assertEquals(xStringList, stringList);
   }
}
```

41.6. Adding extensions

RESTEasy implements a framework that supports extensions for additional reactive classes. To understand the framework, it is necessary to understand the existing support for `CompletionStage` and other reactive classes.

Server side. When a resource method returns a `CompletionStage`, RESTEasy subscribes to it using the class `org.jboss.resteasy.core.AsyncResponseConsumer.CompletionStageResponseConsumer`. When the `CompletionStage` completes, it calls `CompletionStageResponseConsumer.accept()`, which sends the result back to the client.

Support for `CompletionStage` is built in to RESTEasy, but it's not hard to extend that support to a class like `Single` by providing a mechanism for transforming a `Single` into a `CompletionStage`. In module resteasy-rxjava2, that mechanism is supplied by `org.jboss.resteasy.rxjava2.SingleProvider`, which implements interface `org.jboss.resteasy.spi.AsyncResponseProviderSingle?`:

```
public interface AsyncResponseProvider<T> {
   CompletionStage<T> toCompletionStage(T asyncResponse);
}
```

Given SingleProvider, RESTEasy can take a Single, transform it into a CompletionStage, and then use CompletionStageResponseConsumer to handle the eventual value of the Single.

Similarly, when a resource method returns a streaming reactive class like Flowable, RESTEasy subscribes to it, receives a stream of data elements, and sends them to the client. AsyncResponseConsumer has several supporting classes, each of which implements a different mode of streaming. For example, AsyncResponseConsumer.AsyncGeneralStreamingSseResponseConsumer handles general streaming and SSE streaming. Subscribing is done by calling org.reactivestreams.Publisher.subscribe(), so a mechanism is needed for turning, say, a Flowable into a Publisher. That is, an implementation of org.jboss.resteasy.spi.AsyncStreamProviderFlowable is called for, where AsyncStreamProvider is defined:

```
public interface AsyncStreamProvider<T> {
   Publisher<T> toAsyncStream(T asyncResponse);
}
```

In module resteasy-rxjava2, org.jboss.resteasy.FlowableProvider provides that mechanism for Flowable. \[Actually, that's not too hard since, in rxjava2, a Flowable *is* a Provider.]

So, on the server side, adding support for other reactive types can be done by declaring a @Provider for the interface AsyncStreamProvider (for streams) or AsyncResponseProvider (for single values), which both have a single method to convert the new reactive type into (respectively) a Publisher (for streams) or a CompletionStage (for single values).

Client side. The Jakarta RESTful Web Services specification imposes two requirements for support of reactive classes on the client side:

1. support for CompletionStage in the form of an implementation of the interface jakarta.ws.rs.client.CompletionStageRxInvoker, and
2. extensibility in the form of support for registering providers that implement

   ```
   public interface RxInvokerProvider<T> extends RxInvoker {
       boolean isProviderFor(Class<T> clazz);
       T getRxInvoker(SyncInvoker syncInvoker, ExecutorService executorService);
   }
   ```

   Once an RxInvokerProvider is registered, an RxInvoker can be requested by calling the jakarta.ws.rs.client.Invocation.Builder method

   ```
   public <T extends RxInvoker> T rx(Class<T> clazz);
   ```

That `RxInvoker` can then be used for making an invocation that returns the appropriate reactive class. For example,

```
FlowableRxInvoker invoker = client.target(generateURL("/get/string")).request().rx
(FlowableRxInvoker.class);
FlowableString flowable = (FlowableString) invoker.get();
```

RESTEasy provides partial support for implementing a `RxInvoker`. For example, `SingleProvider`, mentioned above, also implements `org.jboss.resteasy.spi.AsyncClientResponse<Provider<Single<?>>>`, where `AsyncClientResponseProvider` is defined

```
public interface AsyncClientResponseProvider<T> {
   T fromCompletionStage(CompletionStage<?> completionStage);
}
```

`SingleProvider` has the ability to turn a `CompletionStage` into a `Single` is used in the implementation of `org.jboss.resteasy.rxjava2.SingleRxInvokerImpl`.

The same concept might be useful in implementing other `RxInvoker` implementations. Note, though, that `ObservableRxInvokerImpl` and `FlowableRxInvokerImpl` in module resteasy-rxjava2 are each derived directly from the SSE implementation.

Chapter 42. Jakarta RESTful Web Services SeBootstrap

In Jakarta RESTful Web Services 3.1 the new `jakarta.ws.rs.SeBootstrap` API was introduced to run Jakarta RESTful Web Services applications in a Java SE environment. As an implementation of the specification RESTEasy includes an implementation for this API.

42.1. Overview

Its suggested by default that the `org.jboss.resteasy:resteasy-undertow-cdi` implementation be used. However, the other `org.jboss.resteasy.plugins.server.embedded.EmbeddedServer` will work excluding the `org.jboss.resteasy:resteasy-jdk-http`.

The `org.jboss.resteasy:resteasy-undertow-cdi` implementation also uses Weld to create a CDI container. This allows the CDI in the SE environment to be used. If CDI is not required or desired the `org.jboss.resteasy:resteasy-undertow` implementation could be used instead.

Example POM Dependencies:

```
<dependencies>
    <dependency>
        <groupId>jakarta.ws.rs</groupId>
        <artifactId>jakarta.ws.rs-api</artifactId>
    </dependency>
    <dependency>
        <groupId>org.jboss.resteasy</groupId>
        <artifactId>resteasy-core</artifactId>
    </dependency>
    <dependency>
        <groupId>org.jboss.resteasy</groupId>
        <artifactId>resteasy-client</artifactId>
    </dependency>
    <dependency>
        <groupId>org.jboss.resteasy</groupId>
        <artifactId>resteasy-undertow-cdi</artifactId>
    </dependency>
</dependencies>
```

Its also suggested that if you do not explicitly define the resources to be used in your application that you use the `io.smallrye:jandex-maven-plugin` to create a Jandex Index. Without this the class path will be scanned for resources which could have significant performance impacts.

```
<plugin>
    <groupId>io.smallrye</groupId>
    <artifactId>jandex-maven-plugin</artifactId>
    <executions>
```

```
        <execution>
            <id>make-index</id>
            <goals>
                <goal>jandex</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

42.2. Usage

Example of using the `jakarta.ws.rs.SeBootstrap` API:

```
public static void main(final String[] args) throws Exception {
    SeBootstrap.start(ExampleApplication.class, SeBootstrap.Configuration.builder()
                    .build())
            .thenApply((instance) -> {
                try (Client client = ClientBuilder.newClient()) {
                    final WebTarget target = client.target(instance.configuration()
.baseUriBuilder());
                    final Response response = client.target(instance.configuration()
                                    .baseUriBuilder()
                                    .path("/api/product/widget"))
                            .request()
                            .get();
                    System.out.printf("Response: %d - %s%n", response.getStatus(),
response.readEntity(String.class));
                }
                return instance;
            })
            .whenComplete((instance, t) -> instance.stop());
}
```

42.3. Configuration Options

Configuration options are represented by the `org.jboss.resteasy.core.se.ConfigurationOption` enum. This enum includes all the supported configuration options.

```
final SeBootstrap.Configuration configuration = SeBootstrap.Configuration.builder()
        .port(8443)
        .protocol("HTTPS")
        .property(ConfigurationOption.JANDEX_CLASS_PATH_FILTER.key(), Index.of
(ItemResource.class, OrderResource.class));
```

42.4. Custom EmbeddedServer Implementations

RESTEasy uses a ServiceLoader to discover `EmbeddedServer` implementations. Custom embedded server implementations can be created by implementing the `org.jboss.resteasy.plugins.server.embedded.EmbeddedServer` interface and registering it via the ServiceLoader mechanism.

42.4.1. Constructor Requirements

Custom `EmbeddedServer` implementations can provide one or both of the following constructors:

- A constructor accepting `jakarta.ws.rs.SeBootstrap.Configuration` - allows access to configuration during initialization
- A no-argument constructor - used as a fallback if the Configuration constructor is not available

The ServiceLoader will prefer the Configuration constructor when available, allowing implementations to use configuration information (such as host and port) during object construction.

Example:

```
public class CustomEmbeddedServer implements EmbeddedServer {

    // Preferred: Constructor with Configuration
    public CustomEmbeddedServer(SeBootstrap.Configuration configuration) {
        // Can use configuration.host(), configuration.port(), etc.
    }

    // Fallback: No-argument constructor
    public CustomEmbeddedServer() {
        // Default initialization
    }

    @Override
    public void start(Configuration configuration) {
        // Start the server
    }

    @Override
    public void stop() {
        // Stop the server
    }

    @Override
    public ResteasyDeployment getDeployment() {
        // Return the deployment
    }
}
```

Register the implementation in `META-INF/services/org.jboss.resteasy.plugins.server.embedded.EmbeddedServer`:

```
com.example.CustomEmbeddedServer
```

Chapter 43. Embedded Containers



The `org.jboss.resteasy.plugins.server.embedded.EmbeddedJaxrsServer` is deprecated and replaced by `org.jboss.resteasy.plugins.server.embedded.EmbeddedServer` and `jakarta.ws.rs.SeBootstrap.Instance`. The preference should be to use the SeBootstrap API

RESTEasy has a few different plugins for different embeddable HTTP and/or Servlet containers, if using RESTEasy in a test environment or within an environment where you do not want a Servlet engine dependency.

43.1. Undertow

Undertow is a new Servlet Container that is used by WildFly (JBoss Community Server). It can be embedded if desired. Here's a test that shows it in action.

```
import io.undertow.servlet.api.DeploymentInfo;
import org.jboss.resteasy.plugins.server.undertow.UndertowJaxrsServer;
import org.jboss.resteasy.test.TestPortProvider;
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.Test;

import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.client.Client;
import jakarta.ws.rs.client.ClientBuilder;
import jakarta.ws.rs.core.Application;
import java.util.Set;

/**
 * @author <a href="mailto:bill@burkecentral.com">Bill Burke</a>
 * @version $Revision: 1 $
 */
public class UndertowTest {
   private static UndertowJaxrsServer server;

   @Path("/test")
   public static class Resource {
      @GET
      @Produces("text/plain")
      public String get() {
         return "hello world";
      }
```

```
    }

    @ApplicationPath("/base")
    public static class MyApp extends Application {
       @Override
       public Set<Class<?>> getClasses() {
          return Set.of(Resource.class);
       }
    }

    @BeforeAll
    public static void init() throws Exception {
       server = new UndertowJaxrsServer().start();
    }

    @AfterAll
    public static void stop() throws Exception {
       server.stop();
    }

    @Test
    public void testApplicationPath() throws Exception {
       server.deployOldStyle(MyApp.class);
       try (Client client = ClientBuilder.newClient()) {
           String val = client.target(TestPortProvider.generateURL("/base/test"))
                              .request().get(String.class);
           Assertions.assertEquals("hello world", val);
       }
    }

    @Test
    public void testApplicationContext() throws Exception {
       server.deployOldStyle(MyApp.class, "/root");
       try (Client client = ClientBuilder.newClient()) {
           String val = client.target(TestPortProvider.generateURL("/root/test"))
                              .request().get(String.class);
           Assertions.assertEquals("hello world", val);
       }
    }

    @Test
    public void testDeploymentInfo() throws Exception {
       DeploymentInfo di = server.undertowDeployment(MyApp.class);
       di.setContextPath("/di");
       di.setDeploymentName("DI");
       server.deploy(di);
       try (Client client = ClientBuilder.newClient()) {
           String val = client.target(TestPortProvider.generateURL("/di/base/test"))
                              .request().get(String.class);
           Assertions.assertEquals("hello world", val);
       }
```

```
    }
}
```

43.2. Oracle JDK HTTP Server



This module is deprecated and should not be used.

The Oracle JDK comes with a simple HTTP server implementation (`com.sun.net.httpserver.HttpServer`) on which RESTEasy can be run on top of.

```
public static void main(final String[] args) throws Exception {
    HttpServer httpServer = HttpServer.create(new InetSocketAddress(port), 10);
    contextBuilder = new HttpContextBuilder();
    contextBuilder.getDeployment().getActualResourceClasses().add(SimpleResource.
class);
    HttpContext context = contextBuilder.bind(httpServer);
    context.getAttributes().put("some.config.info", "42");
    httpServer.start();

    contextBuilder.cleanup();
    httpServer.stop(0);
}
```

Create the HttpServer as desired, then use the `org.jboss.resteasy.plugins.server.sun.http.HttpContextBuilder` to initialize Resteasy and bind it to an `HttpContext`. The `HttpContext` attributes are available by injecting an `org.jboss.resteasy.spi.ResteasyConfiguration` interface using `@Context` within the provider and resource classes.

Maven project you must include is:

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-jdk-http</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

43.3. Netty

RESTEasy has integration with the popular Netty project.

```
public static void start(ResteasyDeployment deployment) throws Exception {
    netty = new NettyJaxrsServer();
    netty.setDeployment(deployment);
    netty.setPort(TestPortProvider.getPort());
```

```
    netty.setRootResourcePath("");
    netty.setSecurityDomain(null);
    netty.start();
}
```

Include this archive to use netty4

```
<dependency>
    <groupId>dev.resteasy.netty</groupId>
    <artifactId>resteasy-embedded-server</artifactId>
</dependency>
```



The Reactor Netty client has moved to its own project which can be found at https://github.com/resteasy/resteasy-netty. The packages have remained the same, but the Maven GAV has changed:

```
<dependency>
    <groupId>dev.resteasy.netty</groupId>
    <artifactId>resteasy-embedded-server</artifactId>
</dependency>
```

43.4. Reactor-Netty

RESTEasy integrates with the reactor-netty project. This server adapter was created to pair with RESTEasy's reactor-netty based Jakarta RESTful Web Services client integration. Ultimately, if using reactor-netty for both the server and server-contained clients it will be possible to do things like share the same event loop for both server and client calls.

```
public static void start(ResteasyDeployment deployment) throws Exception {
    ReactorNettyJaxrsServer server =  new ReactorNettyJaxrsServer();
    server.setDeployment(new ResteasyDeploymentImpl());
    server.setDeployment(deployment);
    server.setPort(TestPortProvider.getPort());
    server.setRootResourcePath("");
    server.setSecurityDomain(null);
    server.start();
}
```

Include this archive to use the reactor netty feature

```
<dependency>
    <groupId>dev.resteasy.netty</groupId>
    <artifactId>resteasy-embedded-server-rector</artifactId>
</dependency>
```



The Reactor Netty embedded server has moved to its own project which can be found at https://github.com/resteasy/resteasy-netty. The packages have remained the same, but the Maven GAV has changed:

```
<dependency>
    <groupId>dev.resteasy.netty</groupId>
    <artifactId>resteasy-embedded-server-rector</artifactId>
</dependency>
```

43.5. Jetty

RESTEasy provides an embedded server implementation backed by Eclipse Jetty.

For RESTEasy Jetty embedded server documentation, see docs.resteasy.dev/resteasy-jetty.

43.6. Vert.x

RESTEasy provides an embedded server implementation backed by Eclipse Vert.x.

For RESTEasy Vert.x embedded server documentation, see docs.resteasy.dev/resteasy-vertx.

43.7. EmbeddedJaxrsServer

`EmbeddedJaxrsServer` is a deprecated interface provided to enable each embedded container wrapper class to configure, start and stop its container in a standard fashion. Each server `UndertowJaxrsServer`, `SunHttpJaxrsServer`, `NettyJaxrsServer`, and `VertxJaxrsServer` implements `EmbeddedJaxrsServer`.

```
public interface EmbeddedJaxrsServer<T> {
    T deploy();
    T start();
    void stop();
    ResteasyDeployment getDeployment();
    T setDeployment(ResteasyDeployment deployment);
    T setPort(int port);
    T setHostname(String hostname);
    T setRootResourcePath(String rootResourcePath);
    T setSecurityDomain(SecurityDomain sc);
}
```

Chapter 44. Server-side Mock Framework

Although RESTEasy has an Embeddable Container, you may not be comfortable with the idea of starting and stopping a web server within unit tests. In reality, the embedded container starts in milliseconds. You might not like the idea of using Apache HTTP Client or `java.net.URL` to test your code. RESTEasy provides a mock framework so that you can invoke on your resource directly.

```
public void testMocking() throws Exception {

    Dispatcher dispatcher = MockDispatcherFactory.createDispatcher();

    POJOResourceFactory noDefaults = new POJOResourceFactory(LocatingResource.class);
    dispatcher.getRegistry().addResourceFactory(noDefaults);
    MockHttpRequest request = MockHttpRequest.get("/locating/basic");
    MockHttpResponse response = new MockHttpResponse();

    dispatcher.invoke(request, response);

    Assertions.assertEquals(HttpServletResponse.SC_OK, response.getStatus());
    Assertions.assertEquals("basic", response.getContentAsString());
}
```

See the RESTEasy Javadoc for all the ease-of-use methods associated with `MockHttpRequest`, and `MockHttpResponse`.

Chapter 45. Securing Jakarta RESTful Web Services and RESTEasy

Because RESTEasy is deployed as a servlet, standard web.xml constraints must be used to enable authentication and authorization.

Unfortunately, web.xml constraints do not mesh very well with Jakarta RESTful Web Services in some situations. The problem is that web.xml URL pattern matching is limited. URL patterns in web.xml only support simple wildcards, so Jakarta RESTful Web Services resources such as the following:

```
/{pathparam1}/foo/bar/{pathparam2}
```

Cannot be mapped as a web.xml URL pattern like:

```
/*/foo/bar/*
```

To resolve this issue, use the security annotations defined below on your Jakarta RESTful Web Services methods. Some general security constraint elements will still need to be declared in web.xml to turn on authentication.

RESTEasy supports the `@RolesAllowed`, `@PermitAll` and `@DenyAll` annotations on Jakarta RESTful Web Services methods. By default, RESTEasy does not recognize these annotations. RESTEasy must be configured to turn on role-based security by setting the appropriate parameter. The code fragment below show how to enable security.



Do not turn on this switch if using Jakarta Enterprise Beans. The Jakarta Enterprise Beans container will provide this functionality instead of RESTEasy.

```
<web-app>
   <context-param>
      <param-name>resteasy.role.based.security</param-name>
      <param-value>true</param-value>
   </context-param>
</web-app>
```

See Configuration for more information about application configuration.

RESTEasy requires that all roles used within the application be declared in the war's web.xml file. A security constraint that permits all of those roles access to every URL handled by the Jakarta RESTful Web Services runtime must also be declared in the web.xml.

RESTEasy performs authorization by checking the method annotations. If a method is annotated with @RolesAllowed. It calls HttpServletRequest.isUserInRole. If one of the @RolesAllowed passes, the request is processed, otherwise, a response is returned with a 401 (Unauthorized) response

code.

Here is an example of a modified RESTEasy WAR file. Notice that every role declared is allowed access to every URL controlled by the RESTEasy servlet.

```
<web-app>
   <context-param>
      <param-name>resteasy.role.based.security</param-name>
      <param-value>true</param-value>
   </context-param>

   <security-constraint>
      <web-resource-collection>
         <web-resource-name>Resteasy</web-resource-name>
         <url-pattern>/security</url-pattern>
      </web-resource-collection>
       <auth-constraint>
         <role-name>admin</role-name>
         <role-name>user</role-name>
      </auth-constraint>
  </security-constraint>

   <login-config>
      <auth-method>BASIC</auth-method>
      <realm-name>Test</realm-name>
   </login-config>

   <security-role>
      <role-name>admin</role-name>
   </security-role>
   <security-role>
      <role-name>user</role-name>
   </security-role>
</web-app>
```

Chapter 46. JSON Web Signature and Encryption (JOSE-JWT)

JSON Web Signature and Encryption (JOSE JWT) specification, rfc7517, defines how to encode content as a string and either digitally sign or encrypt it.

Chapter 47. JSON Web Signature (JWS)

To digitally sign content using JWS, use the `org.jboss.resteasy.jose.jws.JWSBuilder` class. To unpack and verify a JWS, use the `org.jboss.resteasy.jose.jws.JWSInput` class. Here's an example:

```
@Test
public void testRSAWithContentType() throws Exception {
  KeyPair keyPair = KeyPairGenerator.getInstance("RSA").generateKeyPair();

  String encoded = new JWSBuilder()
          .contentType(MediaType.TEXT_PLAIN_TYPE)
          .content("Hello World", MediaType.TEXT_PLAIN_TYPE)
          .rsa256(keyPair.getPrivate());

  System.out.println(encoded);

  JWSInput input = new JWSInput(encoded, ResteasyProviderFactory.getInstance());
  System.out.println(input.getHeader());
  String msg = (String)input.readContent(String.class);
  Assert.assertEquals("Hello World", msg);
  Assert.assertTrue(RSAProvider.verify(input, keyPair.getPublic()));

}
```

Chapter 48. JSON Web Encryption (JWE)

To encrypt content using JWE, use the `org.jboss.resteasy.jose.jwe.JWEBuilder` class. To decrypt content using JWE, use the `org.jboss.resteasy.jose.jwe.JWEInput` class. Here's an example:

```
@Test
public void testRSA() throws Exception {
  KeyPair keyPair = KeyPairGenerator.getInstance("RSA").generateKeyPair();

  String content = "Live long and prosper.";

  {
      String encoded = new JWEBuilder().contentBytes(content.getBytes()).RSA1_5(
(RSAPublicKey)keyPair.getPublic());
      System.out.println("encoded: " + encoded);
      byte[] raw = new JWEInput(encoded).decrypt((RSAPrivateKey)keyPair.getPrivate())
.getRawContent();
      String from = new String(raw);
      Assert.assertEquals(content, from);
  }
  {
     String encoded = new JWEBuilder().contentBytes(content.getBytes()).RSA_OAEP(
(RSAPublicKey)keyPair.getPublic());
     System.out.println("encoded: " + encoded);
     byte[] raw = new JWEInput(encoded).decrypt((RSAPrivateKey)keyPair.getPrivate())
.getRawContent();
     String from = new String(raw);
     Assert.assertEquals(content, from);
  }
  {
     String encoded = new JWEBuilder().contentBytes(content.getBytes()).A128CBC_HS256
().RSA1_5((RSAPublicKey)keyPair.getPublic());
     System.out.println("encoded: " + encoded);
     byte[] raw = new JWEInput(encoded).decrypt((RSAPrivateKey)keyPair.getPrivate())
.getRawContent();
     String from = new String(raw);
     Assert.assertEquals(content, from);
  }
  {
     String encoded = new JWEBuilder().contentBytes(content.getBytes()).A128CBC_HS256
().RSA_OAEP((RSAPublicKey)keyPair.getPublic());
     System.out.println("encoded: " + encoded);
     byte[] raw = new JWEInput(encoded).decrypt((RSAPrivateKey)keyPair.getPrivate())
.getRawContent();
     String from = new String(raw);
     Assert.assertEquals(content, from);
  }
}

@Test
```

```
public void testDirect() throws Exception {
  String content = "Live long and prosper.";
  String encoded = new JWEBuilder().contentBytes(content.getBytes()).dir("geheim");
  System.out.println("encoded: " + encoded);
  byte[] raw = new JWEInput(encoded).decrypt("geheim").getRawContent();
  String from = new String(raw);
  Assert.assertEquals(content, from);

}
```

Chapter 49. Doseta Digital Signature Framework

Digital signatures allow the protection of the integrity of a message. They are used to verify that a transmitted message was sent by the actual user and the sent message was not modified in transit. Most web applications handle message integrity by using TLS, like HTTPS, to secure the connection between the client and server. Sometimes there will be representations that are going to be forwarded to more than one recipient. Some representations may hop around from server to server. In this case, TLS is not enough. There needs to be a mechanism to verify who sent the original representation and that they actually sent that message. This is where digital signatures come in.

While the mime type multiple/signed exists, it does have drawbacks. Most importantly it requires the receiver of the message body to understand how to unpack it. A receiver may not understand this mime type. A better approach would be to put signatures in an HTTP header so that receivers that don't need to worry about the digital signature, don't have to.

The email world has a protocol called Domain Keys Identified Mail (DKIM). Work is underway to apply this header to protocols other than email (i.e. HTTP) through the DOSETA specifications. It allows the user to sign a message body and attach the signature via a DKIM-Signature header. Signatures are calculated by first hashing the message body then combining this hash with an arbitrary set of metadata included within the DKIM-Signature header. Other request or response headers can be added to the calculation of the signature. Adding metadata to the signature calculation gives a lot of flexibility to piggyback various features like expiration and authorization. Here's what an example DKIM-Signature header might look like.

```
DKIM-Signature: v=1;
                a=rsa-sha256;
                d=example.com;
                s=burke;
                c=simple/simple;
                h=Content-Type;
                x=0023423111111;
                bh=2342322111;
                b=M232234=
```

You can see it is a set of name value pairs delimited by a ';'. While it is not THAT important to know the structure of the header, here's an explanation of each parameter:

v

Protocol version. Always 1.

a

Algorithm used to hash and sign the message. RSA signing and SHA256 hashing is the only supported algorithm at the moment by RESTEasy.

d

Domain of the signer. This is used to identify the signer as well as discover the public key to use to verify the signature.

s

Selector of the domain. Also used to identify the signer and discover the public key.

c

Canonical algorithm. Only simple/simple is supported at the moment. This allows the transform of the message body before calculating the hash

h

Semi-colon delimited list of headers that are included in the signature calculation.

x

Signature expiration. This is a numeric long value of the time in seconds since epoch. Allows signer to control when a signed message's signature expires

t

Signature timestamp. Numeric long value of the time in seconds since epoch. Allows the verifier to control when a signature expires.

bh

Base 64 encoded hash of the message body.

b

Base 64 encoded signature.

To verify a signature a public key is needed. DKIM uses DNS text records to discover a public key. To find a public key, the verifier concatenates the Selector (s parameter) with the domain (d parameter)

```
<selector>._domainKey.<domain>
```

It then takes that string and does a DNS request to retrieve a TXT record under that entry. In the above example burke._domainKey.example.com would be used as a string. This is an interesting way to publish public keys. For one, it becomes very easy for verifiers to find public keys. There's no real central store that is needed. DNS is an infrastructure IT knows how to deploy. Verifiers can choose which domains they allow requests from. RESTEasy supports discovering public keys via DNS. It also allows the discovery of public keys within a local Java KeyStore if you do not want to use DNS. It also allows the user to plug in their own mechanism to discover keys.

If interested in learning the possible use cases for digital signatures, here's an informative blog.

49.1. Maven settings

Archive, resteasy-crypto must include the project to use the digital signature framework.

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-crypto</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

49.2. Signing API

To sign a request or response using the RESTEasy client or server framework create an instance of `org.jboss.resteasy.security.doseta.DKIMSignature`. This class represents the DKIM-Signature header. Instantiate the DKIMSignature object and then set the "DKIM-Signature" header of the request or response. Here's an example of using it on the server-side:

```
import org.jboss.resteasy.security.doseta.DKIMSignature;
import java.security.PrivateKey;


@Path("/signed")
public static class SignedResource {
   @GET
   @Path("manual")
   @Produces("text/plain")
   public Response getManual() {
      PrivateKey privateKey = getKey(); // get the private key to sign message

      DKIMSignature signature = new DKIMSignature();
      signature.setSelector("test");
      signature.setDomain("samplezone.org");
      signature.setPrivateKey(privateKey);

      Response.ResponseBuilder builder = Response.ok("hello world");
      builder.header(DKIMSignature.DKIM_SIGNATURE, signature);
      return builder.build();
   }
}
```

```
public static void main(final String[] args) throws Exception {
    // client example
    DKIMSignature signature = new DKIMSignature();
    PrivateKey privateKey = getKey(); // go find it
    signature.setSelector("test");
    signature.setDomain("samplezone.org");
    signature.setPrivateKey(privateKey);

    ClientRequest request = new ClientRequest("http://...");
    request.header("DKIM-Signature", signature);
    request.body("text/plain", "some body to sign");
```

```
    ClientResponse response = request.put();
}
```

To sign a message a PrivateKey is needed. This can be generated by KeyTool or manually using regular, standard JDK Signature APIs. RESTEasy currently only supports RSA key pairs. The DKIMSignature class allows the user to add and control how various pieces of metadata are added to the DKIM-Signature header and the signature calculation. See the javadoc for more details.

If including more than one signature, then add additional DKIMSignature instances to the headers of the request or response.

49.2.1. `@Signed` annotation

Instead of using the API, RESTEasy provides an annotation alternative to the manual way of signing using a DKIMSignature instances. RESTEasy provides annotation @org.jboss.resteasy.annotations.security.doseta.Signed. It is required that a KeyRepository be configured as described later in this chapter. Here's an example:

```
@GET
@Produces("text/plain")
@Path("signedresource")
@Signed(selector="burke", domain="sample.com", timestamped=true, expires=@After(hours
=24))
public String getSigned() {
  return "hello world";
}
```

The above example uses optional annotation attributes of @Signed to create the following Content-Signature header:

```
DKIM-Signature: v=1;
                a=rsa-sha256;
                c=simple/simple;
                domain=sample.com;
                s=burke;
                t=02342342341;
                x=02342342322;
                bh=m0234fsefasf==;
                b=mababaddbb==
```

This annotation also works with the client proxy framework.

49.3. Signature Verification API

RESTEasy supports fine grain control over verification with an API to verify signatures manually. To verify the signature the raw bytes of the HTTP message body are needed. Using org.jboss.resteasy.spi.MarshalledEntity injection will provide access to the unmarshalled message

body and the underlying raw bytes. Here is an example of doing this on the server side:

```
import org.jboss.resteasy.spi.MarshalledEntity;

@POST
@Consumes("text/plain")
@Path("verify-manual")
public void verifyManual(@HeaderParam("Content-Signature") DKIMSignature signature,
                         @Context KeyRepository repository,
                         @Context HttpHeaders headers,
                         MarshalledEntity<String> input) throws Exception {
      Verifier verifier = new Verifier();
      Verification verification = verifier.addNew();
      verification.setRepository(repository);
      verification.setStaleCheck(true);
      verification.setStaleSeconds(100);
      try {
          verifier.verifySignature(headers.getRequestHeaders(), input
.getMarshalledBytes, signature);
      } catch (SignatureException ex) {
      }
      System.out.println("The text message posted is: " + input.getEntity());
}
```

MarshalledEntity is a generic interface. The template parameter should be the Java type of the message body to be converted into. A KeyRepository will have to be configured. This is described later in this chapter.

The client side is a little different:

```
ClientRequest request = new ClientRequest("http://localhost:9095/signed"));

ClientResponse<String> response = request.get(String.class);
Verifier verifier = new Verifier();
Verification verification = verifier.addNew();
verification.setRepository(repository);
response.getProperties().put(Verifier.class.getName(), verifier);

// signature verification happens when you get the entity
String entity = response.getEntity();
```

On the client side, create a verifier and add it as a property to the ClientResponse. This will trigger the verification interceptors.

49.3.1. Annotation-based verification

The easiest way to verify a signature sent in an HTTP request on the server side is to use the @@org.jboss.resteasy.annotations.security.doseta.Verify (or @Verifications which is used to verify

multiple signatures). Here's an example:

```
@POST
@Consumes("text/plain")
@Verify
public void post(String input) {
}
```

In the above example, any DKIM-Signature headers attached to the posted message body will be verified. The public key to verify is discovered using the configured KeyRepository (discussed later in this chapter). The user can specify which specific signatures to be verified as well as define multiple verifications want via the @Verifications annotation. Here's a complex example:

```
@POST
@Consumes("text/plain")
@Verifications({
   @Verify(identifierName="d", identiferValue="inventory.com", stale=@After(days=2)),
   @Verify(identifierName="d", identiferValue="bill.com")
})
public void post(String input) {}
```

The above is expecting 2 different signature to be included within the DKIM-Signature header.

Failed verifications will throw an `org.jboss.resteasy.security.doseta.UnauthorizedSignatureException`. This causes a 401 error code to be sent back to the client. Catching this exception using an ExceptionHandler allows the user to browse the failure results.

49.4. Managing Keys via a KeyRepository

RESTEasy manages keys through an `org.jboss.resteasy.security.doseta.KeyRepository`. By default, the KeyRepository is backed by a Java KeyStore. Private keys are always discovered by looking into this KeyStore. Public keys may also be discovered via a DNS text (TXT) record lookup if configured to do so. The user can also implement and plug in their own implementation of KeyRepository.

49.4.1. Create a KeyStore

Use the Java keytool to generate RSA key pairs. Key aliases MUST HAVE the form of:

```
<selector>._domainKey.<domain>
```

For example:

```
$ keytool -genkeypair -alias burke._domainKey.example.com -keyalg RSA -keysize 1024
-keystore my-apps.jks
```

You can always import your own official certificates too. See the JDK documentation for more details.

49.4.2. Configure RESTEasy to use the KeyRepository

Three `context-param` elements must be declared in the application's web.xml in order for RESTEasy to properly be configured to use the KeyRepository. This information enables the KeyRepository to be created and made available to RESTEasy, which will use it to discover private and public keys.

For example:

```
<web-app>
    <context-param> ①
        <param-name>resteasy.doseta.keystore.classpath</param-name>
        <param-value>test.jks</param-value>
    </context-param>
    <context-param> ②
        <param-name>resteasy.doseta.keystore.password</param-name>
        <param-value>geheim</param-value>
    </context-param>
    <context-param> ③
        <param-name>resteasy.context.objects</param-name>
        <param-value>org.jboss.resteasy.security.doseta.KeyRepository :
org.jboss.resteasy.security.doseta.ConfiguredDosetaKeyRepository</param-value>
    </context-param>
</web-app>
```

① The Java key store to be referenced by the Resteasy signature framework must be identified using either `resteasy.keystore.classpath` or `resteasy.keystore.filename` context parameters.

② The password must be specified using the `resteasy.keystore.password` context parameter. Unfortunately the password must be in clear text.

③ The `resteasy.context.objects parameter` is used to identify the classes used in creating the repository

The user can manually register their own instance of a KeyRepository within an Application class. For example:

```
import org.jboss.resteasy.core.Dispatcher;
import org.jboss.resteasy.security.doseta.KeyRepository;
import org.jboss.resteasy.security.doseta.DosetaKeyRepository;

import jakarta.ws.rs.core.Application;
import jakarta.ws.rs.core.Context;

public class SignatureApplication extends Application {
   private final HashSet<Class<?>> classes;
   private final KeyRepository repository;
```

```
   public SignatureApplication(@Context Dispatcher dispatcher) {
      classes = Set.of(SignedResource.class);

      repository = new DosetaKeyRepository();
      repository.setKeyStorePath("test.jks");
      repository.setKeyStorePassword("password");
      repository.setUseDns(false);
      repository.start();

      dispatcher.getDefaultContextObjects().put(KeyRepository.class, repository);
   }

   @Override
   public Set<Class<?>> getClasses() {
      return classes;
   }
}
```

On the client side, a KeyStore can be loaded manually, by instantiating an instance of org.jboss.resteasy.security.doseta.DosetaKeyRepository. Then set a request attribute, "org.jboss.resteasy.security.doseta.KeyRepository", with the value of the created instance. Use the ClientRequest.getAttributes() method to do this. For example:

```
DosetaKeyRepository keyRepository = new DoestaKeyRepository();
repository.setKeyStorePath("test.jks");
repository.setKeyStorePassword("password");
repository.setUseDns(false);
repository.start();

DKIMSignature signature = new DKIMSignature();
signature.setDomain("example.com");

ClientRequest request = new ClientRequest("http://...");
request.getAttributes().put(KeyRepository.class.getName(), repository);
request.header("DKIM-Signature", signatures);
```

49.4.3. Using DNS to Discover Public Keys

Public keys can be discovered by a DNS text record lookup. The web.xml must be configured to enable this feature:

```
<web-app>
    <context-param>
        <param-name>resteasy.doseta.use.dns</param-name>
        <param-value>true</param-value>
    </context-param>
    <context-param>
        <param-name>resteasy.doseta.dns.uri</param-name>
```

```
        <param-value>dns://localhost:9095</param-value>
    </context-param>
</web-app>
```

The resteasy.doseta.dns.uri context-param is optional and allows pointing to a specific DNS server to locate text records.

Configuring DNS TXT Records

DNS TXT Records are stored via a format described by the DOSETA specification. The public key is defined via a base 64 encoding. Text encoding can be obtained by exporting the public keys from your keystore and then using a tool like openssl to get the text-based format. For example:

```
$ keytool -export -alias bill._domainKey.client.com -keystore client.jks -file
bill.der
$ openssl x509 -noout -pubkey -in bill.der -inform der > bill.pem
```

The output will look something like:

```
-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQCKxct5GHz8dFw0mzAMfvNju2b3
oeAv/EOPfVb9mD73Wn+CJYXvnryhqo99Y/q47urWYWAF/bqH9AMyMfibPr6IlP8m
O9pNYf/Zsqup/7oJxrvzJU7T0IGdLN1hHcC+qRnwkKddNmD8UPEQ4BXiX4xFxbTj
NvKWLZVKGQMyy6EFVQIDAQAB
-----END PUBLIC KEY-----
```

The DNS text record entry would look like this:

```
test2._domainKey        IN      TXT     "v=DKIM1;
p=MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQCIKFLFWuQfDfBug688BJ0dazQ/x+GEnH443KpnBK8agpJX
SgFAPhlRvf0yhqHeuI+J5onsSOo9Rn4fKaFQaQNBfCQpHSMnZpBC3X0G5Bc1HWq1AtBl6Z1rbyFen4CmGYOyRz
DBUOIW6n8QK47bf3hvoSxqpY1pHdgYoVK0YdIP+wIDAQAB; t=s"
```

Notice that the newlines are take out. Also, notice that the text record is a name value ';' delimited list of parameters. The p field contains the public key.

Chapter 50. Body Encryption and Signing via SMIME

S/MIME (Secure/Multipurpose Internet Mail Extensions) is a standard for public key encryption and signing of MIME data. MIME data being a set of headers and a message body. Its most often seen in the email world when somebody wants to encrypt and/or sign an email message they are sending across the internet. It can also be used for HTTP requests as well which is what the RESTEasy integration with S/MIME is all about. RESTEasy allows you to easily encrypt and/or sign an email message using the S/MIME standard.

50.1. Maven settings

Reference the resteasy-crypto archive to use the smime framework.

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-crypto</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

50.2. Message Body Encryption

While HTTPS is used to encrypt the entire HTTP message, S/MIME encryption is used solely for the message body of the HTTP request or response. This is very useful if there is a representation that may be forwarded by multiple parties (for example, HornetQ's REST Messaging integration!) and the message needed to be protected from prying eyes as it travels across the network. RESTEasy has two different interfaces for encrypting message bodies. One for output, one for input. If the client or server wants to send an HTTP request or response with an encrypted body, it uses the `org.jboss.resteasy.security.smime.EnvelopedOutput` type. Encrypting a body also requires an X509 certificate which can be generated by the Java keytool command-line interface, or the openssl tool that comes installed on many OS's. Here's an example of using the `EnvelopedOutput` interface:

Server Side

```
@Path("encrypted")
@GET
public EnvelopedOutput getEncrypted(){
   Customer cust = new Customer();
   cust.setName("Bill");

   X509Certificate certificate = createCert();
   EnvelopedOutput output = new EnvelopedOutput(cust, MediaType.APPLICATION_XML_TYPE);
   output.setCertificate(certificate);
   return output;
}
```

Client Side

```
// client side
X509Certificate cert = createCert();
Customer cust = new Customer();
cust.setName("Bill");
EnvelopedOutput output = new EnvelopedOutput(cust, "application/xml");
output.setCertificate(cert);
Response res = target.request().post(Entity.entity(output, "application/pkcs7-mime")
.post();
```

An EnvelopedOutput instance is created passing in the entity to be marshaled and the media type to marshal into. In this example, a Customer class is being marshaled into XML before it is encrypted. RESTEasy will then encrypt the EnvelopedOutput using the BouncyCastle framework's SMIME integration. The output is a Base64 encoding and would look something like this:

```
Content-Type: application/pkcs7-mime; smime-type=enveloped-data; name="smime.p7m"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="smime.p7m"

MIAGCSqGSIb3DQEHA6CAMIACAQAxgewwgekCAQAwUjBFMQswCQYDVQQGEwJBVTETMBEGA1UECBMK
U29tZS1TdGF0ZTEhMB8GA1UEChMYSW50ZXJuZXQgV2lkZ2l0cyBQdHkgTHRkAgkA7oW81OriflAw
DQYJKoZIhvcNAQEBBQAEgYCfnqPK/O34DFl2p2zm+xZQ6R+94BqZHdtEWQN2evrcgtAng+f2ltIL
xr/PiK+8bE8wDO5GuCg+k92uYp2rLKlZ5BxCGb8tRM4kYC9sHbH2dPaqzUBhMxjgWdMCX6Q7E130
u9MdGcP74Ogwj8fNl3lD4sx/0k02/QwgaukeY7uNHzCABgkqhkiG9w0BBwEwFAYIKoZIhvcNAwcE
CDRozFLsPnSgoIAEQHmqjSKAWlQbuGQL9w4nKw4l+44WgTjKf7mGWZvYY8tOCcdmhDxRSM1Ly682
Imt+LTZf0LXzuFGTsCGOUo742N8AAAAAAAAAAAAA
```

Decrypting an S/MIME encrypted message requires using the org.jboss.resteasy.security.smime.EnvelopedInput interface. Both the private key and X509Certificate used to encrypt the message are also needed. Here's an example:

Server Side

```
@Path("encrypted")
@POST
public void postEncrypted(EnvelopedInput<Customer> input) {
   PrivateKey privateKey = createKey();
   X509Certificate certificate = createCert();
   Customer cust = input.getEntity(privateKey, certificate);
}
```

Client Side

```
ClientRequest request = new ClientRequest("http://localhost:9095/smime/encrypted");
EnvelopedInput input = request.getTarget(EnvelopedInput.class);
Customer cust = (Customer)input.getEntity(Customer.class, privateKey, cert);
```

Both examples simply call the `getEntity()` method passing in the `PrivateKey` and `X509Certificate` instances requires to decrypt the message. On the server side, a generic is used with `EnvelopedInput` to specify the type to marshal to. On the server side this information is passed as a parameter to `getEntity()`. The message is in MIME format: a Content-Type header and body, so the `EnvelopedInput` class has everything it needs to both decrypt and unmarshall the entity.

50.3. Message Body Signing

S/MIME allows the user to digitally sign a message. It is different than the Doseta Digital Signing Framework. Doseta is an HTTP header that contains the signature. S/MIME uses the multipart/signed data format which is a multipart message that contains the entity and the digital signature. Doseta is a header, S/MIME is its own media type. S/MIME signatures require the client to know how to parse a multipart message and Doseta doesn't. It is left to the user to select the method to use.

RESTEasy has two different interfaces for creating a multipart/signed message. One for input, one for output. If the client or server is to send an HTTP request or response with a multipart/signed body, it uses the `org.jboss.resteasy.security.smime.SignedOutput` type. This type requires both the PrivateKey and X509Certificate to create the signature. Here's an example of signing an entity and sending a multipart/signed entity.

Server Side

```
@Path("signed")
@GET
@Produces("multipart/signed")
public SignedOutput getSigned() {
    Customer cust = new Customer();
    cust.setName("Bill");

    SignedOutput output = new SignedOutput(cust, MediaType.APPLICATION_XML_TYPE);
    output.setPrivateKey(privateKey);
    output.setCertificate(certificate);
    return output;
}
```

Client Side

```
Client client = ClientBuilder.newClient();
WebTarget target = client.target("http://localhost:9095/smime/signed");
Customer cust = new Customer();
cust.setName("Bill");
SignedOutput output = new SignedOutput(cust, "application/xml");
output.setPrivateKey(privateKey);
output.setCertificate(cert);
Response res = target.request().post(Entity.entity(output, "multipart/signed");
```

An SignedOutput instance is created passing in the entity wanted to be marshaled and the media

type to marshal into. In this example, a Customer class is marshaled into XML before it is sign. RESTEasy will then sign the SignedOutput using the BouncyCastle framework's SMIME integration. The output would look like this:

```
Content-Type: multipart/signed; protocol="application/pkcs7-signature"; micalg=sha1;
boundary="----=_Part_0_1083228271.1313024422098"

------=_Part_0_1083228271.1313024422098
Content-Type: application/xml
Content-Transfer-Encoding: 7bit

<customer name="bill"/>
------=_Part_0_1083228271.1313024422098
Content-Type: application/pkcs7-signature; name=smime.p7s; smime-type=signed-data
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="smime.p7s"
Content-Description: S/MIME Cryptographic Signature

MIAGCSqGSIb3DQEHAqCAMIACAQExCzAJBgUrDgMCGgUAMIAGCSqGSIb3DQEHAQAAMYIBVzCCAVMC
AQEwUjBFMQswCQYDVQQGEwJBVTETMBEGA1UECBMKU29tZS1TdGF0ZTEhMB8GA1UEChMYSW50ZXJu
ZXQgV2lkZ2l0cyBQdHkgTHRkAgkA7oW81OriflAwCQYFKw4DAhoFAKBdMBgGCSqGSIb3DQEJAzEL
BgkqhkiG9w0BBwEwHAYJKoZIhvcNAQkFMQ8XDTExMDgxMTAxMDAyMlowIwYJKoZIhvcNAQkEMRYE
FH32BfR1l1vzDshtQvJrgvpGvjADMA0GCSqGSIb3DQEBAQUABIGAL3KVi3ul9cPRUMYcGgQmWtsZ
0bLbAldO+okrt8mQ87SrUv2LGkIJbEhGHsOlsgSU80/YumP+Q4lYsVanVfoI8GgQH3Iztp+Rce2c
y42f86ZypE7ueynI4HTPNHfr78EpyKGzWuZHW4yMo70LpXhk5RqfM9a/n4TEa9QuTU76atAAAAAA
AAA=
------=_Part_0_1083228271.1313024422098--
```

To unmarshal and verify a signed message requires using the `org.jboss.resteasy.security.smime.SignedInput` interface. Only the X509Certificate is needed to verify the message. Here's an example of unmarshalling and verifying a multipart/signed entity.

Server Side

```
@Path("signed")
@POST
@Consumes("multipart/signed")
public void postSigned(SignedInput<Customer> input) throws Exception {
    Customer cust = input.getEntity();
    if (!input.verify(certificate)) {
        throw new WebApplicationException(500);
    }
}
```

Client Side

```
Client client = ClientBuilder.newClient();
WebTarget target = client.target("http://localhost:9095/smime/signed");
SignedInput input = target.request().get(SignedInput.class);
```

```
Customer cust = (Customer)input.getEntity(Customer.class)
input.verify(cert);
```

50.4. application/pkcs7-signature

application/pkcs7-signature is a data format that includes both the data and the signature in one ASN.1 binary encoding.

SignedOutput and SignedInput can be used to return application/pkcs7-signature format in binary form. Just change the @Produces or @Consumes to that media type to send back that format.

Also, if the @Produces or @Consumes is text/plain instead, SignedOutput will be base64 encoded and sent as a string.

Chapter 51. Jakarta Enterprise Beans Integration

RESTEasy provides tight integration with Jakarta Enterprise Beans (EJB) components. This allows you to use standard Jakarta REST annotations directly on your Session Beans, leveraging EJB features like transaction management and security.

51.1. Automatic Discovery via CDI (Recommended)

In accordance with the Jakarta REST specification (Section 11.2.4), RESTEasy automatically discovers and registers EJBs that use Jakarta REST annotations.

51.1.1. 1. Standard Configuration

The specification mandates that Resource class annotations (such as `@Path`) MUST be applied to the EJB implementation class, while method annotations (such as `@GET`, `@POST`, `@Produces`) can remain on the Local Interface.

Example: The Standard Pattern

```
// 1. The Local Interface defines the business contract
@Local
public interface Library {
    // JAX-RS method annotations can go here (preferred) or on the class
    @GET
    @Path("/books/{isbn}")
    String getBook(@PathParam("isbn") String isbn);
}

// 2. The Implementation Class defines the Resource Path
@Stateless
@Path("/library") // Resource-level annotations MUST be here
public class LibraryBean implements Library {

    @Override
    public String getBook(String isbn) {
        return "Book: " + isbn;
    }
}
```

51.1.2. 2. View Resolution and Multiple Interfaces

RESTEasy's CDI integration automatically determines the correct EJB View (Interface) to use for the JAX-RS resource lookup.

- **Single Interface:** If your EJB implements a single `@Local` interface, RESTEasy will automatically proxy requests through that interface. No additional configuration is required.

- **Multiple Interfaces:** If your EJB implements multiple interfaces, RESTEasy will attempt to use the first interface it discovers.
- **Warning:** If your JAX-RS methods are split across multiple interfaces, or if the "first discovered" interface does not contain the JAX-RS methods, the lookup may fail or result in 404s.
- **Solution:** In complex multi-interface scenarios, it is recommended to consolidate your JAX-RS methods into a single interface or use `@LocalBean` to expose the class view directly, ensuring all methods are accessible via a single type.

51.2. Manual JNDI Registration (Legacy)

If you are strictly avoiding CDI, you can manually register EJBs in `web.xml`.

```
<context-param>
    <param-name>resteasy.jndi.resources</param-name>
    <param-value>java:global/myApp/myEjbModule/LibraryBean!com.example.Library</param-
value>
</context-param>
```

Chapter 52. Spring Integration

52.1. Overview

There are three RESTEasy GitHub projects that provide components for RESTEasy's Spring Framework support.

- RESTEasy provides core components for Spring Framework integration.
- RESTEasy Spring: This project has been created to separate RESTEasy's Spring integration extensions from RESTEasy's core code. It contains RESTEasy's modules `resteasy-spring-web`, `resteasy-spring`, `resteasy-undertow-spring` and related tests. These modules were moved out of the RESTEasy project as of version 5.0.0. In maven the GAV groupId is now `org.jboss.resteasy.spring`. The artifactIds remain the same.

```
<dependency>
    <groupId>org.jboss.resteasy.spring</groupId>
    <artifactId>resteasy-spring</artifactId>
    <version>${version.org.jboss.resteasy.spring}</version>
</dependency>
```

- resteasy-spring-boot: provides two Spring Boot starters.

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-spring-boot-starter</artifactId>
    <version>${version.org.jboss.resteasy.spring.boot}</version>
</dependency>
```



RESTEasy currently supports Spring version 6.1

RESTEasy provides integrated support for Spring Framework. It supplies a default `org.springframework.web.context.ContextLoaderListener` and `org.springframework.beans.factory.config.BeanPostProcessor` implementation that is used to run Spring applications. Alternatively RESTEasy provides support for registering a custom `org.springframework.beans.factory.config.BeanFactoryPostProcessor` thus enabling the user to create their own bean factories.

Spring MVC Framework support is provided in RESTEasy. A default implementation of Spring's `org.springframework.web.servlet.DispatcherServlet` is supplied.

RESTEasy furnishes an Undertow based embedded Spring container in which Spring applications can run. It supplies class `org.jboss.resteasy.plugins.server.undertow.spring.UndertowJaxrsSpringServer` which accepts a Spring context configuration file and preforms the appropriate wiring of Spring to RESTEasy.

Two types of Spring Boot starters are provided by resteasy-spring-boot. These can be used by any

regular Spring Boot application that wants to have REST endpoints and prefers RESTEasy as the JAX-RS implementation. The starters integrate with Spring, thus Spring beans will be automatically auto-scanned, integrated, and available. Two types of starters are available, one Servlet based for Tomcat and one for Reactor Netty.

RESTEasy supports Spring's singleton and prototype scopes and Spring Web REST annotations.

The following subsections discuss ways of configuring these features for use in RESTEasy. Code examples can be found in project resteasy-examples

52.2. Basic Integration

Basic integration makes use of RESTEasy's default implementations of interface `jakarta.servlet.ServletContextListener`, and classes:

- `org.springframework.web.context.ContextLoaderListener`
- `org.springframework.beans.factory.config.BeanPostProcessor`
- `org.jboss.resteasy.plugins.server.servlet.ResteasyBootstrap`
- `org.jboss.resteasy.plugins.spring.SpringContextLoaderListener`
- `org.jboss.resteasy.plugins.spring.ResteasyBeanPostProcessor`.

To use this feature the user must add the maven resteasy-spring dependency to their project.

```
<dependency>
    <groupId>org.jboss.resteasy.spring</groupId>
    <artifactId>resteasy-spring</artifactId>
    <version>${version.org.jboss.resteasy.spring}</version>
</dependency>
```

RESTEasy's `SpringContextLoaderListener` registers its `ResteasyBeanPostProcessor`. It processes Jakarta RESTful Web Services annotations when a bean is created by a `BeanFactory`. It automatically scans for @Provider and Jakarta RESTful Web Services resource annotations on bean classes and registers them as Jakarta RESTful Web Services resources.

A user's application must be configured via a `web.xml` and optionally a `bean.xml` file to use these classes. The configuration files discussed below can be found in example, Basic Example

Optionally Spring Framework can be configured to scan for the Jakarta RESTful Web Services resources and beans with a Spring configuration file. Here is an example bean.xml that declares component scanning. In the example code this file is named, `resteasy-spring-basic.xml`.

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
```

```
    http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <context:component-scan base-package="org.jboss.resteasy.examples.springbasic">
        <context:include-filter type="annotation" expression="jakarta.ws.rs.Path"/>
    </context:component-scan>
    <context:annotation-config/>
</beans>
```

Two or three elements will need to be added to the application's `web.xml`. Two listener classes must be declared. The RESTEasy servlet must be identified. If the user provided an optional bean.xml file, that needs to be referenced also.

Here is the content that should add into the `web.xml` file:

```
<web-app>
    <listener>
        <listener-class>
org.jboss.resteasy.plugins.server.servlet.ResteasyBootstrap</listener-class> ①
    </listener>

    <listener>
        <listener-class>
org.jboss.resteasy.plugins.spring.SpringContextLoaderListener</listener-class> ①
    </listener>

    <servlet>
        <servlet-name>resteasy-dispatcher</servlet-name>
        <servlet-class>
org.jboss.resteasy.plugins.server.servlet.HttpServletDispatcher</servlet-class> ②
    </servlet>

    <context-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>classpath:resteasy-spring-basic.xml</param-value> ③
    </context-param>
</web-app>
```

① Two listener classes must be declared, `ResteasyBootstrap` and `SpringContextLoaderListener`. The declaration order is critical. ResteasyBootstrap must be declared first because `SpringContextLoaderListener` relies on `ServletContext` attributes initialized by it.

② The RESTEasy servlet is declared.

③ A reference to the bean.xml file must be added if the application provides one.

An alternative for using `HttpServletDispatcher` for deployment, a `FilterDispatcher` can be declared instead:

```
<filter>
    <filter-name>resteasy-filter</filter-name>
    <filter-class>
        org.jboss.resteasy.plugins.server.servlet.FilterDispatcher
    </filter-class>
</filter>
```

52.3. Customized Configuration

The user is not limited to using RESTEasy's `ContextLoaderListener` implementation. The user may provide their own implementation, however such a customization will require the creation of two additional custom classes, to facilitate the wiring of needed RESTEasy classes into the Spring configuration. An implementation of `org.springframework.web.WebApplicationInitializer` and a class that provides an instance of `org.jboss.resteasy.plugins.spring.SpringBeanProcessorServletAware` will be needed. Spring and Resteasy Customized Example provides an example of the wiring that is required.

There are four RESTEasy classes that must be registered with Spring in the implementation of `WebApplicationInitializer`, `ResteasyBootstrap`, `HttpServletDispatcher`, `ResteasyDeployment`, and `SpringBeanProcessorServletAware`.

```
@Override
public void onStartup(ServletContext servletContext) throws ServletException {
    servletContext.addListener(ResteasyBootstrap.class); ①
    AnnotationConfigWebApplicationContext context = new
AnnotationConfigWebApplicationContext();
    context.register(MyConfig.class); ②
    servletContext.addListener(new MyContextLoaderListener(context)); ③
    ServletRegistration.Dynamic dispatcher = servletContext.addServlet("resteasy-
dispatcher", new HttpServletDispatcher()); ④
    dispatcher.setLoadOnStartup(1);
    dispatcher.addMapping("/rest/*");
}
```

① `ResteasyBootstrap` needs to be registered in the `servletContext` so that the RESTEasy container can gain access to it.

② The user's `MyConfig` class provides RESTEasy's implementation of `BeanProcessorServletAware` as a @Bean to the Spring container.

③ The user's implementation of `ContextLoaderListener` is registered in `servletContext` so that the RESTEasy container can gain access to it.

④ RESTEasy's servlet, `HttpServletDispatcher` is registered in `servletContext` so that the RESTEasy container can gain access to it.

The user's implementation of `ContextLoaderListener` performs two important actions.

```
@Override
```

```
protected void customizeContext(ServletContext servletContext,
ConfigurableWebApplicationContext configurableWebApplicationContext) {
        super.customizeContext(servletContext, configurableWebApplicationContext);

        ResteasyDeployment deployment = (ResteasyDeployment) servletContext
.getAttribute(ResteasyDeployment.class.getName()); ①
        if (deployment == null) {
            throw new RuntimeException(Messages.MESSAGES.deploymentIsNull());
        }
        SpringBeanProcessor processor = new SpringBeanProcessor(deployment); ②
        configurableWebApplicationContext.addBeanFactoryPostProcessor(processor);
        configurableWebApplicationContext.addApplicationListener(processor);
    }
```

① an instance of RESTEasy's `ResteasyDeployment` must be retrieved from the `servletContext`.

② and register with Spring

RESTEasy's Spring integration supports both singleton and prototype scope. It handles injecting `@Context` references. Constructor injection is not supported. With the prototype scope RESTEasy will inject any `@*Param` annotated fields or setters before the request is dispatched.



Only auto-proxied beans can be used with RESTEasy's Spring integration. There will be undesirable affects if you use hardcoded proxying with Spring, i.e., with `ProxyFactoryBean`.

52.4. Spring MVC Integration

RESTEasy can be integrated with the Spring MVC Framework. Generally speaking, Jakarta RESTful Web Services can be combined with a Spring `org.springframework.web.servlet.DispatcherServlet` and used in the same web application. An application combined in this way allows the application to dispatch to either the Spring controller or the Jakarta RESTful Web Services resource using the same base URL. In addition Spring `ModelAndView` objects can be returned as arguments from `@GET` resource methods.

The Spring MVC Integration Example demonstrates how to configure an application using Spring MVC.

`resteasy-spring` provides bean xml file, `springmvc-resteasy.xml`. It resides in the `org.jboss.resteasy.spring:resteasy-spring` archive. The file defines the beans for the Spring MVC/RESTEasy integration. The file is required to be imported into the user's MVC application. This bean file can be used as a template to define more advanced functionality, such as configuring multiple RESTEasy factories, dispatchers and registries.

In the example, the reference to `springmvc-resteasy.xml` is declared in an application provided bean xml named, `resteasy-spring-mvc-servlet.xml`. This file imports `springmvc-resteasy.xml`.

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="
        http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-2.5.xsd
        http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
    ">
       <context:component-scan base-package="org.jboss.resteasy.examples.springmvc"/>
①
       <context:annotation-config/>
       <import resource="classpath:springmvc-resteasy.xml"/> ②
....
</beans>
```

① The application must tell Spring the package to scan for its Jakarta RESTful Web Services resource classes

② A reference to resteasy-spring project's `springmvc-resteasy.xml`

The setup requires the application to provide a `web.xml` file in which a Spring `DispatcherServlet` implementation is declared. Project resteasy-spring provides a default Spring `DispatcherServlet` implementation, `org.jboss.resteasy.springmvc.ResteasySpringDispatcherServlet`. This is the `DispatcherServlet` used in the example code.

The `DispatcherServlet` takes as an input parameter a reference to the application's bean file. This reference is declared as an `init-param` to the `servlet`.

The application's web.xml should define the servlet as follows:

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
   version="5.0">
    <display-name>resteasy-spring-mvc</display-name>

    <servlet>
        <servlet-name>resteasy-spring-mvc</servlet-name>
        <servlet-class>
org.jboss.resteasy.springmvc.ResteasySpringDispatcherServlet</servlet-class> ①
        <init-param>
            <param-name>contextConfigLocation</param-name>
            <param-value>classpath:resteasy-spring-mvc-servlet.xml</param-value> ②
        </init-param>
    </servlet>
    ....
</web-app>
```

① An implementation of Spring's `DispatcherServlet`

② The application's bean xml file that imports `org.jboss.resteasy.spring:resteasy-spring`

archive's `springmvc-resteasy.xml` file.

A `jakarta.ws.rs.core.Application` subclass can be combined with a Spring `DispatcherServlet` and used in the same web application. In this scenario a servlet declaration is required for the Spring `DispatcherServlet` and the `jakarta.ws.rs.core.Application` subclass. A RESTEasy Configuration Switch, `resteasy.scan.resources` must be declared as a context-param in the `web.xml`. Here is an example of the minimum configuration information needed in the `web.xml`.

```
<web-app>
    <servlet>
        <servlet-name>mySpring</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-
class> ①
    </servlet>
    <servlet-mapping>
        <servlet-name>mySpring</servlet-name>
        <url-pattern>/*</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>myAppSubclass</servlet-name>
        <servlet-class>org.my.app.EntryApplicationSubclass</servlet-class> ②
    </servlet>
    <servlet-mapping>
        <servlet-name>myAppSubclass</servlet-name>
        <url-pattern>/*</url-pattern>
    </servlet-mapping>

    <!-- required RESTEasy Configuration Switch directs auto scanning
         of the archive for Jakarta RESTful Web Services resource files
    -->
    <context-param>
        <param-name>resteasy.scan.resources</param-name> ③
        <param-value>true</param-value>
    </context-param>
</web-app>
```

① `DispatcherServlet` declaration

② `Application` declaration

③ scanning configuration switch



RESTEasy parameters like `resteasy.scan.resources` may be set in a variety of ways. See Configuration for more information about application configuration.

If the web application contains Jakarta RESTful Web Services provider classes the RESTEasy Configuration Switch, `resteasy.scan.providers`, will also be needed. If the url-pattern for the Jakarta RESTful Web Services `Application` subclass is other than `/` a declaration of RESTEasy Configuration Switch, `resteasy.servlet.mapping.prefix` will be required. This switch can be declared either as a context-param or as a servlet init-param. It's value must be the text that precedes the `/`.

Here is an example of such a `web.xml`:

```
<web-app>
    <servlet>
        <servlet-name>myAppSubclass</servlet-name>
        <servlet-class>org.my.app.EntryApplicationSubclass</servlet-class>

        <init-param>
            <param-name>resteasy.servlet.mapping.prefix</param-name> ①
            <param-value>/resources</param-value>
        </init-param>
    </servlet>
    <servlet-mapping>
        <servlet-name>myAppSubclass</servlet-name>
        <url-pattern>/resources/*</url-pattern> ②
    </servlet-mapping>

    <context-param>
        <param-name>resteasy.scan.resources</param-name> ③
        <param-value>true</param-value>
    </context-param>
    <context-param>
        <param-name>resteasy.scan.providers</param-name> ③
        <param-value>true</param-value>
    </context-param>
</web-app>
```

① Configuration switch, resteasy.servlet.mapping.prefix, specified in an init-param

② The url-pattern `/*` is preceded by /resources.

③ Configuration switches specified as context-params

52.5. Undertow Embedded Spring Container

Project `resteasy-spring` provides an Undertow based embedded Spring container module, `resteasy-undertow-spring`. It provides class, `org.jboss.resteasy.plugins.server.undertow.spring.UndertowJaxrsSpringServer`. This class has a single method, `undertowDeployment` which requires an input parameter that references a Spring context configuration file. The Spring context configuration data is used to wire Spring into Undertow.

An example of setting up the Undertow embedded Spring container can be found in example, resteasy-spring-undertow

To use this container the user must add the following two archives to their maven project.

```
<dependency>
  <groupId>org.jboss.resteasy</groupId>
  <artifactId>resteasy-undertow</artifactId>
```

```
  <version>8.0.0.Alpha1</version>
</dependency>
<dependency>
  <groupId>org.jboss.resteasy</groupId>
  <artifactId>resteasy-undertow-spring</artifactId>
  <version>8.0.0.Alpha1</version>
</dependency>
```

The user's application must provide a bean xml file that imports a reference to the `org.jboss.resteasy.spring:resteasy-spring` archive's bean file, `springmvc-resteasy.xml`. In the example code the user's provided file is named, `spring-servlet.xml` Here is the information needed in this file.

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:p="http://www.springframework.org/schema/p"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:util="http://www.springframework.org/schema/util"
       xsi:schemaLocation="
        http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-2.5.xsd
        http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util-2.5.xsd
        http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
        ">
    <context:component-scan base-package="org.jboss.resteasy.springmvc.test"/>
    <context:annotation-config/>
    <import resource="classpath:springmvc-resteasy.xml"/>
</beans>
```

Below is a code snippet that shows the creation and configuration of the Undertow embedded Spring container.

```
public static void main(final String[] args) throws Exception {
    UndertowJaxrsSpringServer server = new UndertowJaxrsSpringServer(); ①
    server.start();

    DeploymentInfo deployment = server.undertowDeployment("classpath:spring-
servlet.xml", null); ②
    deployment.setDeploymentName(BasicSpringTest.class.getName());
    deployment.setContextPath("/");
    deployment.setClassLoader(BasicSpringTest.class.getClassLoader());
    server.deploy(deployment);
}
```

① Create an instance of the Undertow Spring server

② Provide the server a reference to the user's application bean.xml

52.6. Processing Spring Web REST annotations in RESTEasy

RESTEasy also provides the ability to process Spring Web REST annotations (i.e. Spring classes annotated with `@RestController`) and handle related REST requests without delegating to Spring MVC. This functionality is currently experimental.

In order for RESTEasy to be able to process Spring `@RestController`, the user must include the following maven dependency.

```
<dependency>
    <groupId>org.jboss.resteasy.spring</groupId>
    <artifactId>resteasy-spring-web</artifactId>
    <version>${version.org.jboss.resteasy.spring}</version>
</dependency>
```

RESTEasy does not auto-scan for `@RestController` annotated classes, so all `@RestController` annotated classes need to be declared in the application's `web.xml` file as shown below.

```
 <web-app>
   <display-name>RESTEasy application using Spring REST annotations</display-name>

    <context-param>
        <param-name>resteasy.scanned.resource.classes.with.builder</param-name>
        <param-
value>org.jboss.resteasy.spi.metadata.SpringResourceBuilder:org.example.Controller1,or
g.example.Controller2</param-value>
    </context-param>
</web-app>
```

In the example above, `Controller1` and `Controller2` are registered and expected to be annotated with `@RestController`.

Currently supported annotations:

Annotation	Comment
`@RestController`	
`@RequestMapping`	
`@GetMapping`	
`@PostMapping`	
`@PutMapping`	
`@DeleteMapping`	
`@PatchMapping`	

Annotation	Comment
`@RequestParam`	
`@RequestHeader`	
`@MatrixVariable`	
`@PathVariable`	
`@CookieValue`	
`@RequestBody`	
`@ResponseStatus`	Only supported as a method annotation
`@RequestParam`	

The use of `org.springframework.http.ResponseEntity`, `jakarta.servlet.http.HttpServletRequest` and `jakarta.servlet.http.HttpServletResponse` are supported as return values of method parameters.

A usage example can be found in sample project resteasy-spring-rest

52.7. Spring Boot starter

RESTEasy supports Spring Boot integration. The code was developed by PayPal and donated to the RESTEasy community. The project is maintained in the RESTEasy Spring Boot Starter Project.

Here is the usage in brief:

Add this maven dependency to the Spring Boot application:

```
<dependency>
   <groupId>org.jboss.resteasy</groupId>
   <artifactId>resteasy-spring-boot-starter</artifactId>
   <version>${latest_version_of_restesy_spring_boot}</version>
   <scope>runtime</scope>
</dependency>
```

Use Spring annotation `@Component` to register a Jakarta RESTful Web Services `Application` class:

```
package com.sample.app;

import org.springframework.stereotype.Component;
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@Component
@ApplicationPath("/sample-app/")
public class JaxrsApplication extends Application {
}
```

To register Jakarta RESTful Web Services resources and providers, define them as Spring beans.

They will be automatically registered. Notice that Jakarta RESTful Web Services resources can be singleton or request scoped, while Jakarta RESTful Web Services providers must be singletons.

To see an example, please check the sample-app.

52.8. Upgrading in Wildfly

As noted in Upgrading RESTEasy within WildFly, Galleon is used in updating RESTEasy distributions in Wildfly. RESTEasy Spring also uses Galleon for the same task. Follow the directions in that section to install Galleon, then run command:

```
galleon.sh install org.jboss.resteasy.spring:galleon-feature-pack:{CURRENT-VERSION}
--dir=${WILDFLY-HOME}
```

Note Installing resteasy-spring feature pack, also installs the corresponding RESTEasy archives do to transitive dependencies.

Chapter 53. CDI Integration

This module provides integration with (Contexts and Dependency Injection for the Jakarta EE platform)

53.1. Using CDI beans as Jakarta RESTful Web Services components

Both the Jakarta RESTful Web Services and CDI specifications introduce their own component model. On the one hand, every class placed in a CDI archive that fulfills a set of basic constraints is implicitly a CDI bean. On the other hand, explicit decoration of a Java class with `@Path` or `@Provider` is required for it to become a Jakarta RESTful Web Services component. Without the integration code, annotating a class suitable for being a CDI bean with annotations leads into a faulty result (Jakarta RESTful Web Services component not managed by CDI). Jakarta RESTful Web Services The resteasy-cdi module is a bridge that allows RESTEasy to work with class instances obtained from the CDI container.

During a web service invocation, resteasy-cdi asks the CDI container for the managed instance of a Jakarta RESTful Web Services component. Then, this instance is passed to RESTEasy. If a managed instance is not available for some reason (the class is placed in a jar which is not a bean deployment archive), RESTEasy falls back to instantiating the class itself.

As a result, CDI services like injection, lifecycle management, events, decoration and interceptor bindings can be used in Jakarta RESTful Web Services components.

53.2. Default scopes

A CDI bean that does not explicitly define a scope is `@Dependent` scoped by default. This pseudo scope means that the bean adapts to the lifecycle of the bean it is injected into. Normal scopes (request, session, application) are more suitable for Jakarta RESTful Web Services components as they designate component's lifecycle boundaries explicitly. Therefore, the resteasy-cdi module alters the default scoping in the following way:

- If a Jakarta RESTful Web Services root resource does not define a scope explicitly, it is bound to the Request scope.
- If a Jakarta RESTful Web Services Provider or `jakarta.ws.rs.Application` subclass does not define a scope explicitly, it is bound to the Application scope.



Since the scope of all beans that do not declare a scope is modified by resteasy-cdi, this affects session beans as well. As a result, a conflict occurs if the scope of a stateless session bean or singleton is changed automatically as the spec prohibits these components to be @RequestScoped. Therefore, you need to explicitly define a scope when using stateless session beans or singletons. This requirement is likely to be removed in future releases.

53.3. Constructor Injection

CDI resources support constructor injection without requiring a public no-arg constructor. According to the CDI specification, normal scoped beans (such as `@RequestScoped` and `@ApplicationScoped`) require a non-private constructor with no parameters for proxy creation. This constructor can be `public`, `protected`, or package-private (default access).

RESTEasy supports combining a non-public no-arg constructor (for CDI proxy creation) with an `@Inject`-annotated constructor for dependency injection. This allows you to hide the no-arg constructor since it should only be used internally by CDI for proxy generation.

```
@Path("/example")
@RequestScoped
public class ExampleResource {

    private final ExampleService service;

    @Inject
    private UriInfo uriInfo;

    // Package-private no-arg constructor for CDI proxy generation
    // This constructor is never called directly - only used by CDI for creating the
proxy subclass
    ExampleResource() {
        this.service = null;
    }

    // Public constructor with CDI dependency injection
    // This is the constructor CDI actually uses to create the bean instance
    @Inject
    public ExampleResource(ExampleService service) {
        this.service = service;
    }

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getMessage() {
        return service.getMessage();
    }
}
```

This pattern provides several benefits:

- **Encapsulation**: The no-arg constructor is hidden (package-private or protected), making it clear that it shouldn't be called directly.
- **Required dependencies are explicit**: The `@Inject` constructor clearly shows which dependencies are required for the resource to function.
- **CDI handles all instantiation**: RESTEasy delegates resource creation entirely to CDI, allowing

full use of CDI features including interceptors and decorators.



For normal scoped beans (`@RequestScoped`, `@ApplicationScoped`, etc.), CDI creates a client proxy that extends your bean class. The proxy requires a non-private no-arg constructor to call `super()`. The `@Inject` constructor is used by CDI when creating the actual bean instance behind the proxy.

Resources can use `public`, `protected`, or package-private no-arg constructors. Using `protected` or package-private constructors follows CDI best practices by hiding implementation details.

53.4. Parameter Injection with CDI Qualifiers

Jakarta RESTful Web Services parameter annotations (`@PathParam`, `@QueryParam`, `@HeaderParam`, `@MatrixParam`, `@CookieParam`, `@FormParam`, and `@BeanParam`) are registered as CDI qualifiers by RESTEasy. This allows parameter values to be injected directly via CDI without requiring a no-arg constructor at all.

53.4.1. Constructor Parameter Injection

Resource classes can receive Jakarta RESTful Web Services parameter values directly in an `@Inject` constructor:

```
@Path("/items/{id}")
@RequestScoped
public class ItemResource {

    private final String id;
    private final ItemService service;

    @Inject
    public ItemResource(@PathParam("id") String id, ItemService service) {
        this.id = id;
        this.service = service;
    }

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public Item getItem() {
        return service.findById(id);
    }
}
```

No no-arg constructor is needed. CDI resolves the `@PathParam` qualifier through a RESTEasy-provided producer that extracts the value from the current request context. Standard CDI beans (like `ItemService`) and Jakarta RESTful Web Services parameter values can be mixed freely in the same constructor.

53.4.2. Field Injection

Fields annotated with `@Inject` and a Jakarta RESTful Web Services parameter annotation receive parameter values from the current request:

```
@Path("/items/{id}")
@RequestScoped
public class ItemResource {

    @Inject
    @PathParam("id")
    private String id;

    @Inject
    @QueryParam("expand")
    private boolean expand;

    @Inject
    @HeaderParam("Accept-Language")
    private String language;

    @Inject
    @DefaultValue("10")
    @QueryParam("limit")
    private int limit;
}
```

53.4.3. Setter Method Injection

Setter methods annotated with `@Inject` can receive parameter values. The parameter annotation may be placed on either the method or the method parameter:

```
@Path("/items/{id}")
@RequestScoped
public class ItemResource {

    private String id;
    private String filter;

    // Annotation on parameter
    @Inject
    public void setId(@PathParam("id") String id) {
        this.id = id;
    }

    // Annotation on method
    @Inject
    @QueryParam("filter")
    public void setFilter(String filter) {
```

```
        this.filter = filter;
    }
}
```

When the annotation is placed on the method, the parameter name is derived from the method name using JavaBean conventions (e.g. `setUserId` resolves to `userId`). When the annotation is placed on the parameter, the parameter name is taken from the annotation value.

53.4.4. Bean Parameter Injection

The `@BeanParam` annotation aggregates multiple parameters into a single object:

```
public class SearchParams {
    @Inject
    @QueryParam("q")
    private String query;

    @Inject
    @QueryParam("limit")
    @DefaultValue("10")
    private int limit;

    @Inject
    @HeaderParam("Accept-Language")
    private String language;
}

@Path("/search")
@RequestScoped
public class SearchResource {

    @Inject
    @BeanParam
    private SearchParams params;

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Item> search() {
        return searchItems(params);
    }
}
```

53.4.5. Type Conversion

Parameter values are converted from their string representation to the target Java type using the same conversion rules as standard Jakarta RESTful Web Services parameter handling (see the Jakarta RESTful Web Services specification for details on `ParamConverterProvider`, `valueOf`, and `fromString` resolution).

53.5. Configuration within WildFly

CDI integration is provided with no additional configuration with WildFly.

53.6. Configuration with different distributions

Provided there is an existing RESTEasy application, all that needs to be done is to add the resteasy-cdi jar into the project's `WEB-INF/lib` directory. When using maven, this can be achieved by defining the following dependency.

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-cdi</artifactId>
    <version>${project.version}</version>
</dependency>
```

Furthermore, when running a pre-Servlet 3 container, the following context parameter needs to be specified in web.xml. (This is done automatically via web-fragment in a Servlet 3 environment)

```
<context-param>
    <param-name>resteasy.injector.factory</param-name>
    <param-value>org.jboss.resteasy.cdi.CdiInjectorFactory</param-value>
</context-param>
```

When deploying an application to a Servlet container that does not support CDI out of the box (Tomcat, Jetty, Google App Engine), a CDI implementation needs to be added first. Weld-servlet module can be used for this purpose.

Chapter 54. RESTEasy Client API

54.1. Jakarta RESTful Web Services Client API

The Jakarta RESTful Web Services includes a client API so the user can make http requests to remote RESTful web services. It is a 'fluent' request building API with really 3 main classes: `Client`, `WebTarget`, and `Response`. The `Client` interface is a builder of `WebTarget` instances. `WebTarget` represents a distinct URL or URL template from which the user can build more sub-resource WebTargets or invoke requests on.

There are really two ways to create a `Client`. Standard way, or to use the `ResteasyClientBuilder` class. The advantage of the latter is that it provides a few more helper methods to configure the client.

```
Client client = ClientBuilder.newClient();
// or
client = ClientBuilder.newBuilder().build();
WebTarget target = client.target("http://foo.com/resource");
Response response = target.request().get();
String value = response.readEntity(String.class);
response.close();  // You should close connections!

Client client = ClientBuilder.newClient();
WebTarget target = client.target("http://foo.com/resource");
```

RESTEasy will automatically load a set of default providers. (Basically all classes listed in all `META-INF/services/jakarta.ws.rs.ext.Providers` files). Additionally, other providers, filters, and interceptors can be manually registered through the Configuration object provided by the method call `Client.configuration()`. Configuration also lets the user set various configuration properties that may be needed.

Each WebTarget has its own Configuration instance which inherits the components and properties registered with its parent. This allows the user to set specific configuration options per target resource. For example, username and password.

One RESTEasy extension to the client API is the ability to specify that requests should be sent in "chunked" transfer mode. There are two ways of doing that. One is to configure an `org.jboss.resteasy.client.jaxrs.ResteasyWebTarget` so that all requests to that target are sent in chunked mode:

```
ResteasyClient client = (ResteasyClient)ClientBuilder.newClient();
ResteasyWebTarget target = client.target("http://localhost:8081/test");
target.setChunked(b.booleanValue());
Invocation.Builder request = target.request();
```

Alternatively, it is possible to configure a particular request to be sent in chunked mode:

```
ResteasyClient client = (ResteasyClient)ClientBuilder.newClient();
ResteasyWebTarget target = client.target("http://localhost:8081/test");
ClientInvocationBuilder request = (ClientInvocationBuilder) target.request();
request.setChunked(b);
```

Note that `org.jboss.resteasy.client.jaxrs.internal.ClientInvocationBuilder`, unlike `jakarta.ws.rs.client.Invocation.Builder`, is a RESTEasy class.



The ability to send in chunked mode depends on the underlying transport layer; in particular, it depends on which implementation of `org.jboss.resteasy.client.jaxrs.ClientHttpEngine` is being used. Currently, only the default implementation, `ApacheHttpClient43Engine`, supports chunked mode. See Section Apache HTTP Client 4.x and other backends for more information.



To follow REST principles and avoid introducing state management in applications, `jakarta.ws.rs.client.Client` instances do not provide support for cookie management by default. However, the user can enable it if necessary using `ResteasyClientBuilder`:

```
Client client = ((ResteasyClientBuilder)
ClientBuilder.newBuilder()).enableCookieManagement().build();
```

54.2. RESTEasy Proxy Framework

The RESTEasy Proxy Framework is the mirror opposite of the Jakarta RESTful Web Services server-side specification. Instead of using Jakarta RESTful Web Services annotations to map an incoming request to Jakarta RESTful Web Services methods, the client framework builds an HTTP request that it uses to invoke a remote Jakarta RESTful Web Services. This remote service does not have to be a Jakarta RESTful Web Services service and can be any web resource that accepts HTTP requests.

RESTEasy has a client proxy framework that allows the use of Jakarta RESTful Web Services annotations to invoke a remote HTTP resource. Write a Java interface and use Jakarta RESTful Web Services annotations on methods and the interface. For example:

```
public interface SimpleClient {
   @GET
   @Path("basic")
   @Produces("text/plain")
   String getBasic();

   @PUT
   @Path("basic")
   @Consumes("text/plain")
   void putBasic(String body);
```

```
    @GET
    @Path("queryParam")
    @Produces("text/plain")
    String getQueryParam(@QueryParam("param")String param);

    @GET
    @Path("matrixParam")
    @Produces("text/plain")
    String getMatrixParam(@MatrixParam("param")String param);

    @GET
    @Path("uriParam/{param}")
    @Produces("text/plain")
    int getUriParam(@PathParam("param")int param);
}
```

RESTEasy has a simple API to generate a proxy, then invoke methods on the proxy. The invoked method gets translated to an HTTP request based on how the method was annotated and posted to the server. Here's how to set this up:

```
public void invokeClient() {
    try (Client client = ClientBuilder.newClient()) {
          WebTarget target = client.target("http://example.com/base/uri");
          ResteasyWebTarget rtarget = (ResteasyWebTarget)target;

          SimpleClient simple = rtarget.proxy(SimpleClient.class);
          simple.putBasic("hello world");
    }
}
```

Alternatively use the RESTEasy client extension interfaces directly:

```
public void invokeClient() {
    try (ResteasyClient client = (ResteasyClient)ClientBuilder.newClient()) {
        ResteasyWebTarget target = client.target("http://example.com/base/uri");

        SimpleClient simple = target.proxy(SimpleClient.class);
        simple.putBasic("hello world");
    }
}
```

`@CookieParam` works the mirror opposite of its server-side counterpart and creates a cookie header to send to the server. It is not needed to use `@CookieParam` if you allocate your own `jakarta.ws.rs.core.Cookie` object and pass it as a parameter to a client proxy method. The client framework understands the cookie is being passed to the server so no extra metadata is needed.

The framework supports the Jakarta RESTful Web Services locator pattern, but on the client side. So, if there is a method annotated only with @Path, that proxy method will return a new proxy of

the interface returned by that method.

54.2.1. Abstract Responses

Sometimes there is interest in viewing the response status code and/or the response headers in addition to the response body of the client request. The Client-Proxy framework has two ways to get at this information

A `jakarta.ws.rs.core.Response.Status` enumeration can be returned from the method.

```
@Path("/")
public interface MyProxy {
   @POST
   Response.Status updateSite(MyPojo pojo);
}
```

Internally, after invoking the server, the client proxy internals will convert the HTTP response code into a `Response.Status` enum.

The `jakarta.ws.rs.core.Response` class provides all accessible information:

```
@Path("/")
public interface LibraryService {

   @GET
   @Produces("application/xml")
   Response getAllBooks();
}
```

54.2.2. Response proxies

A further extension implemented by the RESTEasy client proxy framework is the "response proxy facility", where a client proxy method returns an interface that represents the information contained in a `jakarta.ws.rs.core.Response`. Such an interface must be annotated with `@ResponseObject` from package `org.jboss.resteasy.annotations`, and its methods may be further annotated with `@Body`, `@LinkHeaderParam`, and `@Status` from the same package, as well as `jakarta.ws.rs.HeaderParam`. Consider the following example.

```
@ResponseObject
public interface TestResponseObject {

   @Status
   int status();

   @Body
   String body();
```

```
    @HeaderParam("Content-Type")
    String contentType();

    ClientResponse response();
 }

 @Path("test")
 public interface TestClient {

    @GET
    TestResponseObject get();
 }

 @Path("test")
 public static class TestResource {

    @GET
    @Produces("text/plain")
    public String get() {
       return "ABC";
    }
 }
```

Here, `TestClient` will define the client side proxy for `TestResource`. Note that `TestResource.get()` returns a `String` but the proxy based on `TestClient` will return a `TestResponseObject` on a call to `get()`:

```
Client client = ClientBuilder.newClient();
TestClient ClientInterface = ProxyBuilder.builder(TestClient.class, client.target(
"http://localhost:8081")).build();
TestResponseObject tro = ClientInterface.get();
```

The methods of `TestResponseObject` provide access to various pieces of information about the response received from `TestResponse.get()`. This is where the annotations on those methods come into play. `status()` is annotated with `@Status`, and a call to `status()` returns the HTTP status. Similarly, `body()` returns the returned entity, and `contentType()` returns the value of the response header Content-Type:

```
System.out.printf("status: %s", tro.status());
System.out.printf("entity: %s%n", tro.body());
System.out.printf("Content-Type: %s%n", tro.contentType());
```

will yield

```
status: 200
entity: ABC
```

```
Content-Type: text/plain;charset=UTF-8
```

Note that there is one other method in `TestResponseObject`, `response()`, that has no annotation. When RESTEasy sees a method in an interface annotated with `@ResponseObject` that returns a `jakarta.ws.rs.core.Response` (or a subclass thereof), it will return a `org.jboss.resteasy.client.jaxrs.internal.ClientResponse`. For example,

```
ClientResponse clientResponse =  tro.response();
System.out.printf("Content-Length: %d%n", clientResponse.getLength());
```

Perhaps the most interesting piece of the response proxy facility is the treatment of methods annotated with `@LinkHeaderParam`. Its simplest use is to assist in accessing a `jakarta.ws.rs.core.Link` returned by a resource method. For example, let's add

```
@GET
@Path("/link-header")
public Response getWithHeader(@Context UriInfo uri) {
   URI subUri = uri.getAbsolutePathBuilder().path("next-link").build();
   Link link = new LinkBuilderImpl().uri(subUri).rel("nextLink").build();
   return Response.noContent().header("Link", link.toString()).build();
}
```

to `TestResource`, add

```
@GET
@Path("link-header")
ResponseObjectInterface performGetBasedOnHeader();
```

to `ClientInterface`, and add

```
@LinkHeaderParam(rel = "nextLink")
URI nextLink();
```

to `ResponseObjectInterface`. Then calling

```
ResponseObjectInterface obj = ClientInterface.performGetBasedOnHeader();
System.out.printf("nextLink(): %s%n", obj.nextLink());
```

will access the `LinkHeader` returned by `TestResource.getWithHeader()`:

```
nextlink: http://localhost:8081/test/link-header/next-link
```

Last but not least, let's add

```
@GET
@Produces("text/plain")
@Path("/link-header/next-link")
public String getHeaderForward() {
   return "forwarded";
}
```

to `TestResource` and

```
@GET
@LinkHeaderParam(rel = "nextLink")
String followNextLink();
```

to `ResponseObjectInterface`. Note that, unlike `ResponseObjectInterface.nextLink()`, `followNextLink()` is annotated with `@GET`; that is, it qualifies as (the client proxy to) a resource method. When executing `followNextLink()`, RESTEasy will retrieve the value of the `Link` returned by `TestResource.getWithHeader()` and then will make a GET invocation on the `URL` in that `Link`. Calling

```
System.out.printf("followNextLink(): %s%n", obj.followNextLink());
```

causes RESTEasy to retrieve the `URL` http://localhost:8081/test/link-header/next-link from the call to `TestResource.getWithHeader()` and then perform a GET on it, invoking `TestResource.getHeaderForward()`:

```
followNextLink(): forwarded
```



This facility for extracting a `URL` and following it is a step toward supporting the Representation State Transfer principle of HATEOAS. For more information, see RESTful Java with JAX-RS 2.0, 2nd Edition by Bill Burke.

54.2.3. Giving client proxy an ad hoc URI

Client proxies figure out appropriate URIs for targeting resource methods by looking at `@Path` annotations in the client side interface, but it is also possible to pass URIs explicitly to the proxy through the use of the `org.jboss.resteasy.annotations.ClientURI` annotation. For example, let `TestResource` be a client side interface and `TestResourceImpl` a server resource:

```
@Path("")
public interface TestResource {

  @GET
  @Path("dispatch")
  String dispatch(@ClientURI String uri);
}
```

```
@Path("")
public static class TestResourceImpl {

  @GET
  @Path("a")
  public String a() {
     return "a";
  }

  @GET
  @Path("b")
  public String b() {
     return "b";
  }
}
```

Calling `TestResource.dispatch()` allows specifying a specific URI for accessing a resource method. In the following, let BASE_URL be the address of the `TestResourceImpl` resource.

```
 private static String BASE_URL = "http://localhost:8081/";

 public void test() throws Exception {
    try (ResteasyClient client = (ResteasyClient)ClientBuilder.newClient()) {
        TestResource proxy = client.target(BASE_URL).proxy(TestResource.class);
        String name = proxy.dispatch(BASE_URL + "a");
        System.out.printf("name: %s%n", name);
        name = proxy.dispatch(BASE_URL + "b");
        System.out.println("name: %s%n", name);
    }
}
```

Then passing "http://localhost:8081/a" and "http://localhost/b" to `dispatch()` invokes `TestResourceImp.a()` and `TestResourceImpl.b()` respectively, yielding the output

```
name: a
name: b
```

54.2.4. Sharing an interface between client and server

It is generally possible to share an interface between the client and server. In this scenario, just have the Jakarta RESTful Web Services services implement an annotated interface and then reuse that same interface to create client proxies to invoke on the client-side.

54.3. Apache HTTP Client 4.x and other backends



The Apache HTTP Client support is deprecated in RESTEasy. The exposed API's will

> eventually be removed. However, it's still the default client implementation as we prepare for a replacement backing HTTP client.

Network communication between the client and server is handled by default in RESTEasy. The interface between the RESTEasy Client Framework and the network is defined by RESTEasy's `ClientHttpEngine` interface. RESTEasy ships with multiple implementations of this interface.

The default implementation is `ApacheHttpClient43Engine`, which uses version 4.3 of the `HttpClient` from the Apache `HttpComponents` project.

`ApacheHttpAsyncClient4Engine`, instead, is built on top of *HttpAsyncClient* (still from the Apache *HttpComponents* project) with internally dispatches requests using a non-blocking IO model.

`JettyClientEngine` is built on top of *Eclipse Jetty* HTTP engine, which is possibly an interesting option for those already running on the Jetty server.

`VertxClientHttpEngine` is built on top of *Eclipse Vert.x*, which provides a non-blocking HTTP client based on Vert.x framework.

`ReactorNettyClientHttpEngine` is built on top of *Reactor Netty*, which provides a non-blocking HTTP client based on Netty framework.

RESTEasy ClientHttpEngine implementations	
ApacheHttpClient43Engine (deprecated)	Uses HttpComponents HttpClient 4.3+
ApacheHttpAsyncClient4Engine (deprecated)	Uses HttpComponents HttpAsyncClient
JettyClientEngine	Uses Eclipse Jetty
ReactorNettyClientHttpEngine	Uses Reactor Netty
VertxClientHttpEngine	Uses Eclipse Vert.x
URLConnectionEngine	Uses java.net.HttpURLConnection

The RESTEasy Client Framework can also be customized. The user can provide their own implementations of `ClientHttpEngine` to the `ResteasyClient`.

```
ClientHttpEngine myEngine = new ClientHttpEngine() {

    @Override
    public ClientResponse invoke(ClientInvocation request) {
        // implement your processing code and return a
        // org.jboss.resteasy.client.jaxrs.internal.ClientResponse
        // object.
    }

    @Override
    public SSLContext getSslContext() {
       return sslContext;
    }
```

```
    @Override
    public HostnameVerifier getHostnameVerifier() {
       return hostnameVerifier;
    }

    @Override
    public void close() {
       // do nothing
    }
};

Client client = ClientBuilder.newBuilder().register(myEngine).build();
```



If you include a `META-INF/services/org.jboss.resteasy.client.jaxrs.engine.ClientHttpEngineFactory` and an implementation of the interface, you do not need to register your engine in the `ClientBuilder`. You can simply do `Client client = ClientBuilder.newClient()` and your backing engine will be used.



You can also add a `jakarta.annotations.Priority` annotation on your `org.jboss.resteasy.client.jaxrs.engine.ClientHttpEngineFactory` implementation to rank the priorities.

RESTEasy and `HttpClient` make reasonable default decisions so that it is possible to use the client framework without ever referencing `HttpClient`. For some applications it may be necessary to drill down into the `HttpClient` details. `ApacheHttpClient43Engine` can be supplied with an instance of `org.apache.http.client.HttpClient` and an instance of `org.apache.http.protocol.HttpContext`, which can carry additional configuration details into the `HttpClient` layer.

`HttpContextProvider` is a RESTEasy provided interface through which a custom `HttpContext` is supplied to `ApacheHttpClient43Engine`.

```
package org.jboss.resteasy.client.jaxrs.engines;

import org.apache.http.protocol.HttpContext;

public interface HttpContextProvider {
   HttpContext getContext();
}
```

Here is an example of providing a custom HttpContext

```
DefaultHttpClient httpClient = new DefaultHttpClient();
ApacheHttpClient43Engine engine = new ApacheHttpClient43Engine(httpClient,
   new HttpContextProvider() {
           @Override
           public HttpContext getContext() {
```

```
            // Configure HttpClient to authenticate preemptively
            // by prepopulating the authentication data cache.
            // 1. Create AuthCache instance
            AuthCache authCache = new BasicAuthCache();
            // 2. Generate BASIC scheme object and add it to the local auth cache
            BasicScheme basicAuth = new BasicScheme();
            authCache.put(getHttpHost(url), basicAuth);
            // 3. Add AuthCache to the execution context
            BasicHttpContext localContext = new BasicHttpContext();
            localContext.setAttribute(ClientContext.AUTH_CACHE, authCache);
            return localContext;
         }
});
```

54.3.1. HTTP redirect

The `ClientHttpEngine` implementations based on Apache `HttpClient` support HTTP redirection. The feature is disabled by default and has to be enabled by users explicitly. Either by setting up the following property:

- `dev.resteasy.client.follow.redirects`

```
Client client = ClientBuilder.newBuilder().property(
"dev.resteasy.client.follow.redirects", "true").build();
```

or by explicitly calling the API method as following:

```
ApacheHttpClient43Engine engine = new ApacheHttpClient43Engine();
engine.setFollowRedirects(true);
Client client = ((ResteasyClientBuilder)ClientBuilder.newBuilder()).httpEngine(engine
).build();
```

54.3.2. Configuring SSL

To enable SSL on client, a `ClientHttpEngine` containing a SSLContext can be created to build client as in the following example:

```
public Client createClient() {
  ClientHttpEngine myEngine = new ClientHttpEngine() {
     public void setSslContext(SSLContext sslContext) {
        this.sslContext = sslContext;
     }

     @Override
     public HostnameVerifier getHostnameVerifier() {
        return hostnameVerifier;
     }
```

```
  };
  myEngine.setSslContext(mySslContext);
  return ((ResteasyClientBuilder)ClientBuilder.newBuilder()).httpEngine(myEngine)
.build();
}
```

An alternative is to set up a keystore and truststore and pass a custom SslContext to ClientBuilder:

```
Client sslClient = ClientBuilder.newBuilder().sslContext(mySslContext).build();
```

If you don't want to create a SSLContext, you can build client with a keystore and truststore. Note if both SSLContext and keystore/truststore are configured, the later will be ignored by Resteasy ClientBuilder.

```
Client sslClient = ClientBuilder.newBuilder().keystore(keystore,mypassword).
                     trustKeystore(trustStore).build();
```

During handshaking, a custom HostNameVerifier can be called to allow the connection if URL's hostname and the server's identification hostname match.

```
Client sslClient =  ((ResteasyClientBuilder)ClientBuilder.newBuilder()).sslContext
(mysslContext)
                      .hostnameVerifier(myhostnameVerifier).build();
```

Resteasy provides another simple way to set up a `HostnameVerifier`. It allows configuring `ResteasyClientBuilder` with a `HostnameVerificationPolicy` without creating a custom `HostNameVerifier`:

```
Client sslClient =  ((ResteasyClientBuilder)ClientBuilder.newBuilder()).sslContext
(mysslContext)
                      .hostnameVerification(ResteasyClientBuilder
.HostnameVerificationPolicy.ANY).build();
```

- Setting `HostnameVerificationPolicy.ANY` will allow all connections without a check.
- `HostnameVerificationPolicy.WILDCARD` only allows wildcards in subdomain names i.e. *.foo.com.
- `HostnameVerificationPolicy.STRICT` checks if DNS names match the content of the Public Suffix List (https://publicsuffix.org/list/public_suffix_list.dat). Please note if this public suffix list isn't the check wanted, create your own `HostNameVerifier` instead of this policy setting.

54.3.3. HTTP proxy

The `ClientHttpEngine` implementations based on Apache `HttpClient` support HTTP proxy. This feature can be enabled by setting specific properties on the builder:

- org.jboss.resteasy.jaxrs.client.proxy.host
- org.jboss.resteasy.jaxrs.client.proxy.port
- org.jboss.resteasy.jaxrs.client.proxy.scheme

```
Client client = ClientBuilder.newBuilder().property(
"org.jboss.resteasy.jaxrs.client.proxy.host", "someproxy.com").property(
"org.jboss.resteasy.jaxrs.client.proxy.port", 8080).build();
```

54.3.4. Apache HTTP Client 4.3 APIs

The RESTEasy Client framework automatically creates and properly configures the underlying Apache HTTP Client engine. When the `ApacheHttpClient43Engine` is manually created, though, the user can either let it build and use a default `HttpClient` instance or provide a custom one:

```
public ApacheHttpClient43Engine() {
}

public ApacheHttpClient43Engine(HttpClient httpClient) {
}

public ApacheHttpClient43Engine(HttpClient httpClient, boolean closeHttpClient) {
}
```

The *closeHttpClient* parameter on the last constructor above allows controlling whether the Apache `HttpClient` is to be closed upon engine finalization. The default value is *true*. When a custom `HttpClient` instance is not provided, the default instance will always be closed together with the engine.

For more information about HttpClient (4.x), see the documentation at https://hc.apache.org/index.html/.



It is important to understand the difference between "releasing" a connection and "closing" a connection.

- Releasing a connection makes it available for reuse.
- Closing a connection frees its resources and makes it unusable.

If an execution of a request or a call on a proxy returns a class other than `Response`, then RESTEasy will take care of releasing the connection. For example, in the fragments

```
WebTarget target = client.target("http://localhost:8081/customer/123");
String answer = target.request().get(String.class);
```

or

```
ResteasyWebTarget target = client.target("http://localhost:8081/customer/123");
RegistryStats stats = target.proxy(RegistryStats.class);
RegistryData data = stats.get();
```

RESTEasy will release the connection under the covers. The only counterexample is the case in which the response is an instance of `InputStream`, which must be closed explicitly.

On the other hand, if the result of an invocation is an instance of `Response`, then the Response.close() method must be used to release the connection.

```
WebTarget target = client.target("http://localhost:8081/customer/123");
Response response = target.request().get();
System.out.println(response.getStatus());
response.close();
```

It is advisable to execute this in a try-with-resources block. Again, releasing a connection only makes it available for another use. **It does not normally close the socket.**

On the other hand, `ApacheHttpClient43Engine.finalize()` will close any open sockets, unless the user set *closeHttpClient* as *false* when building the engine, in which case he is responsible for closing the connections.

Note that if `ApacheHttpClient43Engine` has created its own instance of `HttpClient`, it is not necessary to wait for `finalize()` to close open sockets. The `ClientHttpEngine` interface has a `close()` method for this purpose.

If the user's `jakarta.ws.rs.client.Client` class has created the engine automatically, the user should call `Client.close()` and this will clean up any socket connections.

Finally, having explicit `finalize()` methods can badly affect performances, the `org.jboss.resteasy.client.jaxrs.engines.ManualClosingApacheHttpClient43Engine` flavour of `org.jboss.resteasy.client.jaxrs.engines.ApacheHttpClient43Engine` can be used. With that the user is always responsible for calling `close()` as no `finalize()` is there to do that before object garbage collection.

54.3.5. Asynchronous HTTP Request Processing

RESTEasy's default async engine implementation class is `ApacheHttpAsyncClient4Engine`. It can be set as the active engine by calling method `useAsyncHttpEngine` in `ResteasyClientBuilder`.

```
public void doTest() {
    Client asyncClient = ((ResteasyClientBuilder)ClientBuilder.newBuilder())
.useAsyncHttpEngine()
                            .build();
    FutureResponse future = asyncClient
                            .target("http://locahost:8080/test").request()
                            .async().get();
```

```
    Response res = future.get();
    Assertions.assertEquals(HttpResponseCodes.SC_OK, res.getStatus());
    String entity = res.readEntity(String.class);
}
```

InvocationCallbacks

InvocationCallbacks are called from within the io-threads and thus must not block or else the application may slow down to a halt. Reading the response is safe because the response is buffered in memory, as are other async and in-memory client-invocations that submit-calls returning a future not containing Response, InputStream or Reader.

```
public void doTest() {
   final CountDownLatch latch = new CountDownLatch(1);
   FutureString future = nioClient.target(generateURL("/test")).request()
         .async().get(new InvocationCallbackString()
               {
                  @Override
                  public void completed(String s)
                  {
                     Assertions.assertEquals("get", s);
                     latch.countDown();
                     throw new RuntimeException("for the test of it");
                  }

                  @Override
                  public void failed(Throwable error)
                  {
                  }
               });
   String entity = future.get();
   Assertions.assertEquals("get", entity);
}
```

InvocationCallbacks may be called seemingly "after" the future-object returns. Thus, responses should be handled solely in the InvocationCallback.

InvocationCallbacks will see the same result as the future-object and vice versa. Thus, if the invocationcallback throws an exception, the future-object will not see it. This is the reason to handle responses only in the InvocationCallback.

Async Engine Usage Considerations

Asynchronous IO means non-blocking IO utilizing few threads, typically at most as many threads as number of cores. As such, performance may profit from fewer thread switches and less memory usage due to fewer thread-stacks. But doing synchronous, blocking IO (the invoke-methods not returning a future) may suffer, because the data has to be transferred piecewise to/from the io-threads.

Request-Entities are fully buffered in memory, thus `HttpAsyncClient` is unsuitable for very large uploads. Response-Entities are buffered in memory, except if requesting a Response, InputStream or Reader as Result. For large downloads or COMET, one of these three return types must be requested, but there may be a performance penalty because the response-body is transferred piecewise from the io-threads. When using InvocationCallbacks, the response is always fully buffered in memory.

54.3.6. Jetty Client Engine

RESTEasy provides an Eclipse Jetty-backed HTTP client engine with support for HTTP/1.1, HTTP/2, SSL/TLS, and asynchronous invocations.

For RESTEasy Jetty documentation, see docs.resteasy.dev/resteasy-jetty.

54.3.7. Vertx Client Engine

RESTEasy provides a Vert.x-backed HTTP client engine with support for HTTP/1.1, HTTP/2, SSL/TLS, and asynchronous invocations.

For RESTEasy Vert.x client documentation, see docs.resteasy.dev/resteasy-vertx.

54.3.8. Reactor Netty Client Engine

Still as a drop in replacement, RESTEasy allows selecting a Reactor Netty based HTTP engine. The Reactor Netty implementation is newer and less tested, but can be a good choice if the user application is already depending on Netty and performs asynchronous client invocations.

The Reactor Netty engine is enabled by adding a dependency to the `org.jboss.resteasy:resteasy-client-reactor-netty` artifact to the Maven project; then the client can be built as follows:

```
ReactorNettyClientHttpEngine engine = new ReactorNettyClientHttpEngine(
   HttpClient.create(),
   new DefaultChannelGroup(new DefaultEventExecutor()),
   HttpResources.get());
ResteasyClient client = ((ResteasyClientBuilder)ClientBuilder.newBuilder())
   .clientEngine(engine).build();
```

When coupled with the MonoRxInvoker, this has several benefits. It supports things like this:

```
webTarget.path("/foo").get().rx(MonoRxInvoker.class).map(...).subscribe()
```

in order to achieve non-blocking HTTP client calls. This allows leveraging some reactor features:

- the ability for a `Mono#timeout` set on the response to aggressively terminate the HTTP request;
- the ability to pass a (reactor) context from client calls into `ReactorNettyClientHttpEngine`.

For some sample code, see `org.jboss.resteasy.reactor.ReactorTest` in the RESTEasy module

resteasy-reactor.



The Reactor Netty client has moved to its own project which can be found at https://github.com/resteasy/resteasy-netty. The packages and GAV's have changed. The new package name is `dev.resteasy.netty.reactor.engine` and only contains `ReactorNettyClientHttpEngine`. The new GAV is as follows:

```
<dependency>
    <groupId>dev.resteasy.netty</groupId>
    <artifactId>resteasy-reactor-netty-client</artifactId>
</dependency>
```

54.4. Client Utilities

The client utilities contain various client side helpers that can be registered on a client. These utilities do not require RESTEasy and can be used with any Jakarta RESTful Web Services implementation.

54.4.1. Client Authentication

The client authentication utilities can be used on a client when an endpoint requires authentication. Currently, BASIC and DIGEST authentication are supported.

```
public Response invoke() {
  try (
          Client client = ClientBuilder.newBuilder()
                  .register(HttpAuthenticators.basic(UserCredentials.clear("user", new
char[] {'p', 'a', 's', 's', 'w', 'o', 'r', 'd'})))
                  .build()
      ) {
      return client.target("https://example.com/api/info")
              .request(MediaType.APPLICATION_JSON_TYPE)
              .get();
    }
}
```

```
final Response response = client.target("https://example.com/api/info")
        .register(HttpAuthenticators.digest(UserCredentials.clear("user", new char[]
{'p', 'a', 's', 's', 'w', 'o', 'r', 'd'})))
        .request(MediaType.APPLICATION_JSON_TYPE)
        .get();
```

Chapter 55. MicroProfile Rest Client

As the microservices style of system architecture (see, for example, Microservices by Martin Fowler) gains increasing traction, new API standards are coming along to support it. One set of such standards comes from the Microprofile Project supported by the Eclipse Foundation, and among Jakarta RESTful Web Services those is one, MicroProfile Rest Client, of particular interest to RESTEasy and . In fact, it is intended to be based on, and consistent with, Jakarta RESTful Web Services, and it includes ideas already implemented in RESTEasy. For a more detailed description of MicroProfile Rest Client, see https://github.com/eclipse/microprofile-rest-client. In particular, the API code is in https://github.com/eclipse/microprofile-rest-client/tree/master/api. and the specification is in https://github.com/eclipse/microprofile-rest-client/tree/master/spec.



As of RESTEasy 5.0.0 the MicroProfile integration has moved to a new project, group id, artifact id and version. The new group id is `org.jboss.resteasy.microprofile`.The artifact id for the client is now `microprofile-client`. To use the MicroProfile Config sources the artifact id is `microprofile-config`. Finally, for context propagation the new artifact is is `microprofile-context-propagation`

You could also use the RESTEasy MicroProfile BOM:

```
<dependency>
    <groupId>org.jboss.resteasy.microprofile</groupId>
    <artifactId>resteasy-microprofile-bom</artifactId>
    <version>${version.org.jboss.resteasy.microprofile}</version>
</dependency>
```

55.1. Client proxies

One of the central ideas in MicroProfile Rest Client is a version of *distributed object communication*, a concept implemented in, among other places, CORBA, Java RMI, the JBoss Remoting project, and RESTEasy. Consider the resource

```
@Path("resource")
public class TestResource {

   @Path("test")
   @GET
   String test() {
      return "test";
   }
}
```

The Jakarta RESTful Web Services native way of accessing `TestResource` looks like

```
public String checkResponse() {
    try (Client client = ClientBuilder.newClient()) {
        return client.target("http://localhost:8081/test").request().get(String.class
);
    }
}
```

The call to `TestResource.test()` is not particularly onerous, but calling `test()` directly allows a more natural syntax. That is exactly what Microprofile Rest Client supports:

```
@Path("resource")
public interface TestResourceClient {

   @Path("test")
   @GET
   String test();
}

TestResourceClient service = RestClientBuilder.newBuilder()
                             .baseUrl("http://localhost:8081/")
                             .build(TestResourceClient.class);
String s = service.test();
```

The first four lines of executable code are spent creating a proxy, `service`, that implements `TestResourceIntf`, but once that is done, calls on `TestResource` can be made very naturally in terms of `TestResourceIntf`, as illustrated by the call `service.test()`.

Beyond the natural syntax, another advantage of proxies is the way the proxy construction process quietly gathers useful information from the implemented interface and makes it available for remote invocations. Consider a more elaborate version of `TestResourceIntf`:

```
@Path("resource")
public interface TestResourceClient {

   @Path("test/{path}")
   @Consumes("text/plain")
   @Produces("text/html")
   @POST
   String test(@PathParam("path") String path, @QueryParam("query") String query,
String entity);
}
```

Calling `service.test("p", "q", "e")` results in an HTTP message that looks like

```
POST /resource/test/p/?query=q HTTP/1.1
Accept: text/html
```

```
Content-Type: text/plain
Content-Length: 1

e
```

The HTTP verb is derived from the `@POST` annotation, the request URI is derived from the two instances of the `@Path` annotation (one on the class, one on the method) plus the first and second parameters of `test()`, the Accept header is derived from the `@Produces` annotation, and the Content-Type header is derived from the `@Consumes` annotation,

Using the Jakarta RESTful Web Services API, `service.test("p", "q", "e")` would look like the more verbose

```
public String checkResponse() {
    try (Client client = ClientBuilder.newClient()) {
        return client.target("http://localhost:8081/resource/test/p")
                             .queryParam("query", "q")
                             .request()
                             .accept("text/html")
                             .post(Entity.entity("e", "text/plain"), String.class);
    }
}
```

One other basic facility offered by MicroProfile Rest Client is the ability to configure the client environment by registering providers:

```
TestResourceClient service = RestClientBuilder.newBuilder()
                              .baseUrl("http://localhost:8081/")
                              .register(MyClientResponseFilter.class)
                              .register(MyMessageBodyReader.class)
                              .build(TestResourceClient.class);
```

Naturally, the registered providers should be relevant to the client environment, rather than, say, a `ContainerResponseFilter`.



So far, the MicroProfile Rest Client should look familiar to anyone who has used the RESTEasy client proxy facility (Section "RESTEasy Proxy Framework"). The construction in the previous listing would look like

```
ResteasyClient client = (ResteasyClient) ResteasyClientBuilder
.newClient();
TestResourceClient service = client.target("http://localhost:8081/")
                                .register(MyClientResponseFilter.class)
                                .register(MyMessageBodyReader.class)
                                .proxy(TestResourceClient.class);
```

in RESTEasy.

55.2. Concepts imported from Jakarta RESTful Web Services

Beyond the central concept of the client proxy, some basic concepts in MicroProfile Client originate in Jakarta RESTful Web Services. Some of these have already been introduced in the previous section, since the interface implemented by a client proxy represents the facilities provided by a Jakarta RESTful Web Services server. For example, the HTTP verb annotations and the `@Consumes` and `@Produces` annotations originate on the Jakarta RESTful Web Services server side. Injectable parameters annotated with `@PathParameter`, `@QueryParameter`, etc., also come from Jakarta RESTful Web Services.

Nearly all of the provider concepts supported by MicroProfile Client also originate in Jakarta RESTful Web Services. These are:

- `jakarta.ws.rs.client.ClientRequestFilter`
- `jakarta.ws.rs.client.ClientResponseFilter`
- `jakarta.ws.rs.ext.MessageBodyReader`
- `jakarta.ws.rs.ext.MessageBodyWriter`
- `jakarta.ws.rs.ext.ParamConverter`
- `jakarta.ws.rs.ext.ReaderInterceptor`
- `jakarta.ws.rs.ext.WriterInterceptor`

Like Jakarta RESTful Web Services, MicroProfile Client also has the concept of mandated providers. These are

- JSON-P `MessageBodyReader` and `MessageBodyWriter` must be provided.
- JSON-B `MessageBodyReader` and `MessageBodyWriter` must be provided if the implementation supports JSON-B.
- `MessageBodyReader` and `MessageBodyWriter` must be provided for the following types:
  - `byte[]`
  - `String`
  - `InputStream`
  - `Reader`
  - `File`

55.3. Beyond Jakarta RESTful Web Services and RESTEasy

Some concepts in MicroProfile Rest Client do not appear in either Jakarta RESTful Web Services or RESTEasy.

55.3.1. Default media type

Whenever no media type is specified by, for example, `@Consumes` or `@Produces` annotations, the media type of a request entity or response entity is "application/json". This is different than Jakarta RESTful Web Services, where the media type defaults to "application/octet-stream".

55.3.2. Declarative registration of providers

In addition to programmatic registration of providers as illustrated above, it is also possible to register providers declaratively with annotations introduced in MicroProfile Rest Client. In particular, providers can be registered by adding the `org.eclipse.microprofile.rest.client.annotation.RegisterProvider` annotation to the target interface:

```
@Path("resource")
@RegisterProvider(MyClientResponseFilter.class)
@RegisterProvider(MyMessageBodyReader.class)
public interface TestResourceClient {

   @Path("test/{path}")
   @Consumes("text/plain")
   @Produces("text/html")
   @POST
   String test(@PathParam("path") String path, @QueryParam("query") String query,
String entity);
}
```

Declaring `MyClientResponseFilter` and `MyMessageBodyReader` with annotations eliminates the need to call `RestClientBuilder.register()`.

55.3.3. Global registration of providers

One more way to register providers is by implementing one or both of the listeners in package `org.eclipse.microprofile.rest.client.spi`:

```
public interface RestClientBuilderListener {

    void onNewBuilder(RestClientBuilder builder);
}

public interface RestClientListener {

    void onNewClient(Class<?> serviceInterface, RestClientBuilder builder);
}
```

which can access a `RestClientBuilder` upon creation of a new `RestClientBuilder` or upon the execution of `RestClientBuilder.build()`, respectively. Implementations must be declared in

```
META-INF/services/org.eclipse.microprofile.rest.client.spi.RestClientBuilderListener
```

or

```
META-INF/services/org.eclipse.microprofile.rest.client.spi.RestClientListener
```

55.3.4. Declarative specification of headers

One way of declaring a header to be included in a request is by annotating one of the resource method parameters with `@HeaderValue`:

```
@POST
@Produces(MediaType.TEXT_PLAIN)
@Consumes(MediaType.TEXT_PLAIN)
String contentLang(@HeaderParam(HttpHeaders.CONTENT_LANGUAGE) String contentLanguage,
String subject);
```

That option is available with RESTEasy client proxies as well, but in case it is inconvenient or otherwise inappropriate to include the necessary parameter, MicroProfile Client makes a declarative alternative available through the use of the `org.eclipse.microprofile.rest.client.annotation.ClientHeaderParam` annotation:

```
@POST
@Produces(MediaType.TEXT_PLAIN)
@Consumes(MediaType.TEXT_PLAIN)
@ClientHeaderParam(name=HttpHeaders.CONTENT_LANGUAGE, value="en")
String contentLang(String subject);
```

In this example, the header value is hardcoded, but it is also possible to compute a value:

```
@POST
@Produces(MediaType.TEXT_PLAIN)
@Consumes(MediaType.TEXT_PLAIN)
@ClientHeaderParam(name=HttpHeaders.CONTENT_LANGUAGE, value="{getLanguage}")
String contentLang(String subject);

default String getLanguage() {
   return headers.getFirst(HttpHeaders.CONTENT_LANGUAGE);
}
```

55.3.5. Propagating headers on the server

An instance of `org.eclipse.microprofile.rest.client.ext.ClientHeadersFactory`,

```
public interface ClientHeadersFactory {

/**
 * Updates the HTTP headers to send to the remote service. Note that providers
 * on the outbound processing chain could further update the headers.
 *
 * @param incomingHeaders - the map of headers from the inbound Jakarta RESTful Web
Services request. This will
 * be an empty map if the associated client interface is not part of a Jakarta RESTful
Web Services request.
 * @param clientOutgoingHeaders - the read-only map of header parameters specified on
the
 * client interface.
 * @return a map of HTTP headers to merge with the clientOutgoingHeaders to be sent to
 * the remote service.
 */
MultivaluedMap<String, String> update(MultivaluedMap<String, String> incomingHeaders,
                                      MultivaluedMap<String, String>
clientOutgoingHeaders);
}
```

if activated, can do a bulk transfer of incoming headers to an outgoing request. The default instance `org.eclipse.microprofile.rest.client.ext.DefaultClientHeadersFactoryImpl` will return a map consisting of those incoming headers listed in the comma separated configuration property

```
org.eclipse.microprofile.rest.client.propagateHeaders
```

In order for an instance of `ClientHeadersFactory` to be activated, the interface must be annotated with `org.eclipse.microprofile.rest.client.annotation.RegisterClientHeaders`. Optionally, the annotation may include a value field set to an implementation class; without an explicit value, the default instance will be used.

Although a `ClientHeadersFactory` is not officially designated as a provider, it is now (as of MicroProfile REST Client specification 1.4) subject to injection. In particular, when an instance of `ClientHeadersFactory` is managed by CDI, then CDI injection is mandatory. When a REST Client is executing in the context of a Jakarta RESTful Web Services implementation, then `@Context` injection into a `ClientHeadersFactory` is currently optional. RESTEasy supports CDI injection and does not currently support @Context injection.

55.3.6. ResponseExceptionMapper

The `org.eclipse.microprofile.rest.client.ext.ResponseExceptionMapper` is the client side inverse of the `jakarta.ws.rs.ext.ExceptionMapper` defined in Jakarta RESTful Web Services. That is, where `ExceptionMapper.toResponse()` turns an `Exception` thrown during server side processing into a `Response`, `ResponseExceptionMapper.toThrowable()` turns a `Response` received on the client side with an HTTP error status into an `Exception`. `ResponseExceptionMapper` can be registered in the same manner as other providers, that is, either programmatically or declaratively. In the absence of a registered

`ResponseExceptionMapper`, a default `ResponseExceptionMapper` will map any response with status >= 400 to a `WebApplicationException`.

55.3.7. Proxy injection by CDI

MicroProfile Rest Client mandates that implementations must support CDI injection of proxies. At first, the concept might seem odd in that CDI is more commonly available on the server side. However, the idea is very consistent with the microservices philosophy. If an application is composed of a number of small services, then it is to be expected that services will often act as clients to other services.

CDI (Contexts and Dependency Injection) is a fairly rich subject and beyond the scope of this Guide. For more information, see Jakarta Contexts and Dependency Injection (the specification), Jakarta EE Tutorial, or WELD - CDI Reference Implementation.

The fundamental thing to know about CDI injection is that annotating a variable with `jakarta.inject.Inject` will lead the CDI runtime (if it is present and enabled) to create an object of the appropriate type and assign it to the variable. For example, in

```
public interface Book {
   String getTitle();
    void setTitle(String title);
}

@Dependent
public class BookImpl implements Book {

   private String title;

   @Override
   public String getTitle() {
      return title;
   }

   @Override
   public void setTitle(String title) {
      this.title = title;
   }
}

public class Author {

   @Inject
   private Book book;

   public Book getBook() {
      return book;
   }
}
```

The CDI runtime will create an instance of `BookImpl` and assign it to the private field `book` when an instance of `Author` is created;

In this example, the injection is done because `BookImpl` is assignable to `book`, but greater discrimination can be imposed by annotating the interface and the field with **qualifier** annotations. For the injection to be legal, every qualifier on the field must be present on the injected interface. For example:

```
@Qualifier
@Target({ElementType.TYPE, ElementType.METHOD, ElementType.PARAMETER, ElementType
.FIELD})
@Retention(RetentionPolicy.RUNTIME)
public @interface Text {}

@Qualifier
@Target({ElementType.TYPE, ElementType.METHOD, ElementType.PARAMETER, ElementType
.FIELD})
@Retention(RetentionPolicy.RUNTIME)
public @interface Graphic {}

@Text
public class TextBookImpl extends BookImpl { }

@Graphic
public class GraphicNovelImpl extends BookImpl { }

public class Genius {

  @Inject @Graphic
  Book book;
}
```

Here, the class `TextBookImpl` is annotated with the `@Text` qualifier and `GraphicNovelImpl` is annotated with `@Graphic`. It follows that an instance of `GraphicNovelImpl` is eligible for assignment to the field `book` in the `Genius` class, but an instance of `TextBookImpl` is not.

Now, in MicroProfile Rest Client, any interface that is to be managed as a CDI bean must be annotated with `@RegisterRestClient`:

```
@Path("resource")
@RegisterProvider(MyClientResponseFilter.class)
public static class TestResourceImpl {

  @Inject TestDataBase db;

  @Path("test/{path}")
  @Consumes("text/plain")
  @Produces("text/html")
  @POST
```

```
  public String test(@PathParam("path") String path, @QueryParam("query") String
query, String entity) {
      return db.getByName(query);
  }
}

@Path("database")
@RegisterRestClient
public interface TestDataBase {

  @Path("")
  @POST
  public String getByName(String name);
}
```

Here, the MicroProfile Rest Client implementation creates a proxy for a `TestDataBase` service, allowing easy access by `TestResourceImpl`. Notice, though, that there's no indication of where the `TestDataBase` implementation lives. That information can be supplied by the optional `@RegisterProvider` parameter `baseUri`:

```
@Path("database")
@RegisterRestClient(baseUri="https://localhost:8080/webapp")
public interface TestDataBase {

  @Path("")
  @POST
  String getByName(String name);
}
```

which indicates that an implementation of `TestDatabase` can be accessed at https://localhost:8080/webapp. The same information can be supplied externally with the system variable

```
fqn of TestDataBase/mp-rest/uri=URL
```

or

```
fqn of TestDataBase/mp-rest/url=URL
```

which will override the value hardcoded in `@RegisterRestClient`. For example,

```
com.bluemonkeydiamond.TestDatabase/mp-rest/url=https://localhost:8080/webapp
```

A number of other properties will be examined in the course of creating the proxy, including, for example

```
com.bluemonkeydiamond.TestDatabase/mp-rest/providers
```

a comma separated list of provider classes to be registered with the proxy. See the MicroProfile Client documentation for more such properties.

These properties can be simplified through the use of the `configKey` field in `@RegisterRestClient`. For example, setting the `configKey` as in

```
@Path("database")
@RegisterRestClient(configKey="bmd")
public interface TestDataBase {}
```

allows the use of properties like

```
bmd/mp-rest/url=https://localhost:8080/webapp
```

Note that, since the configKey is not tied to a particular interface name, multiple proxies can be configured with the same properties.

55.3.8. Proxy lifecycle

Proxies should be closed so that any resources they hold can be released. Every proxy created by `RestClientBuilder` implements the `java.io.Closeable` interface, so it is always possible to cast a proxy to `Closeable` and call `close()`. A nice trick to have the proxy interface explicitly extend `Closeable`, which not only avoids the need for a cast but also makes the proxy eligible to use in a try-with-resources block:

```
@Path("resource")
public interface TestResourceIntf extends Closeable {

   @Path("test")
   @GET
   String test();
}
```

```
public String checkResponse() {
    TestResourceClient service = RestClientBuilder.newBuilder()
                                 .baseUrl("http://localhost:8081/")
                                 .build(TestResourceClient.class);
    try (TestResourceClient tr = service) {
       return service.test();
    }
}
```

55.3.9. Asynchronous support

An interface method can be designated as asynchronous by having it return a `java.util.concurrent.CompletionStage`. For example, in

```
public interface TestResourceClient extends Closeable {

   @Path("test")
   @GET
   String test();

   @Path("testasync")
   @GET
   CompletionStageString testAsync();
}
```

the `test()` method can be turned into the asynchronous method `testAsync()` by having it return a `CompletionStageString` instead of a `String`.

Asynchronous methods are made to be asynchronous by scheduling their execution on a thread distinct from the calling thread. The MicroProfile Client implementation will have a default means of doing that, but `RestClientBuilder.executorService(ExecutorService)` provides a way of substituting an application specific `ExecutorService`.

The classes `AsyncInvocationInterceptorFactory` and `AsyncInvocationInterceptor` in package `org.eclipse.microprofile.rest.client.ext` provides a means of communication between the calling thread and the asynchronous thread:

```
public interface AsyncInvocationInterceptorFactory {

    /**
     * Implementations of this method should return an implementation of the
     * AsyncInvocationInterceptor interface.  The MP Rest Client
     * implementation runtime will invoke this method, and then invoke the
     * prepareContext and applyContext methods of the
     * returned interceptor when performing an asynchronous method invocation.
     * Null return values will be ignored.
     *
     * @return Non-null instance of AsyncInvocationInterceptor
     */
    AsyncInvocationInterceptor newInterceptor();
}

public interface AsyncInvocationInterceptor {

    /**
     * This method will be invoked by the MP Rest Client runtime on the "main"
     * thread (i.e. the thread calling the async Rest Client interface method)
     * prior to returning control to the calling method.
```

```
     */
    void prepareContext();

    /**
     * This method will be invoked by the MP Rest Client runtime on the "async"
     * thread (i.e. the thread used to actually invoke the remote service and
     * wait for the response) prior to sending the request.
     */
    void applyContext();

    /**
     * This method will be invoked by the MP Rest Client runtime on the "async"
     * thread (i.e. the thread used to actually invoke the remote service and
     * wait for the response) after all providers on the inbound response flow
     * have been invoked.
     *
     * @since 1.2
     */
     void removeContext();
}
```

The following sequence of events occurs:

1. `AsyncInvocationInterceptorFactory.newInterceptor()` is called on the calling thread to get an instance of the `AsyncInvocationInterceptor`.
2. `AsyncInvocationInterceptor.prepareContext()` is executed on the calling thread to store information to be used by the request execution.
3. `AsyncInvocationInterceptor.applyContext()` is executed on the asynchronous thread.
4. All relevant outbound providers such as interceptors and filters are executed on the asynchronous thread, followed by the request invocation.
5. All relevant inbound providers are executed on the asynchronous thread, followed by executing `AsyncInvocationInterceptor.removeContext()`
6. The asynchronous thread returns.

An `AsyncInvocationInterceptorFactory` class is enabled by registering it on the client interface with `@RegisterProvider`.

55.3.10. SSL

The MicroProfile Client `RestClientBuilder` interface includes a number of methods that support the use of SSL:

```
RestClientBuilder hostnameVerifier(HostnameVerifier hostnameVerifier);
RestClientBuilder keyStore(KeyStore keyStore, String keystorePassword);
RestClientBuilder sslContext(SSLContext sslContext);
RestClientBuilder trustStore(KeyStore trustStore);
```

For example:

```
KeyStore trustStore = createTrustStore() ;
HostnameVerifier verifier = createHostnameVerifier();
TestResourceIntf service = RestClientBuilder.newBuilder()
                               .baseUrl("http://localhost:8081/")
                               .trustStore(trustStore)
                               .hostnameVerifier(verifier)
                               .build(TestResourceIntf.class);
```

It is also possible to configure `HostnameVerifier`s, `KeyStore`s, and `TrustStore`s using configuration properties:

- com.bluemonkeydiamond.TestResourceIntf/mp-rest/hostnameVerifier
- com.bluemonkeydiamond.TestResourceIntf/mp-rest/keyStore
- com.bluemonkeydiamond.TestResourceIntf/mp-rest/keyStorePassword
- com.bluemonkeydiamond.TestResourceIntf/mp-rest/keyStoreType
- com.bluemonkeydiamond.TestResourceIntf/mp-rest/trustStore
- com.bluemonkeydiamond.TestResourceIntf/mp-rest/trustStorePassword
- com.bluemonkeydiamond.TestResourceIntf/mp-rest/trustStoreType

The values of the ".../mp-rest/keyStore" and "../mp-rest/trustStore" parameters can be either classpath resources (e.g., "classpath:/client-keystore.jks") or files (e.g., "file:/home/user/client-keystore.jks").

Chapter 56. AJAX Client

RESTEasy resources can be accessed in JavaScript using AJAX using a proxy API generated by RESTEasy.

56.1. Generated JavaScript API

RESTEasy can generate a JavaScript API that uses AJAX calls to invoke Jakarta RESTful Web Services operations.

Example 1. First Jakarta RESTful Web Services JavaScript API example

Let's take a simple Jakarta RESTful Web Services API:

```
@Path("orders")
public class Orders {
    @Path("{id}")
    @GET
    public String getOrder(@PathParam("id") String id) {
        return "Hello "+id;
    }
}
```

The preceding API would be accessible using the following JavaScript code:

```
var order = Orders.getOrder({id: 23});
```

56.1.1. JavaScript API servlet

In order to enable the JavaScript API servlet the user must configure it in the web.xml file as such:

```
<web-app>
    <servlet>
     <servlet-name>RESTEasy JSAPI</servlet-name>
     <servlet-class>org.jboss.resteasy.jsapi.JSAPIServlet</servlet-class>
    </servlet>

    <servlet-mapping>
     <servlet-name>RESTEasy JSAPI</servlet-name>
     <url-pattern>/rest-js</url-pattern>
    </servlet-mapping>
</web-app>
```

56.1.2. JavaScript API usage

Each Jakarta RESTful Web Services resource class will generate a JavaScript object of the same name as the declaring class (or interface), which will contain every Jakarta RESTful Web Services method as properties.

Example 2. Structure of Jakarta RESTful Web Services generated JavaScript

For example, if the resource X defines methods Y and Z:

```
@Path("/")
public interface X{
    @GET
    String Y();
    @PUT
    void Z(String entity);
}
```

Then the JavaScript API will define the following functions:

```
var X = {
 Y : function(params){},
 Z : function(params){}
};
```

Each JavaScript API method takes an optional object as single parameter where each property is a cookie, header, path, query or form parameter as identified by their name, or the following special parameters:



The following special parameter names are subject to change.

Property Name	Default	Description
$entity		The entity to send as a PUT, POST request.
$contentType	As determined by `@Consumes`.	The MIME type of the body entity sent as the Content-Type header.
$accepts	Determined by `@Provides`, defaults to /.	The accepted MIME types sent as the Accept header.

Property Name	Default	Description
$callback		Set to a function(httpCode, xmlHttpRequest, value) for an asynchronous call. If not present, the call will be synchronous and return the value.
$apiURL	Determined by container	Set to the base URI of your Jakarta RESTful Web Services endpoint, not including the last slash.
$username		If username and password are set, they will be used for credentials for the request.
$password		If username and password are set, they will be used for credentials for the request.

Example 3. Using the API

Here is an example of Jakarta RESTful Web Services API:

```
@Path("foo")
public class Foo{
    @Path("{id}")
    @GET
    public String get(@QueryParam("order") String order, @HeaderParam("X-Foo")
String header,
                      @MatrixParam("colour") String colour, @CookieParam("Foo-Cookie
") String cookie){
    }
    @POST
    public void post(String text){
    }
}
```

We can use the previous Jakarta RESTful Web Services API in JavaScript using the following code:

```
var text = Foo.get({order: 'desc', 'X-Foo': 'hello',
                    colour: 'blue', 'Foo-Cookie': 123987235444});
Foo.put({$entity: text});
```

56.1.3. Work with @Form

`@Form` is a RESTEasy specific annotation that allows the user to re-use any `@*Param` annotation within an injected class. The generated JavaScript API will expand the parameters for use automatically. Support we have the following form:

```
public class MyForm {
    @FormParam("stuff")
    private String stuff;

    @FormParam("number")
    private int number;

    @HeaderParam("myHeader")
    private String header;
}
```

And the resource is like:

```
@Path("/")
public class MyResource {

    @POST
    public String postForm(@Form MyForm myForm) {}

}
```

Then the method can be called from JavaScript API like following:

```
MyResource.postForm({stuff:"A", myHeader:"B", number:1});
```

Also, `@Form` supports prefix mappings for lists and maps:

```
public static class Person {
    @Form(prefix="telephoneNumbers") List<TelephoneNumber> telephoneNumbers;
    @Form(prefix="address") Map<String, Address> addresses;
}

public static class TelephoneNumber {
    @FormParam("countryCode") private String countryCode;
    @FormParam("number") private String number;
}

public static class Address {
    @FormParam("street") private String street;
    @FormParam("houseNumber") private String houseNumber;
}
```

```
@Path("person")
public static class MyResource {
    @POST
    public void postForm(@Form Person p) {}
}
```

From JavaScript the API could be called like this:

```
MyResource.postForm({
    telephoneNumbers:[
        {"telephoneNumbers[0].countryCode":31},
        {"telephoneNumbers[0].number":12345678},
        {"telephoneNumbers[1].countryCode":91},
        {"telephoneNumbers[1].number":9717738723}
    ],
    address:[
        {"address[INVOICE].street":"Main Street"},
        {"address[INVOICE].houseNumber":2},
        {"address[SHIPPING].street":"Square One"},
        {"address[SHIPPING].houseNumber":13}
    ]
});
```

56.1.4. MIME types and unmarshalling.

The Accept header sent by any client JavaScript function is controlled by the $accepts parameter, which overrides the @Produces annotation on the Jakarta RESTful Web Services endpoint. The returned value however is controlled by the Content-Type header sent in the response as follows:

MIME	Description
text/xml,application/xml,application/*+xml	The response entity is parsed as XML before being returned. The return value is thus a DOM Document.
application/json	The response entity is parsed as JSON before being returned. The return value is thus a JavaScript Object.
Anything else	The response entity is returned raw.

Example 4. Unmarshalling example

The RESTEasy JavaScript client API can automatically unmarshall JSON and XML:

```
@Path("orders")
public interface Orders {
```

```
    @XmlRootElement
    class Order {
        @XmlElement
        private String id;

        public Order(){}

        public Order(String id){
            this.id = id;
        }
        @Path("{id}/xml")
        @GET
        @Produces("application/xml")
        public Order getOrderXML(@PathParam("id") String id){
            return new Order(id);
        }

        @Path("{id}/json")
        @GET
        @Produces("application/json")
        public Order getOrderJSON(@PathParam("id") String id){
            return new Order(id);
        }
    }
}
```

Let's look at what the preceding Jakarta RESTful Web Services API would give on the client side:

```
// this returns a JSON object
var orderJSON = Orders.getOrderJSON({id: "23"});
orderJSON.id == "23";

// this one returns a DOM Document whose root element is the order, with one child
(id)
// whose child is the text node value
var orderXML = Orders.getOrderXML({id: "23"});
orderXML.documentElement.childNodes[0].childNodes[0].nodeValue == "23";
```

56.1.5. MIME types and marshalling.

The Content-Type header sent in the request is controlled by the $contentType parameter which overrides the @Consumes annotation on the Jakarta RESTful Web Services endpoint. The value passed as entity body using the $entity parameter is marshalled according to both its type and content type:

Type	MIME	Description
DOM Element	Empty or text/xml,application/xml,application/*+xml	The DOM Element is marshalled to XML before being sent.
JavaScript Object (JSON)	Empty or application/json	The JSON object is marshalled to a JSON string before being sent.
Anything else	Anything else	The entity is sent as is.

Example 5. Marshalling example

The RESTEasy JavaScript client API can automatically marshall JSON and XML:

```
@Path("orders")
public interface Orders {

    @XmlRootElement
    public static class Order {
        @XmlElement
        private String id;

        public Order(){}

        public Order(String id){
            this.id = id;
        }
    }

    @Path("{id}/xml")
    @PUT
    @Consumes("application/xml")
    default void putOrderXML(Order order){
        // store order
    }

    @Path("{id}/json")
    @PUT
    @Consumes("application/json")
    default void putOrderJSON(Order order){
        // store order
    }
}
```

Let's look at what the preceding Jakarta RESTful Web Services API would give on the client side:

```
// this saves a JSON object
```

```
Orders.putOrderJSON({$entity: {id: "23"}});

// It is a bit more work with XML
var order = document.createElement("order");
var id = document.createElement("id");
order.appendChild(id);
id.appendChild(document.createTextNode("23"));
Orders.putOrderXML({$entity: order});
```

56.2. Using the JavaScript API to build AJAX queries

The RESTEasy JavaScript API can also be used to manually construct your requests.

56.2.1. The REST object

The REST object contains the following read-write properties:

Property	Description
apiURL Set by default to the Jakarta RESTful Web Services root URL, used by every JavaScript client API functions when constructing the requests.	log

Example 6. Using the REST object

The REST object can be used to override RESTEasy JavaScript API client behaviour:

```
// Change the base URL used by the API:
REST.apiURL = "http://api.service.com";

// log everything in a div element
REST.log = function(text){
 jQuery("#log-div").append(text);
};
```

56.2.2. The REST.Request class

The REST.Request class is used to build custom requests. It has the following members:

Member	Description	execute(callback)
Executes the request with all the information set in the current object. The value is never returned but passed to the optional argument callback.	setAccepts(acceptHeader)	Sets the Accept request header. Defaults to */*.
setCredentials(username, password)	Sets the request credentials.	setEntity(entity)
Sets the request entity.	setContentType(contentTypeHeader)	Sets the Content-Type request header.
setURI(uri)	Sets the request URI. This should be an absolute URI.	setMethod(method)
Sets the request method. Defaults to GET.	setAsync(async)	Controls whether the request should be asynchronous. Defaults to true.
addCookie(name, value)	ets the given cookie in the current document when executing the request. Beware that this will be persistent in your browser.	addQueryParameter(name, value)
Adds a query parameter to the URI query part.	addMatrixParameter(name, value)	Adds a matrix parameter (path parameter) to the last path segment of the request URI.

Example 7. Using the REST.Request class

The REST.Request class can be used to build custom requests:

```
var r = new REST.Request();
r.setURI("http://api.service.com/orders/23/json");
r.setMethod("PUT");
r.setContentType("application/json");
r.setEntity({id: "23"});
r.addMatrixParameter("JSESSIONID", "12309812378123");
r.execute(function(status, request, entity){
 log("Response is "+status);
});
```

56.3. Caching Features

RESTEasy AJAX Client works well with server side caching features. But the buggy browsers cache will always prevent the function to work properly. To use the RESTEasy caching feature with its AJAX client, enable the 'antiBrowserCache' option:

```
REST.antiBrowserCache = true;
```

The above setting should be set once before calling any APIs.

Chapter 57. RESTEasy WADL Support

RESTEasy has support to generate WADL for its resources. It supports several different containers. The following text shows how to use this feature in different containers.

57.1. RESTEasy WADL Support for Servlet Container (Deprecated)



The content introduced in this section is outdated, and the `ResteasyWadlServlet` class is deprecated because it doesn't support the GRAMMAR generation. Please check the `ResteasyWadlDefaultResource` introduced in the later section.

RESTEasy WADL uses `ResteasyWadlServlet` to support servlet container. It must be registered in the web.xml to enable WADL feature. Here is an example to show the usages of ResteasyWadlServlet in web.xml:

```
<web-app>
    <servlet>
        <servlet-name>RESTEasy WADL</servlet-name>
        <servlet-class>org.jboss.resteasy.wadl.ResteasyWadlServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>RESTEasy WADL</servlet-name>
        <url-pattern>/application.xml</url-pattern>
    </servlet-mapping>
</web-app>
```

The preceding configuration in web.xml shows how to enable ResteasyWadlServlet and map it to /application.xml, so then the WADL can be accessed from the configured URL:

```
/application.xml
```

57.2. RESTEasy WADL Support for Servlet Container(Updated)

This section introduces the recommended way to enable WADL support in a Servlet Container. First, add a class that extends the `ResteasyWadlDefaultResource` to serve a resource path. Here is an example:

```
import org.jboss.resteasy.wadl.ResteasyWadlDefaultResource;
import jakarta.ws.rs.Path;

@Path("/")
```

```
public class MyWadlResource extends ResteasyWadlDefaultResource {
}
```

As the sample shows above, it implements the `ResteasyWadlDefaultResource` and serves this URL by default:

```
/application.xml
```

To enable the GRAMMAR generation, extend the `ResteasyWadlDefaultResource` like this:

```
import org.jboss.resteasy.wadl.ResteasyWadlDefaultResource;
import org.jboss.resteasy.wadl.ResteasyWadlWriter;

import jakarta.ws.rs.Path;

@Path("/")
public class MyWadlResource extends ResteasyWadlDefaultResource {

    public MyWadlResource() {
        ResteasyWadlWriter.ResteasyWadlGrammar wadlGrammar = new ResteasyWadlWriter
.ResteasyWadlGrammar();
        wadlGrammar.enableSchemaGeneration();
        getWadlWriter().setWadlGrammar(wadlGrammar);

    }
}
```

With the above setup, the WADL module will generate GRAMMAR automatically and register the service under this url:

```
/wadl-extended/xsd0.xsd
```

Above is the basic usage of the WADL module under a servlet container deployment.

57.3. RESTEasy WADL support for Sun JDK HTTP Server

RESTEasy provides a `ResteasyWadlDefaultResource` to generate WADL info for its embedded containers. Here is an example to show how to use it with RESTEasy's Sun JDK HTTP Server container:

```
com.sun.net.httpserver.HttpServer httpServer =
    com.sun.net.httpserver.HttpServer.create(new InetSocketAddress(port), 10);

org.jboss.resteasy.plugins.server.sun.http.HttpContextBuilder contextBuilder =
```

```
    new org.jboss.resteasy.plugins.server.sun.http.HttpContextBuilder();

contextBuilder.getDeployment().getActualResourceClasses()
    .add(ResteasyWadlDefaultResource.class);
contextBuilder.bind(httpServer);

ResteasyWadlDefaultResource.getServices()
    .put("/",
        ResteasyWadlGenerator
            .generateServiceRegistry(contextBuilder.getDeployment()));

httpServer.start();
```

From the above code example, we can see how `ResteasyWadlDefaultResource` is registered in the deployment:

```
contextBuilder.getDeployment().getActualResourceClasses()
    .add(ResteasyWadlDefaultResource.class);
```

Another important task is to use `ResteasyWadlGenerator` to generate the WADL info for the resources in the deployment:

```
ResteasyWadlDefaultResource.getServices()
    .put("/",
        ResteasyWadlGenerator
            .generateServiceRegistry(contextBuilder.getDeployment()));
```

After the above configuration is set, users can access "/application.xml" to fetch the WADL info, because `ResteasyWadlDefaultResource` has `@Path` set to "/application.xml" as default:

```
@Path("/application.xml")
public class ResteasyWadlDefaultResource
```

57.4. RESTEasy WADL support for Netty Container

RESTEasy WADL support for Netty Container is similar to the support for JDK HTTP Server. It uses ResteasyWadlDefaultResource to serve '/application.xml' and ResteasyWadlGenerator to generate WADL info for resources. Here is the sample code:

```
ResteasyDeployment deployment = new ResteasyDeploymentImpl();

netty = new NettyJaxrsServer();
netty.setDeployment(deployment);
netty.setPort(port);
netty.setRootResourcePath("");
```

```
netty.setSecurityDomain(null);
netty.start();

deployment.getRegistry()
    .addPerRequestResource(ResteasyWadlDefaultResource.class);
ResteasyWadlDefaultResource.getServices()
    .put("/", ResteasyWadlGenerator.generateServiceRegistry(deployment));
```

Please note for all embedded containers like JDK HTTP Server and Netty Container, if the resources in the deployment changes at runtime, the ResteasyWadlGenerator.generateServiceRegistry() needs to be re-run to refresh the WADL info.

57.5. RESTEasy WADL Support for Undertow Container

The RESTEasy Undertow Container is an embedded Servlet Container, and RESTEasy WADL provides a connector to it. To use RESTEasy Undertow Container together with WADL support, add these three components into project maven dependencies:

```
<dependencies>
    <dependency>
        <groupId>org.jboss.resteasy</groupId>
        <artifactId>resteasy-wadl</artifactId>
        <version>8.0.0.Alpha1</version>
    </dependency>
    <dependency>
        <groupId>org.jboss.resteasy</groupId>
        <artifactId>resteasy-wadl-undertow-connector</artifactId>
        <version>8.0.0.Alpha1</version>
    </dependency>
    <dependency>
        <groupId>org.jboss.resteasy</groupId>
        <artifactId>resteasy-undertow</artifactId>
        <version>8.0.0.Alpha1</version>
    </dependency>
</dependencies>
```

The resteasy-wadl-undertow-connector provides a WadlUndertowConnector to help use WADL in RESTEasy Undertow Container. Here is the code example:

```
UndertowJaxrsServer server = new UndertowJaxrsServer().start();
WadlUndertowConnector connector = new WadlUndertowConnector();
connector.deployToServer(server, MyApp.class);
```

The MyApp class shown in above code is a standard Jakarta RESTful Web Services Application class in your project:

```
@ApplicationPath("/base")
public static class MyApp extends Application {
    @Override
    public Set<Class<?>> getClasses() {
        return Set.of(YourResource.class);
    }
}
```

After the Application is deployed to the `UndertowJaxrsServer` via `WadlUndertowConnector`, the user can access the WADL info at "/application.xml" prefixed by the `@ApplicationPath` in the Application class. To override the `@ApplicationPath`, use the other method in WadlUndertowConnector:

```
public UndertowJaxrsServer deployToServer(UndertowJaxrsServer server, Class<? extends
Application> application, String contextPath)
```

The "deployToServer" method shown above accepts a "contextPath" parameter, which can be used to override the `@ApplicationPath` value in the Application class.

Chapter 58. RESTEasy Tracing Feature

58.1. Overview

Tracing feature is a way for the users of the RESTEasy to understand what's going on internally in the container when a request is processed. It's different from the pure logging system or profiling feature, which provides more general information about the request and response.

The tracing feature provides more internal states of the Jakarta RESTful Web Services container. For example, it could be able to show what filters a request is going through, or how long a request is processed and other kinds of information.

Currently, it doesn't have a standard or spec to define the tracing feature, so the tracing feature is tightly coupled with the concrete Jakarta RESTful Web Services implementation itself. This chapter, discusses the design and usage of the tracing feature.

58.2. Tracing Info Mode

The RESTEasy tracing feature supports three logging mode:

- OFF
- ON_DEMAND
- ALL

"ALL" will enable the tracing feature. "ON_DEMAND" mode will give the control to the client side: A client can send a tracing request via HTTP header and get the tracing info back from response headers. "OFF" mode disables the tracing feature, and is the default mode.

58.3. Tracing Info Level

The tracing info has three levels:

- SUMMARY
- TRACE
- VERBOSE

The "SUMMARY" level will emit some brief tracing information. The "TRACE" level will produce more detailed tracing information, and the "VERBOSE" level will generate extremely detailed tracing information.

The tracing feature relies on the JBoss Logging framework to produce the tracing info. The JBoss Logging configuration controls the final output of the tracing info.

58.4. Basic Usages

By default, the tracing feature is turned off. To enable the tracing feature, add the following

dependency in your project:

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-tracing-api</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

Because the tracing feature is an optional feature, the above dependency is provided by the resteasy-extensions project.

After including the dependency in the project, set the tracing mode and tracing level via the context-param parameters in the project's web.xml file. Here is the example:

```
<context-param>
    <param-name>resteasy.server.tracing.type</param-name>
    <param-value>ALL</param-value>
    <param-name>resteasy.server.tracing.threshold</param-name>
    <param-value>SUMMARY</param-value>
</context-param>
```

Besides the above configuration, the user needs to make sure that the underlying log manager is configured properly, so the tracing output can be viewed. Here is an example of the "logging.properties" for the JBoss Log Manager:

```
# Additional logger names to configure (root logger is always configured)
#loggers=org.foo.bar, org.foo.baz
# Root logger level
logger.level=ALL
# Declare handlers for the root logger
logger.handlers=CONSOLE,FILE
# Declare handlers for additional loggers
#logger.org.foo.bar.handlers=XXX, YYY
# Console handler configuration
handler.CONSOLE=org.jboss.logmanager.handlers.ConsoleHandler
handler.CONSOLE.properties=autoFlush
handler.CONSOLE.level=ALL
handler.CONSOLE.autoFlush=true
handler.CONSOLE.formatter=PATTERN
# File handler configuration
handler.FILE=org.jboss.logmanager.handlers.FileHandler
handler.FILE.level=ALL
handler.FILE.properties=autoFlush,fileName
handler.FILE.autoFlush=true
handler.FILE.fileName=/tmp/jboss.log
handler.FILE.formatter=PATTERN
# The log format pattern for both logs
formatter.PATTERN=org.jboss.logmanager.formatters.PatternFormatter
```

```
formatter.PATTERN.properties=pattern
formatter.PATTERN.pattern=%d{HH:mm:ss,SSS} %-5p [%c{1}] %m%n
```

In above setting, the logger level is set to "ALL", and the output log file to "/tmp/jboss.log". This example is reporting all the tracing information.

After enabling the tracing feature above, the tracing output looks like the following:

```
16:21:40,110 INFO  [general]
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@721299ff START
baseUri=[http://localhost:8081/] requestUri=[http://localhost:8081/type] method=[GET]
authScheme=[n/a] accept=n/a accept-encoding=n/a accept-charset=n/a accept-language=n/a
content-type=n/a content-length=n/a  [ ---- ms]
16:21:40,110 TRACE [general]
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@721299ff
START_HEADERS Other request headers: Connection=[Keep-Alive] Host=[localhost:8081]
User-Agent=[Apache-HttpClient/4.5.4 (Java/1.8.0_201)]  [ ---- ms]
16:21:40,114 INFO  [general]
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@721299ff
PRE_MATCH_SUMMARY PreMatchRequest summary: 0 filters [ 0.04 ms]
16:21:40,118 DEBUG [general]
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@721299ff
REQUEST_FILTER Filter by [io.weli.tracing.HttpMethodOverride @60353244] [ 0.02 ms]
...
16:21:40,164 INFO  [general]
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@721299ff
RESPONSE_FILTER_SUMMARY Response summary: 1 filters [ 8.11 ms]
16:21:40,164 INFO  [general]
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@721299ff FINISHED
Response status: 200 [ ---- ms]
```

There are several part to the entries in the log information above.

- Level Of The Log Entry

  The entries log level is reported, such as "TRACE", "INFO", "DEBUG". The tracing feature maps its own tracing info levels to the JBoss Logger output levels like this.

- The Request Scope Id

  A request id is listed:

  ```
  org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@721299ff
  ```

  This identifies which request the log entry belongs to.

- The Type Of The Tracing Log

  tracing log entries are divided into multiple categories, such as "START_HEADERS",

"REQUEST_FILTER", "FINISHED", etc.

- The Detail Of The Log Entry

  The last part of a log entry is the detail message of this entry.

In next section describes how the tracing info is fetched from client side.

58.5. Client Side Tracing Info

From the client side, request can be sent to the server side. If the server side is configured properly to produce tracing info, the info will be sent back to client side via response headers. For example, a request is sent to the server like this:

```
curl -i http://localhost:8081/foo
```

The tracing information is retrieved from the response header follows:

```
HTTP/1.1 200 OK
X-RESTEasy-Tracing-026:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7a49a8aa MBW
[ ---- / 61.57 ms |  ---- %] [org.jboss.resteasy.plugins.providers.InputStreamProvider
@1cbf0b08] is skipped
...
Date: Wed, 27 Mar 2019 09:39:50 GMT
Connection: keep-alive
X-RESTEasy-Tracing-000:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7a49a8aa START
[ ---- /  ---- ms |  ---- %] baseUri=[http://localhost:8081/]
requestUri=[http://localhost:8081/type] method=[GET] authScheme=[n/a] accept=*/*
accept-encoding=n/a accept-charset=n/a accept-language=n/a content-type=n/a content-
length=n/a
...
X-RESTEasy-Tracing-025:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7a49a8aa MBW
[ ---- / 61.42 ms |  ---- %] [org.jboss.resteasy.plugins.providers.FileRangeWriter
@35b791fa] is skipped
```

From above output, the tracing info is in the response headers, and it is marked in sequence as in the form of "X-RESTEasy-Tracing-nnn".

58.6. Json Formatted Response

The tracing log can be returned to client side in JSON format. To use this feature, the user must specify a JSON provider for the tracing module to generate JSON formatted info. There are two JSON providers to choose from. Both support JSON data marshalling. The first choice is to use the jackson2 provider:

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-jackson2-provider</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

The second choice is to use the json-binding provider:

```
<dependency>
    <groupId>org.jboss.resteasy</groupId>
    <artifactId>resteasy-json-binding-provider</artifactId>
    <version>8.0.0.Alpha1</version>
</dependency>
```

After including one of the above modules, send a request to server. The JSON formatted tracing information will be returned. Here is a request example (the example is provided at last section of this chapter):

```
curl -H "X-RESTEasy-Tracing-Accept-Format: JSON" -i http://localhost:8081/type
```

In the above curl command, "X-RESTEasy-Tracing-Accept-Format: JSON" was added into the request header. This is requesting the json formatted tracing info from server, and the tracing info in response header is as follows:

```
X-RESTEasy-Tracing-000:
[{"event":"START","duration":0,"timestamp":195286694509932,"text":"baseUri=[http://loc
alhost:8081/] requestUri=[http://localhost:8081/type] method=[GET] authScheme=[n/a]
accept=*/* accept-encoding=n/a accept-charset=n/a accept-language=n/a content-type=n/a
content-length=n/a
","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7f8a
33b9"},{"event":"START_HEADERS","duration":0,"timestamp":195286695053606,"text":"Other
request headers: Accept=[*/*] Host=[localhost:8081] User-Agent=[curl/7.54.0] X-
RESTEasy-Tracing-Accept-Format=[JSON]
","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7f8a
33b9"}...{"event":"FINISHED","duration":0,"timestamp":195286729758836,"text":"Response
status:
200","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7
f8a33b9"}]
```

The above text is the raw output from the response. It can be formatted it to make it readable:

```
[{
    "X-RESTEasy-Tracing-000": [
        {
            "event": "START",
```

```
            "duration": 0,
            "timestamp": 195286694509932,
            "text": "baseUri=[http://localhost:8081/]
requestUri=[http://localhost:8081/type] method=[GET] authScheme=[n/a] accept=*/*
accept-encoding=n/a accept-charset=n/a accept-language=n/a content-type=n/a content-
length=n/a ",
            "requestId":
"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7f8a33b9"
        },
        {
            "event": "START_HEADERS",
            "duration": 0,
            "timestamp": 195286695053606,
            "text": "Other request headers: Accept=[*/*] Host=[localhost:8081] User-
Agent=[curl/7.54.0] X-RESTEasy-Tracing-Accept-Format=[JSON] ",
            "requestId":
"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7f8a33b9"
        },
        {
            "event": "PRE_MATCH_SUMMARY",
            "duration": 14563,
            "timestamp": 195286697637157,
            "text": "PreMatchRequest summary: 0 filters",
            "requestId":
"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7f8a33b9"
        },
        {
            "event": "FINISHED",
            "duration": 0,
            "timestamp": 195286729758836,
            "text": "Response status: 200",
            "requestId":
"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7f8a33b9"
        }
    ]
}]
```

From above shows the tracing info is returned as JSON text.

58.7. ON_DEMAND Mode Usage

To use the ON_DEMAND mode, set the tracing mode via the context-param parameters in the project's web.xml file. Here is the example:

```
<context-param>
    <param-name>resteasy.server.tracing.type</param-name>
    <param-value>ON_DEMAND</param-value>
</context-param>
```

From the client side, the user can send a request with additional tracing headers to enable the tracing feature and set the tracing level at per-request level. Here is the trigger header to enable the tracing information for this request:

```
X-RESTEasy-Tracing-Accept
```

By putting this header with any value in your header, the tracing information will be enabled for the request. To control the tracing level, send this header with relative level value in the request:

```
X-RESTEasy-Tracing-Threshold
```

Here is an example showing how to send the request with tracing feature enabled:

```
$ curl --header "X-RESTEasy-Tracing-Accept: ok" --header "X-RESTEasy-Tracing-
Threshold: VERBOSE" -i http://localhost:8081/foo
```

Here is the tracing information:

```
X-RESTEasy-Tracing-026:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  5.95 ms |  ---- %] [org.jboss.resteasy.plugins.providers.FileProvider
@37a3c619] is skipped
X-RESTEasy-Tracing-027:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  5.96 ms |  ---- %] [org.jboss.resteasy.plugins.providers.ByteArrayProvider
@646b8da5] is skipped
X-RESTEasy-Tracing-028:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  5.97 ms |  ---- %]
[org.jboss.resteasy.plugins.providers.StreamingOutputProvider @3b2a4bf4] is skipped
X-RESTEasy-Tracing-029:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  5.98 ms |  ---- %] [org.jboss.resteasy.plugins.providers.ReaderProvider
@24729366] is skipped
X-RESTEasy-Tracing-030:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  5.99 ms |  ---- %] [org.jboss.resteasy.plugins.providers.DataSourceProvider
@d481aff] is skipped
X-RESTEasy-Tracing-031:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  6.00 ms |  ---- %]
[org.jboss.resteasy.plugins.providers.AsyncStreamingOutputProvider @35f6b856] is
skipped
X-RESTEasy-Tracing-032:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  6.01 ms |  ---- %] [org.jboss.resteasy.plugins.providers.FileRangeWriter
@5cea30f7] is skipped
```

```
X-RESTEasy-Tracing-033:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  6.02 ms |  ---- %] [org.jboss.resteasy.plugins.providers.InputStreamProvider
@6c3361af] is skipped
X-RESTEasy-Tracing-034:
org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@5e84478b MBW
[ ---- /  6.02 ms |  ---- %] WriteTo by
org.jboss.resteasy.plugins.providers.StringTextStar
```

In addition, this header can be used to set the tracing info format:

```
X-RESTEasy-Tracing-Accept-Format
```

For example, set the value to 'JSON' to get the JSON formatted tracing info. Here is the command example:

```
curl --header "X-RESTEasy-Tracing-Accept: ok" --header "X-RESTEasy-Tracing-Threshold:
VERBOSE" --header "X-RESTEasy-Tracing-Accept-Format: JSON" -i
http://localhost:8081/foo
```

The JSON formatted tracing info from the response header will look like this:

```
X-RESTEasy-Tracing-000:
[{"event":"START","duration":0,"timestamp":1108675323356714,"text":"baseUri=[http://lo
calhost:8081/] requestUri=[http://localhost:8081/level] method=[GET] authScheme=[n/a]
accept=*/* accept-encoding=n/a accept-charset=n/a accept-language=n/a content-type=n/a
content-length=n/a
","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@78dd
0a57"},{"event":"START_HEADERS","duration":0,"timestamp":1108675323563245,"text":"Othe
r request headers: Accept=[*/*] Host=[localhost:8081] User-Agent=[curl/7.79.1] X-
RESTEasy-Tracing-Accept=[ok] X-RESTEasy-Tracing-Accept-Format=[JSON] X-RESTEasy-
Tracing-Threshold=[VERBOSE]
","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@78dd
0a57"},{"event":"PRE_MATCH_SUMMARY","duration":5361,"timestamp":1108675323671984,"text
":"PreMatchRequest summary: 0
filters","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"REQUEST_FILTER_SUMMARY","duration":3675,"timestamp":1108675324
245024,"text":"Request summary: 0
filters","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MATCH_RUNTIME_RESOURCE","duration":0,"timestamp":1108675324473
886,"text":"Matched resource:
template=[[org.jboss.resteasy.core.registry.ClassExpression @335ce24a]]
regexp=[\\Q\\E(.*)] matches=[[org.jboss.resteasy.core.registry.SegmentNode @3f56df02]]
from=[]","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MATCH_SUMMARY","duration":189460,"timestamp":1108675324593863,
"text":"RequestMatching
summary","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
```

```
st@78dd0a57"},{"event":"MATCH_RESOURCE","duration":0,"timestamp":1108675324720925,"tex
t":"Resource instance: [org.jboss.resteasy.core.ResourceMethodInvoker
@1d4d0e20]","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRe
quest@78dd0a57"},{"event":"MATCH_RESOURCE_METHOD","duration":0,"timestamp":11086753248
97726,"text":"Matched method  : public java.lang.String
org.jboss.resteasy.tracing.examples.TracingConfigResource.level(jakarta.ws.rs.core.Con
figuration)","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpR
equest@78dd0a57"},{"event":"REQUEST_FILTER","duration":9518,"timestamp":11086753250006
92,"text":"Filter by [org.jboss.resteasy.plugins.providers.jackson.PatchMethodFilter
@748a2754
#2147483647]","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttp
Request@78dd0a57"},{"event":"REQUEST_FILTER","duration":3702,"timestamp":1108675325018
731,"text":"Filter by
[org.jboss.resteasy.plugins.providers.sse.SseEventSinkInterceptor @7dad4808
#2147483647]","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttp
Request@78dd0a57"},{"event":"REQUEST_FILTER_SUMMARY","duration":49575,"timestamp":1108
675325026071,"text":"Request summary: 2
filters","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"METHOD_INVOKE","duration":512182,"timestamp":1108675325569769,
"text":"Resource [SINGLETON|class
org.jboss.resteasy.tracing.examples.TracingConfigResource|org.jboss.resteasy.tracing.e
xamples.TracingConfigResource@2634d8ed] method=[public java.lang.String
org.jboss.resteasy.tracing.examples.TracingConfigResource.level(jakarta.ws.rs.core.Con
figuration)]","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttp
Request@78dd0a57"},{"event":"DISPATCH_RESPONSE","duration":0,"timestamp":1108675325849
901,"text":"Response: [org.jboss.resteasy.specimpl.BuiltResponse @1b63a199
<200/SUCCESSFUL|OK|java.lang.String
@2e2fb68e>]","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpR
equest@78dd0a57"},{"event":"RESPONSE_FILTER","duration":7044,"timestamp":1108675326500
226,"text":"Filter by
[org.jboss.resteasy.plugins.interceptors.MessageSanitizerContainerResponseFilter
@bba8498
#4000]","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReques
t@78dd0a57"},{"event":"MBW_FIND","duration":0,"timestamp":1108675326755389,"text":"Fin
d MBW for type=[java.lang.String] genericType=[java.lang.String]
mediaType=[[jakarta.ws.rs.core.MediaType @1cc060ef]]
annotations=[@jakarta.ws.rs.GET(),
@jakarta.ws.rs.Path(\"/level\")]","requestId":"org.jboss.resteasy.plugins.server.servl
et.Servlet3AsyncHttpRequest@78dd0a57"},{"event":"MBW_SELECTED","duration":0,"timestamp
":1108675326771918,"text":"[org.jboss.resteasy.plugins.providers.StringTextStar
@23ffcf54] IS
writeable","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReq
uest@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326783918,"text
":"[org.jboss.resteasy.plugins.providers.FileProvider @37a3c619] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326790367,"text":
"[org.jboss.resteasy.plugins.providers.ByteArrayProvider @646b8da5] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326796138,"text":
"[org.jboss.resteasy.plugins.providers.StreamingOutputProvider @3b2a4bf4] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
```

```
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326801733,"text":
"[org.jboss.resteasy.plugins.providers.ReaderProvider @24729366] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326810040,"text":
"[org.jboss.resteasy.plugins.providers.DataSourceProvider @d481aff] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326820622,"text":
"[org.jboss.resteasy.plugins.providers.AsyncStreamingOutputProvider @35f6b856] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326831804,"text":
"[org.jboss.resteasy.plugins.providers.FileRangeWriter @5cea30f7] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326840405,"text":
"[org.jboss.resteasy.plugins.providers.InputStreamProvider @6c3361af] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_FIND","duration":0,"timestamp":1108675326885669,"text":"Fi
nd MBW for type=[java.lang.String] genericType=[java.lang.String]
mediaType=[[jakarta.ws.rs.core.MediaType @1cc060ef]]
annotations=[@jakarta.ws.rs.GET(),
@jakarta.ws.rs.Path(\"/level\")]","requestId":"org.jboss.resteasy.plugins.server.servl
et.Servlet3AsyncHttpRequest@78dd0a57"},{"event":"MBW_SELECTED","duration":0,"timestamp
":1108675326901119,"text":"[org.jboss.resteasy.plugins.providers.StringTextStar
@23ffcf54] IS
writeable","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReq
uest@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326929864,"text
":"[org.jboss.resteasy.plugins.providers.FileProvider @37a3c619] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326935210,"text":
"[org.jboss.resteasy.plugins.providers.ByteArrayProvider @646b8da5] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326942651,"text":
"[org.jboss.resteasy.plugins.providers.StreamingOutputProvider @3b2a4bf4] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326948715,"text":
"[org.jboss.resteasy.plugins.providers.ReaderProvider @24729366] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326954536,"text":
"[org.jboss.resteasy.plugins.providers.DataSourceProvider @d481aff] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326961563,"text":
"[org.jboss.resteasy.plugins.providers.AsyncStreamingOutputProvider @35f6b856] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326968934,"text":
"[org.jboss.resteasy.plugins.providers.FileRangeWriter @5cea30f7] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_SKIPPED","duration":0,"timestamp":1108675326974075,"text":
"[org.jboss.resteasy.plugins.providers.InputStreamProvider @6c3361af] is
skipped","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"MBW_WRITE_TO","duration":0,"timestamp":1108675326996329,"text"
:"WriteTo by
org.jboss.resteasy.plugins.providers.StringTextStar","requestId":"org.jboss.resteasy.p
```

```
lugins.server.servlet.Servlet3AsyncHttpRequest@78dd0a57"},{"event":"WI_SUMMARY","durat
ion":528614,"timestamp":1108675327196716,"text":"WriteTo summary: 0
interceptors","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttp
Request@78dd0a57"},{"event":"RESPONSE_FILTER_SUMMARY","duration":1671988,"timestamp":1
108675328127602,"text":"Response summary: 1
filters","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpReque
st@78dd0a57"},{"event":"FINISHED","duration":0,"timestamp":1108675328830911,"text":"Re
sponse status:
200","requestId":"org.jboss.resteasy.plugins.server.servlet.Servlet3AsyncHttpRequest@7
8dd0a57"}]
```


Above is the basic usage introduction of the ON_DEMAND tracing mode.

58.8. List Of Tracing Events

The tracing events are defined in RESTEasyServerTracingEvent. Here is a complete list of the tracing events and its descriptions:

- DISPATCH_RESPONSE

  Resource method invocation results to Jakarta RESTful Web Services Response.

- EXCEPTION_MAPPING

  ExceptionMapper invoked.

- FINISHED

  Request processing finished.

- MATCH_LOCATOR

  Matched sub-resource locator method.

- MATCH_PATH_FIND

  Matching path pattern.

- MATCH_PATH_NOT_MATCHED

  Path pattern not matched.

- MATCH_PATH_SELECTED

  Path pattern matched/selected.

- MATCH_PATH_SKIPPED

  Path pattern skipped as higher-priority pattern has been selected already.

- MATCH_RESOURCE

Matched resource instance.

- MATCH_RESOURCE_METHOD

  Matched resource method.

- MATCH_RUNTIME_RESOURCE

  Matched runtime resource.

- MATCH_SUMMARY

  Matching summary.

- METHOD_INVOKE

  Resource method invoked.

- PRE_MATCH

  RESTEasy HttpRequestPreprocessor invoked.

- PRE_MATCH_SUMMARY

  RESTEasy HttpRequestPreprocessor invoked.

- REQUEST_FILTER

  ContainerRequestFilter invoked.

- REQUEST_FILTER_SUMMARY

  ContainerRequestFilter invocation summary.

- RESPONSE_FILTER

  ContainerResponseFilter invoked.

- RESPONSE_FILTER_SUMMARY

  ContainerResponseFilter invocation summary.

- START

  Request processing started.

- START_HEADERS

  All HTTP request headers.

58.9. Tracing Example

The "resteasy-example" project, contains a RESTEasy Tracing Example to show the usages of tracing

features. Please check the example to see the usages in action.

Chapter 59. Validation

RESTEasy provides the support for validation mandated by the Jakarta RESTful Web Services, given the presence of an implementation of the https://jakarta.ee/specifications/bean-validation/3.1/[Bean Validation specification such as Hibernate Validator.

Validation provides a declarative way of imposing constraints on fields and properties of beans, bean classes, and the parameters and return values of bean methods. For example, in

```
@Path("all")
@TestClassConstraint(5)
public class TestResource {
   @Size(min=2, max=4)
   @PathParam("s")
   String s;

   private String t;

   @Size(min=3)
   public String getT() {
      return t;
   }

   @PathParam("t")
   public void setT(String t) {
      this.t = t;
   }

   @POST
   @Path("{s}/{t}/{u}")
   @Pattern(regexp="[a-c]+")
   public String post(@PathParam("u") String u) {
      return u;
   }
}
```

the field `s` is constrained by the Bean Validation built-in annotation `@Size` to have between 2 and 4 characters, the property `t` is constrained to have at least 3 characters, and the `TestResource` object is constrained by the application defined annotation `@TestClassConstraint` to have the combined lengths of `s` and `t` less than 5:

```
@Constraint(validatedBy = TestClassValidator.class)
@Target({TYPE})
@Retention(RUNTIME)
public @interface TestClassConstraint {
   String message() default "Concatenation of s and t must have length > {value}";
   Class<?>[] groups() default {};
   Class<? extends Payload>[] payload() default {};
```

```
   int value();
}

public class TestClassValidator implements ConstraintValidator<TestClassConstraint,
TestResource> {
   int length;

   public void initialize(TestClassConstraint constraintAnnotation) {
      length = constraintAnnotation.value();
   }

   public boolean isValid(TestResource value, ConstraintValidatorContext context) {
      return value.retrieveS().length() + value.getT().length() < length;
   }
}
```

See the links above for more about how to create validation annotations.

Also, the method parameter `u` is constrained to have no more than 5 characters, and the return value of method `post` is constrained by the built-in annotation `@Pattern` to match the regular expression "[a-c]+".

The sequence of validation constraint testing is as follows:

1. Create the resource and validate property, and class constraints.
2. Validate the resource method parameters.
3. If no violations have been detected, call the resource method and validate the return value



Though fields and properties are technically different, they are subject to the same kinds of constraints, so they are treated the same in the context of validation. Together, they will both be referred to as "properties" herein.

59.1. Violation reporting

If a validation problem occurs, either a problem with the validation definitions or a constraint violation, RESTEasy will set the return header `org.jboss.resteasy.api.validation.Validation.VALIDATION_HEADER` ("validation-exception") to "true".

If RESTEasy detects a structural validation problem, such as a validation annotation with a missing validator class, it will return a String representation of a `jakarta.validation.ValidationException`. For example:

```
jakarta.validation.ValidationException: HV000028: Unexpected exception during isValid
call.[org.jboss.resteasy.test.validation.TestValidationExceptions$OtherValidationExcep
tion]
```

If any constraint violations are detected, RESTEasy will return a report in one of a variety of

formats. If one of "application/xml" or "application/json" occur in the "Accept" request header, RESTEasy will return an appropriately marshalled instance of `org.jboss.resteasy.api.validation.ViolationReport`:

```
@XmlRootElement(name="violationReport")
@XmlAccessorType(XmlAccessType.FIELD)
public class ViolationReport {

   public ArrayList<ResteasyConstraintViolation> getPropertyViolations() {
      return propertyViolations;
   }

   public ArrayList<ResteasyConstraintViolation> getClassViolations() {
      return classViolations;
   }

   public ArrayList<ResteasyConstraintViolation> getParameterViolations() {
      return parameterViolations;
   }

   public ArrayList<ResteasyConstraintViolation> getReturnValueViolations() {
      return returnValueViolations;
   }
}
```

where `org.jboss.resteasy.api.validation.ResteasyConstraintViolation` is defined:

```
@XmlRootElement(name="resteasyConstraintViolation")
@XmlAccessorType(XmlAccessType.FIELD)
public class ResteasyConstraintViolation implements Serializable {

   /**
    * @return type of constraint
    */
   public ConstraintType.Type getConstraintType() {
      return constraintType;
   }

   /**
    * @return description of element violating constraint
    */
   public String getPath() {
      return path;
   }

   /**
    * @return description of constraint violation
    */
   public String getMessage() {
```

```
      return message;
   }

   /**
    * @return object in violation of constraint
    */
   public String getValue() {
      return value;
   }

   /**
    * @return String representation of violation
    */
   public String toString() {
      return "[" + type() + "]\r[" + path + "]\r[" + message + "]\r[" + value + "]\r";
   }

   /**
    * @return String form of violation type
    */
   public String type() {
      return constraintType.toString();
   }
}
```

and `org.jboss.resteasy.api.validation.ConstraintType` is the enumeration

```
public class ConstraintType {
   public enum Type {CLASS, PROPERTY, PARAMETER, RETURN_VALUE}
}
```

If both "application/xml" or "application/json" occur in the "Accept" request header, the media type is chosen according to the ranking given by implicit or explicit "q" parameter values. In the case of a tie, the returned media type is indeterminate.

If neither "application/xml" or "application/json" occur in the "Accept" request header, RESTEasy returns a report with a String representation of each `ResteasyConstraintViolation`, where each field is delimited by '[' and ']', followed by a '\r', with a final '\r' at the end. For example,

```
[PROPERTY]
[s]
[size must be between 2 and 4]
[a]

[PROPERTY]
[t]
[size must be between 3 and 5]
[z]
```

```
[CLASS]
[]
[Concatenation of s and t must have length > 5]
[org.jboss.resteasy.validation.TestResource@68467a6f]

[PARAMETER]
[test.<cross-parameter>]
[Parameters must total <= 7]
[[5, 7]]

[RETURN_VALUE]
[g.<return value>]
[size must be between 2 and 4]
[abcde]
```

where the four fields are

1. type of constraint
2. path to violating element (e.g., property name, class name, method name and parameter name)
3. message
4. violating element

The `ViolationReport` can be reconsititued from the `String` as follows:

```
Client client = ClientBuilder.newClient();
Invocation.Builder request = client.target(...).request();
Response response = request.get();
if (Boolean.valueOf(response.getHeaders().getFirst(Validation.VALIDATION_HEADER))) {
   String s = response.readEntity(String.class);
   ViolationReport report = new ViolationReport(s);
}
```

If the path field is considered to be too much server side information, it can be suppressed by setting the parameter "resteasy.validation.suppress.path" to "true". In that case, "*" will be returned in the path fields.

Validation Service Providers

The form of validation mandated by the Jakarta RESTful Web Services specification, based on Bean Validation 1.1 or greater, is supported by the RESTEasy module resteasy-validator-provider, which produces the artifact resteasy-validator-provider-version.jar. Validation is turned on by default (assuming resteasy-validator-provider-version.jar is available), though parameter and return value validation can be turned off or modified in the validation.xml configuration file. See the Hibernate Validator documentation for the details.

RESTEasy obtains a bean validation implementation by looking in the available `META-`

INF/services/jakarta.ws.rs.Providers files for an implementation of ContextResolverGeneralValidator, where org.jboss.resteasy.spi.GeneralValidator is

```
public interface GeneralValidator {
   /**
    * Validates all constraints on {@code object}.
    *
    * @param object object to validate
    * @param groups the group or list of groups targeted for validation (defaults to
    *        {@link Default})
    * @return constraint violations or an empty set if none
    * @throws IllegalArgumentException if object is {@code null}
    *         or if {@code null} is passed to the varargs groups
    * @throws ValidationException if a non recoverable error happens
    *         during the validation process
    */
   void validate(HttpRequest request, Object object, Class<?>... groups);
   /**
    * Validates all constraints placed on the parameters of the given method.
    *
    * @param <T> the type hosting the method to validate
    * @param object the object on which the method to validate is invoked
    * @param method the method for which the parameter constraints is validated
    * @param parameterValues the values provided by the caller for the given method's
    *        parameters
    * @param groups the group or list of groups targeted for validation (defaults to
    *        {@link Default})
    * @return a set with the constraint violations caused by this validation;
    *         will be empty if no error occurs, but never {@code null}
    * @throws IllegalArgumentException if {@code null} is passed for any of the
parameters
    *         or if parameters don't match with each other
    * @throws ValidationException if a non recoverable error happens during the
    *         validation process
    */
   void validateAllParameters(HttpRequest request, Object object, Method method,
Object[] parameterValues, Class<?>... groups);

   /**
    * Validates all return value constraints of the given method.
    *
    * @param <T> the type hosting the method to validate
    * @param object the object on which the method to validate is invoked
    * @param method the method for which the return value constraints is validated
    * @param returnValue the value returned by the given method
    * @param groups the group or list of groups targeted for validation (defaults to
    *        {@link Default})
    * @return a set with the constraint violations caused by this validation;
    *         will be empty if no error occurs, but never {@code null}
    * @throws IllegalArgumentException if {@code null} is passed for any of the
```

```
object,
    *         method or groups parameters or if parameters don't match with each other
    * @throws ValidationException if a non recoverable error happens during the
    *         validation process
    */
   void validateReturnValue(
         HttpRequest request, Object object, Method method, Object returnValue, Class
<?>... groups);

   /**
    * Indicates if validation is turned on for a class.
    *
    * @param clazz Class to be examined
    * @return true if and only if validation is turned on for clazz
    */
   boolean isValidatable(Class<?> clazz);

   /**
    * Indicates if validation is turned on for a method.
    *
    * @param method method to be examined
    * @return true if and only if validation is turned on for method
    */
   boolean isMethodValidatable(Method method);

   void checkViolations(HttpRequest request);
}
```

The methods and the javadoc are adapted from the Bean Validation 1.1 classes `jakarta.validation.Validator` and `jakarta.validation.executable.ExecutableValidator`.

RESTEasy module resteasy-validator-provider supplies an implementation of `GeneralValidator`. An alternative implementation may be supplied by implementing `ContextResolverGeneralValidator` and `org.jboss.resteasy.spi.validation.GeneralValidator`.

A validator intended to function in the presence of CDI must also implement the sub-interface

```
public interface GeneralValidatorCDI extends GeneralValidator {
   /**
    * Indicates if validation is turned on for a class.
    *
    * This method should be called from the resteasy-core module. It should
    * test if injectorFactor is an instance of CdiInjectorFactory, which indicates
    * that CDI is active.  If so, it should return false. Otherwise, it should
    * return the same value returned by GeneralValidator.isValidatable().
    *
    * @param clazz Class to be examined
    * @param injectorFactory the InjectorFactory used for clazz
    * @return true if and only if validation is turned on for clazz
    */
```

```
    boolean isValidatable(Class<?> clazz, InjectorFactory injectorFactory);

    /**
     * Indicates if validation is turned on for a class.
     * This method should be called only from the resteasy-cdi module.
     *
     * @param clazz Class to be examined
     * @return true if and only if validation is turned on for clazz
     */
    boolean isValidatableFromCDI(Class<?> clazz);

    /**
     * Throws a ResteasyViolationException if any validation violations have been
detected.
     * The method should be called only from the resteasy-cdi module.
     * @param request
     */
    void checkViolationsfromCDI(HttpRequest request);

    /**
     * Throws a ResteasyViolationException if either a ConstraintViolationException or
a
     * ResteasyConstraintViolationException is embedded in the cause hierarchy of e.
     *
     * @param request
     * @param e
     */
    void checkForConstraintViolations(HttpRequest request, Exception e);
}
```

The validator in resteasy-validator-provider implements GeneralValidatorCDI.



Since RESTEasy 8.0, CDI is always active and resources are CDI beans. RESTEasy validates the resource instance, the parameters and the return value of resources created by CDI, as it does for resources it creates itself, so `isValidatable(Class, InjectorFactory)` no longer depends on whether CDI is active. The `JaxrsInjectionTarget` of the resteasy-cdi module validates the instances of the classes for which `isValidatableFromCDI()` returns `true`, when CDI creates them. The validator in resteasy-validator-provider returns `true` only for the classes RESTEasy cannot validate itself, i.e. session beans with class, field or property constraints, which the EJB container creates. A custom implementation returning `true` for all classes validates resources created by CDI twice.

59.2. Validation Implementations

As mentioned above, RESTEasy validation requires an implementation of the Bean Validation specification such as Hibernate Validator. Hibernate Validator is supplied automatically when RESTEasy is running in the context of WildFly. Otherwise, it should be made available. For example,

in maven

```
<dependency>
    <groupId>org.hibernate.validator</groupId>
    <artifactId>hibernate-validator/artifactId>
</dependency>
```

Chapter 60. Internationalization and Localization

With the help of the JBoss Logging project, all log and exception messages in RESTEasy can be internationalized. for more about JBoss Logging Tooling, see https://jboss-logging.github.io/jboss-logging-tools/, Chapters 4 and 5.

60.1. Internationalization

Each module in RESTEasy that produces any text in the form of logging messages or exception messages has an interface named org.jboss.resteasy...i18n.Messages which contains the default messages. Those modules which do any logging also have an interface named org.jboss.resteasy...i18n.LogMessages which gives access to an underlying logger. With the exception of the resteasy-core-spi module, all messages are in the `Messages` class. resteasy-core-spi has exception messages in the `Messages` class and log messages in the `LogMessages` class.

Each message is prefixed by the project code "RESTEASY" followed by an ID which is unique to RESTEasy. These IDs belong to the following ranges:

Range	Module
2000-2999	resteasy-core-spi log messages
3000-4499	resteasy-core-spi exception messages
4500-4999	resteasy-client
5000-5499	providers/resteasy-atom
5500-5999	providers/fastinfoset
6000-6499	providers/resteasy-html
6500-6999	providers/jaxb
7500-7999	providers/multipart
8000-8499	providers/resteasy-hibernatevalidator-provider
8500-8999	providers/resteasy-validator-provider
9500-9999	async-http-servlet-3.0
10000-10499	cache-core
10500-10999	resteasy-cdi
11500-11999	resteasy-jsapi
12000-12499	resteasy-links
12500-12999	resteasy-servlet-initializer
13000-13499	resteasy-spring
13500-13999	security/resteasy-crypto
14000-14499	security/jose-jwt

Range	Module
14500-14999	security/keystone/keystone-as7
15000-15499	security/keystone/keystone-core
15500-15999	security/resteasy-oauth
16000-16499	security/skeleton-key-idm/skeleton-key-as7
16500-16999	security/skeleton-key-idm/skeleton-key-core
17000-17499	security/skeleton-key-idm/skeleton-key-idp
17500-17999	server-adapters/resteasy-jdk-http
18500-18999	server-adapters/resteasy-netty4

For example, the Jakarta XML Binding provider contains the interface `org.jboss.resteasy.plugins.providers.jaxb.i18.Messages` which looks like

```
@MessageBundle(projectCode = "RESTEASY")
public interface Messages {
   Messages MESSAGES = org.jboss.logging.Messages.getBundle(Messages.class);
   int BASE = 6500;

   @Message(id = BASE, value = "Collection wrapping failed, expected root element name
of %s got %s")
   String collectionWrappingFailedLocalPart(String element, String localPart);

   @Message(id = BASE + 05, value = "Collection wrapping failed, expect namespace of
%s got %s")
   String collectionWrappingFailedNamespace(String namespace, String uri);
}
```

The value of a message is retrieved by referencing a method and passing the appropriate parameters. For example,

```
throw new JAXBUnmarshalException(Messages.MESSAGES.collectionWrappingFailedLocalPart
(wrapped.element(), ele.getName().getLocalPart()));
```

Chapter 61. Localization

When RESTEasy is built with the "i18n" profile, a template properties file containing the default messages is created in a subdirectory of target/generated-translation-files. In the Jakarta XML Binding provider, for example, the `Messages.i18n_locale_COUNTRY_VARIANT.properties` goes in the `org/jboss/resteasy/plugins/providers/jaxb/i18n` directory, and the first few lines are:

```
# Id: 6500
# Message: Collection wrapping failed, expected root element name of {0} got {1}
# @param 1: element -
# @param 2: localPart -
collectionWrappingFailedLocalPart=Collection wrapping failed, expected root element
name of {0} got {1}
# Id: 6505
# Message: Collection wrapping failed, expect namespace of {0} got {1}
# @param 1: namespace -
# @param 2: uri -
collectionWrappingFailedNamespace=Collection wrapping failed, expect namespace of {0}
got {1}
```

To provide the translation of the messages for a particular locale, the file should be renamed, replacing "locale", "COUNTRY", and "VARIANT" as appropriate (possibly omitting the latter two), and copied to the `src/main/resources` directory. In the Jakarta XML Binding provider, it would go in src/main/resources/org/jboss/resteasy/plugins/providers/jaxb/i18n`.

For testing purposes, each module containing a Messages interface has two sample properties files, for the locale "en" and the imaginary locale "xx", in the `src/test/resources` directory. They are copied to `src/main/resources` when the module is built and deleted when it is cleaned.

The `Messages.i18n_xx.properties` file in the Jakarta XML Binding provider, for example, looks like

```
# Id: 6500
# Message: Collection wrapping failed, expected root element name of {0} got {1}
# @param 1: element -
# @param 2: localPart -
collectionWrappingFailedLocalPart=Collection wrapping failed, expected root element
name of {0} got {1}
# Id: 6505
# Message: Collection wrapping failed, expect namespace of {0} got {1}
# @param 1: namespace -
# @param 2: uri -
collectionWrappingFailedNamespace=aaa {0} bbb {1} ccc
```

Note that the value of `collectionWrappingFailedNamespace` is modified.

Chapter 62. Maven and RESTEasy

JBoss's Maven Repository is at: https://repository.jboss.org/nexus/content/groups/public/. The RESTEasy dependencies from this repository are synchronized to Maven Central. Note it may take up to a day for this synchronization to happen.

RESTEasy is modularized into multiple components. Each component is accessible as a Maven artifact. As a convenience RESTEasy provides a BOM containing the complete set of components with the appropriate versions for the "stack".

It is recommended to declare the BOM in your POM file, that way you will always be sure to get the correct version of the artifacts. In addition, you will not need to declare the version of each RESTEasy artifact called out in the dependencies section.

Declare the BOM file in the dependencyManagement section of the POM file like this. Note that Maven version 3.9.0 or higher is required to process BOM files.

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.jboss.resteasy</groupId>
            <artifactId>resteasy-bom</artifactId>
            <version>8.0.0.Alpha1</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

Declare the specific RESTEasy artifacts you require in the dependencies section of the POM file like this.

```
<dependencies>
    <dependency>
        <groupId>org.jboss.resteasy</groupId>
        <artifactId>resteasy-client</artifactId>
    </dependency>
</dependencies>
```

It is possible to reference a RESTEasy artifact version not in the current BOM by specifying a version in the dependency itself.

```
<dependencies>
    <dependency>
        <groupId>org.jboss.resteasy</groupId>
        <artifactId>resteasy-client</artifactId>
        <version>8.0.0.Alpha1<version>
```

```
    </dependency>
</dependencies>
```

Chapter 63. Migrating to RESTEasy 8

This chapter covers upgrading from RESTEasy 6.2 or 7.0 to RESTEasy 8.0. When upgrading from 6.2, the changes of Jakarta RESTful Web Services 4.0 and Jakarta EE 11 apply as well. They require no RESTEasy specific changes.

RESTEasy 8 implements Jakarta RESTful Web Services 5.0, which requires CDI. The `@Path`, `@Provider` and `@ApplicationPath` annotations are bean defining annotations, so resources, providers and `Application` subclasses are CDI beans, even in an archive without a `beans.xml`. Resources are `@RequestScoped` by default, and providers and `Application` subclasses are `@ApplicationScoped`. Most of the changes below follow from this. See CDI Integration for details.

63.1. Changes that fail the deployment

63.1.1. Providers declared `@Dependent`

A provider class annotated with `@Provider` and explicitly declared `@Dependent` fails the deployment (RESTEASY010670). Providers are application singletons.

```
@Provider
@Dependent // fails the deployment
public class MyFilter implements ContainerRequestFilter {
    // ...
}
```

Declare the provider `@ApplicationScoped` or `@Singleton`, or remove the scope annotation, as `@Provider` defaults to `@ApplicationScoped`. A provider class which declares type variables is excluded, as CDI requires it to be `@Dependent`.

63.1.2. Public fields of normal scoped resources and providers

A normal scoped bean must not declare a non-static public field. A resource or provider with such a field fails the deployment (e.g. `WELD-000075`), unless it declares `@Dependent` or `@Singleton`.

Make the field private, and add an accessor if other classes read it, e.g. a `ConstraintValidator` of a class level constraint.

63.1.3. Unsatisfied injection points of resources and providers

A resource or provider in an archive with `bean-discovery-mode="annotated"`, or without a `beans.xml`, was not a CDI bean before, so CDI did not validate its injection points. An `@Inject` field without a matching bean was silently left `null`. Now the resource or provider is a bean, and an unsatisfied or ambiguous injection point fails the deployment (e.g. `WELD-001408`).

```
@Path("/echo")
public class EchoResource {
    @Inject
```

```
    EchoService service; // fails the deployment if EchoService is not a bean
}
```

Make the injected type a bean, e.g. by adding a bean defining annotation such as `@Dependent` or `@ApplicationScoped`, or a `beans.xml` to its archive. An EJB in an `ejb-jar.xml` with `metadata-complete="true"` is not a session bean, as its EJB annotations are ignored, and needs a bean defining annotation as well.

The same applies to a `@BeanParam` class which was not a bean before, as it's now made a bean. See `@BeanParam` classes.

63.1.4. Beans for Jakarta RESTful Web Services context types

RESTEasy provides the CDI beans for the Jakarta RESTful Web Services context types, e.g. `UriInfo`, `HttpHeaders` and `Sse`. A bean of the application for one of these types with the `@Default` or `@Context` qualifier, e.g. a producer method, fails the deployment (RESTEASY010660). Remove the bean, or give it a qualifier of its own.

63.1.5. Invalid constraint definitions and declarations

The Jakarta Validation CDI integration inspects the constraints of resources during bean discovery. An invalid constraint definition, e.g. a constraint annotation without a `message` member, or an invalid constraint declaration, e.g. an overriding method redefining the parameter constraints, fails the deployment with a `ConstraintDefinitionException` (e.g. `HV000074`) or a `ConstraintDeclarationException` (e.g. `HV000151`).

Before, RESTEasy reported these errors on the request as a `500 Internal Server Error`, which an `ExceptionMapper` of the application could handle. Such a mapper is no longer invoked for these errors.

63.2. Default scopes and warnings

63.2.1. Unproxyable and generic resources and providers

Some classes cannot have the default normal scopes:

- A class which cannot be proxied, e.g. a final class, a class with a final method or a class without a non-private no-argument constructor. A class with only an `@Inject` constructor cannot be proxied.
- A class which declares type variables, e.g. a generic base resource `CrudResource<T>` annotated with `@Path`. CDI only allows such a bean to be `@Dependent`.

If such a class does not declare a scope itself, RESTEasy is lenient and logs a warning:

Class	Scope used	Warning
Unproxyable resource	`@Dependent`	RESTEASY011010
Unproxyable provider	`@Singleton`	RESTEASY011005

Class	Scope used	Warning
Resource or provider declaring type variables	`@Dependent`	RESTEASY011020

Declare `@Dependent` or `@Singleton` on the class to avoid the warning and to be portable. A class which explicitly declares a normal scope it cannot have fails the deployment, following the CDI rules.

63.2.2. Other `@Dependent` providers

A provider whose CDI bean is `@Dependent` and which is not covered above logs a warning (RESTEASY011015) when it is registered, e.g. a provider class without `@Provider` declared `@Dependent`, or a provider created by a producer method without a scope. RESTEasy still creates a single instance of it. Declare a scope to avoid the warning.

63.2.3. Providers without `@Provider` in `bean-discovery-mode="all"` archives

In an archive with `bean-discovery-mode="all"`, a discovered class which implements a provider contract, is not annotated with `@Provider` and declares no scope is made `@ApplicationScoped`. It's made `@Singleton` if it cannot be proxied, or `@Dependent` if it declares type variables. Before, it was a `@Dependent` bean.

63.2.4. `@BeanParam` classes

A class used for a `@BeanParam` field, setter method or parameter of a resource, which is not a bean and has a no-argument or `@Inject` constructor, is made a `@Dependent` bean. CDI creates it and injects all its members, including `@Inject` members, which were not injected before.

A `@BeanParam` class with a normal scope, e.g. `@RequestScoped`, logs a warning (RESTEASY011025). The injected instance is a client proxy, whose fields do not hold the parameter values, e.g. when a `@Valid` class is validated. Declare `@Dependent` on the class instead.

63.3. Behavior changes

63.3.1. `@Dependent` resources are destroyed when the request completes

A resource declared `@Dependent` is created through CDI for each request and destroyed, including its `@PreDestroy` method, when the request completes. For a suspended request, this is when the asynchronous response completes, e.g. after it was resumed, cancelled or timed out. The same applies to `@Dependent` beans returned from `ResourceContext.getResource()`. The application must not keep references to these instances beyond the request.

63.3.2. Resource instances are CDI client proxies

The instance of a normal scoped resource that RESTEasy hands to the application, e.g. an element of `UriInfo.getMatchedResources()`, is the CDI client proxy. The proxy is an instance of the resource class, its superclasses and its interfaces, but its class is a generated subclass:

```
Object resource = uriInfo.getMatchedResources().get(0);
resource instanceof MyResource;              // true
resource.getClass() == MyResource.class;     // false
resource.getClass().getAnnotation(Path.class); // null, @Path is not @Inherited
```

Use `instanceof` rather than comparing classes, and don't reflect on `getClass()`.

63.3.3. The injected `Application`

An injected `Application`, e.g. a `@Context Application` field, delegates to the `Application` of the current request. It's not an instance of the application's `Application` subclass, so it cannot be cast to it. Inject the subclass instead:

```
@Context
Application application;
((MyApplication) application).getConfig(); // ClassCastException

@Context
MyApplication application;
application.getConfig();                   // works
```

63.3.4. When resource instances are created

The instance of a normal scoped resource is created after the request filters were invoked and before the parameters of the resource method are injected. Its `@PostConstruct` method is invoked before the entity is read, i.e. before reader interceptors and message body readers are invoked.

63.3.5. Validation

Resources created by CDI are validated as before: RESTEasy validates the resource instance (fields, property getters and class level constraints), then the parameters of the resource method, and reports the violations together. The return value is validated after the method is invoked. Session bean resources are validated as before. Differences:

- If a resource class declares method constraints, the Jakarta Validation CDI interceptor is bound to it, and the instance is a CDI subclass. The value of a class level violation, the instance's `toString()`, may be e.g. `MyResource$Proxy$_$$_WeldSubclass@1234` rather than `MyResource@1234`.
- The Jakarta Validation CDI interceptor validates the return value before RESTEasy does. The violations are still reported with a `ViolationReport`. If the request has an `Accept-Language` header, the messages are in the request's language, as before.
- A sub-resource returned from a sub-resource locator which is a CDI client proxy, e.g. an injected `@RequestScoped` bean, is not validated. Other sub-resources are validated.

See Validation for details.

63.3.6. `CompletionCallback` on client disconnect

In a servlet container, the registered `CompletionCallback` instances are invoked, with the failure, when the asynchronous processing ends because of an error, e.g. the client closed the connection.

63.3.7. Known limitation

Per-request objects created outside resources, e.g. `AsyncResponse.register(Class)` callbacks, classes returned from sub-resource locators and `@Form` or `@Query` classes, which are `@Dependent` CDI beans are not destroyed, so their `@PreDestroy` methods are not invoked. This is the same as in RESTEasy 7.

63.4. Configuration

- `dev.resteasy.cdi.enhanced.enabled` has been removed. The CDI integration, including the injection of `@Context` and `@*Param` annotated members through CDI, can no longer be disabled.
- `resteasy.proxy.implement.all.interfaces` is deprecated.

63.5. Requirements

RESTEasy 8 requires Java SE 21 or later.

A Jakarta Validation provider's CDI integration, e.g. `hibernate-validator-cdi`, must be available for validation. It's included in WildFly. `resteasy-validator-provider` now depends on Hibernate Validator, its CDI integration and the Jakarta Expression Language implementation Expressly, so nothing needs to be added when using the Java SE bootstrap. To use another Jakarta Validation provider, exclude these dependencies and add the provider and its CDI integration.

63.6. Changes for integrations

Applications are not affected by these changes. They concern integrations of RESTEasy, e.g. a runtime such as Quarkus, or a custom `InjectorFactory` or `GeneralValidatorCDI` implementation.

- New `org.jboss.resteasy.spi.RequestListener`, discovered with `ServiceLoader`, which is notified once a request has completed.
- `ResteasyAsynchronousResponse.completionCallbacks(Throwable)`: implementations which do not extend `AbstractAsynchronousResponse` do not notify the `RequestListener` instances.
- New default method `InjectorFactory.unwrapProxy(Object)`. RESTEasy invokes it on the resource before the parameters are injected, which creates the contextual instance of a CDI client proxy, and validates the returned instance.
- New default method `InjectorFactory.resolveType(Class)`, which returns the type a container proxy class delegates to, e.g. the bean class of a CDI client proxy class. RESTEasy uses it where the class of a provider or resource instance matters, e.g. for provider contracts, annotations and the resource metadata. The default implementation resolves synthetic proxy classes to their superclass, which covers Weld and OpenWebBeans. An integration with a CDI container which does not mark its proxy classes as synthetic must override it.
- New `org.jboss.resteasy.spi.StreamValueInjector`, a `ValueInjector` which indicates if content

was written to the injected stream. If a parameter of a `void` resource method is injected by a `StreamValueInjector` which reports content, e.g. a `@Context HttpServletResponse` written to, the response is not replaced with a `204 No Content`.

- `GeneralValidatorCDI.isValidatableFromCDI(Class)`: the `JaxrsInjectionTarget` validates the instances of the classes for which it returns `true` when CDI creates them. The validator of `resteasy-validator-provider` returns `true` only for the classes RESTEasy cannot validate itself, i.e. session beans with class, field or property constraints. A custom implementation returning `true` for all classes validates resources created by CDI twice.
- `GeneralValidatorImpl.isValidatable(Class)` no longer depends on whether CDI is active.
- `resteasy-validator-provider` no longer depends on `resteasy-cdi` or the CDI API.

Chapter 64. Migration from older versions

64.1. Migration to RESTEasy 3.0 series

Many facilities from RESTEasy 2 appear in a different form in RESTEasy 3. For example, much of the client framework in RESTEasy 2 is formalized, in modified form, in JAX-RS 2.0. RESTEasy versions 3.0.x implement both the older deprecated form and the newer conformant form. The deprecated form is moved to legacy module in RESTEasy 3.1 and finally removed in RESTEasy 4. For more information on upgrading from various deprecated facilities in RESTEasy 2, see http://docs.jboss.org/resteasy/docs/resteasy-upgrade-guide-en-US.pdf

64.2. Migration to RESTEasy 3.1 series

RESTEasy 3.1.0.Final release comes with many changes compared to previous 3.0 point releases. User discernible changes in RESTEasy 3.1.0.Final include

- module reorganization
- package reorganization
- new features
- minor behavioral changes
- miscellaneous changes

In this chapter we focus on changes that might cause existing code to fail or behave in new ways. The audience for this discussion may be partitioned into three subsets, depending on the version of RESTEasy currently in use, the API currently in use, and the API to be used after an upgrade to RESTEasy 3.1. The following APIs are available:

1. **RESTEasy 2**: RESTEasy 2 implements the JAX-RS 1 specification, and adds a variety of additional facilities, such as a client API, a caching system, an interceptor framework, etc. All of these user facing classes and interfaces comprise the RESTEasy 2 API.
2. **RESTEasy 3**: RESTEasy 3 implements the JAX-RS 2 specification, and adds some additional facilities. Many of the non-spec facilities from the RESTEasy 2 API are formalized, in altered form, in JAX-RS 2, in which case the older facilites are deprecated. The non-deprecated user facing classes and interfaces in RESTEasy 3 comprise the RESTEasy 3 API.

These definitions are rather informal and imprecise, since the user facing classes / interfaces in Resteasy 3.0.19.Final, for example, are a proper superset of the user facing classes / interfaces in RESTEasy 3.0.1.Final. For this discussion, we identify the API with the version currently in use in a given project.

Now, there are three potential target audiences of users planning to upgrade to RESTEasy 3.1.0.Final:

1. Those currently using RESTEasy API 3 with some RESTEasy 3.0.x release
2. Those currently using RESTEasy API 2 with some RESTEasy 2.x or 3.0.x release and planning to

upgrade to RESTEasy API 3

3. Those currently using RESTEasy API 2 with some RESTEasy 2.x or 3.0.x release and planning to continue to use RESTEasy API 2

Of these, users in Group 2 have the most work to do in upgrading from RESTEasy API 2 to RESTEasy API 3. They should consult the separate guide Upgrading from RESTEasy 2 to RESTEasy 3.

Ideally, users in Groups 1 and 3 might make some changes to take advantage of new features but would have no changes forced on them by reorganization or altered behavior. Indeed, that is almost the case, but there are a few changes that they should be aware of.

64.2.1. Upgrading with RESTEasy 3 API

All RESTEasy changes are documented in JIRA issues. Issues that describe detectable changes in release 3.1.0.Final that might impact existing applications include

- RESTEASY-1341: Build method of org.jboss.resteasy.client.jaxrs.internal.ClientInvocationBuilder always return the same instance.

  When a `build()` method from

  - `org.jboss.resteasy.client.jaxrs.internal.ClientInvocationBuilder` in resteasy-client,
  - `org.jboss.resteasy.specimpl.LinkBuilderImpl` in resteasy-core,
  - `org.jboss.resteasy.specimpl.ResteasyUriBuilder` in resteasy-jaxrs

    is called, it will return a new object. This behavior might be seen indirectly. For example,

    ```
    Builder builder = client.target(generateURL(path)).request();
    Link link = new LinkBuilderImpl().uri(href).build();
    URI uri = uriInfo.getBaseUriBuilder().path("test").build();
    ```

- RESTEASY-1433: Compile with JDK 1.8 source/target version

  As it says. Depending on the application, it might be necessary to recompile with a target of JDK 1.8 so that calls to RESTEasy code can work.

- RESTEASY-1484: CVE-2016-6346: Abuse of GZIPInterceptor in can lead to denial of service attack

  Prior to release 3.1.0.Final, the default behavior of RESTEasy was to use GZIP to compress and decompress messages whenever "gzip" appeared in the Content-Encoding header. However, decompressing messages can lead to security issues, so, as of release 3.1.0.Final, GZIP compression has to be enabled explicitly. For details, see Chapter GZIP Compression/Decompression.



Because of some package reorganization due to RESTEASY-1531 (see below), the GZIP interceptors, which used to be in package `org.jboss.resteasy.plugins.interceptors.encoding` are now in `org.jboss.resteasy.plugins.interceptors`.

- RESTEASY-1531: Restore removed RESTEasy internal classes into a deprecated/disabled module

  This issue is related to refactoring deprecated elements of the RESTEasy 2 API into a separate module, and, ideally, would have no bearing at all on RESTEasy 3. However, a reorganization of packages has led to moving some non-deprecated API elements in the resteasy-core module:

  - `org.jboss.resteasy.client.ClientURI` is now `org.jboss.resteasy.annotations.ClientURI`
  - `org.jboss.resteasy.core.interception.JaxrsInterceptorRegistryListener` is now `org.jboss.resteasy.core.interception.jaxrs.JaxrsInterceptorRegistryListener`
  - `org.jboss.resteasy.spi.interception.DecoratorProcessor` is now `org.jboss.resteasy.spi.DecoratorProcessor`
  - All of the dynamic features and interceptors in the package `org.jboss.resteasy.plugins.interceptors.encoding` are now in `org.jboss.resteasy.plugins.interceptors`

64.3. Migration to RESTEasy 3.5+ series

RESTEasy 3.5 series is a spin-off of the old RESTEasy 3.0 series, featuring Jakarta RESTful Web Services implementation.

The reason why 3.5 comes from 3.0 instead of the 3.1 / 4.0 development streams is basically providing users with a selection of RESTEasy 4 critical / strategic new features, while ensuring full backward compatibility. As a consequence, no major issues are expected when upgrading RESTEasy from 3.0.x to 3.5.x. The 3.6 and all other 3.x minors after that are backward compatible evolutions of 3.5 series.

The natural upgrade path for users already on RESTEasy 3.1 series is straight to RESTEasy 4 instead.

64.4. Migration to RESTEasy 4 series

User migrating from RESTEasy 3.0 and 3.5+ series should be aware of the changes mentioned in the RESTEasy 3.1 migration section. In addition to that, the aspects from the following sections are to be considered.

64.4.1. Public / private API

The `resteasy-jaxrs` and `resteasy-client` modules in RESTEasy 3 contain most of the framework classes and there's no real demarcation between what is internal implementation detail and what is for public consumption. In WildFly, the artifact archives from those modules are also included in a public module. Given the common expectation of full backward compatibility of whatever comes from public modules, to allow for easier project evolution and maintenance, in RESTEasy 4.0.0.Final those big components have been split as follows:

resteasy-core-spi

The public classes of the former `resteasy-jaxrs` module; the following packages are included:

- `org.jboss.resteasy.annotations`
- `org.jboss.resteasy.api.validation`
- `org.jboss.resteasy.spi`
- `org.jboss.resteasy.plugins.providers.validation`

resteasy-core

The internal details of the former `resteasy-jaxrs` module, including classes from the following packages:

- `org.jboss.resteasy.core`
- `org.jboss.resteasy.mock`
- `org.jboss.resteasy.plugins`
- `org.jboss.resteasy.specimpl`
- `org.jboss.resteasy.tracing`
- `org.jboss.resteasy.util`

resteasy-client-api

The public classes from the former `resteasy-client` module, basically whatever is used for configuring the RESTEasy client additions:

- `ClientHttpEngine` and `ClientHttpEngineBuilder`
- `ProxyBuilder` and `ProxyConfig`
- `ResteasyClient`
- `ResteasyClientBuilder`
- `ResteasyWebTarget`



The `ClientHttpEngineBuilder` has been deprecated and the `ClientHttpEngineFactory` should be used instead.

resteasy-client

The remainings of the former `resteasy-client` module, internal details.

As a consequence of the split, all modules except `resteasy-core-spi` and `resteasy-client-api` are effectively private / internal. User applications and integration code should not directly rely on classes from those modules, which can be changed without going through any formal deprecation process.

Unfortunately, the refactoring that led to this implied some unavoidable class moves and changes breaking backward compatibility. A detailed list of the potentially problematic changes is available on the refactoring PR.

64.4.2. Deprecated classes and modules removal

All classes and modules that were deprecated in RESTEasy 3 have been dropped in 4. In particular, this includes the legacy modules (`resteasy-legacy`, `security-legacy`) that were introduced in 3.1.

In addition to the legacy modules, few other modules have been dropped for multiple different reasons, including dependency on unsupported / abandoned libraries, better options available, etc:

- `resteasy-jackson-provider`, users should rely on `resteasy-jackson2-provider` instead;
- `resteasy-jettison-provider`, users should rely on `resteasy-jackson2-provider` instead;
- `abdera-atom-provider`;
- `resteasy-yaml-provider`;
- `resteasy-rx-java`, users should rely on `resteasy-rx-java2` instead;
- `tjws`.

The `resteasy-validator-provider-11` is also gone, with the `resteasy-validator-provider` one now supporting Bean Validation 2.0.

64.4.3. Behavior changes

With the `ClientHttpEngine` based on Apache HTTP Client 4.0 having gone (it was previously deprecated) and the engine based on version 4.3 of the same library being the default, the user might want to double check the notes about connection close in Apache HTTP Client 4.3 APIs.

The conversion of `String` objects to `MediaType` objects is quite common in RESTEasy; for performances reasons a cache has been added to store the results of that conversion; by default the cache keeps the result of 200 conversions, but the number can be configured by setting the `org.jboss.resteasy.max_mediatype_cache_size` system property.

64.4.4. Other changes

- In releases 3.x, when bean validation Validation threw instances of exceptions
  - `jakarta.validation.ConstraintDefinitionException`
  - `jakarta.validation.ConstraintDeclarationException`
  - `jakarta.validation.GroupDefinitionException`

  they were wrapped in a `org.jboss.resteasy.api.validation.Resteasy.ResteasyViolationException`, which `org.jboss.resteasy.api.validation.ResteasyViolationExceptionMapper`, the built-in implementation of `jakarta.ws.rs.ext.ExceptionMapperjakarta.validation.ValidationException`, then turned into descriptive text. As of release 4.0.0, instances of `ConstraintDefinitionException`, etc., are thrown as is. They are still caught by `ResteasyViolationExceptionMapper`, so, in general, there is no detectable change. It should be noted, however, that an implementation of `ExceptionMapperResteasyViolationException`, which, prior to release 4.0.0, would have caught wrapped instances of `ConstraintDefinitionException`, will not catch unwrapped instances.

- The `ResteasyProviderFactory` is now an abstract class and is meant to be created using its `getInstance()` and `newInstance()` methods. Moreover, on client side, the resolution of the current instance is cached for each thread local context classloader.
- The `ResteasyClient` and `ResteasyClientBuilder` are now abstract classes (from `resteasy-client-api`) and are not meant for user direct instantiation; plain Jakarta RESTful Web Services API usage is expected instead:

```
//ResteasyClient client = new ResteasyClientBuilder().build(); NO!
//if plain Jakarta RESTful Web Services is enough ...
Client client = ClientBuilder.newClient();

//if RESTEasy API is needed ...
ResteasyClient client = (ResteasyClient)ClientBuilder.newClient();


//ResteasyClientBuilder builder = new ResteasyClientBuilder(); NO!
//if plain Jakarta RESTful Web Services is enough ...
ClientBuilder builder = ClientBuilder.newBuilder();

//if RESTEasy API is needed ...
ResteayClientBuilder builder = (ResteasyClientBuilder)ClientBuilder.newBuilder();
```

- The package `org.jboss.resteasy.plugins.stats` (which contains a resource and some related classes) has been moved out of the `resteasy-jaxb-provider` module into a `new resteasy-stats` module.

Chapter 65. Books You Can Read

There are a number of great books that you can learn REST and Jakarta RESTful Web Services from

- RESTful Web Services by Leonard Richardson and Sam Ruby. A great introduction to REST.
- RESTful Java with JAX-RS by Bill Burke. Overview of REST and detailed explanation of JAX-RS. Book examples are distributed with RESTEasy.
- RESTful Web Services Cookbook by Subbu Allamaraju and Mike Amundsen. Detailed cookbook on how to design RESTful services.